Qwen Forecast
Código, arquitetura e inferência
Guia completo comentado

Edição de 8 de outubro de 2026. Leitura linha a linha do projeto atual, com foco na construção de prompts e na inferência local pelo Ollama.

Inclui conversão e ingestão de CSVs textuais, prompts narrativos, adaptadores Qwen/Chronos/TimesFM, avaliação Qwen 7/15 dias, comparação de prompts, baselines clássicos 355/30/30, gráficos, CLIs, configurações, scripts e testes.

A documentação preserva o comportamento atual e evidencia limitações. O livro descreve as mudanças solicitadas no pipeline; sua geração não executa modelos nem modifica previsões.

Retrato Git: 5d64c1c. O retrato inclui alterações locais ainda não commitadas; os hashes individuais do manifesto identificam o conteúdo exato documentado.

Sumário navegável

Qwen Forecast — Código, arquitetura e inferência — Guia completo comentado 1

1. Como usar este livro e o que mudou 5

2. Arquitetura e responsabilidades 6

3. Mapa de chamadas: do CSV textual aos resultados 7

4. Uma janela concreta e a proteção temporal 8

5. Calendário em palavras: exportação e leitura 9

6. Estrutura narrativa dos prompts 10

7. Python aplicado e contratos de funções 11

8. Ollama local, recursos e reprodutibilidade 12

9. Métricas, gráficos, artefatos e limites 13

10. Mini-exemplo: palavras, HTTP e avaliação 14

11. Comparação entre prompt numérico e linguagem natural 15

12. Baselines clássicos: seleção, reajuste e janela expansiva 16

13. Como funcionam regressão linear, KNN e ARIMA neste código 17

14. Inventário e dependências entre módulos 18

15. pyproject.toml 21

16. experiments/start_ollama_context_server.sh 22

17. experiments/run_benchmark.sh 23

18. configs/benchmark.json 24

19. configs/qwen_context_15_days.json 29

20. configs/qwen_text_1_7b_7.json 34

21. configs/qwen_text_1_7b_15.json 39

22. configs/qwen_text_4b_7.json 44

23. configs/qwen_text_4b_15.json 49

24. src/pipelines/qwen_text_experiment.py 54

25. src/pipelines/forecasting_pipeline.py 64

26. src/pipelines/data_pipeline.py 78

27. src/data/ingestion/files.py 80

28. src/data/validation/schema.py 82

29. src/data/preprocessing/calendar_text.py 85

30. src/data/preprocessing/resampling.py 98

31. src/data/preprocessing/missing_values.py 99

32. src/evaluation/cross_validation.py 101

33. src/data/features/temporal.py 107

34. src/data/features/covariates.py 108

35. src/data/preprocessing/generate_prompts.py 112

36. src/models/promptcast.py 125

37. src/models/chronos.py 133

38. src/models/timesfm.py 137

39. src/evaluation/metrics.py 141

40. src/pipelines/compare_models.py 144

41. src/training/train.py 151

42. src/visualization/qwen_text_results.py 152

43. src/visualization/forecast_comparison.py 157

44. src/visualization/plots.py 163

45. src/visualization/colors.py 170

46. tests/test_calendar_prompts.py 174

47. tests/test_forecasting.py 183

48. tests/test_qwen_fixed_context.py 200

49. tests/test_seven_day_metrics.py 204

50. tests/test_text_dataset_input.py 209

51. configs/classical_baselines.json 213

52. src/data/features/autoregressive.py 226

53. src/models/classical.py 229

54. src/training/classical.py 234

55. src/pipelines/classical_baselines.py 238

56. src/visualization/comparison_results.py 249

57. tests/test_classical_baselines.py 259

Catálogo de funções e métodos de biblioteca 265

Como executar e localizar o resultado 288

Cobertura, fontes e reprodução do documento 289

1. Como usar este livro e o que mudou

Este guia mantém o formato didático da edição anterior: visão de arquitetura, uma previsão acompanhada do início ao fim, conceitos de Python aplicados, contratos de funções, código real comentado linha a linha e catálogo das APIs utilizadas. O PDF possui sumário e marcadores navegáveis; o HTML permite pesquisar nomes e trechos. Os números de linha e os hashes são extraídos dos arquivos atuais, sem reescrever suas listagens.

A edição anterior, com 174 páginas, foi analisada antes desta atualização. Sua geração combinava conteúdo editorial com análise sintática AST do Python e composição A4 pelo ReportLab. Esta edição conserva esse método, mas atualiza inventário, contratos, comentários semânticos, configurações e comandos. Não reutiliza anotações antigas em linhas cujo significado mudou.

As mudanças centrais são: calendário escrito em inglês; exportação e leitura direta dos CSVs textuais; prompts narrativos; contexto turístico de Guaratuba; Qwen restrito a sete ou quinze dias; execução comparável dos checkpoints 1.7B e 4B; métricas e gráficos específicos dessa avaliação. Não há treinamento dos pesos Qwen. Os novos baselines clássicos são ajustados por fold. O módulo train.py é uma extensão auxiliar.

Dados completos, prompts de todas as janelas e respostas extensas não são reproduzidos. O guia descreve seu esquema, sua origem e onde encontrá-los. O próprio gerador documental fica fora de src para não alterar a identidade das inferências enquanto a documentação é atualizada.

2. Arquitetura e responsabilidades

src/data/ingestion lê os arquivos; validation exige datas únicas, ordem diária e demanda válida; preprocessing transforma calendário, expõe lacunas, preenche somente com passado e constrói prompts. features define o contrato de covariáveis e separa história de futuro. models adapta a interface local do Ollama e as bibliotecas de Chronos/TimesFM. evaluation fornece origens cronológicas e métricas determinísticas. pipelines conecta essas camadas; visualization transforma previsões já concluídas em gráficos.

O padrão de adaptador está nas classes dos modelos, que expõem predict apesar de usar backends diferentes. create_model funciona como fábrica simples. Dataclasses como ForecastWindow e OllamaConfig agrupam contratos explícitos. Configurações JSON separam parâmetros dos algoritmos. Não há uma infraestrutura de treinamento escondida na conversão textual: trocar a representação do calendário não ajusta pesos nem aprende estatísticas do conjunto de teste.

O benchmark tradicional continua documentado porque Chronos e TimesFM utilizam seus CSVs numéricos e histórico crescente. O experimento textual do Qwen tem entrada e diretório próprios; a configuração marca textual_calendar=true e classifica dias, estações e indicadores como categorias textuais. As duas representações não devem ser confundidas nem passadas indiscriminadamente ao mesmo adaptador.

3. Mapa de chamadas: do CSV textual aos resultados

A entrada recomendada é python -m src.pipelines.qwen_text_experiment. main chama prepare, que percorre as quatro combinações de checkpoint/contexto, lê os dois datasets textuais, gera os 632 prompts e verifica seu tamanho conservador. --prepare-only termina nessa etapa: não faz HTTP nem inferência.

Sem essa opção, main chama run_dataset para cada uma das oito avaliações. run_dataset valida primeiro que qwen_history_days vale 7 ou 15; prepara dados/janelas; cria o adaptador Qwen; grava procedência; monta história/futuro em cada origem; escreve o prompt; verifica cache; chama predict quando necessário; valida a saída e só depois consulta a demanda real do horizonte para registrar y_true.

QwenForecaster consulta /api/tags para localizar o checkpoint instalado e /api/show para conhecer seu limite. predict monta um POST para /api/generate no Ollama local. A resposta de transporte contém outra string JSON em response; parse_prediction interpreta esse JSON interno e devolve sete floats finitos e não negativos.

Após as oito avaliações completas, summarize exige horizontes completos e alvos iguais, calcula métricas agregadas e por lead, e escreve comparações simples. --summarize-only recalcula resultados salvos sem chamar modelos. python -m src.visualization.qwen_text_results valida os resultados por summarize e gera seis gráficos. A atualização no Canva usa esses gráficos e as métricas calculadas; não existe publicação automática no código de inferência.

```
python -m src.pipelines.qwen_text_experiment
  -> prepare() -> configuration(checkpoint, dias)
     -> prepare_dataset(textual_calendar=True)
        -> read_series() -> date_from_text() -> validate_series()
        -> daily_grid() -> fill_past_only() -> build_windows()
     -> model_inputs(history_days=7 ou 15)
     -> build_prompt() -> _narrate() -> calendar_to_text()
     -> prompts/*.txt + prompt_manifest.json
  -> run_dataset() -> qwen_history_days() -> create_model()
     -> QwenForecaster: /api/tags + /api/show
     -> para cada origem: recorte -> prompt -> identidade/cache
        -> predict() -> post_json() -> /api/generate local
        -> parse_prediction() -> sete floats
        -> somente depois: y_true e registros por lead
     -> predictions.csv + status.json + responses/
  -> summarize() -> alvos iguais, sete leads, metricas e CSVs
python -m src.visualization.qwen_text_results
  -> summarize() -> quatro series em paineis + duas barras RMSE
```

4. Uma janela concreta e a proteção temporal

Em Joinville, a primeira origem é o índice 365; o alvo é df.iloc[365:372]. São sete linhas, pois o limite direito do slice é exclusivo. Para Qwen com quinze dias, history é df.iloc[350:365]; com sete dias, df.iloc[358:365]. O alvo permanece o mesmo nas duas configurações e nos dois checkpoints. Em Guaratuba, a origem inicial é 730 e aplicam-se os mesmos recortes relativos.

Na origem seguinte, o índice avança sete dias e o início da história do Qwen também avança sete. Seu tamanho permanece sete ou quinze; somente Chronos e TimesFM recebem todo o histórico desde o começo. context do dataset escolhe a primeira origem; qwen_history_days escolhe a quantidade de observações fornecidas ao LLM; num_ctx limita tokens. Essas três medidas têm funções diferentes.

model_inputs constrói future selecionando timestamp e covariáveis, sem water_produced. Verifica também que a maior data histórica precede a menor data futura. build_prompt rejeita uma coluna futura de demanda e exige exatamente as datas seguintes ao histórico. Os testes alteram alvos futuros por uma sentinela e exigem que o prompt não mude. Isso testa isolamento dos alvos; não comprova a disponibilidade operacional de toda covariável.

As configurações novas usam zero-shot. O seletor de exemplos permanece disponível: quando solicitado, recebe apenas o prefixo congelado anterior ao primeiro teste, evitando transformar alvos testados em demonstrações de treinamento. Observações de janelas anteriores podem compor o histórico observado da previsão seguinte, como definido pelo protocolo walk-forward; isso não implica ajuste de pesos do Qwen.

Guaratuba tem uma lacuna em 29/02/2016. daily_grid a expõe; observed é calculado antes do preenchimento; fill_past_only propaga dados anteriores. Alvos artificiais são excluídos das janelas de teste. Esse preenchimento não reconstrói automaticamente o calendário: se a lacuna entrasse em um contexto usado, seria necessário conferir seus atributos. Os contextos fixos avaliados começam muito depois dessa lacuna; o guia registra a limitação sem ocultar o comportamento real.

5. Calendário em palavras: exportação e leitura

calendar_text.py é a única implementação da conversão. number_to_words escreve inteiros de zero a 9999 sem depender do locale. date_to_text transforma 2016-01-01 em “two thousand and sixteen, day one of January”. O ano não é alterado: 2026 seria “two thousand and twenty-six”. Dias são cardinais, como “one”, conforme a representação escolhida.

Os pares dayofweek_sin/cos e season_sin/cos são decodificados pelo ângulo atan2(seno, cosseno), normalizado ao ciclo. O índice arredondado identifica segunda a domingo ou verão, outono, inverno e primavera, em inglês. A comparação allclose com seno/cosseno reconstruídos verifica se os valores correspondem a uma categoria válida. Estações são decodificadas do valor fornecido, em vez de aproximadas somente pelo mês; isso preserva transições como 20 de março. Um par incompleto é rejeitado.

Indicadores booleanos ou zero/um recebem expressões como “a public holiday”, “not a public holiday”, “during Carnival” e “during the Paraná school recess”. As colunas de demanda e meteorologia conservam seus números. Os pares cíclicos viram as colunas dayofweek e season; por isso o contrato textual possui colunas diferentes do contrato numérico original.

O CLI de calendar_text escreve cópias em data/processed/calendar_text, sem sobrescrever as fontes. Para prever a partir delas, read_series recebe textual_calendar=true e aplica date_from_text somente ao timestamp. words_to_number aceita a grafia canônica emitida pelo exportador; datas inválidas são rejeitadas. A conversão da data de volta a Timestamp existe para ordenação e recortes temporais; os demais campos de calendário continuam em palavras. Ao narrar, calendar_to_text aceita os indicadores e dias já convertidos, sem convertê-los novamente em códigos numéricos.

A exportação inteira não é um ajuste estatístico. Cada linha depende apenas de seus próprios valores. Ler o arquivo completo para administrar janelas não significa fornecê-lo completo ao LLM: somente os recortes permitidos são serializados em cada pedido.

6. Estrutura narrativa dos prompts

build_prompt recebe history, horizon, demonstrations opcionais, future_covariates opcionais e region. LABELS traduz nomes técnicos em descrições: temperature_mean vira average temperature e water_produced vira water production (water_produced). _narrate escreve uma sentença por variável, com valores em ordem cronológica: “From {t1} to {tobs}, the average temperature of region {region} was {valores} degrees Celsius on each day, respectively.”

A lista de datas por extenso informa a correspondência entre posição e dia. Cada sequência tem exatamente um valor por observação; respectively deixa explícito o alinhamento. Demanda é formatada com duas casas decimais; números meteorológicos usam seis algarismos significativos. Esse arredondamento pertence ao texto do prompt, sem substituir os números usados para avaliação.

As covariáveis futuras são narradas em bloco próprio como supplied inputs. Sua demanda não está presente. Ao fim, há uma pergunta “What is the water_produced going to be on ...?” para cada uma das sete datas futuras. A ordem das perguntas é a ordem exigida no array predictions. Não há tabela CSV embutida nem função alternativa de prompt tabular no pipeline atual.

Quando region começa por Guaratuba, o prompt informa que a cidade é turística e tem forte sazonalidade durante férias e recessos brasileiros, especialmente de dezembro a março. Joinville não recebe essa afirmação. Esse contexto é uma instrução geral solicitada para o modelo, não um valor calculado a partir dos alvos de teste.

Texto mais natural não garante previsão melhor. O modelo pode confundir grandezas ou produzir valores de escala inadequada e ainda respeitar o JSON. O validador verifica estrutura e domínio numérico; não altera previsões para aproximá-las da verdade observada. O desempenho precisa ser medido nos CSVs e não inferido da fluência do prompt.

7. Python aplicado e contratos de funções

Um argumento posicional é associado pela posição; um argumento nomeado é associado pelo nome. Nas assinaturas, * obriga os argumentos seguintes a serem nomeados, reduzindo trocas acidentais entre horizon, future_covariates e region. None é ausência de valor, mas não é permitido como qwen_history_days: o validador exige type(days) is int e days em (7, 15). bool é subclasse de int em Python; a comparação exata de tipo impede True de ser tratado como um número de dias.

DataFrame guarda colunas com tipos distintos: demanda e meteorologia numéricas, timestamp temporal e calendário textual. .iloc utiliza posições, enquanto .loc utiliza rótulos. slice(start, stop) não materializa dados; o recorte ocorre quando aplicado à tabela. .copy evita que a apresentação textual modifique a fonte. Iterar uma Series mantém sua ordem, essencial para que valores e datas permaneçam alinhados.

Dataclasses geram inicialização e representação; frozen=True impede reatribuir campos normais da instância, mas não torna objetos internos recursivamente imutáveis. Decoradores property expõem history e target como atributos calculados. List comprehensions criam coleções; geradores de any/all podem parar cedo. json.dumps serializa objetos; encode produz bytes; json.load lê o envelope HTTP; json.loads interpreta o texto interno do modelo.

Exceções não são previsões. run_dataset registra uma falha em status/responses e não inventa valores substitutos. O resumo exige execuções completas para impedir comparação de subconjuntos diferentes. As chamadas de mock dos testes validam contratos sem carregar modelos; os CSVs da execução real comprovam a execução do Ollama.

8. Ollama local, recursos e reprodutibilidade

As quatro configurações qwen_text usam http://127.0.0.1:11435, num_ctx=16384, num_predict=256, temperature=0 e seed=42. Os checkpoints são qwen3:1.7b e qwen3:4b-instruct-2507-q4_K_M. Os pesos precisam estar instalados localmente. O código não baixa pesos e o trabalho não instala nem modifica ambientes Conda.

num_ctx é capacidade de tokens por sequência, não número de dias; num_predict limita tokens de resposta, não a quantidade de previsões. O JSON Schema é quem exige sete números. A checagem anterior ao pedido usa bytes UTF-8 + num_predict + 512 como limite superior conservador; a contagem real de tokens pode ser menor. O adaptador também rejeita respostas incompletas e situações em que a contagem retornada alcança o limite.

think=False desativa o modo de raciocínio configurável no pedido; stream=False solicita um envelope completo; format recebe o schema; truncate=False pede que a entrada não seja truncada; keep_alive mantém o modelo carregado entre pedidos. Essas opções não constituem treinamento. Timeout controla espera HTTP e retries controla novas tentativas após falhas de transporte ou validação. Uma previsão válida porém ruim não é tentada novamente por causa de seu erro posterior.

As variáveis de start_ollama_context_server.sh configuram o processo servidor, incluindo Flash Attention, cache KV quantizado e paralelismo. Os campos runtime em configurações antigas descrevem essas opções; passá-los ao Python não reinicia o servidor. OllamaConfig reúne opções aceitas pelo adaptador.

run_id incorpora configuração, hashes de dataset e código, identidade/digest do modelo e protocolo. request_id inclui run_id, janela e prompt. Cache é reutilizado apenas para o mesmo identificador e status ok. Mudança em qualquer .py de src altera source_sha256 e invalida cache; por isso o gerador do guia fica em experiments/reports/guide_source. Determinismo configurado favorece reprodução, mas hardware/backend podem influenciar detalhes numéricos.

9. Métricas, gráficos, artefatos e limites

Cada configuração salva predictions.csv por cidade, com origin, timestamp, lead de um a sete, y_true, y_pred, contexto e procedência. status.json registra completude; responses preserva pedido, tentativas e resposta bruta; prompts guarda o texto enviado. prompt_manifest.json descreve as 632 janelas. input_audit.json registra igualdade dos prompts entre checkpoints, invariância à mutação dos alvos futuros e correspondência das datas de teste com Chronos.

comparison_<cidade>.csv possui somente modelo, contexto inicial, RMSE, R2, MAE e MAPE. Nesse resumo, contexto inicial é sete ou quinze dias efetivamente fornecidos ao LLM; em predictions.csv, initial_context continua sendo 365 ou 730, isto é, o limiar inicial das origens de avaliação. context registra o tamanho real da entrada. metrics.csv acrescenta cidade, sMAPE e contagens; metrics_by_horizon.csv separa os sete leads.

MAE é a média de |previsto-real|. RMSE é a raiz da média dos erros quadráticos sobre todos os pontos, não a média dos RMSEs das janelas. $R^2 = 1$ - soma dos erros quadráticos / soma dos desvios quadráticos da verdade em relação à sua média. R^2 negativo é permitido e indica desempenho pior que essa referência constante; não deve ser cortado para zero. MAPE é expresso em porcentagem e exclui alvos iguais a zero, registrando sua contagem; sMAPE usa denominador |real|+|previsto| e define contribuição zero quando ambos são zero.

São 54 janelas/378 pontos por configuração em Joinville e 104/728 em Guaratuba: oito avaliações, 632 janelas e 4.424 previsões. summarize verifica todos os sete leads e alvos idênticos entre configurações. O módulo gráfico produz quatro figuras com painéis separados para sete/quinze dias, uma por cidade/checkpoint, e duas comparações de RMSE. Não suaviza nem exclui previsões ruins.

O adaptador barra alvos futuros nos inputs, mas as covariáveis meteorológicas futuras são fornecidas pelos CSVs e sua disponibilidade na origem não está comprovada. As métricas são condicionais a essas entradas; não demonstram desempenho operacional livre de informação futura. Não confundir ausência de treinamento com prova de causalidade de todas as entradas.

Resultados numéricos pertencem aos CSVs gerados, não a constantes no guia. O Canva é atualizado com valores e imagens extraídos desses artefatos após conferência. Alterar o PDF não executa inferências nem atualiza automaticamente uma apresentação.

10. Mini-exemplo: palavras, HTTP e avaliação

Considere duas observações didáticas: 1 e 2 de janeiro de 2016, demandas 100 e 110 e temperaturas 20 e 21. O horizonte deste exemplo é dois apenas para acompanhar os objetos; a avaliação real mantém horizonte sete e contexto mínimo sete. O CSV textual guarda “two thousand and sixteen, day one of January” e o dia “one”. read_series interpreta a data para Timestamp, mantendo o dia em palavras.

build_prompt produz, entre outras frases: “From two thousand and sixteen, day one of January to two thousand and sixteen, day two of January, the average temperature of region Joinville was 20; 21 degrees Celsius on each day, respectively.” Outra frase descreve a demanda histórica. As perguntas usam os dias três e quatro; nenhuma delas contém a demanda real dessas datas.

prediction_schema(2) exige um objeto com predictions de dois números. json.dumps(payload).encode converte o pedido em bytes; urlopen envia o POST. Suponha response contendo a string {"predictions":[112,114]}. Primeiro json.load interpreta o envelope HTTP; depois parse_prediction interpreta essa string e devolve [112.0, 114.0]. São dois níveis de JSON, não duas previsões independentes.

Só depois da inferência a avaliação consulta a verdade [111,117]. Os erros são +1 e -3; MAE=2 e RMSE=sqrt(5), aproximadamente 2,236. Se a resposta trouxer a string “112” em lugar do número 112, o parser rejeita; se trouxer apenas um valor, rejeita antes do zip. Se trouxer dois números válidos porém muito errados, preserva-os: o erro pertence à avaliação e não deve ser corrigido usando o alvo.

11. Comparação entre prompt numérico e linguagem natural

As previsões numéricas anteriores foram preservadas: Qwen 1.7B em experiments/benchmark e Qwen 4B em experiments/qwen_15_days. As novas avaliações estão em experiments/qwen_text. comparison_results.py exige execuções completas, igualdade de origin, timestamp, lead e y_true e o mesmo digest do checkpoint. A comparação pareada usa quinze dias; a tabela também mostra linguagem natural com sete dias, sem inventar uma execução numérica de sete dias.

A redução percentual é 100 × (RMSE anterior − RMSE natural) / RMSE anterior. Para 4B, foi 5,16% em Joinville e 7,86% em Guaratuba. Para 1.7B, foi −522,33% e −400,04%, respectivamente: erro maior, não ganho. Previsões extremas permanecem nos resultados e nos gráficos, cujos painéis usam escalas verticais automáticas.

Esta é uma comparação histórica descritiva. Além da redação, mudaram calendário, conteúdo turístico e a capacidade da janela de contexto num_ctx (não o limite de tokens gerados): 8192 para 16384 no 1.7B e 131072 para 16384 no 4B. Não isola causalmente o efeito da linguagem natural. Covariáveis meteorológicas futuras continuam condicionadas à disponibilidade suposta no dataset.

experiments/prompt_comparison contém metrics.csv, prompt_change.csv, comparison_<cidade>.csv, quatro figuras Qwen e duas figuras de baselines. canva_update.json registra a apresentação atualizada: tabela clássica no slide 22, tabelas Qwen nos slides 27 e 28 e gráficos nos slides 20, 21 e 23 a 26. Esses números descrevem o estado salvo, podendo mudar se a apresentação for reorganizada.

12. Baselines clássicos: seleção, reajuste e janela expansiva

Por confirmação explícita do usuário, 355 dias de treino, 30 de validação e 30 de teste aplicam-se somente a regressão linear, ARIMA e KNN. Qwen continua com contexto fixo 7/15 e horizonte/passo 7. Chronos e TimesFM conservam o protocolo anterior. As métricas de horizonte 30 não são diretamente comparáveis às de horizonte 7.

A divisão é cronológica, sem embaralhamento. Fold 1: treino [0,355), validação [355,385), teste [385,415). Fold 2: treino [0,385), validação [385,415), teste [415,445). Fold 3: treino [0,415), validação [415,445), teste [445,475). O limite direito é exclusivo. O passo é 30 dias e o início do treino permanece zero.

Cada candidato ajusta somente o treino e prevê a validação inteira. O menor RMSE de validação escolhe os parâmetros; empates mantêm o primeiro candidato da grade. Depois há reajuste com treino + validação, ambos anteriores ao teste: 385 observações no primeiro fold, 415 no segundo. Por isso training_days e fit_days não são sinônimos. initial_context é 355 nos CSVs simples.

Durante os 30 dias de teste, nenhum valor real desse bloco entra no ajuste ou nos atrasos. Ao terminar o bloco, seus 30 dados reais tornam-se passado disponível na origem seguinte; na divisão adotada, ocupam primeiro a nova validação. Isso é avaliação temporal expansiva, não validação cruzada aleatória. select_and_forecast nem aceita alvos de teste como argumento.

Só folds completos são avaliados: 12 em Joinville e 35 em Guaratuba por modelo. A cauda de 26 dias de Guaratuba fica fora e é registrada. A preparação usa grade diária e preenchimento com passado; a construção dos folds rejeita validação/teste com demanda originalmente ausente. Foram concluídas seis execuções, 141 folds e 4230 previsões, sem falhas. O registro audit.json verifica limites e seleção das execuções salvas.

13. Como funcionam regressão linear, KNN e ARIMA neste código

Os três baselines são univariados: usam somente water_produced histórico, sem clima futuro ou calendário textual. lagged_samples cria exemplos supervisionados: com atrasos 3, [10,11,12,13] fornece X=[10,11,12] e y=13. Cada atributo precede seu alvo. A grade usa atrasos 7, 15 ou 30.

Regressão linear ajusta uma combinação linear dos atrasos com intercepto. KNN busca exemplos históricos próximos no espaço desses atrasos; testa 3, 5 ou 10 vizinhos e pesos uniformes ou inversamente proporcionais à distância. StandardScaler e o regressor ficam na mesma Pipeline, ajustada exclusivamente nos exemplos do prefixo autorizado. São três candidatos lineares e dezoito candidatos KNN por fold.

recursive_forecast prevê um dia, acrescenta a própria previsão ao histórico temporário e repete até completar 30 dias. Não usa teacher forcing com valores reais do horizonte. Essa recursão vale tanto para validação quanto para teste e pode acumular erro.

ARIMA(p,d,q) combina termos autorregressivos, diferenciação e médias móveis dos erros. Não há componente sazonal SARIMA nesta implementação. A grade tem oito ordens: (1,0,0), (2,0,0), (1,0,1), (2,0,1), (1,1,0), (2,1,0), (0,1,1), (1,1,1). Média e desvio são calculados somente no prefixo de ajuste; o desvio mínimo é 1e-8. A previsão de 30 passos é transformada de volta à escala original. O ajuste permite até 200 iterações; aviso de não convergência é tratado como falha de candidato.

Falhas ficam registradas em trials; se todos os candidatos falham, o fold falha. Valores não finitos são rejeitados. Não há substituição por previsões inventadas, recorte de negativos ou remoção de extremos nos baselines. run.json identifica configuração, hashes e limites; folds/*.json registra seleção, erros, previsões e duração. status.json aponta completude. predictions.csv contém origem, data, lead, verdade, previsão e tamanhos de treino/reajuste.

RMSE agregado em Joinville: regressão linear 13634,16; ARIMA 14719,54; KNN 14135,82. Em Guaratuba: regressão linear 2670,04; ARIMA 2920,12; KNN 2967,14. A regressão teve o menor RMSE entre estes três candidatos de modelo no protocolo executado. Os números completos estão em experiments/classical_baselines/metrics.csv. Os testes cobrem limites dos folds, escala ajustada somente no treino, seleção/reajuste sem alvos de teste e recursão real dos regressores.

14. Inventário e dependências entre módulos

pyproject.toml — 18 linhas. Configuração declarativa.

experiments/start_ollama_context_server.sh — 8 linhas. Script de execução.

experiments/run_benchmark.sh — 11 linhas. Script de execução.

configs/benchmark.json — 59 linhas. Configuração declarativa.

configs/qwen_context_15_days.json — 65 linhas. Configuração declarativa.

configs/qwen_text_1_7b_7.json — 57 linhas. Configuração declarativa.

configs/qwen_text_1_7b_15.json — 57 linhas. Configuração declarativa.

configs/qwen_text_4b_7.json — 57 linhas. Configuração declarativa.

configs/qwen_text_4b_15.json — 57 linhas. Configuração declarativa.

src/pipelines/qwen_text_experiment.py — 105 linhas. CLI das oito avaliações textuais: gera todos os prompts, executa checkpoint/contexto/cidade e resume métricas completas, alinhadas e por lead.

src/pipelines/forecasting_pipeline.py — 148 linhas. Orquestra inferência e cache. Exige Qwen com 7/15 dias, mantém histórias de baseline e preserva somente covariáveis no frame futuro. Não calcula métricas agregadas sozinho.

src/pipelines/data_pipeline.py — 11 linhas. Composição de leitura, validação, grade diária, preenchimento causal e janelas. textual_calendar seleciona a interpretação das datas do CSV de palavras.

src/data/ingestion/files.py — 22 linhas. Lê CSV numérico ou textual conforme textual_calendar. Interpreta datas escritas antes da validação e oferece escrita JSON atômica para artefatos.

src/data/validation/schema.py — 21 linhas. Contrato do CSV: datas diárias únicas e demanda finita, não negativa. Chamado por read_series. Retorna cópia ordenada, preservando covariáveis; erro invalida o dataset antes do modelo.

src/data/preprocessing/calendar_text.py — 152 linhas. Converte calendário para inglês, valida codificações cíclicas, exporta CSVs textuais e interpreta suas datas para os recortes temporais. Não treina nem consulta outras linhas para calcular uma categoria.

src/data/preprocessing/resampling.py — 8 linhas. Converte datas observadas em grade diária. Chamado por prepare_dataset antes do preenchimento. Adiciona observed para distinguir alvos reais de linhas inseridas.

src/data/preprocessing/missing_values.py — 11 linhas. Preenchimento causal. Chamado após daily_grid. Retorna tabela sem lacunas iniciais de demanda; o preenchimento indiscriminado do calendário é um defeito confirmado, não uma prática recomendada.

src/evaluation/cross_validation.py — 76 linhas. Define ForecastWindow e origens walk-forward. prepare_dataset chama build_windows. validation_split é usado por training_partition e testes, não para ajustar os modelos neste benchmark.

src/data/features/temporal.py — 6 linhas. Geração determinística das datas-alvo a partir da última data histórica. Chamado pelos dois construtores de prompt; não consulta a demanda futura.

src/data/features/covariates.py — 43 linhas. Contrato comum dos modelos: nomes de covariáveis, validação, separação history/future e conversão para TimesFM. model_inputs é a fronteira que exclui estruturalmente o alvo da entrada futura.

src/data/preprocessing/generate_prompts.py — 145 linhas. Constrói exclusivamente prompts narrativos. Recebe história e covariáveis futuras sem alvo, descreve todas as variáveis e aplica o contexto turístico somente a Guaratuba.

src/models/promptcast.py — 99 linhas. Adaptador HTTP do Qwen: configuração, schema de saída, parser, cliente JSON e classe QwenForecaster. forecasting_pipeline chama predict; post_json encapsula transporte, e parse_prediction valida sem corrigir artificialmente valores.

src/models/chronos.py — 34 linhas. Adapter de Chronos2Pipeline. Importações pesadas ficam dentro do construtor, permitindo importar o pipeline em outros ambientes. Usa covariáveis passadas/futuras e devolve o quantil 0,5 como previsão pontual.

src/models/timesfm.py — 41 linhas. Adapter de TimesFM 2.5 com XReg. Prepara capacidade suficiente para todo o histórico a cada origem e devolve somente os primeiros horizon valores. Não corta a história ao contexto inicial.

src/evaluation/metrics.py — 20 linhas. Função numérica sem I/O. Recebe vetores reais e previstos já alinhados e retorna dicionário de métricas. É chamada pelos comparadores, testes e auditoria.

src/pipelines/compare_models.py — 64 linhas. Compara execuções completas do benchmark Qwen/Chronos/TimesFM. Valida contextos fixos do Qwen e rotula corretamente sete ou quinze dias.

src/training/train.py — 11 linhas. Ponto auxiliar para uma extensão de treinamento: oferece slices cronológicos treino/validação. Não é chamado pela inferência atual nem treina pesos.

src/visualization/qwen_text_results.py — 49 linhas. Produz seis gráficos das oito avaliações textuais concluídas: quatro pares observado/previsto e duas comparações de RMSE. Usa todos os sete leads.

src/visualization/forecast_comparison.py — 60 linhas. Renderiza os resultados históricos preservados dos baselines e dos Qwen de quinze dias. É distinto dos novos gráficos do experimento textual.

src/visualization/plots.py — 90 linhas. Funções de desenho e exportação usadas pela visualização histórica, mantendo o estilo copiado de Siaqua.

src/visualization/colors.py — 46 linhas. Paleta e seleção de cores das funções de visualização histórica.

tests/test_calendar_prompts.py — 78 linhas. Testes automatizados de contratos do pipeline.

tests/test_forecasting.py — 160 linhas. Testes automatizados de contratos do pipeline.

tests/test_qwen_fixed_context.py — 40 linhas. Testes automatizados de contratos do pipeline.

tests/test_seven_day_metrics.py — 47 linhas. Testes automatizados de contratos do pipeline.

tests/test_text_dataset_input.py — 38 linhas. Testes automatizados de contratos do pipeline.

configs/classical_baselines.json — 179 linhas. Define séries, protocolo e grades dos três baselines univariados.

src/data/features/autoregressive.py — 25 linhas. Constrói atrasos anteriores ao alvo e previsão recursiva sem observações do horizonte.

src/models/classical.py — 61 linhas. Adapta regressão linear, KNN e ARIMA ao contrato fit(history)/predict(horizon).

src/training/classical.py — 32 linhas. Seleciona parâmetros pelo RMSE de validação e reajusta somente o prefixo anterior ao teste.

src/pipelines/classical_baselines.py — 106 linhas. Executa o protocolo exclusivo 355/30/30, salva procedência, auditorias e previsões.

src/visualization/comparison_results.py — 96 linhas. Compara os prompts Qwen históricos e gera gráficos separados dos baselines.

tests/test_classical_baselines.py — 53 linhas. Verifica isolamento temporal, seleção, escala e previsão recursiva.

src/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/data/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/data/features/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/data/ingestion/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/data/preprocessing/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/data/validation/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/evaluation/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/models/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/pipelines/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/training/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

src/visualization/__init__.py — vazio; identifica/organiza um pacote Python, sem função, efeito de inicialização ou chamada de inferência a explicar.

15. pyproject.toml

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: c85a2368ec54a37f42e229907d8c6070958d9fa4dc4b2ae72c3fd89b5c1178f7

Código comentado, linha a linha

```
0001  [build-system]
```

Seção que configura a ferramenta de construção do pacote.

```
0002  requires = ["setuptools>=61"]
```

Dependência de build setuptools>=61; não é comando executado no benchmark.

```
0003  build-backend = "setuptools.build_meta"
```

Backend setuptools.build_meta usado para construir/instalar o pacote.

```
0005  [project]
```

Metadados do projeto instalável.

```
0006  name = "promptcast-forecast"
```

Nome de distribuição, diferente do pacote importado src.

```
0007  version = "0.2.0"
```

Versão declarada do projeto.

```
0008  description = "Auditable rolling-origin water demand forecasting with Qwen, Chronos and
    TimesFM"
```

Descrição textual do pacote.

```
0009  requires-python = ">=3.10"
```

Versão mínima de Python declarada. A sintaxe de união | exige Python 3.10+.

```
0010  dependencies = ["numpy>=1.24", "pandas>=2"]
```

Dependências comuns; PyTorch/Chronos/TimesFM pertencem aos ambientes especializados, não são instalados por esta lista.

```
0012  [project.scripts]
```

Tabela de comandos de console que a instalação disponibiliza.

```
0013  qwen_forecast = "src.pipelines.forecasting_pipeline:main"
```

Cria qwen_forecast apontando para main de forecasting_pipeline. Não procura arquivo qwen_forecast.py.

```
0014  forecast-compare = "src.pipelines.compare_models:main"
```

Cria forecast-compare apontando para main de compare_models.

```
0016  [tool.setuptools.packages.find]
```

Configura descoberta de pacotes do setuptools.

```
0017  where = ["."]
```

Busca pacotes a partir da raiz atual.

```
0018  include = ["src*"]
```

Inclui pacotes cujo nome começa com src. A estrutura real, não nomes de arquivos antigos, determina os imports.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

16. experiments/start_ollama_context_server.sh

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: 00184c26225c9a45960d3e6a6c9afbf2094f74a251c7cd77fdca6842e9db15ef

Código comentado, linha a linha

```
0001  #!/usr/bin/env bash
```

Escolhe Bash pelo PATH via /usr/bin/env; o sistema usa essa primeira linha quando executa o script diretamente.

```
0002  set -euo pipefail
```

-e encerra em falhas não tratadas; -u rejeita variáveis inexistentes; pipefail propaga falhas de componentes de pipelines shell.

```
0003  export OLLAMA_HOST=127.0.0.1:11435
```

export disponibiliza a variável aos processos filhos. HOST escolhe interface local e porta 11435; não publica o modelo em uma interface externa.

```
0004  export OLLAMA_FLASH_ATTENTION=1
```

Habilita a opção de Flash Attention do servidor, quando suportada pelo backend.

```
0005  export OLLAMA_KV_CACHE_TYPE=q4_0
```

Configura quantização q4_0 para cache de atenção K/V; não é a tag Q4_K_M dos pesos.

```
0006  export OLLAMA_NUM_PARALLEL=1
```

Limita a uma sequência simultânea para reduzir memória e controlar a execução sequencial.

```
0007  export LLAMA_ARG_CACHE_RAM=0
```

Desabilita RAM extra de snapshots de prompts no backend usado; o KV da sequência ativa continua existindo.

```
0008  exec ollama serve
```

exec substitui o processo Bash por ollama serve. O servidor fica ativo e atende HTTP; não chama o pipeline Python por conta própria.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

17. experiments/run_benchmark.sh

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: 7c9226c3ec2387aa24ae8b98758c065b32dce9b6595aed63be59765f0870c7bf

Código comentado, linha a linha

```
0001  #!/usr/bin/env bash
```

Seleciona o interpretador Bash.

```
0002  # Use existing conda environments; never install or update packages here.
```

Comentário de política: reutilizar ambientes existentes sem instalar ou atualizar bibliotecas.

```
0003  set -euo pipefail
```

Configura tratamento estrito de falhas e variáveis.

```
0004  cd "$(dirname "$0")/.."
```

Muda à raiz do projeto usando a localização do script, não a pasta de onde o usuário o chamou.

```
0005  CONDA_ENVS="${CONDA_ENVS:-/home/wagner-barbosa/miniconda3/envs}"
```

Escolhe raiz dos ambientes conda por variável opcional ou padrão; mantém os ambientes intactos.

```
0006  mkdir -p experiments/benchmark/logs
```

Cria diretório de logs do benchmark.

```
0007  "$CONDA_ENVS/promptcast/bin/python" -m src.pipelines.forecasting_pipeline --model qwen >
    experiments/benchmark/logs/qwen.log 2>&1
```

Executa Qwen no ambiente promptcast com configuração padrão benchmark.json e redireciona toda saída ao log. Isso usa Qwen3 1.7B, não o JSON do 4B.

```
0008  ollama stop qwen3:1.7b
```

Solicita ao servidor Ollama padrão descarregar o modelo 1.7B para liberar recursos antes das baselines. Não apaga pesos em disco.

```
0009  HF_HUB_OFFLINE=1 "$CONDA_ENVS/chronos/bin/python" -m src.pipelines.forecasting_pipeline
    --model chronos > experiments/benchmark/logs/chronos.log 2>&1
```

Executa Chronos no ambiente chronos; HF_HUB_OFFLINE=1 vale para esse processo e impede acesso ao Hub. Log captura saída/erros.

```
0010  HF_HUB_OFFLINE=1 XLA_PYTHON_CLIENT_PREALLOCATE=false "$CONDA_ENVS/timesfm/bin/python" -m
    src.pipelines.forecasting_pipeline --model timesfm > experiments/benchmark/logs/timesfm.log
    2>&1
```

Executa TimesFM no ambiente timesfm. HF_HUB_OFFLINE vale para o Hub; XLA_PYTHON_CLIENT_PREALLOCATE=false desabilita pré-alocação XLA se esse backend for usado. O caminho principal do modelo aqui é PyTorch.

```
0011  "$CONDA_ENVS/promptcast/bin/python" -m src.pipelines.compare_models
```

Invoca comparador dos três modelos após todas as inferências terminarem com sucesso.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

18. configs/benchmark.json

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: 95c8957574f7180762877b8d22ae2ccde79bb2e0c7f70248fc81e485d9f18d88

Código comentado, linha a linha

```
0001  {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0002    "datasets": {
```

Mapeamento dos dois datasets; cada bloco fornece caminho, primeira origem e contrato de covariáveis. JSON é lido por json.loads; não executa chamadas por si só.

```
0003      "joinville": {
```

Bloco da série de Joinville. Contexto inicial de avaliação: 365 dias; total atual 54 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0004        "path": "data/processed/joinville_processed_1day.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0005        "context": 365,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0006        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0007          "precipitation_median",
```

Nome de covariável precipitation_median. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0008          "temperature_2m_previsao_mean",
```

Nome de covariável temperature_2m_previsao_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0009          "shortwave_radiation_previsao_std",
```

Nome de covariável shortwave_radiation_previsao_std. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0010          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0011          "dayofweek_sin",
```

Nome de covariável dayofweek_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0012          "dayofweek_cos",
```

Nome de covariável dayofweek_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0013          "season_sin",
```

Nome de covariável season_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0014          "season_cos"
```

Nome de covariável season_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0015        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0016        "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0017          "is_holiday"
```

Nome de covariável is_holiday. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0018        ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0019      },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0020      "guaratuba": {
```

Bloco da série de Guaratuba. Contexto inicial de avaliação: 730 dias; total atual 104 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0021        "path": "data/processed/guaratuba_processed_1daysin.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0022        "context": 730,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0023        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0024          "relative_humidity_mean",
```

Nome de covariável relative_humidity_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0025          "temperature_mean",
```

Nome de covariável temperature_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0026          "radiation_mean",
```

Nome de covariável radiation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0027          "precipitation_mean",
```

Nome de covariável precipitation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0028          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0029         "dayofweek_sin",
```

Nome de covariável dayofweek_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0030         "dayofweek_cos",
```

Nome de covariável dayofweek_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0031         "season_sin",
```

Nome de covariável season_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0032         "season_cos"
```

Nome de covariável season_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0033       ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0034       "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0035         "is_holiday_ctba_gtba_jve",
```

Nome de covariável is_holiday_ctba_gtba_jve. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0036         "is_weekend",
```

Nome de covariável is_weekend. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0037         "is_carnival",
```

Nome de covariável is_carnival. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0038         "is_school_recess_pr"
```

Nome de covariável is_school_recess_pr. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0039       ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0040     }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0041   },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0042   "horizon": 7,
```

Sete valores futuros por origem; o alvo é [origin, origin+7). JSON é lido por json.loads; não executa chamadas por si só.

```
0043   "step": 7,
```

Avança sete dias entre origens; com horizonte sete as janelas avaliadas não se sobrepõem. JSON é lido por json.loads; não executa chamadas por si só.

```
0044    "validation_fraction": 0.2,
```

Proporção 0,2 para validação em uma extensão de treino. A inferência zero-shot atual não a usa para otimizar parâmetros. JSON é lido por json.loads; não executa chamadas por si só.

```
0045    "seed": 42,
```

Semente global transmitida ao Qwen/seleção de exemplos. Os adaptadores numéricos usam 42 internamente. JSON é lido por json.loads; não executa chamadas por si só.

```
0046    "qwen": {
```

Parâmetros que **config["qwen"] expande em OllamaConfig; chaves precisam corresponder aos campos da dataclass. JSON é lido por json.loads; não executa chamadas por si só.

```
0047      "model": "qwen3:1.7b",
```

Tag exata do checkpoint instalado no Ollama; o digest é consultado no servidor e guardado em run.json. JSON é lido por json.loads; não executa chamadas por si só.

```
0048      "base_url": "http://127.0.0.1:11434",
```

Endereço HTTP local do Ollama. As quatro configurações qwen_text usam a porta 11435; benchmark.json conserva sua porta própria. JSON é lido por json.loads; não executa chamadas por si só.

```
0049      "temperature": 0,
```

Temperatura de geração enviada em options; zero não equivale a treinamento nem a garantia absoluta de determinismo. JSON é lido por json.loads; não executa chamadas por si só.

```
0050      "num_ctx": 16384,
```

Capacidade de tokens do servidor por sequência. Configurações textuais usam 16384, verificados contra limite do modelo e tamanho conservador do prompt. JSON é lido por json.loads; não executa chamadas por si só.

```
0051      "num_predict": 256,
```

Limite de tokens gerados pelo Ollama. Não substitui horizon, que é quantidade de números previstos. JSON é lido por json.loads; não executa chamadas por si só.

```
0052      "timeout": 600,
```

Limite de espera HTTP do cliente em segundos. Pode durar mais de uma hora no caso 7200. JSON é lido por json.loads; não executa chamadas por si só.

```
0053      "retries": 2
```

Tentativas adicionais depois da primeira; retries=1 permite duas chamadas para a mesma janela. JSON é lido por json.loads; não executa chamadas por si só.

```
0054    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0055    "strategy": "zero",
```

zero retorna nenhuma demonstração; few_random/few_similar só são aceitos no layout legado. JSON é lido por json.loads; não executa chamadas por si só.

```
0056    "few_shot_examples": 3,
```

Quantidade de exemplos para estratégias few-shot. Ignorada efetivamente quando strategy=zero. JSON é lido por json.loads; não executa chamadas por si só.

```
0057    "qwen_history_days": 15,
```

Inteiro obrigatório 7 ou 15 para Qwen. Representa histórico fixo em observações diárias; valor nulo ou outro tamanho é rejeitado. JSON é lido por json.loads; não executa chamadas por si só.

```
0058    "future_covariates_policy": "supplied CSV values, as in original scripts; availability at
    forecast origin is an unverified input assumption"
```

Texto de procedência armazenado com o experimento. A redação antiga diz hipótese não verificada; posteriormente o usuário confirmou disponibilidade na origem. Preservar o JSON preserva o hash histórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0059  }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

19. configs/qwen_context_15_days.json

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: f1c1dae3c49ccde40b14eb5be5fa42887f62354001ff5b3a33489e8e11569406

Código comentado, linha a linha

```
0001  {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0002    "datasets": {
```

Mapeamento dos dois datasets; cada bloco fornece caminho, primeira origem e contrato de covariáveis. JSON é lido por json.loads; não executa chamadas por si só.

```
0003      "joinville": {
```

Bloco da série de Joinville. Contexto inicial de avaliação: 365 dias; total atual 54 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0004        "path": "data/processed/joinville_processed_1day.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0005        "context": 365,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0006        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0007          "precipitation_median",
```

Nome de covariável precipitation_median. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0008          "temperature_2m_previsao_mean",
```

Nome de covariável temperature_2m_previsao_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0009          "shortwave_radiation_previsao_std",
```

Nome de covariável shortwave_radiation_previsao_std. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0010          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0011          "dayofweek_sin",
```

Nome de covariável dayofweek_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0012          "dayofweek_cos",
```

Nome de covariável dayofweek_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0013          "season_sin",
```

Nome de covariável season_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0014          "season_cos"
```

Nome de covariável season_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0015        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0016        "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0017          "is_holiday"
```

Nome de covariável is_holiday. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0018        ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0019      },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0020      "guaratuba": {
```

Bloco da série de Guaratuba. Contexto inicial de avaliação: 730 dias; total atual 104 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0021        "path": "data/processed/guaratuba_processed_1daysin.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0022        "context": 730,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0023        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0024          "relative_humidity_mean",
```

Nome de covariável relative_humidity_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0025          "temperature_mean",
```

Nome de covariável temperature_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0026          "radiation_mean",
```

Nome de covariável radiation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0027          "precipitation_mean",
```

Nome de covariável precipitation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0028          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0029         "dayofweek_sin",
```

Nome de covariável dayofweek_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0030         "dayofweek_cos",
```

Nome de covariável dayofweek_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0031         "season_sin",
```

Nome de covariável season_sin. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0032         "season_cos"
```

Nome de covariável season_cos. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0033       ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0034       "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0035         "is_holiday_ctba_gtba_jve",
```

Nome de covariável is_holiday_ctba_gtba_jve. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0036         "is_weekend",
```

Nome de covariável is_weekend. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0037         "is_carnival",
```

Nome de covariável is_carnival. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0038         "is_school_recess_pr"
```

Nome de covariável is_school_recess_pr. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0039       ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0040     }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0041   },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0042   "horizon": 7,
```

Sete valores futuros por origem; o alvo é [origin, origin+7). JSON é lido por json.loads; não executa chamadas por si só.

```
0043   "step": 7,
```

Avança sete dias entre origens; com horizonte sete as janelas avaliadas não se sobrepõem. JSON é lido por json.loads; não executa chamadas por si só.

```
0044    "validation_fraction": 0.2,
```

Proporção 0,2 para validação em uma extensão de treino. A inferência zero-shot atual não a usa para otimizar parâmetros. JSON é lido por json.loads; não executa chamadas por si só.

```
0045    "seed": 42,
```

Semente global transmitida ao Qwen/seleção de exemplos. Os adaptadores numéricos usam 42 internamente. JSON é lido por json.loads; não executa chamadas por si só.

```
0046    "qwen": {
```

Parâmetros que **config["qwen"] expande em OllamaConfig; chaves precisam corresponder aos campos da dataclass. JSON é lido por json.loads; não executa chamadas por si só.

```
0047      "model": "qwen3:4b-instruct-2507-q4_K_M",
```

Tag exata do checkpoint instalado no Ollama; o digest é consultado no servidor e guardado em run.json. JSON é lido por json.loads; não executa chamadas por si só.

```
0048      "base_url": "http://127.0.0.1:11435",
```

Endereço HTTP local do Ollama. As quatro configurações qwen_text usam a porta 11435; benchmark.json conserva sua porta própria. JSON é lido por json.loads; não executa chamadas por si só.

```
0049      "temperature": 0,
```

Temperatura de geração enviada em options; zero não equivale a treinamento nem a garantia absoluta de determinismo. JSON é lido por json.loads; não executa chamadas por si só.

```
0050      "num_ctx": 131072,
```

Capacidade de tokens do servidor por sequência. Configurações textuais usam 16384, verificados contra limite do modelo e tamanho conservador do prompt. JSON é lido por json.loads; não executa chamadas por si só.

```
0051      "num_predict": 256,
```

Limite de tokens gerados pelo Ollama. Não substitui horizon, que é quantidade de números previstos. JSON é lido por json.loads; não executa chamadas por si só.

```
0052      "timeout": 7200,
```

Limite de espera HTTP do cliente em segundos. Pode durar mais de uma hora no caso 7200. JSON é lido por json.loads; não executa chamadas por si só.

```
0053      "retries": 1
```

Tentativas adicionais depois da primeira; retries=1 permite duas chamadas para a mesma janela. JSON é lido por json.loads; não executa chamadas por si só.

```
0054    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0055    "strategy": "zero",
```

zero retorna nenhuma demonstração; few_random/few_similar só são aceitos no layout legado. JSON é lido por json.loads; não executa chamadas por si só.

```
0056    "few_shot_examples": 3,
```

Quantidade de exemplos para estratégias few-shot. Ignorada efetivamente quando strategy=zero. JSON é lido por json.loads; não executa chamadas por si só.

```
0057    "qwen_history_days": 15,
```

Inteiro obrigatório 7 ou 15 para Qwen. Representa histórico fixo em observações diárias; valor nulo ou outro tamanho é rejeitado. JSON é lido por json.loads; não executa chamadas por si só.

```
0058    "future_covariates_policy": "supplied CSV values, as in original scripts; availability at
    forecast origin is an unverified input assumption",
```

Texto de procedência armazenado com o experimento. A redação antiga diz hipótese não verificada; posteriormente o usuário confirmou disponibilidade na origem. Preservar o JSON preserva o hash histórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0059    "runtime": {
```

Registro declarativo das opções pretendidas de servidor. forecasting_pipeline não exporta essas variáveis automaticamente; o script do servidor precisa aplicá-las. JSON é lido por json.loads; não executa chamadas por si só.

```
0060      "ollama_flash_attention": true,
```

Registro da intenção de usar Flash Attention; aplicada por OLLAMA_FLASH_ATTENTION no script do servidor. JSON é lido por json.loads; não executa chamadas por si só.

```
0061      "ollama_kv_cache_type": "q4_0",
```

Registro de cache KV q4_0; diferente da quantização Q4_K_M dos pesos. JSON é lido por json.loads; não executa chamadas por si só.

```
0062      "ollama_num_parallel": 1,
```

Uma sequência simultânea neste experimento para limitar memória e facilitar reutilização de prefixo. JSON é lido por json.loads; não executa chamadas por si só.

```
0063      "llama_arg_cache_ram": 0
```

Zero desabilita cache adicional de snapshots; não desliga estados KV da sequência ativa. JSON é lido por json.loads; não executa chamadas por si só.

```
0064    }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0065  }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

20. configs/qwen_text_1_7b_7.json

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: 5c080ec9f035263d2f07df25b8d2cc674d3970cf5e9ba41aae4442da4b45922e

Código comentado, linha a linha

```
0001  {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0002    "datasets": {
```

Mapeamento dos dois datasets; cada bloco fornece caminho, primeira origem e contrato de covariáveis. JSON é lido por json.loads; não executa chamadas por si só.

```
0003      "joinville": {
```

Bloco da série de Joinville. Contexto inicial de avaliação: 365 dias; total atual 54 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0004        "path": "data/processed/calendar_text/joinville_processed_1day_calendar_text.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0005        "context": 365,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0006        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0007          "precipitation_median",
```

Nome de covariável precipitation_median. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0008          "temperature_2m_previsao_mean",
```

Nome de covariável temperature_2m_previsao_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0009          "shortwave_radiation_previsao_std"
```

Nome de covariável shortwave_radiation_previsao_std. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0010        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0011        "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0012          "is_holiday",
```

Nome de covariável is_holiday. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0013          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0014          "dayofweek",
```

Nome de covariável dayofweek. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0015          "season"
```

Nome de covariável season. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0016        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0017        "textual_calendar": true
```

True ativa date_from_text na leitura do CSV para permitir os recortes temporais. Demais covariáveis de calendário permanecem palavras. JSON é lido por json.loads; não executa chamadas por si só.

```
0018      },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0019      "guaratuba": {
```

Bloco da série de Guaratuba. Contexto inicial de avaliação: 730 dias; total atual 104 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0020        "path": "data/processed/calendar_text/guaratuba_processed_1daysin_calendar_text.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0021        "context": 730,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0022        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0023          "relative_humidity_mean",
```

Nome de covariável relative_humidity_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0024          "temperature_mean",
```

Nome de covariável temperature_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0025          "radiation_mean",
```

Nome de covariável radiation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0026          "precipitation_mean"
```

Nome de covariável precipitation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0027        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0028        "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0029         "is_holiday_ctba_gtba_jve",
```

Nome de covariável is_holiday_ctba_gtba_jve. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0030         "is_weekend",
```

Nome de covariável is_weekend. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0031         "is_carnival",
```

Nome de covariável is_carnival. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0032         "is_school_recess_pr",
```

Nome de covariável is_school_recess_pr. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0033         "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0034         "dayofweek",
```

Nome de covariável dayofweek. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0035         "season"
```

Nome de covariável season. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0036       ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0037       "textual_calendar": true
```

True ativa date_from_text na leitura do CSV para permitir os recortes temporais. Demais covariáveis de calendário permanecem palavras. JSON é lido por json.loads; não executa chamadas por si só.

```
0038     }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0039   },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0040   "horizon": 7,
```

Sete valores futuros por origem; o alvo é [origin, origin+7). JSON é lido por json.loads; não executa chamadas por si só.

```
0041   "step": 7,
```

Avança sete dias entre origens; com horizonte sete as janelas avaliadas não se sobrepõem. JSON é lido por json.loads; não executa chamadas por si só.

```
0042   "validation_fraction": 0.2,
```

Proporção 0,2 para validação em uma extensão de treino. A inferência zero-shot atual não a usa para otimizar parâmetros. JSON é lido por json.loads; não executa chamadas por si só.

```
0043   "seed": 42,
```

Semente global transmitida ao Qwen/seleção de exemplos. Os adaptadores numéricos usam 42 internamente. JSON é lido por json.loads; não executa chamadas por si só.

```
0044    "qwen": {
```

Parâmetros que **config["qwen"] expande em OllamaConfig; chaves precisam corresponder aos campos da dataclass. JSON é lido por json.loads; não executa chamadas por si só.

```
0045      "model": "qwen3:1.7b",
```

Tag exata do checkpoint instalado no Ollama; o digest é consultado no servidor e guardado em run.json. JSON é lido por json.loads; não executa chamadas por si só.

```
0046      "base_url": "http://127.0.0.1:11435",
```

Endereço HTTP local do Ollama. As quatro configurações qwen_text usam a porta 11435; benchmark.json conserva sua porta própria. JSON é lido por json.loads; não executa chamadas por si só.

```
0047      "temperature": 0,
```

Temperatura de geração enviada em options; zero não equivale a treinamento nem a garantia absoluta de determinismo. JSON é lido por json.loads; não executa chamadas por si só.

```
0048      "num_ctx": 16384,
```

Capacidade de tokens do servidor por sequência. Configurações textuais usam 16384, verificados contra limite do modelo e tamanho conservador do prompt. JSON é lido por json.loads; não executa chamadas por si só.

```
0049      "num_predict": 256,
```

Limite de tokens gerados pelo Ollama. Não substitui horizon, que é quantidade de números previstos. JSON é lido por json.loads; não executa chamadas por si só.

```
0050      "timeout": 600,
```

Limite de espera HTTP do cliente em segundos. Pode durar mais de uma hora no caso 7200. JSON é lido por json.loads; não executa chamadas por si só.

```
0051      "retries": 2
```

Tentativas adicionais depois da primeira; retries=1 permite duas chamadas para a mesma janela. JSON é lido por json.loads; não executa chamadas por si só.

```
0052    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0053    "strategy": "zero",
```

zero retorna nenhuma demonstração; few_random/few_similar só são aceitos no layout legado. JSON é lido por json.loads; não executa chamadas por si só.

```
0054    "few_shot_examples": 3,
```

Quantidade de exemplos para estratégias few-shot. Ignorada efetivamente quando strategy=zero. JSON é lido por json.loads; não executa chamadas por si só.

```
0055    "qwen_history_days": 7,
```

Inteiro obrigatório 7 ou 15 para Qwen. Representa histórico fixo em observações diárias; valor nulo ou outro tamanho é rejeitado. JSON é lido por json.loads; não executa chamadas por si só.

```
0056    "future_covariates_policy": "supplied CSV values, as in original scripts; availability at
    forecast origin is an unverified input assumption"
```

Texto de procedência armazenado com o experimento. A redação antiga diz hipótese não verificada; posteriormente o usuário confirmou disponibilidade na origem. Preservar o JSON preserva o hash histórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0057  }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

21. configs/qwen_text_1_7b_15.json

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: bf4f49f761d9d98591a9df5adfa0b64411189bdbb9f987d706be5718e9608ee8

Código comentado, linha a linha

```
0001  {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0002    "datasets": {
```

Mapeamento dos dois datasets; cada bloco fornece caminho, primeira origem e contrato de covariáveis. JSON é lido por json.loads; não executa chamadas por si só.

```
0003      "joinville": {
```

Bloco da série de Joinville. Contexto inicial de avaliação: 365 dias; total atual 54 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0004        "path": "data/processed/calendar_text/joinville_processed_1day_calendar_text.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0005        "context": 365,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0006        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0007          "precipitation_median",
```

Nome de covariável precipitation_median. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0008          "temperature_2m_previsao_mean",
```

Nome de covariável temperature_2m_previsao_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0009          "shortwave_radiation_previsao_std"
```

Nome de covariável shortwave_radiation_previsao_std. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0010        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0011        "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0012          "is_holiday",
```

Nome de covariável is_holiday. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0013          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0014          "dayofweek",
```

Nome de covariável dayofweek. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0015          "season"
```

Nome de covariável season. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0016        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0017        "textual_calendar": true
```

True ativa date_from_text na leitura do CSV para permitir os recortes temporais. Demais covariáveis de calendário permanecem palavras. JSON é lido por json.loads; não executa chamadas por si só.

```
0018      },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0019      "guaratuba": {
```

Bloco da série de Guaratuba. Contexto inicial de avaliação: 730 dias; total atual 104 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0020        "path": "data/processed/calendar_text/guaratuba_processed_1daysin_calendar_text.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0021        "context": 730,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0022        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0023          "relative_humidity_mean",
```

Nome de covariável relative_humidity_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0024          "temperature_mean",
```

Nome de covariável temperature_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0025          "radiation_mean",
```

Nome de covariável radiation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0026          "precipitation_mean"
```

Nome de covariável precipitation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0027        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0028        "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0029         "is_holiday_ctba_gtba_jve",
```

Nome de covariável is_holiday_ctba_gtba_jve. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0030         "is_weekend",
```

Nome de covariável is_weekend. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0031         "is_carnival",
```

Nome de covariável is_carnival. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0032         "is_school_recess_pr",
```

Nome de covariável is_school_recess_pr. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0033         "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0034         "dayofweek",
```

Nome de covariável dayofweek. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0035         "season"
```

Nome de covariável season. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0036       ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0037       "textual_calendar": true
```

True ativa date_from_text na leitura do CSV para permitir os recortes temporais. Demais covariáveis de calendário permanecem palavras. JSON é lido por json.loads; não executa chamadas por si só.

```
0038     }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0039   },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0040   "horizon": 7,
```

Sete valores futuros por origem; o alvo é [origin, origin+7). JSON é lido por json.loads; não executa chamadas por si só.

```
0041   "step": 7,
```

Avança sete dias entre origens; com horizonte sete as janelas avaliadas não se sobrepõem. JSON é lido por json.loads; não executa chamadas por si só.

```
0042   "validation_fraction": 0.2,
```

Proporção 0,2 para validação em uma extensão de treino. A inferência zero-shot atual não a usa para otimizar parâmetros. JSON é lido por json.loads; não executa chamadas por si só.

```
0043   "seed": 42,
```

Semente global transmitida ao Qwen/seleção de exemplos. Os adaptadores numéricos usam 42 internamente. JSON é lido por json.loads; não executa chamadas por si só.

```
0044    "qwen": {
```

Parâmetros que **config["qwen"] expande em OllamaConfig; chaves precisam corresponder aos campos da dataclass. JSON é lido por json.loads; não executa chamadas por si só.

```
0045      "model": "qwen3:1.7b",
```

Tag exata do checkpoint instalado no Ollama; o digest é consultado no servidor e guardado em run.json. JSON é lido por json.loads; não executa chamadas por si só.

```
0046      "base_url": "http://127.0.0.1:11435",
```

Endereço HTTP local do Ollama. As quatro configurações qwen_text usam a porta 11435; benchmark.json conserva sua porta própria. JSON é lido por json.loads; não executa chamadas por si só.

```
0047      "temperature": 0,
```

Temperatura de geração enviada em options; zero não equivale a treinamento nem a garantia absoluta de determinismo. JSON é lido por json.loads; não executa chamadas por si só.

```
0048      "num_ctx": 16384,
```

Capacidade de tokens do servidor por sequência. Configurações textuais usam 16384, verificados contra limite do modelo e tamanho conservador do prompt. JSON é lido por json.loads; não executa chamadas por si só.

```
0049      "num_predict": 256,
```

Limite de tokens gerados pelo Ollama. Não substitui horizon, que é quantidade de números previstos. JSON é lido por json.loads; não executa chamadas por si só.

```
0050      "timeout": 600,
```

Limite de espera HTTP do cliente em segundos. Pode durar mais de uma hora no caso 7200. JSON é lido por json.loads; não executa chamadas por si só.

```
0051      "retries": 2
```

Tentativas adicionais depois da primeira; retries=1 permite duas chamadas para a mesma janela. JSON é lido por json.loads; não executa chamadas por si só.

```
0052    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0053    "strategy": "zero",
```

zero retorna nenhuma demonstração; few_random/few_similar só são aceitos no layout legado. JSON é lido por json.loads; não executa chamadas por si só.

```
0054    "few_shot_examples": 3,
```

Quantidade de exemplos para estratégias few-shot. Ignorada efetivamente quando strategy=zero. JSON é lido por json.loads; não executa chamadas por si só.

```
0055    "qwen_history_days": 15,
```

Inteiro obrigatório 7 ou 15 para Qwen. Representa histórico fixo em observações diárias; valor nulo ou outro tamanho é rejeitado. JSON é lido por json.loads; não executa chamadas por si só.

```
0056    "future_covariates_policy": "supplied CSV values, as in original scripts; availability at
    forecast origin is an unverified input assumption"
```

Texto de procedência armazenado com o experimento. A redação antiga diz hipótese não verificada; posteriormente o usuário confirmou disponibilidade na origem. Preservar o JSON preserva o hash histórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0057  }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

22. configs/qwen_text_4b_7.json

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: ab650eee3dbc4bd67941e06ba720d20212292c04ee8ebaa46f1758b2f159c0cc

Código comentado, linha a linha

```
0001  {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0002    "datasets": {
```

Mapeamento dos dois datasets; cada bloco fornece caminho, primeira origem e contrato de covariáveis. JSON é lido por json.loads; não executa chamadas por si só.

```
0003      "joinville": {
```

Bloco da série de Joinville. Contexto inicial de avaliação: 365 dias; total atual 54 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0004        "path": "data/processed/calendar_text/joinville_processed_1day_calendar_text.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0005        "context": 365,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0006        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0007          "precipitation_median",
```

Nome de covariável precipitation_median. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0008          "temperature_2m_previsao_mean",
```

Nome de covariável temperature_2m_previsao_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0009          "shortwave_radiation_previsao_std"
```

Nome de covariável shortwave_radiation_previsao_std. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0010        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0011        "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0012          "is_holiday",
```

Nome de covariável is_holiday. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0013          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0014          "dayofweek",
```

Nome de covariável dayofweek. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0015          "season"
```

Nome de covariável season. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0016        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0017        "textual_calendar": true
```

True ativa date_from_text na leitura do CSV para permitir os recortes temporais. Demais covariáveis de calendário permanecem palavras. JSON é lido por json.loads; não executa chamadas por si só.

```
0018      },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0019      "guaratuba": {
```

Bloco da série de Guaratuba. Contexto inicial de avaliação: 730 dias; total atual 104 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0020        "path": "data/processed/calendar_text/guaratuba_processed_1daysin_calendar_text.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0021        "context": 730,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0022        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0023          "relative_humidity_mean",
```

Nome de covariável relative_humidity_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0024          "temperature_mean",
```

Nome de covariável temperature_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0025          "radiation_mean",
```

Nome de covariável radiation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0026          "precipitation_mean"
```

Nome de covariável precipitation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0027        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0028        "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0029         "is_holiday_ctba_gtba_jve",
```

Nome de covariável is_holiday_ctba_gtba_jve. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0030         "is_weekend",
```

Nome de covariável is_weekend. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0031         "is_carnival",
```

Nome de covariável is_carnival. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0032         "is_school_recess_pr",
```

Nome de covariável is_school_recess_pr. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0033         "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0034         "dayofweek",
```

Nome de covariável dayofweek. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0035         "season"
```

Nome de covariável season. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0036       ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0037       "textual_calendar": true
```

True ativa date_from_text na leitura do CSV para permitir os recortes temporais. Demais covariáveis de calendário permanecem palavras. JSON é lido por json.loads; não executa chamadas por si só.

```
0038     }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0039   },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0040   "horizon": 7,
```

Sete valores futuros por origem; o alvo é [origin, origin+7). JSON é lido por json.loads; não executa chamadas por si só.

```
0041   "step": 7,
```

Avança sete dias entre origens; com horizonte sete as janelas avaliadas não se sobrepõem. JSON é lido por json.loads; não executa chamadas por si só.

```
0042   "validation_fraction": 0.2,
```

Proporção 0,2 para validação em uma extensão de treino. A inferência zero-shot atual não a usa para otimizar parâmetros. JSON é lido por json.loads; não executa chamadas por si só.

```
0043   "seed": 42,
```

Semente global transmitida ao Qwen/seleção de exemplos. Os adaptadores numéricos usam 42 internamente. JSON é lido por json.loads; não executa chamadas por si só.

```
0044    "qwen": {
```

Parâmetros que **config["qwen"] expande em OllamaConfig; chaves precisam corresponder aos campos da dataclass. JSON é lido por json.loads; não executa chamadas por si só.

```
0045      "model": "qwen3:4b-instruct-2507-q4_K_M",
```

Tag exata do checkpoint instalado no Ollama; o digest é consultado no servidor e guardado em run.json. JSON é lido por json.loads; não executa chamadas por si só.

```
0046      "base_url": "http://127.0.0.1:11435",
```

Endereço HTTP local do Ollama. As quatro configurações qwen_text usam a porta 11435; benchmark.json conserva sua porta própria. JSON é lido por json.loads; não executa chamadas por si só.

```
0047      "temperature": 0,
```

Temperatura de geração enviada em options; zero não equivale a treinamento nem a garantia absoluta de determinismo. JSON é lido por json.loads; não executa chamadas por si só.

```
0048      "num_ctx": 16384,
```

Capacidade de tokens do servidor por sequência. Configurações textuais usam 16384, verificados contra limite do modelo e tamanho conservador do prompt. JSON é lido por json.loads; não executa chamadas por si só.

```
0049      "num_predict": 256,
```

Limite de tokens gerados pelo Ollama. Não substitui horizon, que é quantidade de números previstos. JSON é lido por json.loads; não executa chamadas por si só.

```
0050      "timeout": 600,
```

Limite de espera HTTP do cliente em segundos. Pode durar mais de uma hora no caso 7200. JSON é lido por json.loads; não executa chamadas por si só.

```
0051      "retries": 2
```

Tentativas adicionais depois da primeira; retries=1 permite duas chamadas para a mesma janela. JSON é lido por json.loads; não executa chamadas por si só.

```
0052    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0053    "strategy": "zero",
```

zero retorna nenhuma demonstração; few_random/few_similar só são aceitos no layout legado. JSON é lido por json.loads; não executa chamadas por si só.

```
0054    "few_shot_examples": 3,
```

Quantidade de exemplos para estratégias few-shot. Ignorada efetivamente quando strategy=zero. JSON é lido por json.loads; não executa chamadas por si só.

```
0055    "qwen_history_days": 7,
```

Inteiro obrigatório 7 ou 15 para Qwen. Representa histórico fixo em observações diárias; valor nulo ou outro tamanho é rejeitado. JSON é lido por json.loads; não executa chamadas por si só.

```
0056    "future_covariates_policy": "supplied CSV values, as in original scripts; availability at
    forecast origin is an unverified input assumption"
```

Texto de procedência armazenado com o experimento. A redação antiga diz hipótese não verificada; posteriormente o usuário confirmou disponibilidade na origem. Preservar o JSON preserva o hash histórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0057  }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

23. configs/qwen_text_4b_15.json

Arquivo de configuração/execução que conecta parâmetros e módulos.

SHA-256: d2a0fa7ae26bf8c71ca8fed04a0718afa3caa1cc4ea20d11deca61e16e151397

Código comentado, linha a linha

```
0001  {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0002    "datasets": {
```

Mapeamento dos dois datasets; cada bloco fornece caminho, primeira origem e contrato de covariáveis. JSON é lido por json.loads; não executa chamadas por si só.

```
0003      "joinville": {
```

Bloco da série de Joinville. Contexto inicial de avaliação: 365 dias; total atual 54 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0004        "path": "data/processed/calendar_text/joinville_processed_1day_calendar_text.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0005        "context": 365,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0006        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0007          "precipitation_median",
```

Nome de covariável precipitation_median. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0008          "temperature_2m_previsao_mean",
```

Nome de covariável temperature_2m_previsao_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0009          "shortwave_radiation_previsao_std"
```

Nome de covariável shortwave_radiation_previsao_std. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0010        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0011        "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0012          "is_holiday",
```

Nome de covariável is_holiday. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0013          "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0014          "dayofweek",
```

Nome de covariável dayofweek. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0015          "season"
```

Nome de covariável season. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0016        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0017        "textual_calendar": true
```

True ativa date_from_text na leitura do CSV para permitir os recortes temporais. Demais covariáveis de calendário permanecem palavras. JSON é lido por json.loads; não executa chamadas por si só.

```
0018      },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0019      "guaratuba": {
```

Bloco da série de Guaratuba. Contexto inicial de avaliação: 730 dias; total atual 104 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0020        "path": "data/processed/calendar_text/guaratuba_processed_1daysin_calendar_text.csv",
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0021        "context": 730,
```

Tamanho inicial que define a primeira origem do dataset: 365 ou 730. Qwen recebe o recorte fixo de 7/15; baselines recebem histórico crescente. JSON é lido por json.loads; não executa chamadas por si só.

```
0022        "numerical_covariates": [
```

Colunas numéricas preservadas nos inputs. Nas configurações textuais, contém somente meteorologia; o calendário convertido é categórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0023          "relative_humidity_mean",
```

Nome de covariável relative_humidity_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0024          "temperature_mean",
```

Nome de covariável temperature_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0025          "radiation_mean",
```

Nome de covariável radiation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0026          "precipitation_mean"
```

Nome de covariável precipitation_mean. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0027        ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0028        "categorical_covariates": [
```

Categorias e indicadores. Na entrada textual inclui day, dayofweek, season e descrições de feriado/recesso; não são reconvertidos em códigos. JSON é lido por json.loads; não executa chamadas por si só.

```
0029         "is_holiday_ctba_gtba_jve",
```

Nome de covariável is_holiday_ctba_gtba_jve. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0030         "is_weekend",
```

Nome de covariável is_weekend. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0031         "is_carnival",
```

Nome de covariável is_carnival. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0032         "is_school_recess_pr",
```

Nome de covariável is_school_recess_pr. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0033         "day",
```

Nome de covariável day. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0034         "dayofweek",
```

Nome de covariável dayofweek. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0035         "season"
```

Nome de covariável season. O CSV precisa conter exatamente esta coluna; ela é preservada na história e no futuro sem o alvo.

```
0036       ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0037       "textual_calendar": true
```

True ativa date_from_text na leitura do CSV para permitir os recortes temporais. Demais covariáveis de calendário permanecem palavras. JSON é lido por json.loads; não executa chamadas por si só.

```
0038     }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0039   },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0040   "horizon": 7,
```

Sete valores futuros por origem; o alvo é [origin, origin+7). JSON é lido por json.loads; não executa chamadas por si só.

```
0041   "step": 7,
```

Avança sete dias entre origens; com horizonte sete as janelas avaliadas não se sobrepõem. JSON é lido por json.loads; não executa chamadas por si só.

```
0042   "validation_fraction": 0.2,
```

Proporção 0,2 para validação em uma extensão de treino. A inferência zero-shot atual não a usa para otimizar parâmetros. JSON é lido por json.loads; não executa chamadas por si só.

```
0043   "seed": 42,
```

Semente global transmitida ao Qwen/seleção de exemplos. Os adaptadores numéricos usam 42 internamente. JSON é lido por json.loads; não executa chamadas por si só.

```
0044    "qwen": {
```

Parâmetros que **config["qwen"] expande em OllamaConfig; chaves precisam corresponder aos campos da dataclass. JSON é lido por json.loads; não executa chamadas por si só.

```
0045      "model": "qwen3:4b-instruct-2507-q4_K_M",
```

Tag exata do checkpoint instalado no Ollama; o digest é consultado no servidor e guardado em run.json. JSON é lido por json.loads; não executa chamadas por si só.

```
0046      "base_url": "http://127.0.0.1:11435",
```

Endereço HTTP local do Ollama. As quatro configurações qwen_text usam a porta 11435; benchmark.json conserva sua porta própria. JSON é lido por json.loads; não executa chamadas por si só.

```
0047      "temperature": 0,
```

Temperatura de geração enviada em options; zero não equivale a treinamento nem a garantia absoluta de determinismo. JSON é lido por json.loads; não executa chamadas por si só.

```
0048      "num_ctx": 16384,
```

Capacidade de tokens do servidor por sequência. Configurações textuais usam 16384, verificados contra limite do modelo e tamanho conservador do prompt. JSON é lido por json.loads; não executa chamadas por si só.

```
0049      "num_predict": 256,
```

Limite de tokens gerados pelo Ollama. Não substitui horizon, que é quantidade de números previstos. JSON é lido por json.loads; não executa chamadas por si só.

```
0050      "timeout": 600,
```

Limite de espera HTTP do cliente em segundos. Pode durar mais de uma hora no caso 7200. JSON é lido por json.loads; não executa chamadas por si só.

```
0051      "retries": 2
```

Tentativas adicionais depois da primeira; retries=1 permite duas chamadas para a mesma janela. JSON é lido por json.loads; não executa chamadas por si só.

```
0052    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0053    "strategy": "zero",
```

zero retorna nenhuma demonstração; few_random/few_similar só são aceitos no layout legado. JSON é lido por json.loads; não executa chamadas por si só.

```
0054    "few_shot_examples": 3,
```

Quantidade de exemplos para estratégias few-shot. Ignorada efetivamente quando strategy=zero. JSON é lido por json.loads; não executa chamadas por si só.

```
0055    "qwen_history_days": 15,
```

Inteiro obrigatório 7 ou 15 para Qwen. Representa histórico fixo em observações diárias; valor nulo ou outro tamanho é rejeitado. JSON é lido por json.loads; não executa chamadas por si só.

```
0056    "future_covariates_policy": "supplied CSV values, as in original scripts; availability at
    forecast origin is an unverified input assumption"
```

Texto de procedência armazenado com o experimento. A redação antiga diz hipótese não verificada; posteriormente o usuário confirmou disponibilidade na origem. Preservar o JSON preserva o hash histórico. JSON é lido por json.loads; não executa chamadas por si só.

```
0057  }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

24. src/pipelines/qwen_text_experiment.py

CLI das oito avaliações textuais: gera todos os prompts, executa checkpoint/contexto/cidade e resume métricas completas, alinhadas e por lead.

SHA-256: 3dc87eecf2ea633aea5d1b7692d8c93fb5c8b413d795fe75c3c52bd8532a769e

Funções, contratos e pontos de conexão

```
L19: configuration(slug, days)
```

slug identifica 1_7b ou 4b e days identifica 7/15. Lê o arquivo JSON correspondente em configs e devolve o dicionário.

```
L23: prepare()
```

Gera e grava todos os prompts do experimento textual, verificando covariáveis e limite conservador de entrada. Não chama Ollama. Escreve prompt_manifest.json.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:96; configuration → src/pipelines/qwen_text_experiment.py:19; model_inputs → src/data/features/covariates.py:25; prepare_dataset → src/pipelines/data_pipeline.py:8; validate_covariates → src/data/features/covariates.py:13; write_json → src/data/ingestion/files.py:16

```
L49: summarize()
```

Exige as oito avaliações completas, contextos corretos, sete leads e alvos iguais. Escreve CSVs simples por cidade/execução, métricas agregadas e por lead. Não altera previsões.

Chamadas do projeto: deterministic_metrics → src/evaluation/metrics.py:5

```
L86: main()
```

Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada.

Chamadas do projeto: configuration → src/pipelines/qwen_text_experiment.py:19; prepare → src/pipelines/qwen_text_experiment.py:23; run_dataset → src/pipelines/forecasting_pipeline.py:52; summarize → src/pipelines/qwen_text_experiment.py:49

APIs externas usadas neste arquivo

RuntimeError, ValueError, add_argument, all, any, append, apply, argparse.ArgumentParser, capitalize, dict, duplicated, encode, eq, groupby, int, items, json.loads, len, list, max, mkdir, parse_args, pd.DataFrame, pd.read_csv, pd.testing.assert_frame_equal, print, range, read_text, reset_index, sort_values, sorted, strftime, sum, to_csv, to_string, write_text. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Prepare, execute and summarize Qwen runs sourced from the textual CSVs."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import argparse
```

Importa argparse. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  from src.data.features.covariates import model_inputs, validate_covariates
```

Importa model_inputs, validate_covariates de src.data.features.covariates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  from src.data.ingestion.files import write_json
```

Importa write_json de src.data.ingestion.files. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.data.preprocessing.generate_prompts import build_prompt
```

Importa build_prompt de src.data.preprocessing.generate_prompts. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  from src.evaluation.metrics import deterministic_metrics
```

Importa deterministic_metrics de src.evaluation.metrics. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0012  from src.pipelines.data_pipeline import prepare_dataset
```

Importa prepare_dataset de src.pipelines.data_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0013  from src.pipelines.forecasting_pipeline import ROOT, run_dataset
```

Importa ROOT, run_dataset de src.pipelines.forecasting_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0015  OUTPUT = ROOT / 'experiments/qwen_text'
```

Atribui a OUTPUT o resultado de ROOT / 'experiments/qwen_text'. A expressão à direita é avaliada antes da atribuição.

```
0016  RUNS = [('1_7b', 7), ('1_7b', 15), ('4b', 7), ('4b', 15)]
```

Atribui a RUNS o resultado de [('1_7b', 7), ('1_7b', 15), ('4b', 7), ('4b', 15)]. A expressão à direita é avaliada antes da atribuição.

```
0019  def configuration(slug, days):
```

Define configuration(slug, days). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0020      return json.loads((ROOT / f'configs/qwen_text_{slug}_{days}.json').read_text())
```

Encerra esta função e devolve json.loads((ROOT / f'configs/qwen_text_{slug}_{days}.json').read_text()) ao chamador; não imprime o resultado por si só.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (ROOT / f'configs/qwen_text_{slug}_{days}.json').read_text().

(ROOT / f'configs/qwen_text_{slug}_{days}.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0023  def prepare():
```

Define prepare(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0024      records = []
```

Atribui a records o resultado de []. A expressão à direita é avaliada antes da atribuição.

```
0025      for slug, days in RUNS:
```

Percorre RUNS; em cada iteração associa o item a (slug, days). O corpo indentado aplica a operação a esse item/grupo.

```
0026          config = configuration(slug, days)
```

Atribui a config o resultado de configuration(slug, days). A expressão à direita é avaliada antes da atribuição.

configuration — slug identifica 1_7b ou 4b e days identifica 7/15. Lê o arquivo JSON correspondente em configs e devolve o dicionário. Argumentos usados: posicional 1: slug; posicional 2: days.

```
0027          for name, dataset in config['datasets'].items():
```

Percorre config['datasets'].items(); em cada iteração associa o item a (name, dataset). O corpo indentado aplica a operação a esse item/grupo.

config['datasets'].items — Percorre pares (chave, valor) do dicionário. Argumentos usados: nenhum argumento explícito.

```
0028              frame, windows = prepare_dataset(ROOT / dataset['path'], dataset['context'],
```

Atribui a (frame, windows) o resultado de prepare_dataset(ROOT / dataset['path'], dataset['context'], textual_calendar=True). A expressão à direita é avaliada antes da atribuição.

prepare_dataset — path aponta o CSV; context define a primeira origem; horizon/step definem grade de avaliação; textual_calendar escolhe interpretação de datas em inglês. Retorna tabela preparada e lista de ForecastWindow. Argumentos usados: posicional 1: ROOT / dataset['path']; posicional 2: dataset['context']; textual_calendar: True.

```
0029                                              textual_calendar=True)
```

Continua os argumentos nomeados: textual_calendar = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0030              validate_covariates(frame, dataset)
```

Executa validate_covariates(frame, dataset). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

validate_covariates — df: tabela preparada; dataset: contrato de colunas. Retorna None; falha em ausentes/NaN/números não finitos. Argumentos usados: posicional 1: frame; posicional 2: dataset.

```
0031              destination = OUTPUT / f'{slug}_{days}' / name / 'prompts'
```

Atribui a destination o resultado de OUTPUT / f'{slug}_{days}' / name / 'prompts'. A expressão à direita é avaliada antes da atribuição.

```
0032              destination.mkdir(parents=True, exist_ok=True)
```

Executa destination.mkdir(parents=True, exist_ok=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

destination.mkdir — Cria diretório. Argumentos usados: parents: True; exist_ok: True.

```
0033              maximum = 0
```

Atribui a maximum o resultado de 0. A expressão à direita é avaliada antes da atribuição.

```
0034              for window in windows:
```

Percorre windows; em cada iteração associa o item a window. O corpo indentado aplica a operação a esse item/grupo.

```
0035                  history, future = model_inputs(frame, window, dataset, history_days=days)
```

Atribui a (history, future) o resultado de model_inputs(frame, window, dataset, history_days=days). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: frame; posicional 2: window; posicional 3: dataset; history_days: days.

```
0036                  prompt = build_prompt(history, 7, future_covariates=future,
    region=name.capitalize())
```

Atribui a prompt o resultado de build_prompt(history, 7, future_covariates=future, region=name.capitalize()). A expressão à direita é avaliada antes da atribuição.

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: history; posicional 2: 7; future_covariates: future;

region: name.capitalize().

name.capitalize — Capitaliza a primeira letra do texto. Argumentos usados: nenhum argumento explícito.

```
0037                 required = len(prompt.encode()) + config['qwen']['num_predict'] + 512
```

Atribui a required o resultado de len(prompt.encode()) + config['qwen']['num_predict'] + 512. A expressão à direita é avaliada antes da atribuição.

len — Conta elementos. Argumentos usados: posicional 1: prompt.encode().

prompt.encode — Codifica string em bytes. Argumentos usados: nenhum argumento explícito.

```
0038                 if required > config['qwen']['num_ctx']:
```

Avalia a condição required > config['qwen']['num_ctx']. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0039                     raise ValueError(f'Prompt does not fit: {slug}/{days}/{name}')
```

Interrompe o caminho atual lançando ValueError(f'Prompt does not fit: {slug}/{days}/{name}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Prompt does not fit: {slug}/{days}/{name}'.

```
0040                 maximum = max(maximum, required)
```

Atribui a maximum o resultado de max(maximum, required). A expressão à direita é avaliada antes da atribuição.

max — Retorna maior valor. Argumentos usados: posicional 1: maximum; posicional 2: required.

```
0041                 origin = future.timestamp.iloc[0].strftime('%Y-%m-%d')
```

Atribui a origin o resultado de future.timestamp.iloc[0].strftime('%Y-%m-%d'). A expressão à direita é avaliada antes da atribuição.

future.timestamp.iloc[0].strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0042                 (destination / f'{origin}.txt').write_text(prompt)
```

Executa (destination / f'{origin}.txt').write_text(prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

(destination / f'{origin}.txt').write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: prompt.

```
0043             records.append(dict(model=slug, context=days, dataset=name, windows=len(windows),
```

Executa records.append(dict(model=slug, context=days, dataset=name, windows=len(windows), source=dataset['path'], maximum_byte_budget=maximum)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

records.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(model=slug, context=days, dataset=name, windows=len(windows), source=dataset['path'], maximum_byte_budget=maximum).

dict — Cria dicionário. Argumentos usados: model: slug; context: days; dataset: name; windows: len(windows); source: dataset['path']; maximum_byte_budget: maximum.

len — Conta elementos. Argumentos usados: posicional 1: windows.

```
0044                                 source=dataset['path'], maximum_byte_budget=maximum))
```

Continua os argumentos nomeados: source = dataset['path']; maximum_byte_budget = maximum. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0045     write_json(OUTPUT / 'prompt_manifest.json', records)
```

Executa write_json(OUTPUT / 'prompt_manifest.json', records). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

write_json — path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino. Argumentos usados: posicional 1: OUTPUT / 'prompt_manifest.json'; posicional 2: records.

```
0046      print(f'Prepared {sum(r["windows"] for r in records)} prompts from textual CSVs.',
    flush=True)
```

Executa print(f"Prepared {sum((r['windows'] for r in records))} prompts from textual CSVs.", flush=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: f"Prepared {sum((r['windows'] for r in records))} prompts from textual CSVs."; flush: True.

sum — Soma elementos de iterável. Argumentos usados: posicional 1: (r['windows'] for r in records).

```
0049  def summarize():
```

Define summarize(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0050      rows, per_lead = [], []
```

Atribui a (rows, per_lead) o resultado de ([], []). A expressão à direita é avaliada antes da atribuição.

```
0051      for name in ('joinville', 'guaratuba'):
```

Percorre ('joinville', 'guaratuba'); em cada iteração associa o item a name. O corpo indentado aplica a operação a esse item/grupo.

```
0052          reference = None
```

Atribui a reference o resultado de None. A expressão à direita é avaliada antes da atribuição.

```
0053          simple = []
```

Atribui a simple o resultado de []. A expressão à direita é avaliada antes da atribuição.

```
0054          for slug, days in RUNS:
```

Percorre RUNS; em cada iteração associa o item a (slug, days). O corpo indentado aplica a operação a esse item/grupo.

```
0055              folder = OUTPUT / f'{slug}_{days}' / name / 'qwen'
```

Atribui a folder o resultado de OUTPUT / f'{slug}_{days}' / name / 'qwen'. A expressão à direita é avaliada antes da atribuição.

```
0056              status = json.loads((folder / 'status.json').read_text())
```

Atribui a status o resultado de json.loads((folder / 'status.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'status.json').read_text().

(folder / 'status.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0057              run = json.loads((folder / 'run.json').read_text())
```

Atribui a run o resultado de json.loads((folder / 'run.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'run.json').read_text().

(folder / 'run.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0058              if status['failures'] or status['successful_windows'] !=
    status['expected_windows']:
```

Avalia a condição status['failures'] or status['successful_windows'] != status['expected_windows']. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0059                  raise ValueError(f'Incomplete run: {folder}')
```

Interrompe o caminho atual lançando ValueError(f'Incomplete run: {folder}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Incomplete run: {folder}'.

```
0060             frame = pd.read_csv(folder / 'predictions.csv').sort_values(['origin',
    'lead']).reset_index(drop=True)
```

Atribui a frame o resultado de pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index(drop=True). A expressão à direita é avaliada antes da atribuição.

pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

pd.read_csv(folder / 'predictions.csv').sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: ['origin', 'lead'].

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: folder / 'predictions.csv'.

```
0061             if not frame.context.eq(days).all() or frame.duplicated(['origin',
    'lead']).any():
```

Avalia a condição not frame.context.eq(days).all() or frame.duplicated(['origin', 'lead']).any(). O bloco indentado seguinte só é executado quando ela é verdadeira.

frame.duplicated(['origin', 'lead']).any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

frame.context.eq(days).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

frame.duplicated — Marca repetições. Argumentos usados: posicional 1: ['origin', 'lead'].

frame.context.eq — Compara igualdade elemento a elemento. Argumentos usados: posicional 1: days.

```
0062                 raise ValueError(f'Invalid contexts/duplicate forecasts: {folder}')
```

Interrompe o caminho atual lançando ValueError(f'Invalid contexts/duplicate forecasts: {folder}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Invalid contexts/duplicate forecasts: {folder}'.

```
0063             if len(frame) != status['expected_windows'] * 7 or not
    frame.groupby('origin').lead.apply(
```

Avalia a condição len(frame) != status['expected_windows'] * 7 or not frame.groupby('origin').lead.apply(lambda values: sorted(values) == list(range(1, 8))).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: frame.

frame.groupby('origin').lead.apply(lambda values: sorted(values) == list(range(1, 8))).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

frame.groupby('origin').lead.apply — Aplica uma função por grupo/objeto. Argumentos usados: posicional 1: lambda values: sorted(values) == list(range(1, 8)).

frame.groupby — Cria grupos sem calcular métricas por si só. Argumentos usados: posicional 1: 'origin'.

```
0064                     lambda values: sorted(values) == list(range(1, 8))).all():
```

Continuação da instrução na linha 63: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

sorted — Devolve uma lista ordenada. Argumentos usados: posicional 1: values.

list — Materializa um iterável em lista. Argumentos usados: posicional 1: range(1, 8).

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 1; posicional 2: 8.

```
0065                 raise ValueError('Missing forecast leads')
```

Interrompe o caminho atual lançando ValueError('Missing forecast leads'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Missing forecast leads'.

```
0066             keys = frame[['origin', 'timestamp', 'lead', 'y_true']]
```

Atribui a keys o resultado de frame[['origin', 'timestamp', 'lead', 'y_true']]. A expressão à direita é avaliada antes da atribuição.

```
0067             if reference is not None:
```

Avalia a condição reference is not None. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0068                 pd.testing.assert_frame_equal(reference, keys)
```

Executa pd.testing.assert_frame_equal(reference, keys). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_frame_equal — Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem. Argumentos usados: posicional 1: reference; posicional 2: keys.

```
0069             reference = keys
```

Atribui a reference o resultado de keys. A expressão à direita é avaliada antes da atribuição.

```
0070             metrics = deterministic_metrics(frame.y_true, frame.y_pred)
```

Atribui a metrics o resultado de deterministic_metrics(frame.y_true, frame.y_pred). A expressão à direita é avaliada antes da atribuição.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R² e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: frame.y_true; posicional 2: frame.y_pred.

```
0071             label = 'Qwen3 1.7B' if slug == '1_7b' else 'Qwen3 4B'
```

Atribui a label o resultado de 'Qwen3 1.7B' if slug == '1_7b' else 'Qwen3 4B'. A expressão à direita é avaliada antes da atribuição.

```
0072             rows.append(dict(dataset=name, model=label, context=days, slug=slug,
```

Executa rows.append(dict(dataset=name, model=label, context=days, slug=slug, windows=status['successful_windows'], predictions=len(frame), **metrics)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

rows.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(dataset=name, model=label, context=days, slug=slug, windows=status['successful_windows'], predictions=len(frame), **metrics).

dict — Cria dicionário. Argumentos usados: dataset: name; model: label; context: days; slug: slug; windows: status['successful_windows']; predictions: len(frame); **desempacotamento: metrics.

```
0073                              windows=status['successful_windows'], predictions=len(frame),
    **metrics))
```

Continua os argumentos nomeados: windows = status['successful_windows']; predictions = len(frame); ** = metrics. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

len — Conta elementos. Argumentos usados: posicional 1: frame.

```
0074             simple.append({'modelo': label, 'contexto_inicial': days, 'RMSE':
    metrics['rmse'],
```

Executa simple.append({'modelo': label, 'contexto_inicial': days, 'RMSE': metrics['rmse'], 'R2': metrics['r2'], 'MAE': metrics['mae'], 'MAPE': metrics['mape_percent']}). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

simple.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: {'modelo': label, 'contexto_inicial': days, 'RMSE': metrics['rmse'], 'R2': metrics['r2'], 'MAE': metrics['mae'], 'MAPE': metrics['mape_percent']}.

```
0075                            'R2': metrics['r2'], 'MAE': metrics['mae'], 'MAPE':
    metrics['mape_percent']})
```

Define 'R2': campo do registro/coleção em construção; recebe metrics['r2']; 'MAE': campo do registro/coleção em construção; recebe metrics['mae']; 'MAPE': campo do registro/coleção em construção; recebe metrics['mape_percent']. Os valores são avaliados nesta janela/iteração.

```
0076             pd.DataFrame([simple[-1]]).to_csv(folder / 'comparison.csv', index=False)
```

Executa pd.DataFrame([simple[-1]]).to_csv(folder / 'comparison.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame([simple[-1]]).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: folder / 'comparison.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: [simple[-1]].

```
0077             for lead, subset in frame.groupby('lead'):
```

Percorre frame.groupby('lead'); em cada iteração associa o item a (lead, subset). O corpo indentado aplica a operação a esse item/grupo.

frame.groupby — Cria grupos sem calcular métricas por si só. Argumentos usados: posicional 1: 'lead'.

```
0078                 per_lead.append(dict(dataset=name, model=label, context=days, lead=int(lead),
```

Executa per_lead.append(dict(dataset=name, model=label, context=days, lead=int(lead), **deterministic_metrics(subset.y_true, subset.y_pred))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

per_lead.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(dataset=name, model=label, context=days, lead=int(lead), **deterministic_metrics(subset.y_true, subset.y_pred)).

dict — Cria dicionário. Argumentos usados: dataset: name; model: label; context: days; lead: int(lead); **desempacotamento: deterministic_metrics(subset.y_true, subset.y_pred).

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: lead.

```
0079                                     **deterministic_metrics(subset.y_true, subset.y_pred)))
```

Continua os argumentos nomeados: ** = deterministic_metrics(subset.y_true, subset.y_pred). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R^2 e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: subset.y_true; posicional 2: subset.y_pred.

```
0080         pd.DataFrame(simple).to_csv(OUTPUT / f'comparison_{name}.csv', index=False)
```

Executa pd.DataFrame(simple).to_csv(OUTPUT / f'comparison_{name}.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(simple).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: OUTPUT / f'comparison_{name}.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: simple.

```
0081     pd.DataFrame(rows).to_csv(OUTPUT / 'metrics.csv', index=False)
```

Executa pd.DataFrame(rows).to_csv(OUTPUT / 'metrics.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(rows).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: OUTPUT / 'metrics.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: rows.

```
0082     pd.DataFrame(per_lead).to_csv(OUTPUT / 'metrics_by_horizon.csv', index=False)
```

Executa pd.DataFrame(per_lead).to_csv(OUTPUT / 'metrics_by_horizon.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(per_lead).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: OUTPUT / 'metrics_by_horizon.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: per_lead.

```
0083     print(pd.DataFrame(rows)[['dataset', 'model', 'context', 'rmse',
    'r2']].to_string(index=False), flush=True)
```

Executa print(pd.DataFrame(rows)[['dataset', 'model', 'context', 'rmse', 'r2']].to_string(index=False), flush=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: pd.DataFrame(rows)[['dataset', 'model', 'context', 'rmse', 'r2']].to_string(index=False); flush: True.

pd.DataFrame(rows)[['dataset', 'model', 'context', 'rmse', 'r2']].to_string — Formata tabela como texto para terminal/log. Argumentos usados: index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: rows.

```
0086  def main():
```

Define main(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0087      parser = argparse.ArgumentParser(description=__doc__)
```

Atribui a parser o resultado de argparse.ArgumentParser(description=__doc__). A expressão à direita é avaliada antes da atribuição.

argparse.ArgumentParser — Cria o parser da interface de linha de comando. Argumentos usados: description: __doc__.

```
0088      parser.add_argument('--prepare-only', action='store_true')
```

Executa parser.add_argument('--prepare-only', action='store_true'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--prepare-only'; action: 'store_true'.

```
0089      parser.add_argument('--summarize-only', action='store_true')
```

Executa parser.add_argument('--summarize-only', action='store_true'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--summarize-only'; action: 'store_true'.

```
0090      args = parser.parse_args()
```

Atribui a args o resultado de parser.parse_args(). A expressão à direita é avaliada antes da atribuição.

parser.parse_args — Interpreta argumentos do processo e retorna Namespace. Argumentos usados: nenhum argumento explícito.

```
0091      if not args.summarize_only:
```

Avalia a condição not args.summarize_only. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0092          prepare()
```

Executa prepare(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

prepare — Gera e grava todos os prompts do experimento textual, verificando covariáveis e limite conservador de entrada. Não chama Ollama. Escreve prompt_manifest.json. Argumentos usados: nenhum argumento explícito.

```
0093          if args.prepare_only:
```

Avalia a condição args.prepare_only. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0094              return
```

Encerra esta função e devolve None ao chamador; não imprime o resultado por si só.

```
0095          for slug, days in RUNS:
```

Percorre RUNS; em cada iteração associa o item a (slug, days). O corpo indentado aplica a operação a esse item/grupo.

```
0096              config = configuration(slug, days)
```

Atribui a config o resultado de configuration(slug, days). A expressão à direita é avaliada antes da atribuição.

configuration — slug identifica 1_7b ou 4b e days identifica 7/15. Lê o arquivo JSON correspondente em configs e devolve o dicionário. Argumentos usados: posicional 1: slug; posicional 2: days.

```
0097              for name, dataset in config['datasets'].items():
```

Percorre config['datasets'].items(); em cada iteração associa o item a (name, dataset). O corpo indentado aplica a operação a esse item/grupo.

config['datasets'].items — Percorre pares (chave, valor) do dicionário. Argumentos usados: nenhum argumento explícito.

```
0098                  failures = run_dataset(name, dataset, config, 'qwen', OUTPUT /
    f'{slug}_{days}')
```

Atribui a failures o resultado de run_dataset(name, dataset, config, 'qwen', OUTPUT / f'{slug}_{days}'). A expressão à direita é avaliada antes da atribuição.

run_dataset — name, dataset e config identificam cidade e parâmetros; model_name seleciona adaptador; output define pasta; limit só limita testes de fumaça. Valida contexto, executa janelas, salva previsões e status; retorna número de falhas. Argumentos usados: posicional 1: name; posicional 2: dataset; posicional 3: config; posicional 4: 'qwen'; posicional 5: OUTPUT / f'{slug}_{days}'.

```
0099                 if failures:
```

Avalia a condição failures. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0100                     raise RuntimeError(f'{failures} failed windows in {slug}/{days}/{name}')
```

Interrompe o caminho atual lançando RuntimeError(f'{failures} failed windows in {slug}/{days}/{name}'). O chamador precisa capturar a exceção ou ela encerra a execução.

RuntimeError — Cria exceção para estado de execução inesperado. Argumentos usados: posicional 1: f'{failures} failed windows in {slug}/{days}/{name}'.

```
0101     summarize()
```

Executa summarize(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

summarize — Exige as oito avaliações completas, contextos corretos, sete leads e alvos iguais. Escreve CSVs simples por cidade/execução, métricas agregadas e por lead. Não altera previsões. Argumentos usados: nenhum argumento explícito.

```
0104 if __name__ == '__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0105     main()
```

Executa main(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

main — Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

25. src/pipelines/forecasting_pipeline.py

Orquestra inferência e cache. Exige Qwen com 7/15 dias, mantém histórias de baseline e preserva somente covariáveis no frame futuro. Não calcula métricas agregadas sozinho.

SHA-256: b9531d5ea72b9b6fcfa18d469dc90c9352989147bbea0019733d800452a0c30a

Funções, contratos e pontos de conexão

```
L18: fingerprint(value) → str
```

value: objeto serializável. Retorna SHA-256 hexadecimal do JSON com chaves ordenadas.

```
L22: source_fingerprint() → str
```

Sem parâmetros. Retorna digest incremental de nomes e bytes de todos os .py de src, em ordem fixa.

```
L31: create_model(name, context, config, dataset)
```

name: qwen/chronos/timesfm; context: contexto inicial; config: configuração global; dataset: configuração da cidade. Retorna adaptador apropriado. A fábrica não prevê por si só.

Chamadas do projeto: ChronosForecaster → src/models/chronos.py:5; OllamaConfig → src/models/promptcast.py:11; QwenForecaster → src/models/promptcast.py:53; TimesFMForecaster → src/models/timesfm.py:7

```
L44: qwen_history_days(config) → int
```

config precisa conter inteiro exato 7 ou 15. Ausência, None, strings, bool, floats e outros comprimentos são rejeitados antes de carregar dados/modelo.

```
L52: run_dataset(name, dataset, config, model_name, output, limit = 0)
```

name, dataset e config identificam cidade e parâmetros; model_name seleciona adaptador; output define pasta; limit só limita testes de fumaça. Valida contexto, executa janelas, salva previsões e status; retorna número de falhas.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:96; covariate_columns → src/data/features/covariates.py:6; create_model → src/pipelines/forecasting_pipeline.py:31; fingerprint → src/pipelines/forecasting_pipeline.py:18; model.predict → src/models/promptcast.py:71; model.predict → src/models/chronos.py:18; model.predict → src/models/timesfm.py:20; model.predict → src/models/classical.py:31; model.predict → src/models/classical.py:51; model_inputs → src/data/features/covariates.py:25; prepare_dataset → src/pipelines/data_pipeline.py:8; qwen_history_days → src/pipelines/forecasting_pipeline.py:44; select_demonstrations → src/data/preprocessing/generate_prompts.py:17; source_fingerprint → src/pipelines/forecasting_pipeline.py:22; validate_covariates → src/data/features/covariates.py:13; write_json → src/data/ingestion/files.py:16

```
L129: main()
```

Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada.

Chamadas do projeto: run_dataset → src/pipelines/forecasting_pipeline.py:52

APIs externas usadas neste arquivo

ChronosForecaster, OllamaConfig, Path, QwenForecaster, SystemExit, TimesFMForecaster, ValueError, add_argument, all, append, argparse.ArgumentParser, asdict, capitalize, dict, encode, enumerate, error, exists, float, get, hashlib.sha256, hexdigest, items, iterrows, json.dumps, json.loads, len, mkdir, np.isfinite, parse_args, pd.DataFrame, predict, print, read_bytes, read_text, relative_to, resolve, rglob, sorted, str, strftime, time.monotonic, to_csv, to_numpy, tolist, type, update, write_text, zip. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Run one model/environment with shared windows and resumable, fingerprinted records."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import argparse
```

Importa argparse. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from dataclasses import asdict
```

Importa asdict de dataclasses. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import hashlib
```

Importa hashlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  import time
```

Importa time. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.data.ingestion.files import write_json
```

Importa write_json de src.data.ingestion.files. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  from src.data.preprocessing.generate_prompts import build_prompt, select_demonstrations
```

Importa build_prompt, select_demonstrations de src.data.preprocessing.generate_prompts. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0012  from src.pipelines.data_pipeline import prepare_dataset
```

Importa prepare_dataset de src.pipelines.data_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0013  from src.data.features.covariates import model_inputs, validate_covariates, covariate_columns
```

Importa model_inputs, validate_covariates, covariate_columns de src.data.features.covariates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0015  ROOT = Path(__file__).resolve().parents[2]
```

Atribui a ROOT o resultado de Path(__file__).resolve().parents[2]. A expressão à direita é avaliada antes da atribuição.

Path(__file__).resolve — Resolve caminho absoluto e links simbólicos. Argumentos usados: nenhum argumento explícito.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: __file__.

```
0018  def fingerprint(value) -> str:
```

Define fingerprint(value) → str. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0019      return hashlib.sha256(json.dumps(value, sort_keys=True).encode()).hexdigest()
```

Encerra esta função e devolve hashlib.sha256(json.dumps(value, sort_keys=True).encode()).hexdigest() ao chamador; não imprime o resultado por si só.

hashlib.sha256(json.dumps(value, sort_keys=True).encode()).hexdigest — Retorna a representação hexadecimal do digest acumulado. Argumentos usados: nenhum argumento explícito.

hashlib.sha256 — Cria calculador de digest SHA-256. Argumentos usados: posicional 1: json.dumps(value, sort_keys=True).encode().

json.dumps(value, sort_keys=True).encode — Codifica string em bytes. Argumentos usados: nenhum argumento explícito.

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: value; sort_keys: True.

```
0022  def source_fingerprint() -> str:
```

Define source_fingerprint() → str. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0023      """Invalidate cached inference when any model/data/pipeline source changes."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0024      digest = hashlib.sha256()
```

Atribui a digest o resultado de hashlib.sha256(). A expressão à direita é avaliada antes da atribuição.

hashlib.sha256 — Cria calculador de digest SHA-256. Argumentos usados: nenhum argumento explícito.

```
0025      for path in sorted((ROOT / 'src').rglob('*.py')):
```

Percorre sorted((ROOT / 'src').rglob('*.py')); em cada iteração associa o item a path. O corpo indentado aplica a operação a esse item/grupo.

sorted — Devolve uma lista ordenada. Argumentos usados: posicional 1: (ROOT / 'src').rglob('*.py').

(ROOT / 'src').rglob — Percorre recursivamente caminhos compatíveis com padrão. Argumentos usados: posicional 1: '*.py'.

```
0026          digest.update(str(path.relative_to(ROOT)).encode())
```

Executa digest.update(str(path.relative_to(ROOT)).encode()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

digest.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: posicional 1: str(path.relative_to(ROOT)).encode().

str(path.relative_to(ROOT)).encode — Codifica string em bytes. Argumentos usados: nenhum argumento explícito.

str — Produz representação textual. Argumentos usados: posicional 1: path.relative_to(ROOT).

path.relative_to — Calcula caminho relativo a um ancestral de um Path. Argumentos usados: posicional 1: ROOT.

```
0027          digest.update(path.read_bytes())
```

Executa digest.update(path.read_bytes()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

digest.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: posicional 1: path.read_bytes().

path.read_bytes — Lê bytes integrais do arquivo. Argumentos usados: nenhum argumento explícito.

```
0028      return digest.hexdigest()
```

Encerra esta função e devolve digest.hexdigest() ao chamador; não imprime o resultado por si só.

digest.hexdigest — Retorna a representação hexadecimal do digest acumulado. Argumentos usados: nenhum argumento explícito.

```
0031  def create_model(name, context, config, dataset):
```

Define create_model(name, context, config, dataset). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0032      if name == 'qwen':
```

Avalia a condição name == 'qwen'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0033          from src.models.promptcast import OllamaConfig, QwenForecaster
```

Importa OllamaConfig, QwenForecaster de src.models.promptcast. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0034          return QwenForecaster(OllamaConfig(**config['qwen'], seed=config['seed']))
```

Encerra esta função e devolve QwenForecaster(OllamaConfig(**config['qwen'], seed=config['seed'])) ao chamador; não imprime o resultado por si só.

QwenForecaster — Constrói adaptador Qwen e consulta servidor local. Argumentos usados: posicional 1: OllamaConfig(**config['qwen'], seed=config['seed']).

OllamaConfig — Constrói configuração tipada do cliente Qwen. Argumentos usados: **desempacotamento: config['qwen']; seed: config['seed'].

```
0035      if name == 'chronos':
```

Avalia a condição name == 'chronos'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0036          from src.models.chronos import ChronosForecaster
```

Importa ChronosForecaster de src.models.chronos. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0037          return ChronosForecaster(context, dataset)
```

Encerra esta função e devolve ChronosForecaster(context, dataset) ao chamador; não imprime o resultado por si só.

ChronosForecaster — Constrói adaptador Chronos2 com pesos locais. Argumentos usados: posicional 1: context; posicional 2: dataset.

```
0038      if name == 'timesfm':
```

Avalia a condição name == 'timesfm'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0039          from src.models.timesfm import TimesFMForecaster
```

Importa TimesFMForecaster de src.models.timesfm. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0040          return TimesFMForecaster(context, dataset)
```

Encerra esta função e devolve TimesFMForecaster(context, dataset) ao chamador; não imprime o resultado por si só.

TimesFMForecaster — Constrói adaptador TimesFM2.5 local. Argumentos usados: posicional 1: context; posicional 2: dataset.

```
0041      raise ValueError(f'Unknown model {name}')
```

Interrompe o caminho atual lançando ValueError(f'Unknown model {name}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Unknown model {name}'.

```
0044  def qwen_history_days(config) -> int:
```

Define qwen_history_days(config) → int. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0045      """LLM inputs must use a fixed seven- or fifteen-day observation window."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0046      days = config.get('qwen_history_days')
```

Atribui a days o resultado de config.get('qwen_history_days'). A expressão à direita é avaliada antes da atribuição.

config.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'qwen_history_days'.

```
0047      if type(days) is not int or days not in (7, 15):
```

Avalia a condição type(days) is not int or days not in (7, 15). O bloco indentado seguinte só é executado quando ela é verdadeira.

type — Obtém a classe concreta do valor. Argumentos usados: posicional 1: days.

```
0048          raise ValueError('qwen_history_days must be 7 or 15')
```

Interrompe o caminho atual lançando ValueError('qwen_history_days must be 7 or 15'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'qwen_history_days must be 7 or 15'.

```
0049      return days
```

Encerra esta função e devolve days ao chamador; não imprime o resultado por si só.

```
0052  def run_dataset(name, dataset, config, model_name, output, limit=0):
```

Define run_dataset(name, dataset, config, model_name, output, limit = 0). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0053      history_days = qwen_history_days(config) if model_name == 'qwen' else None
```

Atribui a history_days o resultado de qwen_history_days(config) if model_name == 'qwen' else None. A expressão à direita é avaliada antes da atribuição.

qwen_history_days — config precisa conter inteiro exato 7 ou 15. Ausência, None, strings, bool, floats e outros comprimentos são rejeitados antes de carregar dados/modelo. Argumentos usados: posicional 1: config.

```
0054      path = ROOT / dataset['path']
```

Atribui a path o resultado de ROOT / dataset['path']. A expressão à direita é avaliada antes da atribuição.

```
0055      df, windows = prepare_dataset(path, dataset['context'], horizon=config['horizon'],
```

Atribui a (df, windows) o resultado de prepare_dataset(path, dataset['context'], horizon=config['horizon'], step=config['step'], textual_calendar=dataset.get('textual_calendar', False)). A expressão à direita é avaliada antes da atribuição.

prepare_dataset — path aponta o CSV; context define a primeira origem; horizon/step definem grade de avaliação; textual_calendar escolhe interpretação de datas em inglês. Retorna tabela preparada e lista de ForecastWindow. Argumentos usados: posicional 1: path; posicional 2: dataset['context']; horizon: config['horizon']; step: config['step']; textual_calendar: dataset.get('textual_calendar', False).

```
0056                                    step=config['step'],
    textual_calendar=dataset.get('textual_calendar', False))
```

Continua os argumentos nomeados: step = config['step']; textual_calendar = dataset.get('textual_calendar', False). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

dataset.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'textual_calendar'; posicional 2: False.

```
0057      validate_covariates(df, dataset)
```

Executa validate_covariates(df, dataset). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

validate_covariates — df: tabela preparada; dataset: contrato de colunas. Retorna None; falha em ausentes/NaN/números não finitos. Argumentos usados: posicional 1: df; posicional 2: dataset.

```
0058      if limit:
```

Avalia a condição limit. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0059          windows = windows[:limit]
```

Atribui a windows o resultado de windows[:limit]. A expressão à direita é avaliada antes da atribuição.

```
0060      model = create_model(model_name, dataset['context'], config, dataset)
```

Atribui a model o resultado de create_model(model_name, dataset['context'], config, dataset). A expressão à direita é avaliada antes da atribuição.

create_model — name: qwen/chronos/timesfm; context: contexto inicial; config: configuração global; dataset: configuração da cidade. Retorna adaptador apropriado. A fábrica não prevê por si só. Argumentos usados: posicional 1: model_name; posicional 2: dataset['context']; posicional 3: config; posicional 4: dataset.

```
0061      provenance = {'config': config, 'dataset_sha256':
    hashlib.sha256(path.read_bytes()).hexdigest(),
```

Atribui a provenance o resultado de {'config': config, 'dataset_sha256': hashlib.sha256(path.read_bytes()).hexdigest(), 'source_sha256': source_fingerprint(), 'model': model.identity, 'dataset': name, 'limit': limit, 'n_windows': len(windows), 'first_test_origin': windows[0].origin, 'missing_days_forward_filled': df.loc[~df.observed, 'timestamp'].dt.strftime('%Y-%m-%d').tolist(), 'protocol': (f'fixed {history_days}-day history' if model_name == 'qwen' else 'expanding history') + ' with historical and supplied future covariates', 'covariates': covariate_columns(dataset), 'future_covariates_policy': config['future_covariates_policy'], 'qwen_history_days': config['qwen_history_days']}. A expressão à direita é avaliada antes da atribuição.

hashlib.sha256(path.read_bytes()).hexdigest — Retorna a representação hexadecimal do digest acumulado. Argumentos usados: nenhum argumento explícito.

hashlib.sha256 — Cria calculador de digest SHA-256. Argumentos usados: posicional 1: path.read_bytes().

path.read_bytes — Lê bytes integrais do arquivo. Argumentos usados: nenhum argumento explícito.

```
0062                     'source_sha256': source_fingerprint(), 'model': model.identity,
```

Define 'source_sha256': hash de todos os .py sob src; recebe source_fingerprint(); 'model': identidade do checkpoint/adaptador; recebe model.identity. Os valores são avaliados nesta janela/iteração.

source_fingerprint — Sem parâmetros. Retorna digest incremental de nomes e bytes de todos os .py de src, em ordem fixa. Argumentos usados: nenhum argumento explícito.

```
0063                     'dataset': name, 'limit': limit, 'n_windows': len(windows),
```

Define 'dataset': campo do registro/coleção em construção; recebe name; 'limit': campo do registro/coleção em construção; recebe limit; 'n_windows': campo do registro/coleção em construção; recebe len(windows). Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: windows.

```
0064                     'first_test_origin': windows[0].origin,
```

Define 'first_test_origin': campo do registro/coleção em construção; recebe windows[0].origin. Os valores são avaliados nesta janela/iteração.

```
0065                     'missing_days_forward_filled': df.loc[~df.observed,
    'timestamp'].dt.strftime('%Y-%m-%d').tolist(),
```

Define 'missing_days_forward_filled': campo do registro/coleção em construção; recebe df.loc[~df.observed, 'timestamp'].dt.strftime('%Y-%m-%d').tolist(). Os valores são avaliados nesta janela/iteração.

df.loc[~df.observed, 'timestamp'].dt.strftime('%Y-%m-%d').tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

df.loc[~df.observed, 'timestamp'].dt.strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0066                     'protocol': (f'fixed {history_days}-day history' if model_name == 'qwen'
```

Define 'protocol': campo do registro/coleção em construção; recebe (f'fixed {history_days}-day history' if model_name == 'qwen' else 'expanding history') + ' with historical and supplied future covariates'. Os valores são avaliados nesta janela/iteração.

```
0067                                 else 'expanding history') + ' with historical and supplied
    future covariates',
```

Continuação da instrução na linha 61: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0068                     'covariates': covariate_columns(dataset),
```

Define 'covariates': nomes das variáveis auxiliares; recebe covariate_columns(dataset). Os valores são avaliados nesta janela/iteração.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0069                     'future_covariates_policy': config['future_covariates_policy'],
```

Define 'future_covariates_policy': campo do registro/coleção em construção; recebe config['future_covariates_policy']. Os valores são avaliados nesta janela/iteração.

```
0070                     'qwen_history_days': config['qwen_history_days']}
```

Define 'qwen_history_days': campo do registro/coleção em construção; recebe config['qwen_history_days']. Os valores são avaliados nesta janela/iteração.

```
0071      run_id = fingerprint(provenance)
```

Atribui a run_id o resultado de fingerprint(provenance). A expressão à direita é avaliada antes da atribuição.

fingerprint — value: objeto serializável. Retorna SHA-256 hexadecimal do JSON com chaves ordenadas. Argumentos usados: posicional 1: provenance.

```
0072      destination = output / name / model_name
```

Atribui a destination o resultado de output / name / model_name. A expressão à direita é avaliada antes da atribuição.

```
0073      write_json(destination / 'run.json', dict(provenance, run_id=run_id))
```

Executa write_json(destination / 'run.json', dict(provenance, run_id=run_id)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

write_json — path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino. Argumentos usados: posicional 1: destination / 'run.json'; posicional 2: dict(provenance, run_id=run_id).

dict — Cria dicionário. Argumentos usados: posicional 1: provenance; run_id: run_id.

```
0074      # This prefix is frozen for every window, including all few-shot strategies.
```

Comentário de implementação: This prefix is frozen for every window, including all few-shot strategies.. Não é executado; as operações efetivas são as linhas seguintes.

```
0075      training_values = df.iloc[:windows[0].origin].water_produced.to_numpy()
```

Atribui a training_values o resultado de df.iloc[:windows[0].origin].water_produced.to_numpy(). A expressão à direita é avaliada antes da atribuição.

df.iloc[:windows[0].origin].water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

```
0076      rows, failures = [], []
```

Atribui a (rows, failures) o resultado de ([], []). A expressão à direita é avaliada antes da atribuição.

```
0077      for index, window in enumerate(windows, 1):
```

Percorre enumerate(windows, 1); em cada iteração associa o item a (index, window). O corpo indentado aplica a operação a esse item/grupo.

enumerate — Percorre iterável com contador. Argumentos usados: posicional 1: windows; posicional 2: 1.

```
0078          history, future = model_inputs(df, window, dataset,
```

Atribui a (history, future) o resultado de model_inputs(df, window, dataset, history_days=history_days). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: df; posicional 2: window; posicional 3: dataset; history_days: history_days.

```
0079              history_days=history_days)
```

Continua os argumentos nomeados: history_days = history_days. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0080          origin_date = df.timestamp.iloc[window.origin].strftime('%Y-%m-%d')
```

Atribui a origin_date o resultado de df.timestamp.iloc[window.origin].strftime('%Y-%m-%d'). A expressão à direita é avaliada antes da atribuição.

df.timestamp.iloc[window.origin].strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0081          target = destination / 'responses' / f'{origin_date}.json'
```

Atribui a target o resultado de destination / 'responses' / f'{origin_date}.json'. A expressão à direita é avaliada antes da atribuição.

```
0082          prompt = None
```

Atribui a prompt o resultado de None. A expressão à direita é avaliada antes da atribuição.

```
0083          if model_name == 'qwen':
```

Avalia a condição model_name == 'qwen'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0084             examples = select_demonstrations(training_values,
    history.water_produced.to_numpy(),
```

Atribui a examples o resultado de select_demonstrations(training_values, history.water_produced.to_numpy(), window.horizon, strategy=config['strategy'], count=config['few_shot_examples'], seed=config['seed']). A expressão à direita é avaliada antes da atribuição.

select_demonstrations — training_values: prefixo congelado anterior ao primeiro teste; history: demanda histórica corrente; horizon: tamanho de alvo dos exemplos; strategy: zero/few_random/few_similar; count=3: quantidade; seed=42: sorteio. Retorna lista de Demonstration. Argumentos usados: posicional 1: training_values; posicional 2: history.water_produced.to_numpy(); posicional 3: window.horizon; strategy: config['strategy']; count: config['few_shot_examples']; seed: config['seed'].

history.water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

```
0085                 window.horizon, strategy=config['strategy'],
    count=config['few_shot_examples'], seed=config['seed'])
```

Continua os argumentos nomeados: strategy = config['strategy']; count = config['few_shot_examples']; seed = config['seed']. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0086             prompt = build_prompt(history, window.horizon, demonstrations=examples,
```

Atribui a prompt o resultado de build_prompt(history, window.horizon, demonstrations=examples, future_covariates=future, region=name.capitalize()). A expressão à direita é avaliada antes da atribuição.

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: history; posicional 2: window.horizon; demonstrations: examples; future_covariates: future; region: name.capitalize().

```
0087                                 future_covariates=future, region=name.capitalize())
```

Continua os argumentos nomeados: future_covariates = future; region = name.capitalize(). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

name.capitalize — Capitaliza a primeira letra do texto. Argumentos usados: nenhum argumento explícito.

```
0088             prompt_path = output / name / 'prompts' / f'{origin_date}.txt'
```

Atribui a prompt_path o resultado de output / name / 'prompts' / f'{origin_date}.txt'. A expressão à direita é avaliada antes da atribuição.

```
0089             prompt_path.parent.mkdir(parents=True, exist_ok=True)
```

Executa prompt_path.parent.mkdir(parents=True, exist_ok=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

prompt_path.parent.mkdir — Cria diretório. Argumentos usados: parents: True; exist_ok: True.

```
0090             prompt_path.write_text(prompt)
```

Executa prompt_path.write_text(prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

prompt_path.write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: prompt.

```
0091         request_id = fingerprint({'run_id': run_id, 'window': asdict(window), 'prompt':
    prompt})
```

Atribui a request_id o resultado de fingerprint({'run_id': run_id, 'window': asdict(window), 'prompt': prompt}). A expressão à direita é avaliada antes da atribuição.

fingerprint — value: objeto serializável. Retorna SHA-256 hexadecimal do JSON com chaves ordenadas. Argumentos usados: posicional 1: {'run_id': run_id, 'window': asdict(window), 'prompt': prompt}.

asdict — Converte dataclass em dicionário de seus campos. Argumentos usados: posicional 1: window.

```
0092         saved = json.loads(target.read_text()) if target.exists() else {}
```

Atribui a saved o resultado de json.loads(target.read_text()) if target.exists() else {}. A expressão à direita é avaliada antes da atribuição.

target.exists — Verifica existência do caminho. Argumentos usados: nenhum argumento explícito.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: target.read_text().

target.read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0093          cached = saved.get('request_id') == request_id and saved.get('status') == 'ok'
```

Atribui a cached o resultado de saved.get('request_id') == request_id and saved.get('status') == 'ok'. A expressão à direita é avaliada antes da atribuição.

saved.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'request_id'.

saved.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'status'.

```
0094          if not cached:
```

Avalia a condição not cached. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0095              started = time.monotonic()
```

Atribui a started o resultado de time.monotonic(). A expressão à direita é avaliada antes da atribuição.

time.monotonic — Relógio crescente para medir duração. Argumentos usados: nenhum argumento explícito.

```
0096              saved = {'request_id': request_id, 'window': asdict(window), 'origin':
    origin_date,
```

Atribui a saved o resultado de {'request_id': request_id, 'window': asdict(window), 'origin': origin_date, 'history_rows': len(history), 'history_start': str(history.timestamp.iloc[0]), 'history_end': str(history.timestamp.iloc[-1]), 'covariates': covariate_columns(dataset)}. A expressão à direita é avaliada antes da atribuição.

asdict — Converte dataclass em dicionário de seus campos. Argumentos usados: posicional 1: window.

```
0097                        'history_rows': len(history), 'history_start':
    str(history.timestamp.iloc[0]),
```

Define 'history_rows': comprimento da história; recebe len(history); 'history_start': primeira data histórica efetivamente fornecida; recebe str(history.timestamp.iloc[0]). Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: history.

str — Produz representação textual. Argumentos usados: posicional 1: history.timestamp.iloc[0].

```
0098                        'history_end': str(history.timestamp.iloc[-1]), 'covariates':
    covariate_columns(dataset)}
```

Define 'history_end': última data conhecida antes da origem; recebe str(history.timestamp.iloc[-1]); 'covariates': nomes das variáveis auxiliares; recebe covariate_columns(dataset). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: history.timestamp.iloc[-1].

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0099              try:
```

Inicia região protegida de execução. Exceções compatíveis serão tratadas pelo except correspondente; sucesso continua normalmente.

```
0100                  predictions = model.predict(history, window.horizon, prompt=prompt,
    future_covariates=future)
```

Atribui a predictions o resultado de model.predict(history, window.horizon, prompt=prompt, future_covariates=future). A expressão à direita é avaliada antes da atribuição.

model.predict — Interface comum dos adaptadores do projeto. Argumentos usados: posicional 1: history; posicional 2: window.horizon; prompt: prompt; future_covariates: future.

```
0101                  if len(predictions) != window.horizon or not np.isfinite(predictions).all():
```

Avalia a condição len(predictions) != window.horizon or not np.isfinite(predictions).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: predictions.

np.isfinite(predictions).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: predictions.

```
0102                     raise ValueError('Invalid model output shape or finite values')
```

Interrompe o caminho atual lançando ValueError('Invalid model output shape or finite values'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Invalid model output shape or finite values'.

```
0103                 saved.update(status='ok', predictions=predictions)
```

Executa saved.update(status='ok', predictions=predictions). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

saved.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: status: 'ok'; predictions: predictions.

```
0104             except Exception as exc:
```

Trata as exceções indicadas na linha e vincula a falha a exc quando declarado. O registro permite diagnosticar a tentativa sem inventar previsões.

```
0105                 saved.update(status='error', error=f'{type(exc).__name__}: {exc}')
```

Executa saved.update(status='error', error=f'{type(exc).__name__}: {exc}'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

saved.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: status: 'error'; error: f'{type(exc).__name__}: {exc}'.

type — Obtém a classe concreta do valor. Argumentos usados: posicional 1: exc.

```
0106             saved.update(elapsed_seconds=time.monotonic()-started, audit=model.last_audit)
```

Executa saved.update(elapsed_seconds=time.monotonic() - started, audit=model.last_audit). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

saved.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: elapsed_seconds: time.monotonic() - started; audit: model.last_audit.

time.monotonic — Relógio crescente para medir duração. Argumentos usados: nenhum argumento explícito.

```
0107             write_json(target, saved)
```

Executa write_json(target, saved). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

write_json — path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino. Argumentos usados: posicional 1: target; posicional 2: saved.

```
0108         print(f'{name}/{model_name} {index}/{len(windows)} {origin_date}: {saved["status"]}'
```

Executa print(f"{name}/{model_name} {index}/{len(windows)} {origin_date}: {saved['status']}" + (' cached' if cached else ''), flush=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: f"{name}/{model_name} {index}/{len(windows)} {origin_date}: {saved['status']}" + (' cached' if cached else ''); flush: True.

len — Conta elementos. Argumentos usados: posicional 1: windows.

```
0109               + (' cached' if cached else ''), flush=True)
```

Continua os argumentos nomeados: flush = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0110         if saved['status'] != 'ok':
```

Avalia a condição saved['status'] != 'ok'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0111             failures.append({'origin': origin_date, 'error': saved['error']})
```

Executa failures.append({'origin': origin_date, 'error': saved['error']}). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

failures.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: {'origin': origin_date, 'error': saved['error']}.

```
0112              continue
```

Avança imediatamente à próxima iteração; esta janela com falha não recebe linhas de previsão.

```
0113          # Ground truth is accessed only AFTER model inference finishes.
```

Comentário de implementação: Ground truth is accessed only AFTER model inference finishes.. Não é executado; as operações efetivas são as linhas seguintes.

```
0114          truth = df.iloc[window.target]
```

Atribui a truth o resultado de df.iloc[window.target]. A expressão à direita é avaliada antes da atribuição.

```
0115          for lead, ((_, observation), prediction) in enumerate(zip(truth.iterrows(),
    saved['predictions']), 1):
```

Percorre enumerate(zip(truth.iterrows(), saved['predictions']), 1); em cada iteração associa o item a (lead, ((_, observation), prediction)). O corpo indentado aplica a operação a esse item/grupo.

enumerate — Percorre iterável com contador. Argumentos usados: posicional 1: zip(truth.iterrows(), saved['predictions']); posicional 2: 1.

zip — Emparelha iteráveis pela posição. Argumentos usados: posicional 1: truth.iterrows(); posicional 2: saved['predictions'].

truth.iterrows — Itera pares (índice, Series da linha). Argumentos usados: nenhum argumento explícito.

```
0116              rows.append({'dataset': name, 'model': model_name, 'origin': origin_date,
```

Executa rows.append({'dataset': name, 'model': model_name, 'origin': origin_date, 'timestamp': observation.timestamp.strftime('%Y-%m-%d'), 'lead': lead, 'y_true': float(observation.water_produced), 'y_pred': prediction, 'initial_context': dataset['context'], 'context': len(history), 'history_start': history.timestamp.iloc[0].strftime('%Y-%m-%d'), 'horizon': window.horizon, 'elapsed_seconds_window': saved['elapsed_seconds'], 'run_id': run_id}). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

rows.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: {'dataset': name, 'model': model_name, 'origin': origin_date, 'timestamp': observation.timestamp.strftime('%Y-%m-%d'), 'lead': lead, 'y_true': float(observation.water_produced), 'y_pred': prediction, 'initial_context': dataset['context'], 'context': len(history), 'history_start': history.timestamp.iloc[0].strftime('%Y-%m-%d'), 'horizon': window.horizon, 'elapsed_seconds_window': saved['elapsed_seconds'], 'run_id': run_id}.

```
0117                              'timestamp': observation.timestamp.strftime('%Y-%m-%d'), 'lead':
    lead,
```

Define 'timestamp': campo do registro/coleção em construção; recebe observation.timestamp.strftime('%Y-%m-%d'); 'lead': posição dentro do horizonte: 1 a 7 no Qwen ou 1 a 30 nos baselines clássicos; recebe lead. Os valores são avaliados nesta janela/iteração.

observation.timestamp.strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0118                              'y_true': float(observation.water_produced), 'y_pred': prediction,
```

Define 'y_true': demanda observada usada somente na avaliação; recebe float(observation.water_produced); 'y_pred': valor retornado pelo modelo para a data correspondente; recebe prediction. Os valores são avaliados nesta janela/iteração.

float — Converte escalar para float Python. Argumentos usados: posicional 1: observation.water_produced.

```
0119                              'initial_context': dataset['context'], 'context': len(history),
```

Define 'initial_context': origem inicial do protocolo de avaliação; recebe dataset['context']; 'context': número de linhas históricas efetivamente fornecidas; recebe len(history). Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0120                              'history_start': history.timestamp.iloc[0].strftime('%Y-%m-%d'),
```

Define 'history_start': primeira data histórica efetivamente fornecida; recebe history.timestamp.iloc[0].strftime('%Y-%m-%d'). Os valores são avaliados nesta janela/iteração.

history.timestamp.iloc[0].strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0121                            'horizon': window.horizon,
```

Define 'horizon': campo do registro/coleção em construção; recebe window.horizon. Os valores são avaliados nesta janela/iteração.

```
0122                            'elapsed_seconds_window': saved['elapsed_seconds'], 'run_id':
    run_id})
```

Define 'elapsed_seconds_window': duração da janela, repetida em seus sete registros; recebe saved['elapsed_seconds']; 'run_id': identificador derivado da procedência da execução; recebe run_id. Os valores são avaliados nesta janela/iteração.

```
0123      pd.DataFrame(rows).to_csv(destination / 'predictions.csv', index=False)
```

Executa pd.DataFrame(rows).to_csv(destination / 'predictions.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(rows).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: destination / 'predictions.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: rows.

```
0124      write_json(destination / 'status.json', {'expected_windows': len(windows),
```

Executa write_json(destination / 'status.json', {'expected_windows': len(windows), 'successful_windows': len(rows) // config['horizon'], 'failures': failures}). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

write_json — path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino. Argumentos usados: posicional 1: destination / 'status.json'; posicional 2: {'expected_windows': len(windows), 'successful_windows': len(rows) // config['horizon'], 'failures': failures}.

len — Conta elementos. Argumentos usados: posicional 1: windows.

```
0125                'successful_windows': len(rows)//config['horizon'], 'failures': failures})
```

Define 'successful_windows': quantidade de janelas com previsões válidas; recebe len(rows) // config['horizon']; 'failures': lista de janelas com falha; recebe failures. Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: rows.

```
0126      return len(failures)
```

Encerra esta função e devolve len(failures) ao chamador; não imprime o resultado por si só.

len — Conta elementos. Argumentos usados: posicional 1: failures.

```
0129  def main():
```

Define main(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0130      parser = argparse.ArgumentParser(description=__doc__)
```

Atribui a parser o resultado de argparse.ArgumentParser(description=__doc__). A expressão à direita é avaliada antes da atribuição.

argparse.ArgumentParser — Cria o parser da interface de linha de comando. Argumentos usados: description: __doc__.

```
0131      parser.add_argument('--model', choices=['qwen', 'chronos', 'timesfm'], default='qwen')
```

Executa parser.add_argument('--model', choices=['qwen', 'chronos', 'timesfm'], default='qwen'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--model'; choices: ['qwen', 'chronos', 'timesfm']; default: 'qwen'.

```
0132      parser.add_argument('--config', type=Path, default=ROOT / 'configs/benchmark.json')
```

Executa parser.add_argument('--config', type=Path, default=ROOT / 'configs/benchmark.json'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--config'; type: Path; default: ROOT / 'configs/benchmark.json'.

```
0133      parser.add_argument('--dataset', choices=['all', 'joinville', 'guaratuba'],
    default='all')
```

Executa parser.add_argument('--dataset', choices=['all', 'joinville', 'guaratuba'], default='all'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--dataset'; choices: ['all', 'joinville', 'guaratuba']; default: 'all'.

```
0134      parser.add_argument('--output', type=Path, default=ROOT / 'experiments/benchmark')
```

Executa parser.add_argument('--output', type=Path, default=ROOT / 'experiments/benchmark'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--output'; type: Path; default: ROOT / 'experiments/benchmark'.

```
0135      parser.add_argument('--limit', type=int, default=0, help='Smoke test only; 0 evaluates
    every window')
```

Executa parser.add_argument('--limit', type=int, default=0, help='Smoke test only; 0 evaluates every window'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--limit'; type: int; default: 0; help: 'Smoke test only; 0 evaluates every window'.

```
0136      args = parser.parse_args()
```

Atribui a args o resultado de parser.parse_args(). A expressão à direita é avaliada antes da atribuição.

parser.parse_args — Interpreta argumentos do processo e retorna Namespace. Argumentos usados: nenhum argumento explícito.

```
0137      if args.limit < 0:
```

Avalia a condição args.limit < 0. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0138          parser.error('--limit must be nonnegative')
```

Executa parser.error('--limit must be nonnegative'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.error — Em ArgumentParser, mostra mensagem e encerra com erro de CLI. Argumentos usados: posicional 1: '--limit must be nonnegative'.

```
0139      config = json.loads(args.config.read_text())
```

Atribui a config o resultado de json.loads(args.config.read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: args.config.read_text().

args.config.read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0140      failures = 0
```

Atribui a failures o resultado de 0. A expressão à direita é avaliada antes da atribuição.

```
0141      for name, dataset in config['datasets'].items():
```

Percorre config['datasets'].items(); em cada iteração associa o item a (name, dataset). O corpo indentado aplica a operação a esse item/grupo.

config['datasets'].items — Percorre pares (chave, valor) do dicionário. Argumentos usados: nenhum argumento explícito.

```
0142          if args.dataset in ('all', name):
```

Avalia a condição args.dataset in ('all', name). O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0143              failures += run_dataset(name, dataset, config, args.model, args.output,
    args.limit)
```

Atualiza failures acumulando o resultado de run_dataset(name, dataset, config, args.model, args.output, args.limit).

run_dataset — name, dataset e config identificam cidade e parâmetros; model_name seleciona adaptador; output define pasta; limit só limita testes de fumaça. Valida contexto, executa janelas, salva previsões e status; retorna número de falhas. Argumentos usados: posicional 1: name; posicional 2: dataset; posicional 3: config; posicional 4: args.model; posicional 5: args.output; posicional 6: args.limit.

```
0144      raise SystemExit(1 if failures else 0)
```

Interrompe o caminho atual lançando SystemExit(1 if failures else 0). O chamador precisa capturar a exceção ou ela encerra a execução.

SystemExit — Solicita encerramento do processo Python. Argumentos usados: posicional 1: 1 if failures else 0.

```
0147  if __name__ == '__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0148      main()
```

Executa main(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

main — Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

26. src/pipelines/data_pipeline.py

Composição de leitura, validação, grade diária, preenchimento causal e janelas. textual_calendar seleciona a interpretação das datas do CSV de palavras.

SHA-256: 8f27d6d9d8838704f5d9bb95887ab9f0b8e928b785031e8041eb34f952be4378

Funções, contratos e pontos de conexão

```
L8: prepare_dataset(path, context, *, horizon = 7, step = 7, textual_calendar = False)
```

path aponta o CSV; context define a primeira origem; horizon/step definem grade de avaliação; textual_calendar escolhe interpretação de datas em inglês. Retorna tabela preparada e lista de ForecastWindow.

Chamadas do projeto: build_windows → src/evaluation/cross_validation.py:20; daily_grid → src/data/preprocessing/resampling.py:5; fill_past_only → src/data/preprocessing/missing_values.py:5; read_series → src/data/ingestion/files.py:8

Código comentado, linha a linha

```
0001  """One data preparation path shared by prompts, models and evaluation."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  from src.data.ingestion.files import read_series
```

Importa read_series de src.data.ingestion.files. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from src.data.preprocessing.resampling import daily_grid
```

Importa daily_grid de src.data.preprocessing.resampling. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from src.data.preprocessing.missing_values import fill_past_only
```

Importa fill_past_only de src.data.preprocessing.missing_values. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  from src.evaluation.cross_validation import build_windows
```

Importa build_windows de src.evaluation.cross_validation. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  def prepare_dataset(path, context, *, horizon=7, step=7, textual_calendar=False):
```

Define prepare_dataset(path, context, *, horizon = 7, step = 7, textual_calendar = False). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0009      df = fill_past_only(daily_grid(read_series(path, textual_calendar=textual_calendar)))
```

Atribui a df o resultado de fill_past_only(daily_grid(read_series(path, textual_calendar=textual_calendar))). A expressão à direita é avaliada antes da atribuição.

fill_past_only — df: grade com possíveis NaN. Retorno: cópia preenchida com passado. Não reconstrói atributos de calendário, limitação conhecida. Argumentos usados: posicional 1: daily_grid(read_series(path, textual_calendar=textual_calendar)).

daily_grid — df: tabela com timestamp. Retorno: grade diária com observed. Dias ausentes são expostos, não removidos. Argumentos usados: posicional 1: read_series(path, textual_calendar=textual_calendar).

read_series — path é o CSV; textual_calendar=False lê datas usuais; True aplica date_from_text à coluna timestamp. Retorna DataFrame validado e ordenado. Calendário categórico continua textual. Argumentos usados: posicional 1: path; textual_calendar: textual_calendar.

```
0010      windows = build_windows(df, context, horizon, step)
```

Atribui a windows o resultado de build_windows(df, context, horizon, step). A expressão à direita é avaliada antes da atribuição.

build_windows — df: tabela preparada; context: primeira origem; horizon=7: quantidade prevista; step=7: avanço entre origens. Retorna lista ForecastWindow, excluindo janelas com alvo imputado. Argumentos usados: posicional 1: df; posicional 2: context; posicional 3: horizon; posicional 4: step.

```
0011      return df, windows
```

Encerra esta função e devolve (df, windows) ao chamador; não imprime o resultado por si só.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

27. src/data/ingestion/files.py

Lê CSV numérico ou textual conforme textual_calendar. Interpreta datas escritas antes da validação e oferece escrita JSON atômica para artefatos.

SHA-256: 80eb05a0f103682889465c1daf24c26385e382d557110dd285bb4b54dfabe275

Funções, contratos e pontos de conexão

```
L8: read_series(path: str | Path, *, textual_calendar: bool = False) → pd.DataFrame
```

path é o CSV; textual_calendar=False lê datas usuais; True aplica date_from_text à coluna timestamp. Retorna DataFrame validado e ordenado. Calendário categórico continua textual.

Chamadas do projeto: validate_series → src/data/validation/schema.py:6

```
L16: write_json(path: str | Path, value: dict) → None
```

path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino.

APIs externas usadas neste arquivo

Path, json.dumps, map, mkdir, pd.read_csv, replace, with_suffix, write_text. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Filesystem input and atomic artifact writes."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  from src.data.validation.schema import validate_series
```

Importa validate_series de src.data.validation.schema. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  def read_series(path: str | Path, *, textual_calendar: bool = False) -> pd.DataFrame:
```

Define read_series(path: str | Path, *, textual_calendar: bool = False) → pd.DataFrame. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0009      frame = pd.read_csv(path)
```

Atribui a frame o resultado de pd.read_csv(path). A expressão à direita é avaliada antes da atribuição.

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: path.

```
0010      if textual_calendar:
```

Avalia a condição textual_calendar. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0011          from src.data.preprocessing.calendar_text import date_from_text
```

Importa date_from_text de src.data.preprocessing.calendar_text. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0012          frame['timestamp'] = frame.timestamp.map(date_from_text)
```

Atribui a frame['timestamp'] o resultado de frame.timestamp.map(date_from_text). A expressão à direita é avaliada antes da atribuição.

frame.timestamp.map — Aplica uma função a cada valor da Series. Argumentos usados: posicional 1: date_from_text.

```
0013      return validate_series(frame)
```

Encerra esta função e devolve validate_series(frame) ao chamador; não imprime o resultado por si só.

validate_series — df: tabela lida. Retorno: cópia ordenada. Rejeita esquema, datas e demanda inválidos antes de qualquer modelo. Argumentos usados: posicional 1: frame.

```
0016  def write_json(path: str | Path, value: dict) -> None:
```

Define write_json(path: str | Path, value: dict) → None. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0017      """Replace a complete JSON file atomically; a crash cannot leave half a cache."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0018      path = Path(path)
```

Atribui a path o resultado de Path(path). A expressão à direita é avaliada antes da atribuição.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: path.

```
0019      path.parent.mkdir(parents=True, exist_ok=True)
```

Executa path.parent.mkdir(parents=True, exist_ok=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

path.parent.mkdir — Cria diretório. Argumentos usados: parents: True; exist_ok: True.

```
0020      temporary = path.with_suffix('.tmp')
```

Atribui a temporary o resultado de path.with_suffix('.tmp'). A expressão à direita é avaliada antes da atribuição.

path.with_suffix — Produz outro Path trocando a extensão. Argumentos usados: posicional 1: '.tmp'.

```
0021      temporary.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) +
    '\n')
```

Executa temporary.write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + '\n'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

temporary.write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False) + '\n'.

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: value; ensure_ascii: False; indent: 2; allow_nan: False.

```
0022      temporary.replace(path)
```

Executa temporary.replace(path). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

temporary.replace — Substitui o destino pelo arquivo representado pelo Path. Argumentos usados: posicional 1: path.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

28. src/data/validation/schema.py

Contrato do CSV: datas diárias únicas e demanda finita, não negativa. Chamado por read_series. Retorna cópia ordenada, preservando covariáveis; erro invalida o dataset antes do modelo.

SHA-256: d5c72310716276c63210b694a2e7f0d0cdc7b267b476dac2ab1e412a1436e12c

Funções, contratos e pontos de conexão

```
L6: validate_series(df: pd.DataFrame) → pd.DataFrame
```

df: tabela lida. Retorno: cópia ordenada. Rejeita esquema, datas e demanda inválidos antes de qualquer modelo.

APIs externas usadas neste arquivo

ValueError, all, any, copy, duplicated, eq, isna, issubset, normalize, np.isfinite, pd.to_datetime, pd.to_numeric, reset_index, set, sort_values, to_numpy. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Validate observed data without silently dropping or repairing observations."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  def validate_series(df: pd.DataFrame) -> pd.DataFrame:
```

Define validate_series(df: pd.DataFrame) → pd.DataFrame. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0007      """Require unique daily dates and finite, nonnegative water demand."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0008      required = {'timestamp', 'water_produced'}
```

Atribui a required o resultado de {'timestamp', 'water_produced'}. A expressão à direita é avaliada antes da atribuição.

```
0009      if not required.issubset(df.columns):
```

Avalia a condição not required.issubset(df.columns). O bloco indentado seguinte só é executado quando ela é verdadeira.

required.issubset — Verifica se todos os elementos estão contidos em outro conjunto/iterável. Argumentos usados: posicional 1: df.columns.

```
0010          raise ValueError(f'Missing columns: {required - set(df.columns)}')
```

Interrompe o caminho atual lançando ValueError(f'Missing columns: {required - set(df.columns)}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Missing columns: {required - set(df.columns)}'.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: df.columns.

```
0011      result = df.copy()
```

Atribui a result o resultado de df.copy(). A expressão à direita é avaliada antes da atribuição.

df.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0012      result['timestamp'] = pd.to_datetime(result['timestamp'], errors='raise')
```

Atribui a result['timestamp'] o resultado de pd.to_datetime(result['timestamp'], errors='raise'). A expressão à direita é avaliada antes da atribuição.

pd.to_datetime — Converte valores em datas pandas. Argumentos usados: posicional 1: result['timestamp']; errors: 'raise'.

```
0013      if result.empty or result.timestamp.isna().any() or result.timestamp.duplicated().any():
```

Avalia a condição result.empty or result.timestamp.isna().any() or result.timestamp.duplicated().any(). O bloco indentado seguinte só é executado quando ela é verdadeira.

result.timestamp.isna().any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

result.timestamp.duplicated().any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

result.timestamp.isna — Retorna máscara booleana de valores ausentes. Argumentos usados: nenhum argumento explícito.

result.timestamp.duplicated — Marca repetições. Argumentos usados: nenhum argumento explícito.

```
0014          raise ValueError('Dates must be nonempty, nonmissing and unique')
```

Interrompe o caminho atual lançando ValueError('Dates must be nonempty, nonmissing and unique'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Dates must be nonempty, nonmissing and unique'.

```
0015      if not result.timestamp.eq(result.timestamp.dt.normalize()).all():
```

Avalia a condição not result.timestamp.eq(result.timestamp.dt.normalize()).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

result.timestamp.eq(result.timestamp.dt.normalize()).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

result.timestamp.eq — Compara igualdade elemento a elemento. Argumentos usados: posicional 1: result.timestamp.dt.normalize().

result.timestamp.dt.normalize — Zera o horário de cada timestamp, preservando sua data. Argumentos usados: nenhum argumento explícito.

```
0016          raise ValueError('Expected daily timestamps at midnight')
```

Interrompe o caminho atual lançando ValueError('Expected daily timestamps at midnight'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Expected daily timestamps at midnight'.

```
0017      result['water_produced'] = pd.to_numeric(result.water_produced, errors='raise')
```

Atribui a result['water_produced'] o resultado de pd.to_numeric(result.water_produced, errors='raise'). A expressão à direita é avaliada antes da atribuição.

pd.to_numeric — Converte texto/objetos em números. Argumentos usados: posicional 1: result.water_produced; errors: 'raise'.

```
0018      values = result.water_produced.to_numpy()
```

Atribui a values o resultado de result.water_produced.to_numpy(). A expressão à direita é avaliada antes da atribuição.

result.water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

```
0019      if not np.isfinite(values).all() or (values < 0).any():
```

Avalia a condição not np.isfinite(values).all() or (values < 0).any(). O bloco indentado seguinte só é executado quando ela é verdadeira.

(values < 0).any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

np.isfinite(values).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: values.

```
0020          raise ValueError('Demand must be finite and nonnegative')
```

Interrompe o caminho atual lançando ValueError('Demand must be finite and nonnegative'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Demand must be finite and nonnegative'.

```
0021      return result.sort_values('timestamp').reset_index(drop=True)
```

Encerra esta função e devolve result.sort_values('timestamp').reset_index(drop=True) ao chamador; não imprime o resultado por si só.

result.sort_values('timestamp').reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

result.sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: 'timestamp'.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

29. src/data/preprocessing/calendar_text.py

Converte calendário para inglês, valida codificações cíclicas, exporta CSVs textuais e interpreta suas datas para os recortes temporais. Não treina nem consulta outras linhas para calcular uma categoria.

SHA-256: 6a6359e538192b00df0d74e568f1b1fbb01d5fbcde84b50befffedd016393ec8

Funções, contratos e pontos de conexão

`L28: number_to_words(value: int) → str`

value é um inteiro de 0 a 9999; retorna grafia inglesa canônica. Rejeita valores fora do domínio ou não inteiros; recursão combina milhares, centenas e dezenas.

`L48: words_to_number(text: str) → int`

text deve seguir a grafia canônica do exportador. Reconstrói o inteiro e compara sua grafia de volta; rejeita palavras ou formas inesperadas.

Chamadas do projeto: number_to_words → src/data/preprocessing/calendar_text.py:28

`L70: date_from_text(text: str) → pd.Timestamp`

text é a data no formato exato do exportador. Valida com fullmatch, converte ano/dia e retorna Timestamp; utilizado somente na organização temporal dos inputs.

Chamadas do projeto: words_to_number → src/data/preprocessing/calendar_text.py:48

`L79: date_to_text(value) → str`

value é uma data ou sua representação textual canônica. Retorna ano, dia e mês em inglês sem mudar o ano original.

Chamadas do projeto: date_from_text → src/data/preprocessing/calendar_text.py:70; number_to_words → src/data/preprocessing/calendar_text.py:28

`L86: _cycle_labels(frame, prefix, labels)`

frame contém os dois componentes prefix_sin/cos; labels define a ordem do ciclo. Decodifica o ângulo, valida seno/cosseno reconstruídos e retorna uma categoria por linha.

`L98: calendar_to_text(frame: pd.DataFrame) → pd.DataFrame`

frame pode conter calendário numérico ou já convertido. Devolve cópia com calendário inglês, mescla pares cíclicos e preserva demanda/meteorologia. Não ajusta estatísticas nem altera o argumento.

Chamadas do projeto: _cycle_labels → src/data/preprocessing/calendar_text.py:86; number_to_words → src/data/preprocessing/calendar_text.py:28; words_to_number → src/data/preprocessing/calendar_text.py:48

`L114: month_name(value)`

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

`L121: flag_label(value)`

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

`L134: main()`

Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada.

Chamadas do projeto: calendar_to_text → src/data/preprocessing/calendar_text.py:98

APIs externas usadas neste arquivo

Path, ValueError, add_argument, all, any, argparse.ArgumentParser, astype, copy, divmod, drop, index, int, isinstance, isna, items, len, lower, map, mkdir, np.allclose, np.arctan2, np.cos, np.mod, np.rint, np.sin, parse_args, pd.Timestamp, pd.read_csv, print, range, re.fullmatch, replace, resolve, split, str, strip, to_csv, to_numpy. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Render calendar features in English without changing the numerical datasets.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0003  Run with ``python -m src.data.preprocessing.calendar_text`` to export text copies.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0004  Cyclic features are decoded from their supplied values, not approximated by month.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0005  """
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0006  import argparse
```

Importa argparse. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  import re
```

Importa re. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0013  MONTHS = ('January', 'February', 'March', 'April', 'May', 'June', 'July',
```

Atribui a MONTHS o resultado de ('January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December'). A expressão à direita é avaliada antes da atribuição.

```
0014            'August', 'September', 'October', 'November', 'December')
```

Continuação da instrução na linha 13: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0015  WEEKDAYS = ('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday')
```

Atribui a WEEKDAYS o resultado de ('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'). A expressão à direita é avaliada antes da atribuição.

```
0016  SEASONS = ('summer', 'autumn', 'winter', 'spring')
```

Atribui a SEASONS o resultado de ('summer', 'autumn', 'winter', 'spring'). A expressão à direita é avaliada antes da atribuição.

```
0017  FLAGS = {
```

Atribui a FLAGS o resultado de {'is_holiday': ('not a public holiday', 'a public holiday'), 'holiday': ('not a public holiday', 'a public holiday'), 'is_holiday_ctba_gtba_jve': ('not a holiday in the Curitiba–Guaratuba–Joinville calendar', 'a holiday in the Curitiba–Guaratuba–Joinville calendar'), 'is_weekend': ('a weekday', 'a weekend day'), 'is_carnival': ('outside Carnival', 'during Carnival'), 'is_school_recess_pr': ('outside the Paraná school recess', 'during the Paraná school recess')}. A expressão à direita é avaliada antes da atribuição.

```
0018      'is_holiday': ('not a public holiday', 'a public holiday'),
```

Define 'is_holiday': campo do registro/coleção em construção; recebe ('not a public holiday', 'a public holiday'). Os valores são avaliados nesta janela/iteração.

```
0019      'holiday': ('not a public holiday', 'a public holiday'),
```

Define 'holiday': campo do registro/coleção em construção; recebe ('not a public holiday', 'a public holiday'). Os valores são avaliados nesta janela/iteração.

```
0020      'is_holiday_ctba_gtba_jve': ('not a holiday in the Curitiba–Guaratuba–Joinville
    calendar',
```

Define 'is_holiday_ctba_gtba_jve': campo do registro/coleção em construção; recebe ('not a holiday in the Curitiba–Guaratuba–Joinville calendar', 'a holiday in the Curitiba–Guaratuba–Joinville calendar'). Os valores são avaliados nesta janela/iteração.

```
0021                                'a holiday in the Curitiba–Guaratuba–Joinville calendar'),
```

Continuação da instrução na linha 17: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0022      'is_weekend': ('a weekday', 'a weekend day'),
```

Define 'is_weekend': campo do registro/coleção em construção; recebe ('a weekday', 'a weekend day'). Os valores são avaliados nesta janela/iteração.

```
0023      'is_carnival': ('outside Carnival', 'during Carnival'),
```

Define 'is_carnival': campo do registro/coleção em construção; recebe ('outside Carnival', 'during Carnival'). Os valores são avaliados nesta janela/iteração.

```
0024      'is_school_recess_pr': ('outside the Paraná school recess', 'during the Paraná school
    recess'),
```

Define 'is_school_recess_pr': campo do registro/coleção em construção; recebe ('outside the Paraná school recess', 'during the Paraná school recess'). Os valores são avaliados nesta janela/iteração.

```
0025  }
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0028  def number_to_words(value: int) -> str:
```

Define number_to_words(value: int) → str. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0029      """Spell out calendar integers (zero through 9999), independent of locale."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0030      small = ('zero', 'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight',
```

Atribui a small o resultado de ('zero', 'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine', 'ten', 'eleven', 'twelve', 'thirteen', 'fourteen', 'fifteen', 'sixteen', 'seventeen', 'eighteen', 'nineteen'). A expressão à direita é avaliada antes da atribuição.

```
0031                'nine', 'ten', 'eleven', 'twelve', 'thirteen', 'fourteen', 'fifteen',
```

Continuação da instrução na linha 30: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0032              'sixteen', 'seventeen', 'eighteen', 'nineteen')
```

Continuação da instrução na linha 30: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0033      tens = ('', '', 'twenty', 'thirty', 'forty', 'fifty', 'sixty', 'seventy', 'eighty',
    'ninety')
```

Atribui a tens o resultado de ('', '', 'twenty', 'thirty', 'forty', 'fifty', 'sixty', 'seventy', 'eighty', 'ninety'). A expressão à direita é avaliada antes da atribuição.

```
0034      integer = int(value)
```

Atribui a integer o resultado de int(value). A expressão à direita é avaliada antes da atribuição.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: value.

```
0035      if integer != value or not 0 <= integer <= 9999:
```

Avalia a condição integer != value or not 0 <= integer <= 9999. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0036          raise ValueError(f'Invalid calendar integer: {value}')
```

Interrompe o caminho atual lançando ValueError(f'Invalid calendar integer: {value}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Invalid calendar integer: {value}'.

```
0037      if integer < 20:
```

Avalia a condição integer < 20. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0038          return small[integer]
```

Encerra esta função e devolve small[integer] ao chamador; não imprime o resultado por si só.

```
0039      if integer < 100:
```

Avalia a condição integer < 100. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0040          quotient, remainder = divmod(integer, 10)
```

Atribui a (quotient, remainder) o resultado de divmod(integer, 10). A expressão à direita é avaliada antes da atribuição.

divmod — Devolve quociente inteiro e resto. Argumentos usados: posicional 1: integer; posicional 2: 10.

```
0041          return tens[quotient] + ('-' + small[remainder] if remainder else '')
```

Encerra esta função e devolve tens[quotient] + ('-' + small[remainder] if remainder else '') ao chamador; não imprime o resultado por si só.

```
0042      base = 100 if integer < 1000 else 1000
```

Atribui a base o resultado de 100 if integer < 1000 else 1000. A expressão à direita é avaliada antes da atribuição.

```
0043      quotient, remainder = divmod(integer, base)
```

Atribui a (quotient, remainder) o resultado de divmod(integer, base). A expressão à direita é avaliada antes da atribuição.

divmod — Devolve quociente inteiro e resto. Argumentos usados: posicional 1: integer; posicional 2: base.

```
0044      result = number_to_words(quotient) + (' hundred' if base == 100 else ' thousand')
```

Atribui a result o resultado de number_to_words(quotient) + (' hundred' if base == 100 else ' thousand'). A expressão à direita é avaliada antes da atribuição.

number_to_words — value é um inteiro de 0 a 9999; retorna grafia inglesa canônica. Rejeita valores fora do domínio ou não inteiros; recursão combina milhares, centenas e dezenas. Argumentos usados: posicional 1: quotient.

```
0045      return result + ((' and ' if remainder < 100 else ' ') + number_to_words(remainder) if
    remainder else '')
```

Encerra esta função e devolve result + ((' and ' if remainder < 100 else ' ') + number_to_words(remainder) if remainder else '') ao chamador; não imprime o resultado por si só.

number_to_words — value é um inteiro de 0 a 9999; retorna grafia inglesa canônica. Rejeita valores fora do domínio ou não inteiros; recursão combina milhares, centenas e dezenas. Argumentos usados: posicional 1: remainder.

```
0048  def words_to_number(text: str) -> int:
```

Define words_to_number(text: str) → int. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0049      """Parse only the canonical integer spelling emitted by this exporter."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0050      units = {number_to_words(n): n for n in (*range(20), *range(20, 100, 10))}
```

Atribui a units o resultado de {number_to_words(n): n for n in (*range(20), *range(20, 100, 10))}. A expressão à direita é avaliada antes da atribuição.

number_to_words — value é um inteiro de 0 a 9999; retorna grafia inglesa canônica. Rejeita valores fora do domínio ou não inteiros; recursão combina milhares, centenas e dezenas. Argumentos usados: posicional 1: n.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 20.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 20; posicional 2: 100; posicional 3: 10.

```
0051      total, group = 0, 0
```

Atribui a (total, group) o resultado de (0, 0). A expressão à direita é avaliada antes da atribuição.

```
0052      for word in text.replace('-', ' ').split():
```

Percorre text.replace('-', ' ').split(); em cada iteração associa o item a word. O corpo indentado aplica a operação a esse item/grupo.

text.replace('-', ' ').split — Divide uma string por separador. Argumentos usados: nenhum argumento explícito.

text.replace — Substitui o destino pelo arquivo representado pelo Path. Argumentos usados: posicional 1: '-'; posicional 2: ' '.

```
0053          if word == 'and':
```

Avalia a condição word == 'and'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0054              continue
```

Avança imediatamente à próxima iteração; esta janela com falha não recebe linhas de previsão.

```
0055          if word == 'hundred':
```

Avalia a condição word == 'hundred'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0056              group *= 100
```

Atualiza group acumulando o resultado de 100.

```
0057          elif word == 'thousand':
```

Avalia a condição word == 'thousand'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0058              total += group * 1000
```

Atualiza total acumulando o resultado de group * 1000.

```
0059              group = 0
```

Atribui a group o resultado de 0. A expressão à direita é avaliada antes da atribuição.

```
0060          elif word in units:
```

Avalia a condição word in units. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0061              group += units[word]
```

Atualiza group acumulando o resultado de units[word].

```
0062         else:
```

Ramo alternativo da condição imediatamente anterior: executa quando o teste anterior não foi satisfeito.

```
0063             raise ValueError(f'Invalid calendar number: {text}')
```

Interrompe o caminho atual lançando ValueError(f'Invalid calendar number: {text}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Invalid calendar number: {text}'.

```
0064     value = total + group
```

Atribui a value o resultado de total + group. A expressão à direita é avaliada antes da atribuição.

```
0065     if number_to_words(value) != text:
```

Avalia a condição number_to_words(value) != text. O bloco indentado seguinte só é executado quando ela é verdadeira.

number_to_words — value é um inteiro de 0 a 9999; retorna grafia inglesa canônica. Rejeita valores fora do domínio ou não inteiros; recursão combina milhares, centenas e dezenas. Argumentos usados: posicional 1: value.

```
0066         raise ValueError(f'Noncanonical calendar number: {text}')
```

Interrompe o caminho atual lançando ValueError(f'Noncanonical calendar number: {text}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Noncanonical calendar number: {text}'.

```
0067     return value
```

Encerra esta função e devolve value ao chamador; não imprime o resultado por si só.

```
0070 def date_from_text(text: str) -> pd.Timestamp:
```

Define date_from_text(text: str) → pd.Timestamp. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0071     """Recover a timestamp solely to align splits; prompts retain word dates."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0072     match = re.fullmatch(r'(.+), day (.+) of ([A-Za-z]+)', text)
```

Atribui a match o resultado de re.fullmatch('(.+), day (.+) of ([A-Za-z]+)', text). A expressão à direita é avaliada antes da atribuição.

re.fullmatch — Exige correspondência da string inteira com expressão regular. Argumentos usados: posicional 1: '(.+), day (.+) of ([A-Za-z]+)'; posicional 2: text.

```
0073     if not match or match[3] not in MONTHS:
```

Avalia a condição not match or match[3] not in MONTHS. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0074         raise ValueError(f'Invalid textual date: {text}')
```

Interrompe o caminho atual lançando ValueError(f'Invalid textual date: {text}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Invalid textual date: {text}'.

```
0075     return pd.Timestamp(year=words_to_number(match[1]), month=MONTHS.index(match[3])+1,
```

Encerra esta função e devolve pd.Timestamp(year=words_to_number(match[1]), month=MONTHS.index(match[3]) + 1, day=words_to_number(match[2])) ao chamador; não imprime o resultado por si só.

pd.Timestamp — Cria instante temporal pandas. Argumentos usados: year: words_to_number(match[1]); month: MONTHS.index(match[3]) + 1; day: words_to_number(match[2]).

words_to_number — text deve seguir a grafia canônica do exportador. Reconstrói o inteiro e compara sua grafia de volta; rejeita palavras ou formas inesperadas. Argumentos usados: posicional 1: match[1].

MONTHS.index — Localiza a posição de um valor numa sequência. Argumentos usados: posicional 1: match[3].

```
0076                            day=words_to_number(match[2]))
```

Continua os argumentos nomeados: day = words_to_number(match[2]). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

words_to_number — text deve seguir a grafia canônica do exportador. Reconstrói o inteiro e compara sua grafia de volta; rejeita palavras ou formas inesperadas. Argumentos usados: posicional 1: match[2].

```
0079  def date_to_text(value) -> str:
```

Define date_to_text(value) → str. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0080      date = date_from_text(value) if isinstance(value, str) and ', day ' in value else
    pd.Timestamp(value)
```

Atribui a date o resultado de date_from_text(value) if isinstance(value, str) and ', day ' in value else pd.Timestamp(value). A expressão à direita é avaliada antes da atribuição.

date_from_text — text é a data no formato exato do exportador. Valida com fullmatch, converte ano/dia e retorna Timestamp; utilizado somente na organização temporal dos inputs. Argumentos usados: posicional 1: value.

pd.Timestamp — Cria instante temporal pandas. Argumentos usados: posicional 1: value.

isinstance — Verifica se um objeto é instância de um tipo ou tupla de tipos. Argumentos usados: posicional 1: value; posicional 2: str.

```
0081      if pd.isna(date):
```

Avalia a condição pd.isna(date). O bloco indentado seguinte só é executado quando ela é verdadeira.

pd.isna — Retorna máscara booleana de valores ausentes. Argumentos usados: posicional 1: date.

```
0082          raise ValueError('Missing calendar date')
```

Interrompe o caminho atual lançando ValueError('Missing calendar date'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Missing calendar date'.

```
0083      return f'{number_to_words(date.year)}, day {number_to_words(date.day)} of
    {MONTHS[date.month - 1]}'
```

Encerra esta função e devolve f'{number_to_words(date.year)}, day {number_to_words(date.day)} of {MONTHS[date.month - 1]}' ao chamador; não imprime o resultado por si só.

number_to_words — value é um inteiro de 0 a 9999; retorna grafia inglesa canônica. Rejeita valores fora do domínio ou não inteiros; recursão combina milhares, centenas e dezenas. Argumentos usados: posicional 1: date.year.

number_to_words — value é um inteiro de 0 a 9999; retorna grafia inglesa canônica. Rejeita valores fora do domínio ou não inteiros; recursão combina milhares, centenas e dezenas. Argumentos usados: posicional 1: date.day.

```
0086  def _cycle_labels(frame, prefix, labels):
```

Define _cycle_labels(frame, prefix, labels). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0087      sine = frame[f'{prefix}_sin'].to_numpy(dtype=float)
```

Atribui a sine o resultado de frame[f'{prefix}_sin'].to_numpy(dtype=float). A expressão à direita é avaliada antes da atribuição.

frame[f'{prefix}_sin'].to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: dtype: float.

```
0088      cosine = frame[f'{prefix}_cos'].to_numpy(dtype=float)
```

Atribui a cosine o resultado de frame[f'{prefix}_cos'].to_numpy(dtype=float). A expressão à direita é avaliada antes da atribuição.

frame[f'{prefix}_cos'].to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: dtype: float.

```
0089      phase = np.mod(np.arctan2(sine, cosine), 2 * np.pi)
```

Atribui a phase o resultado de np.mod(np.arctan2(sine, cosine), 2 * np.pi). A expressão à direita é avaliada antes da atribuição.

np.mod — Calcula resto elemento a elemento. Argumentos usados: posicional 1: np.arctan2(sine, cosine); posicional 2: 2 * np.pi.

np.arctan2 — Calcula ângulo a partir de seno e cosseno. Argumentos usados: posicional 1: sine; posicional 2: cosine.

```
0090      indices = np.rint(phase * len(labels) / (2 * np.pi)).astype(int) % len(labels)
```

Atribui a indices o resultado de np.rint(phase * len(labels) / (2 * np.pi)).astype(int) % len(labels). A expressão à direita é avaliada antes da atribuição.

np.rint(phase * len(labels) / (2 * np.pi)).astype — Converte o tipo dos valores. Argumentos usados: posicional 1: int.

len — Conta elementos. Argumentos usados: posicional 1: labels.

np.rint — Arredonda valores ao inteiro mais próximo. Argumentos usados: posicional 1: phase * len(labels) / (2 * np.pi).

len — Conta elementos. Argumentos usados: posicional 1: labels.

```
0091      angles = indices * 2 * np.pi / len(labels)
```

Atribui a angles o resultado de indices * 2 * np.pi / len(labels). A expressão à direita é avaliada antes da atribuição.

len — Conta elementos. Argumentos usados: posicional 1: labels.

```
0092      if not (np.allclose(sine, np.sin(angles), atol=1e-6) and
```

Avalia a condição not (np.allclose(sine, np.sin(angles), atol=1e-06) and np.allclose(cosine, np.cos(angles), atol=1e-06)). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.allclose — Verifica proximidade numérica elemento a elemento. Argumentos usados: posicional 1: sine; posicional 2: np.sin(angles); atol: 1e-06.

np.sin — Calcula seno em radianos. Argumentos usados: posicional 1: angles.

```
0093              np.allclose(cosine, np.cos(angles), atol=1e-6)):
```

Continua os argumentos nomeados: atol = 1e-06. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

np.allclose — Verifica proximidade numérica elemento a elemento. Argumentos usados: posicional 1: cosine; posicional 2: np.cos(angles); atol: 1e-06.

np.cos — Calcula cosseno em radianos. Argumentos usados: posicional 1: angles.

```
0094          raise ValueError(f'Invalid {prefix} cyclic encoding')
```

Interrompe o caminho atual lançando ValueError(f'Invalid {prefix} cyclic encoding'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Invalid {prefix} cyclic encoding'.

```
0095      return [labels[index] for index in indices]
```

Encerra esta função e devolve [labels[index] for index in indices] ao chamador; não imprime o resultado por si só.

```
0098  def calendar_to_text(frame: pd.DataFrame) -> pd.DataFrame:
```

Define calendar_to_text(frame: pd.DataFrame) → pd.DataFrame. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0099      """Return a copy, replacing only calendar fields and merging cyclic pairs."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0100      result = frame.copy()
```

Atribui a result o resultado de frame.copy(). A expressão à direita é avaliada antes da atribuição.

frame.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0101      result['timestamp'] = result.timestamp.map(date_to_text)
```

Atribui a result['timestamp'] o resultado de result.timestamp.map(date_to_text). A expressão à direita é avaliada antes da atribuição.

result.timestamp.map — Aplica uma função a cada valor da Series. Argumentos usados: posicional 1: date_to_text.

```
0102      for prefix, labels in [('dayofweek', WEEKDAYS), ('season', SEASONS)]:
```

Percorre [('dayofweek', WEEKDAYS), ('season', SEASONS)]; em cada iteração associa o item a (prefix, labels). O corpo indentado aplica a operação a esse item/grupo.

```
0103          pair = [f'{prefix}_sin', f'{prefix}_cos']
```

Atribui a pair o resultado de [f'{prefix}_sin', f'{prefix}_cos']. A expressão à direita é avaliada antes da atribuição.

```
0104          if any(column in result for column in pair):
```

Avalia a condição any((column in result for column in pair)). O bloco indentado seguinte só é executado quando ela é verdadeira.

any — Reduz a existência de pelo menos um True. Argumentos usados: posicional 1: (column in result for column in pair).

```
0105              if not all(column in result for column in pair):
```

Avalia a condição not all((column in result for column in pair)). O bloco indentado seguinte só é executado quando ela é verdadeira.

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (column in result for column in pair).

```
0106                  raise ValueError(f'Both {prefix} cyclic components are required')
```

Interrompe o caminho atual lançando ValueError(f'Both {prefix} cyclic components are required'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Both {prefix} cyclic components are required'.

```
0107              result[prefix] = _cycle_labels(frame, prefix, labels)
```

Atribui a result[prefix] o resultado de _cycle_labels(frame, prefix, labels). A expressão à direita é avaliada antes da atribuição.

_cycle_labels — frame contém os dois componentes prefix_sin/cos; labels define a ordem do ciclo. Decodifica o ângulo, valida seno/cosseno reconstruídos e retorna uma categoria por linha. Argumentos usados: posicional 1: frame; posicional 2: prefix; posicional 3: labels.

```
0108              result = result.drop(columns=pair)
```

Atribui a result o resultado de result.drop(columns=pair). A expressão à direita é avaliada antes da atribuição.

result.drop — Retorna tabela sem as linhas/colunas indicadas. Argumentos usados: columns: pair.

```
0109      for column in ('day', 'year'):
```

Percorre ('day', 'year'); em cada iteração associa o item a column. O corpo indentado aplica a operação a esse item/grupo.

```
0110          if column in result:
```

Avalia a condição column in result. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0111              result[column] = result[column].map(lambda value:
    number_to_words(words_to_number(value))
```

Atribui a result[column] o resultado de result[column].map(lambda value: number_to_words(words_to_number(value)) if isinstance(value, str) else number_to_words(value)). A expressão à direita é avaliada antes da atribuição.

result[column].map — Aplica uma função a cada valor da Series. Argumentos usados: posicional 1: lambda value: number_to_words(words_to_number(value)) if isinstance(value, str) else number_to_words(value).

number_to_words — value é um inteiro de 0 a 9999; retorna grafia inglesa canônica. Rejeita valores fora do domínio ou não inteiros; recursão combina milhares, centenas e dezenas. Argumentos usados: posicional 1: words_to_number(value).

words_to_number — text deve seguir a grafia canônica do exportador. Reconstrói o inteiro e compara sua grafia de volta; rejeita palavras ou formas inesperadas. Argumentos usados: posicional 1: value.

```
0112                                              if isinstance(value, str) else
    number_to_words(value))
```

Filtro da compreensão iniciada acima; apenas elementos que atendem à condição são incluídos.

isinstance — Verifica se um objeto é instância de um tipo ou tupla de tipos. Argumentos usados: posicional 1: value; posicional 2: str.

number_to_words — value é um inteiro de 0 a 9999; retorna grafia inglesa canônica. Rejeita valores fora do domínio ou não inteiros; recursão combina milhares, centenas e dezenas. Argumentos usados: posicional 1: value.

```
0113      if 'month' in result:
```

Avalia a condição 'month' in result. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0114          def month_name(value):
```

Define month_name(value). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0115              if int(value) != value or not 1 <= value <= 12:
```

Avalia a condição int(value) != value or not 1 <= value <= 12. O bloco indentado seguinte só é executado quando ela é verdadeira.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: value.

```
0116                  raise ValueError(f'Invalid month: {value}')
```

Interrompe o caminho atual lançando ValueError(f'Invalid month: {value}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Invalid month: {value}'.

```
0117              return MONTHS[int(value) - 1]
```

Encerra esta função e devolve MONTHS[int(value) - 1] ao chamador; não imprime o resultado por si só.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: value.

```
0118          result['month'] = result.month.map(month_name)
```

Atribui a result['month'] o resultado de result.month.map(month_name). A expressão à direita é avaliada antes da atribuição.

result.month.map — Aplica uma função a cada valor da Series. Argumentos usados: posicional 1: month_name.

```
0119      for column, labels in FLAGS.items():
```

Percorre FLAGS.items(); em cada iteração associa o item a (column, labels). O corpo indentado aplica a operação a esse item/grupo.

FLAGS.items — Percorre pares (chave, valor) do dicionário. Argumentos usados: nenhum argumento explícito.

```
0120          if column in result:
```

Avalia a condição column in result. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0121              def flag_label(value):
```

Define flag_label(value). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0122                  if value in labels:
```

Avalia a condição value in labels. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0123                      return value
```

Encerra esta função e devolve value ao chamador; não imprime o resultado por si só.

```
0124                  normalized = str(value).strip().lower()
```

Atribui a normalized o resultado de str(value).strip().lower(). A expressão à direita é avaliada antes da atribuição.

str(value).strip().lower — Converte letras para minúsculas. Argumentos usados: nenhum argumento explícito.

str(value).strip — Remove espaços e quebras nas bordas do texto. Argumentos usados: nenhum argumento explícito.

str — Produz representação textual. Argumentos usados: posicional 1: value.

```
0125                 if normalized in {'0', '0.0', 'false'}:
```

Avalia a condição normalized in {'0', '0.0', 'false'}. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0126                     return labels[0]
```

Encerra esta função e devolve labels[0] ao chamador; não imprime o resultado por si só.

```
0127                 if normalized in {'1', '1.0', 'true'}:
```

Avalia a condição normalized in {'1', '1.0', 'true'}. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0128                     return labels[1]
```

Encerra esta função e devolve labels[1] ao chamador; não imprime o resultado por si só.

```
0129                 raise ValueError(f'Invalid calendar flag {column}: {value}')
```

Interrompe o caminho atual lançando ValueError(f'Invalid calendar flag {column}: {value}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Invalid calendar flag {column}: {value}'.

```
0130             result[column] = result[column].map(flag_label)
```

Atribui a result[column] o resultado de result[column].map(flag_label). A expressão à direita é avaliada antes da atribuição.

result[column].map — Aplica uma função a cada valor da Series. Argumentos usados: posicional 1: flag_label.

```
0131     return result
```

Encerra esta função e devolve result ao chamador; não imprime o resultado por si só.

```
0134 def main():
```

Define main(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0135     parser = argparse.ArgumentParser(description=__doc__)
```

Atribui a parser o resultado de argparse.ArgumentParser(description=__doc__). A expressão à direita é avaliada antes da atribuição.

argparse.ArgumentParser — Cria o parser da interface de linha de comando. Argumentos usados: description: __doc__.

```
0136     parser.add_argument('--input', type=Path, nargs='+', default=[
```

Executa parser.add_argument('--input', type=Path, nargs='+', default=[Path('data/processed/joinville_processed_1day.csv'), Path('data/processed/guaratuba_processed_1daysin.csv')]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--input'; type: Path; nargs: '+'; default: [Path('data/processed/joinville_processed_1day.csv'), Path('data/processed/guaratuba_processed_1daysin.csv')].

```
0137         Path('data/processed/joinville_processed_1day.csv'),
```

Continuação da instrução na linha 136: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: 'data/processed/joinville_processed_1day.csv'.

```
0138         Path('data/processed/guaratuba_processed_1daysin.csv')])
```

Continuação da instrução na linha 136: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: 'data/processed/guaratuba_processed_1daysin.csv'.

```
0139      parser.add_argument('--output-dir', type=Path,
    default=Path('data/processed/calendar_text'))
```

Executa parser.add_argument('--output-dir', type=Path, default=Path('data/processed/calendar_text')). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--output-dir'; type: Path; default: Path('data/processed/calendar_text').

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: 'data/processed/calendar_text'.

```
0140      args = parser.parse_args()
```

Atribui a args o resultado de parser.parse_args(). A expressão à direita é avaliada antes da atribuição.

parser.parse_args — Interpreta argumentos do processo e retorna Namespace. Argumentos usados: nenhum argumento explícito.

```
0141      args.output_dir.mkdir(parents=True, exist_ok=True)
```

Executa args.output_dir.mkdir(parents=True, exist_ok=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

args.output_dir.mkdir — Cria diretório. Argumentos usados: parents: True; exist_ok: True.

```
0142      for source in args.input:
```

Percorre args.input; em cada iteração associa o item a source. O corpo indentado aplica a operação a esse item/grupo.

```
0143          output = args.output_dir / f'{source.stem}_calendar_text.csv'
```

Atribui a output o resultado de args.output_dir / f'{source.stem}_calendar_text.csv'. A expressão à direita é avaliada antes da atribuição.

```
0144          if output.resolve() == source.resolve():
```

Avalia a condição output.resolve() == source.resolve(). O bloco indentado seguinte só é executado quando ela é verdadeira.

output.resolve — Resolve caminho absoluto e links simbólicos. Argumentos usados: nenhum argumento explícito.

source.resolve — Resolve caminho absoluto e links simbólicos. Argumentos usados: nenhum argumento explícito.

```
0145              raise ValueError('Text export must not overwrite its source')
```

Interrompe o caminho atual lançando ValueError('Text export must not overwrite its source'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Text export must not overwrite its source'.

```
0146          converted = calendar_to_text(pd.read_csv(source))
```

Atribui a converted o resultado de calendar_to_text(pd.read_csv(source)). A expressão à direita é avaliada antes da atribuição.

calendar_to_text — frame pode conter calendário numérico ou já convertido. Devolve cópia com calendário inglês, mescla pares cíclicos e preserva demanda/meteorologia. Não ajusta estatísticas nem altera o argumento. Argumentos usados: posicional 1: pd.read_csv(source).

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: source.

```
0147          converted.to_csv(output, index=False)
```

Executa converted.to_csv(output, index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

converted.to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: output; index: False.

```
0148          print(f'{output}: {len(converted)} rows')
```

Executa print(f'{output}: {len(converted)} rows'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: f'{output}: {len(converted)} rows'.

len — Conta elementos. Argumentos usados: posicional 1: converted.

```
0151  if __name__ == '__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0152      main()
```

Executa main(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

main — Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

30. src/data/preprocessing/resampling.py

Converte datas observadas em grade diária. Chamado por prepare_dataset antes do preenchimento. Adiciona observed para distinguir alvos reais de linhas inseridas.

SHA-256: 33d458e31acaed8a8d826b01d2b7d5b20f45a0825073b66d4092ecf40c4995e0

Funções, contratos e pontos de conexão

```
L5: daily_grid(df: pd.DataFrame) → pd.DataFrame
```

df: tabela com timestamp. Retorno: grade diária com observed. Dias ausentes são expostos, não removidos.

APIs externas usadas neste arquivo

asfreq, notna, rename_axis, reset_index, set_index. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Expose missing calendar days explicitly; do not interpolate future values."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  def daily_grid(df: pd.DataFrame) -> pd.DataFrame:
```

Define daily_grid(df: pd.DataFrame) → pd.DataFrame. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0006      result = df.set_index('timestamp').asfreq('D')
```

Atribui a result o resultado de df.set_index('timestamp').asfreq('D'). A expressão à direita é avaliada antes da atribuição.

df.set_index('timestamp').asfreq — Reindexa a série para uma frequência temporal regular. Argumentos usados: posicional 1: 'D'.

df.set_index — Transforma coluna em índice do DataFrame. Argumentos usados: posicional 1: 'timestamp'.

```
0007      result['observed'] = result.water_produced.notna()
```

Atribui a result['observed'] o resultado de result.water_produced.notna(). A expressão à direita é avaliada antes da atribuição.

result.water_produced.notna — Retorna True onde o valor não é ausente. Argumentos usados: nenhum argumento explícito.

```
0008      return result.rename_axis('timestamp').reset_index()
```

Encerra esta função e devolve result.rename_axis('timestamp').reset_index() ao chamador; não imprime o resultado por si só.

result.rename_axis('timestamp').reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: nenhum argumento explícito.

result.rename_axis — Nomeia o eixo do índice. Argumentos usados: posicional 1: 'timestamp'.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

31. src/data/preprocessing/missing_values.py

Preenchimento causal. Chamado após daily_grid. Retorna tabela sem lacunas iniciais de demanda; o preenchimento indiscriminado do calendário é um defeito confirmado, não uma prática recomendada.

SHA-256: 48d7a521417ef68f727b15797fb8d8c33dfe205f71db9dddd890574ecf1e297a

Funções, contratos e pontos de conexão

```
L5: fill_past_only(df: pd.DataFrame) → pd.DataFrame
```

df: grade com possíveis NaN. Retorno: cópia preenchida com passado. Não reconstrói atributos de calendário, limitação conhecida.

APIs externas usadas neste arquivo

ValueError, any, copy, ffill, isna. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Causal missing-day treatment; imputed targets never enter evaluation."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  def fill_past_only(df: pd.DataFrame) -> pd.DataFrame:
```

Define fill_past_only(df: pd.DataFrame) → pd.DataFrame. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0006      result = df.copy()
```

Atribui a result o resultado de df.copy(). A expressão à direita é avaliada antes da atribuição.

df.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0007      columns = [c for c in result.columns if c not in {'timestamp', 'observed'}]
```

Seleciona TODAS as colunas exceto data e observed. Isso inclui calendário; é a origem do defeito identificado na auditoria de 29/02/2016.

```
0008      result[columns] = result[columns].ffill()
```

Propaga passado sem vazamento futuro, mas copia day/weekday/is_weekend indevidamente para dias inseridos. Preencher demanda causalmente e reconstruir calendário são tarefas distintas.

result[columns].ffill — Propaga o último valor válido anterior para lacunas subsequentes. Argumentos usados: nenhum argumento explícito.

```
0009      if result.water_produced.isna().any():
```

Avalia a condição result.water_produced.isna().any(). O bloco indentado seguinte só é executado quando ela é verdadeira.

result.water_produced.isna().any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

result.water_produced.isna — Retorna máscara booleana de valores ausentes. Argumentos usados: nenhum argumento explícito.

```
0010          raise ValueError('Cannot fill initial missing demand without past data')
```

Interrompe o caminho atual lançando ValueError('Cannot fill initial missing demand without past data'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Cannot fill initial missing demand without past data'.

```
0011      return result
```

Encerra esta função e devolve result ao chamador; não imprime o resultado por si só.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

32. src/evaluation/cross_validation.py

Define ForecastWindow e origens walk-forward. prepare_dataset chama build_windows. validation_split é usado por training_partition e testes, não para ajustar os modelos neste benchmark.

SHA-256: 87e7ae6d7cd50ff53e224a7385bfbc567d503c2b2cb2f53f5624c84cc31f2b1e

Funções, contratos e pontos de conexão

```
L12: history(self) → slice
```

Property calculada de ForecastWindow: retorna slice, sem materializar a tabela até o chamador aplicá-lo com .iloc.

```
L16: target(self) → slice
```

Property calculada de ForecastWindow: retorna slice, sem materializar a tabela até o chamador aplicá-lo com .iloc.

```
L20: build_windows(df, context: int, horizon: int = 7, step: int = 7) → list[ForecastWindow]
```

df: tabela preparada; context: primeira origem; horizon=7: quantidade prevista; step=7: avanço entre origens. Retorna lista ForecastWindow, excluindo janelas com alvo imputado.

Chamadas do projeto: ForecastWindow → src/evaluation/cross_validation.py:6

```
L32: validation_split(training_size: int, fraction: float = 0.2) → tuple[slice, slice]
```

training_size: comprimento do prefixo anterior ao teste; fraction=0.2: parte final reservada à validação. Retorna dois slices sem embaralhar.

```
L48: training(self)
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

```
L52: validation(self)
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

```
L56: test(self)
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

```
L60: expanding_validation_folds(df, initial_training = 355, validation_days = 30, test_days = 30)
```

Gera treino expansivo desde zero, validação móvel e teste completo, com avanço igual ao tamanho do teste. Rejeita alvos ausentes nos blocos pontuados.

Chamadas do projeto: ExpandingValidationFold → src/evaluation/cross_validation.py:41

APIs externas usadas neste arquivo

ForecastWindow, ValueError, all, append, dataclass, enumerate, int, len, max, min, range, slice. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Shared chronological evaluation origins; each model selects its allowed history."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  from dataclasses import dataclass
```

Importa dataclass de dataclasses. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  @dataclass(frozen=True)
```

Aplica o decorador dataclass(frozen=True) à classe declarada em seguida; gera comportamento de dataclass.

dataclass — Decora a classe para gerar métodos a partir dos campos anotados. Argumentos usados: frozen: True.

```
0006  class ForecastWindow:
```

Declara a classe ForecastWindow. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0007      context_start: int
```

Declara o campo context_start com tipo esperado int, a ser fornecido na construção. A anotação não faz conversão automática.

```
0008      origin: int
```

Declara o campo origin com tipo esperado int, a ser fornecido na construção. A anotação não faz conversão automática.

```
0009      horizon: int
```

Declara o campo horizon com tipo esperado int, a ser fornecido na construção. A anotação não faz conversão automática.

```
0011      @property
```

Aplica o decorador property ao método seguinte; permite acessá-lo como atributo calculado.

```
0012      def history(self) -> slice:
```

Define history(self) → slice. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0013          return slice(self.context_start, self.origin)
```

Encerra esta função e devolve slice(self.context_start, self.origin) ao chamador; não imprime o resultado por si só.

slice — Representa intervalo, não materializa dados. Argumentos usados: posicional 1: self.context_start; posicional 2: self.origin.

```
0015      @property
```

Aplica o decorador property ao método seguinte; permite acessá-lo como atributo calculado.

```
0016      def target(self) -> slice:
```

Define target(self) → slice. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0017          return slice(self.origin, self.origin + self.horizon)
```

Encerra esta função e devolve slice(self.origin, self.origin + self.horizon) ao chamador; não imprime o resultado por si só.

slice — Representa intervalo, não materializa dados. Argumentos usados: posicional 1: self.origin; posicional 2: self.origin + self.horizon.

```
0020  def build_windows(df, context: int, horizon: int = 7, step: int = 7) -> list[ForecastWindow]:
```

Define build_windows(df, context: int, horizon: int = 7, step: int = 7) → list[ForecastWindow]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0021      """Context is the initial history length. Every later history starts at zero."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0022      if min(context, horizon, step) < 1:
```

Avalia a condição min(context, horizon, step) < 1. O bloco indentado seguinte só é executado quando ela é verdadeira.

min — Retorna menor valor. Argumentos usados: posicional 1: context; posicional 2: horizon; posicional 3: step.

```
0023          raise ValueError('Window parameters must be positive')
```

Interrompe o caminho atual lançando ValueError('Window parameters must be positive'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Window parameters must be positive'.

```
0024      windows = [ForecastWindow(0, origin, horizon)
```

Atribui a windows o resultado de [ForecastWindow(0, origin, horizon) for origin in range(context, len(df) - horizon + 1, step) if df.iloc[origin:origin + horizon].observed.all()]. A expressão à direita é avaliada antes da atribuição.

ForecastWindow — Constrói dataclass de uma origem temporal. Argumentos usados: posicional 1: 0; posicional 2: origin; posicional 3: horizon.

```
0025                for origin in range(context, len(df) - horizon + 1, step)
```

Cláusula de compreensão: percorre os valores indicados para construir a coleção da instrução anterior, mantendo a ordem desse percurso.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: context; posicional 2: len(df) - horizon + 1; posicional 3: step.

len — Conta elementos. Argumentos usados: posicional 1: df.

```
0026                if df.iloc[origin:origin + horizon].observed.all()]
```

Filtro da compreensão iniciada acima; apenas elementos que atendem à condição são incluídos.

df.iloc[origin:origin + horizon].observed.all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

```
0027      if not windows:
```

Avalia a condição not windows. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0028          raise ValueError('No complete observed evaluation windows')
```

Interrompe o caminho atual lançando ValueError('No complete observed evaluation windows'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'No complete observed evaluation windows'.

```
0029      return windows
```

Encerra esta função e devolve windows ao chamador; não imprime o resultado por si só.

```
0032  def validation_split(training_size: int, fraction: float = 0.2) -> tuple[slice, slice]:
```

Define validation_split(training_size: int, fraction: float = 0.2) → tuple[slice, slice]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0033      """Last 20% of a supplied training prefix; never split a test interval."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0034      if training_size < 2 or not 0 < fraction < 1:
```

Avalia a condição training_size < 2 or not 0 < fraction < 1. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0035          raise ValueError('Invalid training size or validation fraction')
```

Interrompe o caminho atual lançando ValueError('Invalid training size or validation fraction'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Invalid training size or validation fraction'.

```
0036      size = max(1, int(training_size * fraction))
```

Atribui a size o resultado de max(1, int(training_size * fraction)). A expressão à direita é avaliada antes da atribuição.

max — Retorna maior valor. Argumentos usados: posicional 1: 1; posicional 2: int(training_size * fraction).

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: training_size * fraction.

```
0037      return slice(0, training_size - size), slice(training_size - size, training_size)
```

Encerra esta função e devolve (slice(0, training_size - size), slice(training_size - size, training_size)) ao chamador; não imprime o resultado por si só.

slice — Representa intervalo, não materializa dados. Argumentos usados: posicional 1: 0; posicional 2: training_size - size.

slice — Representa intervalo, não materializa dados. Argumentos usados: posicional 1: training_size - size; posicional 2: training_size.

```
0040  @dataclass(frozen=True)
```

Aplica o decorador dataclass(frozen=True) à classe declarada em seguida; gera comportamento de dataclass.

dataclass — Decora a classe para gerar métodos a partir dos campos anotados. Argumentos usados: frozen: True.

```
0041  class ExpandingValidationFold:
```

Declara a classe ExpandingValidationFold. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0042      number: int
```

Declara o campo number com tipo esperado int, a ser fornecido na construção. A anotação não faz conversão automática.

```
0043      train_end: int
```

Declara o campo train_end com tipo esperado int, a ser fornecido na construção. A anotação não faz conversão automática.

```
0044      validation_end: int
```

Declara o campo validation_end com tipo esperado int, a ser fornecido na construção. A anotação não faz conversão automática.

```
0045      test_end: int
```

Declara o campo test_end com tipo esperado int, a ser fornecido na construção. A anotação não faz conversão automática.

```
0047      @property
```

Aplica o decorador property ao método seguinte; permite acessá-lo como atributo calculado.

```
0048      def training(self):
```

Define training(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0049          return slice(0, self.train_end)
```

Encerra esta função e devolve slice(0, self.train_end) ao chamador; não imprime o resultado por si só.

slice — Representa intervalo, não materializa dados. Argumentos usados: posicional 1: 0; posicional 2: self.train_end.

```
0051      @property
```

Aplica o decorador property ao método seguinte; permite acessá-lo como atributo calculado.

```
0052      def validation(self):
```

Define validation(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0053          return slice(self.train_end, self.validation_end)
```

Encerra esta função e devolve slice(self.train_end, self.validation_end) ao chamador; não imprime o resultado por si só.

slice — Representa intervalo, não materializa dados. Argumentos usados: posicional 1: self.train_end; posicional 2: self.validation_end.

```
0055      @property
```

Aplica o decorador property ao método seguinte; permite acessá-lo como atributo calculado.

```
0056      def test(self):
```

Define test(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0057          return slice(self.validation_end, self.test_end)
```

Encerra esta função e devolve slice(self.validation_end, self.test_end) ao chamador; não imprime o resultado por si só.

slice — Representa intervalo, não materializa dados. Argumentos usados: posicional 1: self.validation_end; posicional 2: self.test_end.

```
0060  def expanding_validation_folds(df, initial_training=355, validation_days=30, test_days=30):
```

Define expanding_validation_folds(df, initial_training = 355, validation_days = 30, test_days = 30). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0061      """Chronological folds with a rolling validation block and growing train prefix.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0063      Full test blocks only. Advancing by test_days reveals that block's actual
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0064      observations for the next origin, never for predictions within that block.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0065      """
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0066      if min(initial_training, validation_days, test_days) < 1:
```

Avalia a condição min(initial_training, validation_days, test_days) < 1. O bloco indentado seguinte só é executado quando ela é verdadeira.

min — Retorna menor valor. Argumentos usados: posicional 1: initial_training; posicional 2: validation_days; posicional 3: test_days.

```
0067          raise ValueError('Fold lengths must be positive')
```

Interrompe o caminho atual lançando ValueError('Fold lengths must be positive'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Fold lengths must be positive'.

```
0068      folds = []
```

Atribui a folds o resultado de []. A expressão à direita é avaliada antes da atribuição.

```
0069      for number, train_end in enumerate(range(initial_training, len(df)-validation_days-
    test_days+1, test_days), 1):
```

Percorre enumerate(range(initial_training, len(df) - validation_days - test_days + 1, test_days), 1); em cada iteração associa o item a (number, train_end). O corpo indentado aplica a operação a esse item/grupo.

enumerate — Percorre iterável com contador. Argumentos usados: posicional 1: range(initial_training, len(df) - validation_days - test_days + 1, test_days); posicional 2: 1.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: initial_training; posicional 2: len(df) - validation_days - test_days + 1; posicional 3: test_days.

len — Conta elementos. Argumentos usados: posicional 1: df.

```
0070          validation_end = train_end + validation_days
```

Atribui a validation_end o resultado de train_end + validation_days. A expressão à direita é avaliada antes da atribuição.

```
0071          test_end = validation_end + test_days
```

Atribui a test_end o resultado de validation_end + test_days. A expressão à direita é avaliada antes da atribuição.

```
0072          if df.iloc[train_end:test_end].observed.all():
```

Avalia a condição df.iloc[train_end:test_end].observed.all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

df.iloc[train_end:test_end].observed.all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

```
0073              folds.append(ExpandingValidationFold(number, train_end, validation_end,
    test_end))
```

Executa folds.append(ExpandingValidationFold(number, train_end, validation_end, test_end)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

folds.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: ExpandingValidationFold(number, train_end, validation_end, test_end).

ExpandingValidationFold — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: number; posicional 2: train_end; posicional 3: validation_end; posicional 4: test_end.

```
0074      if not folds:
```

Avalia a condição not folds. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0075          raise ValueError('No complete train/validation/test folds')
```

Interrompe o caminho atual lançando ValueError('No complete train/validation/test folds'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'No complete train/validation/test folds'.

```
0076      return folds
```

Encerra esta função e devolve folds ao chamador; não imprime o resultado por si só.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

33. src/data/features/temporal.py

Geração determinística das datas-alvo a partir da última data histórica. Chamado pelos dois construtores de prompt; não consulta a demanda futura.

SHA-256: 05e1165987546b2320e6dff5a8709cf646b98db9f8f5281c0a8092bce8144c24

Funções, contratos e pontos de conexão

```
L5: future_dates(last_observed: pd.Timestamp, horizon: int) → pd.DatetimeIndex
```

last_observed: último Timestamp histórico; horizon: quantidade de dias. Retorna DatetimeIndex dos próximos dias.

APIs externas usadas neste arquivo

pd.Timedelta, pd.date_range. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Only deterministic calendar data may be computed for future dates."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  def future_dates(last_observed: pd.Timestamp, horizon: int) -> pd.DatetimeIndex:
```

Define future_dates(last_observed: pd.Timestamp, horizon: int) → pd.DatetimeIndex. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0006      return pd.date_range(last_observed + pd.Timedelta(days=1), periods=horizon, freq='D')
```

Encerra esta função e devolve pd.date_range(last_observed + pd.Timedelta(days=1), periods=horizon, freq='D') ao chamador; não imprime o resultado por si só.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: last_observed + pd.Timedelta(days=1); periods: horizon; freq: 'D'.

pd.Timedelta — Representa uma duração temporal. Argumentos usados: days: 1.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

34. src/data/features/covariates.py

Contrato comum dos modelos: nomes de covariáveis, validação, separação history/future e conversão para TimesFM. model_inputs é a fronteira que exclui estruturalmente o alvo da entrada futura.

SHA-256: 265c84d324e98cc1db784ef3d155806d742adb6e360735207cc6f9881fe9eddd

Funções, contratos e pontos de conexão

```
L6: covariate_columns(dataset: dict) → list[str]
```

dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados.

```
L13: validate_covariates(df: pd.DataFrame, dataset: dict) → None
```

df: tabela preparada; dataset: contrato de colunas. Retorna None; falha em ausentes/NaN/números não finitos.

Chamadas do projeto: covariate_columns → src/data/features/covariates.py:6

```
L25: model_inputs(df, window, dataset, *, history_days = None)
```

df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro.

Chamadas do projeto: covariate_columns → src/data/features/covariates.py:6

```
L36: timesfm_covariates(history, future, dataset)
```

history/future: tabelas separadas; dataset: listas numéricas/categóricas. Retorna dois dicts com listas de séries cobrindo passado e futuro, sem demanda futura.

Chamadas do projeto: covariate_columns → src/data/features/covariates.py:6

APIs externas usadas neste arquivo

ValueError, all, any, astype, copy, isna, len, max, min, np.isfinite, pd.concat, pd.to_numeric, set, to_numpy, tolist. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """One explicit covariate contract, shared by Chronos, TimesFM and Qwen."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  def covariate_columns(dataset: dict) -> list[str]:
```

Define covariate_columns(dataset: dict) → list[str]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0007      columns = dataset['numerical_covariates'] + dataset['categorical_covariates']
```

Atribui a columns o resultado de dataset['numerical_covariates'] + dataset['categorical_covariates']. A expressão à direita é avaliada antes da atribuição.

```
0008      if len(columns) != len(set(columns)) or {'timestamp', 'water_produced', 'observed',
    'unique_id'} & set(columns):
```

Avalia a condição len(columns) != len(set(columns)) or {'timestamp', 'water_produced', 'observed', 'unique_id'} & set(columns). O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: columns.

len — Conta elementos. Argumentos usados: posicional 1: set(columns).

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: columns.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: columns.

```
0009          raise ValueError('Covariates must be unique and cannot contain the target or
    metadata')
```

Interrompe o caminho atual lançando ValueError('Covariates must be unique and cannot contain the target or metadata'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Covariates must be unique and cannot contain the target or metadata'.

```
0010      return columns
```

Encerra esta função e devolve columns ao chamador; não imprime o resultado por si só.

```
0013  def validate_covariates(df: pd.DataFrame, dataset: dict) -> None:
```

Define validate_covariates(df: pd.DataFrame, dataset: dict) → None. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0014      columns = covariate_columns(dataset)
```

Atribui a columns o resultado de covariate_columns(dataset). A expressão à direita é avaliada antes da atribuição.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0015      missing = set(columns) - set(df.columns)
```

Atribui a missing o resultado de set(columns) - set(df.columns). A expressão à direita é avaliada antes da atribuição.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: columns.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: df.columns.

```
0016      if missing:
```

Avalia a condição missing. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0017          raise ValueError(f'Missing configured covariates: {missing}')
```

Interrompe o caminho atual lançando ValueError(f'Missing configured covariates: {missing}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Missing configured covariates: {missing}'.

```
0018      if df[columns].isna().any().any():
```

Avalia a condição df[columns].isna().any().any(). O bloco indentado seguinte só é executado quando ela é verdadeira.

df[columns].isna().any().any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

df[columns].isna().any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

df[columns].isna — Retorna máscara booleana de valores ausentes. Argumentos usados: nenhum argumento explícito.

```
0019          raise ValueError('Covariates contain missing values')
```

Interrompe o caminho atual lançando ValueError('Covariates contain missing values'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Covariates contain missing values'.

```
0020      for column in dataset['numerical_covariates']:
```

Percorre dataset['numerical_covariates']; em cada iteração associa o item a column. O corpo indentado aplica a operação a esse item/grupo.

```
0021          if not np.isfinite(pd.to_numeric(df[column], errors='raise').to_numpy()).all():
```

Avalia a condição not np.isfinite(pd.to_numeric(df[column], errors='raise').to_numpy()).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.isfinite(pd.to_numeric(df[column], errors='raise').to_numpy()).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: pd.to_numeric(df[column], errors='raise').to_numpy().

pd.to_numeric(df[column], errors='raise').to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

pd.to_numeric — Converte texto/objetos em números. Argumentos usados: posicional 1: df[column]; errors: 'raise'.

```
0022              raise ValueError(f'Nonfinite covariate: {column}')
```

Interrompe o caminho atual lançando ValueError(f'Nonfinite covariate: {column}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Nonfinite covariate: {column}'.

```
0025  def model_inputs(df, window, dataset, *, history_days=None):
```

Define model_inputs(df, window, dataset, *, history_days = None). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0026      """Exclude target structurally from the future frame, including for Qwen."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0027      columns = covariate_columns(dataset)
```

Atribui a columns o resultado de covariate_columns(dataset). A expressão à direita é avaliada antes da atribuição.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0028      start = window.context_start if history_days is None else max(window.context_start,
    window.origin-history_days)
```

Atribui a start o resultado de window.context_start if history_days is None else max(window.context_start, window.origin - history_days). A expressão à direita é avaliada antes da atribuição.

max — Retorna maior valor. Argumentos usados: posicional 1: window.context_start; posicional 2: window.origin - history_days.

```
0029      history = df.iloc[start:window.origin][['timestamp', 'water_produced'] + columns].copy()
```

Atribui a history o resultado de df.iloc[start:window.origin][['timestamp', 'water_produced'] + columns].copy(). A expressão à direita é avaliada antes da atribuição.

df.iloc[start:window.origin][['timestamp', 'water_produced'] + columns].copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0030      future = df.iloc[window.target][['timestamp'] + columns].copy()
```

Atribui a future o resultado de df.iloc[window.target][['timestamp'] + columns].copy(). A expressão à direita é avaliada antes da atribuição.

df.iloc[window.target][['timestamp'] + columns].copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0031      if len(future) != window.horizon or history.timestamp.max() >= future.timestamp.min():
```

Avalia a condição len(future) != window.horizon or history.timestamp.max() >= future.timestamp.min(). O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: future.

history.timestamp.max — Retorna maior valor. Argumentos usados: nenhum argumento explícito.

future.timestamp.min — Retorna menor valor. Argumentos usados: nenhum argumento explícito.

```
0032          raise ValueError('History and future covariates must be temporally disjoint')
```

Interrompe o caminho atual lançando ValueError('History and future covariates must be temporally disjoint'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'History and future covariates must be temporally disjoint'.

```
0033      return history, future
```

Encerra esta função e devolve (history, future) ao chamador; não imprime o resultado por si só.

```
0036  def timesfm_covariates(history, future, dataset):
```

Define timesfm_covariates(history, future, dataset). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0037      if 'water_produced' in future:
```

Avalia a condição 'water_produced' in future. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0038          raise ValueError('Future targets cannot enter model inputs')
```

Interrompe o caminho atual lançando ValueError('Future targets cannot enter model inputs'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Future targets cannot enter model inputs'.

```
0039      columns = covariate_columns(dataset)
```

Atribui a columns o resultado de covariate_columns(dataset). A expressão à direita é avaliada antes da atribuição.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0040      full = pd.concat([history[columns], future[columns]], ignore_index=True)
```

Atribui a full o resultado de pd.concat([history[columns], future[columns]], ignore_index=True). A expressão à direita é avaliada antes da atribuição.

pd.concat — Concatena tabelas. Argumentos usados: posicional 1: [history[columns], future[columns]]; ignore_index: True.

```
0041      numerical = {c: [full[c].to_numpy(dtype=float)] for c in dataset['numerical_covariates']}
```

Atribui a numerical o resultado de {c: [full[c].to_numpy(dtype=float)] for c in dataset['numerical_covariates']}. A expressão à direita é avaliada antes da atribuição.

full[c].to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: dtype: float.

```
0042      categorical = {c: [full[c].astype(str).tolist()] for c in
    dataset['categorical_covariates']}
```

Atribui a categorical o resultado de {c: [full[c].astype(str).tolist()] for c in dataset['categorical_covariates']}. A expressão à direita é avaliada antes da atribuição.

full[c].astype(str).tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

full[c].astype — Converte o tipo dos valores. Argumentos usados: posicional 1: str.

```
0043      return numerical, categorical
```

Encerra esta função e devolve (numerical, categorical) ao chamador; não imprime o resultado por si só.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

35. src/data/preprocessing/generate_prompts.py

Constrói exclusivamente prompts narrativos. Recebe história e covariáveis futuras sem alvo, descreve todas as variáveis e aplica o contexto turístico somente a Guaratuba.

SHA-256: 148e21bca2ec04ab9840e1799a577acd9745b7a06d51734125dac9797c83619b

Funções, contratos e pontos de conexão

```
L17: select_demonstrations(training_values, history, horizon: int, *, strategy: str = 'zero',
    count: int = 3, seed: int = 42) → list[Demonstration]
```

training_values: prefixo congelado anterior ao primeiro teste; history: demanda histórica corrente; horizon: tamanho de alvo dos exemplos; strategy: zero/few_random/few_similar; count=3: quantidade; seed=42: sorteio. Retorna lista de Demonstration.

Chamadas do projeto: Demonstration → src/data/preprocessing/generate_prompts.py:11; standardized → src/data/preprocessing/generate_prompts.py:39

```
L39: standardized(x)
```

x: vetor de exemplo/consulta histórica. Retorna valores centrados e divididos por desvio-padrão com piso 1e-8; apenas auxilia seleção por forma.

```
L48: format_values(values) → str
```

values: números históricos. Retorna string separada por vírgulas com duas casas decimais.

```
L73: _narrate(frame: pd.DataFrame, region: str, *, future: bool = False) → list[str]
```

frame é um recorte de história ou futuro sem alvo; region identifica a cidade; future altera os tempos verbais e indica covariáveis fornecidas. Retorna lista de frases alinhadas às datas.

Chamadas do projeto: calendar_to_text → src/data/preprocessing/calendar_text.py:98

```
L96: build_prompt(history: pd.DataFrame, horizon: int, *, demonstrations: list[Demonstration] |
    None = None, future_covariates: pd.DataFrame | None = None, region: str = 'the study area') →
    str
```

history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON.

Chamadas do projeto: _narrate → src/data/preprocessing/generate_prompts.py:73; date_to_text → src/data/preprocessing/calendar_text.py:79; format_values → src/data/preprocessing/generate_prompts.py:48; future_dates → src/data/features/temporal.py:5

APIs externas usadas neste arquivo

Demonstration, ValueError, all, append, casefold, choice, dataclass, equals, extend, float, get, int, isinstance, issubset, join, json.dumps, len, list, max, np.arange, np.argsort, np.asarray, np.isfinite, np.mean, np.random.default_rng, np.std, pd.DatetimeIndex, pd.date_range, replace, set, sorted, startswith, str, to_numpy, tuple. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """PromptCast prompts with demand and the same covariates used by the baselines; future
    targets are absent by construction."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from dataclasses import dataclass
```

Importa dataclass de dataclasses. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  from src.data.features.temporal import future_dates
```

Importa future_dates de src.data.features.temporal. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  from src.data.preprocessing.calendar_text import calendar_to_text, date_to_text
```

Importa calendar_to_text, date_to_text de src.data.preprocessing.calendar_text. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  @dataclass(frozen=True)
```

Aplica o decorador dataclass(frozen=True) à classe declarada em seguida; gera comportamento de dataclass.

dataclass — Decora a classe para gerar métodos a partir dos campos anotados. Argumentos usados: frozen: True.

```
0011  class Demonstration:
```

Declara a classe Demonstration. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0012      history: tuple[float, ...]
```

Declara o campo history com tipo esperado tuple[float, ...], a ser fornecido na construção. A anotação não faz conversão automática.

```
0013      target: tuple[float, ...]
```

Declara o campo target com tipo esperado tuple[float, ...], a ser fornecido na construção. A anotação não faz conversão automática.

```
0014      target_end: int
```

Declara o campo target_end com tipo esperado int, a ser fornecido na construção. A anotação não faz conversão automática.

```
0017  def select_demonstrations(training_values, history, horizon: int, *,
```

Define select_demonstrations(training_values, history, horizon: int, *, strategy: str = 'zero', count: int = 3, seed: int = 42) → list[Demonstration]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0018                            strategy: str = 'zero', count: int = 3,
```

Continuação da instrução na linha 17: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção. training_values: prefixo congelado anterior ao primeiro teste; history: demanda histórica corrente; horizon: tamanho de alvo dos exemplos; strategy: zero/few_random/few_similar; count=3: quantidade; seed=42: sorteio. Retorna lista de Demonstration.

```
0019                            seed: int = 42) -> list[Demonstration]:
```

Continuação da instrução na linha 17: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção. training_values: prefixo congelado anterior ao primeiro teste; history: demanda histórica corrente; horizon: tamanho de alvo dos exemplos; strategy: zero/few_random/few_similar; count=3: quantidade; seed=42: sorteio. Retorna lista de Demonstration.

```
0020      """Select only from the frozen prefix preceding the FIRST evaluated target.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0022      Similarity uses historical inputs only. The caller passes an already
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0023      restricted training prefix; test observations can never become examples.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0024      """
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0025      if strategy not in {'zero', 'few_random', 'few_similar'}:
```

Avalia a condição strategy not in {'zero', 'few_random', 'few_similar'}. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0026          raise ValueError(f'Unknown strategy: {strategy}')
```

Interrompe o caminho atual lançando ValueError(f'Unknown strategy: {strategy}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Unknown strategy: {strategy}'.

```
0027      if strategy == 'zero':
```

Avalia a condição strategy == 'zero'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0028          return []
```

Encerra esta função e devolve [] ao chamador; não imprime o resultado por si só.

```
0029      if count < 1 or horizon < 1:
```

Avalia a condição count < 1 or horizon < 1. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0030          raise ValueError('count and horizon must be positive')
```

Interrompe o caminho atual lançando ValueError('count and horizon must be positive'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'count and horizon must be positive'.

```
0031      values = np.asarray(training_values, dtype=float)
```

Atribui a values o resultado de np.asarray(training_values, dtype=float). A expressão à direita é avaliada antes da atribuição.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: training_values; dtype: float.

```
0032      width = len(history)
```

Atribui a width o resultado de len(history). A expressão à direita é avaliada antes da atribuição.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0033      origins = np.arange(width, len(values) - horizon + 1)
```

Atribui a origins o resultado de np.arange(width, len(values) - horizon + 1). A expressão à direita é avaliada antes da atribuição.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: width; posicional 2: len(values) - horizon + 1.

len — Conta elementos. Argumentos usados: posicional 1: values.

```
0034      if len(origins) < count:
```

Avalia a condição len(origins) < count. O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: origins.

```
0035          raise ValueError('Insufficient pre-test data for requested demonstrations')
```

Interrompe o caminho atual lançando ValueError('Insufficient pre-test data for requested demonstrations'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Insufficient pre-test data for requested demonstrations'.

```
0036      if strategy == 'few_random':
```

Avalia a condição strategy == 'few_random'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0037          selected = np.random.default_rng(seed).choice(origins, size=count, replace=False)
```

Atribui a selected o resultado de np.random.default_rng(seed).choice(origins, size=count, replace=False). A expressão à direita é avaliada antes da atribuição.

np.random.default_rng(seed).choice — Sorteia elementos do gerador NumPy. Argumentos usados: posicional 1: origins; size: count; replace: False.

np.random.default_rng — Cria um gerador pseudoaleatório local. Argumentos usados: posicional 1: seed.

```
0038      else:
```

Ramo alternativo da condição imediatamente anterior: executa quando o teste anterior não foi satisfeito.

```
0039          def standardized(x):
```

Define standardized(x). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0040              return (x - np.mean(x)) / max(float(np.std(x)), 1e-8)
```

Encerra esta função e devolve (x - np.mean(x)) / max(float(np.std(x)), 1e-08) ao chamador; não imprime o resultado por si só.

max — Retorna maior valor. Argumentos usados: posicional 1: float(np.std(x)); posicional 2: 1e-08.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: x.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.std(x).

np.std — Calcula desvio-padrão. Argumentos usados: posicional 1: x.

```
0041          query = standardized(np.asarray(history))
```

Atribui a query o resultado de standardized(np.asarray(history)). A expressão à direita é avaliada antes da atribuição.

standardized — x: vetor de exemplo/consulta histórica. Retorna valores centrados e divididos por desvio-padrão com piso 1e-8; apenas auxilia seleção por forma. Argumentos usados: posicional 1: np.asarray(history).

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: history.

```
0042          distances = [np.mean((standardized(values[o-width:o]) - query) ** 2) for o in
    origins]
```

Atribui a distances o resultado de [np.mean((standardized(values[o - width:o]) - query) ** 2) for o in origins]. A expressão à direita é avaliada antes da atribuição.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: (standardized(values[o - width:o]) - query) ** 2.

standardized — x: vetor de exemplo/consulta histórica. Retorna valores centrados e divididos por desvio-padrão com piso 1e-8; apenas auxilia seleção por forma. Argumentos usados: posicional 1: values[o - width:o].

```
0043          selected = origins[np.argsort(distances, kind='stable')[:count]]
```

Atribui a selected o resultado de origins[np.argsort(distances, kind='stable')[:count]]. A expressão à direita é avaliada antes da atribuição.

np.argsort — Retorna índices que ordenariam os valores. Argumentos usados: posicional 1: distances; kind: 'stable'.

```
0044      return [Demonstration(tuple(values[o-width:o]), tuple(values[o:o+horizon]),
    int(o+horizon))
```

Encerra esta função e devolve [Demonstration(tuple(values[o - width:o]), tuple(values[o:o + horizon]), int(o + horizon)) for o in sorted(selected)] ao chamador; não imprime o resultado por si só.

Demonstration — Constrói dataclass de exemplo few-shot. Argumentos usados: posicional 1: tuple(values[o - width:o]); posicional 2: tuple(values[o:o + horizon]); posicional 3: int(o + horizon).

tuple — Materializa sequência imutável. Argumentos usados: posicional 1: values[o - width:o].

tuple — Materializa sequência imutável. Argumentos usados: posicional 1: values[o:o + horizon].

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: o + horizon.

```
0045              for o in sorted(selected)]
```

Cláusula de compreensão: percorre os valores indicados para construir a coleção da instrução anterior, mantendo a ordem desse percurso.

sorted — Devolve uma lista ordenada. Argumentos usados: posicional 1: selected.

```
0048  def format_values(values) -> str:
```

Define format_values(values) → str. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0049      """Round only the text representation, retaining raw values in model baselines."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0050      return ','.join(f'{float(value):.2f}' for value in values)
```

Encerra esta função e devolve ','.join((f'{float(value):.2f}' for value in values)) ao chamador; não imprime o resultado por si só.

','.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: (f'{float(value):.2f}' for value in values).

float — Converte escalar para float Python. Argumentos usados: posicional 1: value.

```
0053  LABELS = {
```

Atribui a LABELS o resultado de {'water_produced': 'water production (water_produced)', 'temperature_mean': 'average temperature', 'temperature_2m_previsao_mean': 'forecast average temperature at two metres', 'relative_humidity_mean': 'average relative humidity', 'radiation_mean': 'average radiation', 'precipitation_mean': 'average precipitation', 'precipitation_median': 'median precipitation', 'shortwave_radiation_previsao_std': 'forecast standard deviation of shortwave radiation', 'day': 'day of the month', 'dayofweek': 'day of the week', 'season': 'season of the year', 'is_holiday': 'public holiday status', 'is_holiday_ctba_gtba_jve': 'regional holiday status', 'is_weekend': 'weekend status', 'is_carnival': 'Carnival status', 'is_school_recess_pr': 'Paraná school recess status'}. A expressão à direita é avaliada antes da atribuição.

```
0054      'water_produced': 'water production (water_produced)',
```

Define 'water_produced': campo do registro/coleção em construção; recebe 'water production (water_produced)'. Os valores são avaliados nesta janela/iteração.

```
0055      'temperature_mean': 'average temperature',
```

Define 'temperature_mean': campo do registro/coleção em construção; recebe 'average temperature'. Os valores são avaliados nesta janela/iteração.

```
0056      'temperature_2m_previsao_mean': 'forecast average temperature at two metres',
```

Define 'temperature_2m_previsao_mean': campo do registro/coleção em construção; recebe 'forecast average temperature at two metres'. Os valores são avaliados nesta janela/iteração.

```
0057      'relative_humidity_mean': 'average relative humidity',
```

Define 'relative_humidity_mean': campo do registro/coleção em construção; recebe 'average relative humidity'. Os valores são avaliados nesta janela/iteração.

```
0058      'radiation_mean': 'average radiation',
```

Define 'radiation_mean': campo do registro/coleção em construção; recebe 'average radiation'. Os valores são avaliados nesta janela/iteração.

```
0059      'precipitation_mean': 'average precipitation',
```

Define 'precipitation_mean': campo do registro/coleção em construção; recebe 'average precipitation'. Os valores são avaliados nesta janela/iteração.

```
0060      'precipitation_median': 'median precipitation',
```

Define 'precipitation_median': campo do registro/coleção em construção; recebe 'median precipitation'. Os valores são avaliados nesta janela/iteração.

```
0061      'shortwave_radiation_previsao_std': 'forecast standard deviation of shortwave radiation',
```

Define 'shortwave_radiation_previsao_std': campo do registro/coleção em construção; recebe 'forecast standard deviation of shortwave radiation'. Os valores são avaliados nesta janela/iteração.

```
0062      'day': 'day of the month',
```

Define 'day': campo do registro/coleção em construção; recebe 'day of the month'. Os valores são avaliados nesta janela/iteração.

```
0063      'dayofweek': 'day of the week',
```

Define 'dayofweek': campo do registro/coleção em construção; recebe 'day of the week'. Os valores são avaliados nesta janela/iteração.

```
0064      'season': 'season of the year',
```

Define 'season': campo do registro/coleção em construção; recebe 'season of the year'. Os valores são avaliados nesta janela/iteração.

```
0065      'is_holiday': 'public holiday status',
```

Define 'is_holiday': campo do registro/coleção em construção; recebe 'public holiday status'. Os valores são avaliados nesta janela/iteração.

```
0066      'is_holiday_ctba_gtba_jve': 'regional holiday status',
```

Define 'is_holiday_ctba_gtba_jve': campo do registro/coleção em construção; recebe 'regional holiday status'. Os valores são avaliados nesta janela/iteração.

```
0067      'is_weekend': 'weekend status',
```

Define 'is_weekend': campo do registro/coleção em construção; recebe 'weekend status'. Os valores são avaliados nesta janela/iteração.

```
0068      'is_carnival': 'Carnival status',
```

Define 'is_carnival': campo do registro/coleção em construção; recebe 'Carnival status'. Os valores são avaliados nesta janela/iteração.

```
0069      'is_school_recess_pr': 'Paraná school recess status',
```

Define 'is_school_recess_pr': campo do registro/coleção em construção; recebe 'Paraná school recess status'. Os valores são avaliados nesta janela/iteração.

```
0070  }
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0073  def _narrate(frame: pd.DataFrame, region: str, *, future: bool = False) -> list[str]:
```

Define _narrate(frame: pd.DataFrame, region: str, *, future: bool = False) → list[str]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0074      text = calendar_to_text(frame)
```

Atribui a text o resultado de calendar_to_text(frame). A expressão à direita é avaliada antes da atribuição.

calendar_to_text — frame pode conter calendário numérico ou já convertido. Devolve cópia com calendário inglês, mescla pares cíclicos e preserva demanda/meteorologia. Não ajusta estatísticas nem altera o argumento. Argumentos usados: posicional 1: frame.

```
0075      start, end = text.timestamp.iloc[0], text.timestamp.iloc[-1]
```

Atribui a (start, end) o resultado de (text.timestamp.iloc[0], text.timestamp.iloc[-1]). A expressão à direita é avaliada antes da atribuição.

```
0076      sentences = [
```

Atribui a sentences o resultado de [('The forecast dates' if future else 'The observation dates') + ' in chronological order are: ' + '; '.join(text.timestamp) + '.']. A expressão à direita é avaliada antes da atribuição.

```
0077          ('The forecast dates' if future else 'The observation dates') +
```

Continuação da instrução na linha 76: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0078          ' in chronological order are: ' + '; '.join(text.timestamp) + '.',
```

Continuação da instrução na linha 76: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

'; '.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: text.timestamp.

```
0079      ]
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0080      for column in text.columns:
```

Percorre text.columns; em cada iteração associa o item a column. O corpo indentado aplica a operação a esse item/grupo.

```
0081          if column in {'timestamp', 'unique_id', 'observed'}:
```

Avalia a condição column in {'timestamp', 'unique_id', 'observed'}. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0082              continue
```

Avança imediatamente à próxima iteração; esta janela com falha não recebe linhas de previsão.

```
0083          label = LABELS.get(column, column.replace('_', ' '))
```

Atribui a label o resultado de LABELS.get(column, column.replace('_', ' ')). A expressão à direita é avaliada antes da atribuição.

LABELS.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: column; posicional 2: column.replace('_', ' ').

column.replace — Substitui o destino pelo arquivo representado pelo Path. Argumentos usados: posicional 1: '_'; posicional 2: ' '.

```
0084          values = '; '.join(f'{value:.2f}' if column == 'water_produced' else
```

Atribui a values o resultado de '; '.join((f'{value:.2f}' if column == 'water_produced' else f'{value:.6g}' if isinstance(value, (float, int)) else str(value) for value in text[column])). A expressão à direita é avaliada antes da atribuição.

'; '.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: (f'{value:.2f}' if column == 'water_produced' else f'{value:.6g}' if isinstance(value, (float, int)) else str(value) for value in text[column]).

```
0085                              f'{value:.6g}' if isinstance(value, (float, int)) else str(value)
```

Continuação da instrução na linha 84: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

isinstance — Verifica se um objeto é instância de um tipo ou tupla de tipos. Argumentos usados: posicional 1: value; posicional 2: (float, int).

str — Produz representação textual. Argumentos usados: posicional 1: value.

```
0086                              for value in text[column])
```

Cláusula de compreensão: percorre os valores indicados para construir a coleção da instrução anterior, mantendo a ordem desse percurso.

```
0087          unit = {'water_produced': ' cubic metres', 'temperature_mean': ' degrees Celsius',
```

Atribui a unit o resultado de {'water_produced': ' cubic metres', 'temperature_mean': ' degrees Celsius', 'temperature_2m_previsao_mean': ' degrees Celsius'}.get(column, ''). A expressão à direita é avaliada antes da atribuição.

{'water_produced': ' cubic metres', 'temperature_mean': ' degrees Celsius', 'temperature_2m_previsao_mean': ' degrees Celsius'}.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: column; posicional 2: ''.

```
0088                  'temperature_2m_previsao_mean': ' degrees Celsius'}.get(column, '')
```

Define 'temperature_2m_previsao_mean': campo do registro/coleção em construção; recebe ' degrees Celsius'. Os valores são avaliados nesta janela/iteração.

```
0089          subject = f'the supplied {label}' if future else f'the {label}'
```

Atribui a subject o resultado de f'the supplied {label}' if future else f'the {label}'. A expressão à direita é avaliada antes da atribuição.

```
0090          verb = 'is' if future else 'was'
```

Atribui a verb o resultado de 'is' if future else 'was'. A expressão à direita é avaliada antes da atribuição.

```
0091          sentences.append(f'From {start} to {end}, {subject} of region {region} '
```

Executa sentences.append(f'From {start} to {end}, {subject} of region {region} {verb} {values}{unit} on each day, respectively.'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

sentences.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: f'From {start} to {end}, {subject} of region {region} {verb} {values}{unit} on each day, respectively.'.

```
0092                              f'{verb} {values}{unit} on each day, respectively.')
```

Continuação da instrução na linha 91: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0093      return sentences
```

Encerra esta função e devolve sentences ao chamador; não imprime o resultado por si só.

```
0096  def build_prompt(history: pd.DataFrame, horizon: int, *,
```

Define build_prompt(history: pd.DataFrame, horizon: int, *, demonstrations: list[Demonstration] | None = None, future_covariates: pd.DataFrame | None = None, region: str = 'the study area') → str. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0097                  demonstrations: list[Demonstration] | None = None,
```

Continuação da instrução na linha 96: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção. history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON.

```
0098                  future_covariates: pd.DataFrame | None = None,
```

Continuação da instrução na linha 96: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção. history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON.

```
0099                  region: str = 'the study area') -> str:
```

Continuação da instrução na linha 96: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção. history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON.

```
0100      """Narrate historical inputs and target-free supplied covariates in English.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0102      Region is passed explicitly by the pipeline. Calendar conversion is purely
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0103      representational: no fitting, aggregation, or access to future targets.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0104      """
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0105      if horizon < 1 or history.empty:
```

Avalia a condição horizon < 1 or history.empty. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0106          raise ValueError('History and positive horizon are required')
```

Interrompe o caminho atual lançando ValueError('History and positive horizon are required'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'History and positive horizon are required'.

```
0107      if not np.isfinite(history.water_produced.to_numpy(dtype=float)).all():
```

Avalia a condição not np.isfinite(history.water_produced.to_numpy(dtype=float)).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.isfinite(history.water_produced.to_numpy(dtype=float)).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: history.water_produced.to_numpy(dtype=float).

history.water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: dtype: float.

```
0108          raise ValueError('History contains invalid values')
```

Interrompe o caminho atual lançando ValueError('History contains invalid values'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'History contains invalid values'.

```
0109      timestamps = pd.DatetimeIndex(history.timestamp)
```

Atribui a timestamps o resultado de pd.DatetimeIndex(history.timestamp). A expressão à direita é avaliada antes da atribuição.

pd.DatetimeIndex — Constrói um índice temporal para comparação. Argumentos usados: posicional 1: history.timestamp.

```
0110      if timestamps.hasnans or not timestamps.equals(pd.date_range(timestamps[0],
    periods=len(history), freq='D')):
```

Avalia a condição timestamps.hasnans or not timestamps.equals(pd.date_range(timestamps[0], periods=len(history), freq='D')). O bloco indentado seguinte só é executado quando ela é verdadeira.

timestamps.equals — Compara conteúdo e estrutura do receptor com outro objeto pandas. Argumentos usados: posicional 1: pd.date_range(timestamps[0], periods=len(history), freq='D').

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: timestamps[0]; periods: len(history); freq: 'D'.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0111          raise ValueError('History must contain ordered consecutive daily observations')
```

Interrompe o caminho atual lançando ValueError('History must contain ordered consecutive daily observations'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'History must contain ordered consecutive daily observations'.

```
0112      dates = future_dates(timestamps[-1], horizon)
```

Atribui a dates o resultado de future_dates(timestamps[-1], horizon). A expressão à direita é avaliada antes da atribuição.

future_dates — last_observed: último Timestamp histórico; horizon: quantidade de dias. Retorna DatetimeIndex dos próximos dias. Argumentos usados: posicional 1: timestamps[-1]; posicional 2: horizon.

```
0113      if future_covariates is not None:
```

Avalia a condição future_covariates is not None. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0114          if 'water_produced' in future_covariates.columns:
```

Avalia a condição 'water_produced' in future_covariates.columns. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0115              raise ValueError('Future target must never appear in a prompt')
```

Interrompe o caminho atual lançando ValueError('Future target must never appear in a prompt'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Future target must never appear in a prompt'.

```
0116          if len(future_covariates) != horizon or not
    dates.equals(pd.DatetimeIndex(future_covariates.timestamp)):
```

Avalia a condição len(future_covariates) != horizon or not dates.equals(pd.DatetimeIndex(future_covariates.timestamp)). O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: future_covariates.

dates.equals — Compara conteúdo e estrutura do receptor com outro objeto pandas. Argumentos usados: posicional 1: pd.DatetimeIndex(future_covariates.timestamp).

pd.DatetimeIndex — Constrói um índice temporal para comparação. Argumentos usados: posicional 1: future_covariates.timestamp.

```
0117              raise ValueError('Future covariate dates must match the requested horizon')
```

Interrompe o caminho atual lançando ValueError('Future covariate dates must match the requested horizon'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Future covariate dates must match the requested horizon'.

```
0118          columns = [c for c in future_covariates.columns if c != 'timestamp']
```

Atribui a columns o resultado de [c for c in future_covariates.columns if c != 'timestamp']. A expressão à direita é avaliada antes da atribuição.

```
0119          if not set(columns).issubset(history.columns):
```

Avalia a condição not set(columns).issubset(history.columns). O bloco indentado seguinte só é executado quando ela é verdadeira.

set(columns).issubset — Verifica se todos os elementos estão contidos em outro conjunto/iterável. Argumentos usados: posicional 1: history.columns.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: columns.

```
0120              raise ValueError('Historical and future covariates must match')
```

Interrompe o caminho atual lançando ValueError('Historical and future covariates must match'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Historical and future covariates must match'.

```
0121      parts = [
```

Atribui a parts o resultado de ['Forecast daily water production in cubic metres using demand, weather and calendar context.', 'Each sequence below contains one value per day, aligned with the listed dates in chronological order.', f'Return ONLY a JSON object with key "predictions" and exactly {horizon} nonnegative numbers.', 'Do not explain. Do not repeat the historical values as an answer.']. A expressão à direita é avaliada antes da atribuição.

```
0122          'Forecast daily water production in cubic metres using demand, weather and calendar
    context.',
```

Continuação da instrução na linha 121: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0123          'Each sequence below contains one value per day, aligned with the listed dates in
    chronological order.',
```

Continuação da instrução na linha 121: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0124          f'Return ONLY a JSON object with key "predictions" and exactly {horizon} nonnegative
    numbers.',
```

Continuação da instrução na linha 121: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0125          'Do not explain. Do not repeat the historical values as an answer.',
```

Continuação da instrução na linha 121: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0126      ]
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0127      if region.casefold().startswith('guaratuba'):
```

Avalia a condição region.casefold().startswith('guaratuba'). O bloco indentado seguinte só é executado quando ela é verdadeira.

region.casefold().startswith — Testa prefixo textual exato. Argumentos usados: posicional 1: 'guaratuba'.

region.casefold — Normaliza caixa de texto para comparação. Argumentos usados: nenhum argumento explícito.

```
0128          parts.append('Guaratuba is a tourist city with strong seasonality during Brazilian
    holidays '
```

Executa parts.append('Guaratuba is a tourist city with strong seasonality during Brazilian holidays and school recesses, especially from December through March. Consider this context when interpreting changes in water demand.'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parts.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: 'Guaratuba is a tourist city with strong seasonality during Brazilian holidays and school recesses, especially from December through March. Consider this context when interpreting changes in water demand.'.

```
0129                        'and school recesses, especially from December through March. '
```

Continuação da instrução na linha 128: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0130                        'Consider this context when interpreting changes in water demand.')
```

Continuação da instrução na linha 128: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0131      for example in demonstrations or []:
```

Percorre demonstrations or []; em cada iteração associa o item a example. O corpo indentado aplica a operação a esse item/grupo.

```
0132          parts.append('In a historical training example, daily water production was ' +
```

Executa parts.append('In a historical training example, daily water production was ' + format_values(example.history) + ' cubic metres, respectively.'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parts.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: 'In a historical training example, daily water production was ' + format_values(example.history) + ' cubic metres, respectively.'.

```
0133                        format_values(example.history) + ' cubic metres, respectively.')
```

Continuação da instrução na linha 132: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

format_values — values: números históricos. Retorna string separada por vírgulas com duas casas decimais. Argumentos usados: posicional 1: example.history.

```
0134          parts.append('The following daily production values were: ' +
```

Executa parts.append('The following daily production values were: ' + json.dumps({'predictions': list(example.target)})). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parts.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: 'The following daily production values were: ' + json.dumps({'predictions': list(example.target)}).

```
0135                        json.dumps({'predictions': list(example.target)}))
```

Define 'predictions': lista ordenada de números retornados; recebe list(example.target). Os valores são avaliados nesta janela/iteração.

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: {'predictions': list(example.target)}.

list — Materializa um iterável em lista. Argumentos usados: posicional 1: example.target.

```
0136      parts.extend(_narrate(history, region))
```

Executa parts.extend(_narrate(history, region)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parts.extend — Adiciona cada elemento de um iterável à lista. Argumentos usados: posicional 1: _narrate(history, region).

_narrate — frame é um recorte de história ou futuro sem alvo; region identifica a cidade; future altera os tempos verbais e indica covariáveis fornecidas. Retorna lista de frases alinhadas às datas. Argumentos usados: posicional 1: history; posicional 2: region.

```
0137      if future_covariates is not None:
```

Avalia a condição future_covariates is not None. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0138          parts.append('The following covariates are supplied inputs for the forecast dates; '
```

Executa parts.append('The following covariates are supplied inputs for the forecast dates; they do not include future water production.'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parts.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: 'The following covariates are supplied inputs for the forecast dates; they do not include future water production.'.

```
0139                        'they do not include future water production.')
```

Continuação da instrução na linha 138: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0140          parts.extend(_narrate(future_covariates, region, future=True))
```

Executa parts.extend(_narrate(future_covariates, region, future=True)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parts.extend — Adiciona cada elemento de um iterável à lista. Argumentos usados: posicional 1: _narrate(future_covariates, region, future=True).

_narrate — frame é um recorte de história ou futuro sem alvo; region identifica a cidade; future altera os tempos verbais e indica covariáveis fornecidas. Retorna lista de frases alinhadas às datas. Argumentos usados: posicional 1: future_covariates; posicional 2: region; future: True.

```
0141      for date in dates:
```

Percorre dates; em cada iteração associa o item a date. O corpo indentado aplica a operação a esse item/grupo.

```
0142          parts.append(f'What is the water_produced going to be on {date_to_text(date)} in
    region {region}?')
```

Executa parts.append(f'What is the water_produced going to be on {date_to_text(date)} in region {region}?'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parts.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: f'What is the water_produced going to be on {date_to_text(date)} in region {region}?'.

date_to_text — value é uma data ou sua representação textual canônica. Retorna ano, dia e mês em inglês sem mudar o ano original. Argumentos usados: posicional 1: date.

```
0143      parts.append(f'Output schema: {{"predictions": [number, ...]}}; array length must be
    {horizon}, '
```

Executa parts.append(f'Output schema: {{"predictions": [number, ...]}}; array length must be {horizon}, in the same order as the questions.'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parts.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: f'Output schema: {{"predictions": [number, ...]}}; array length must be {horizon}, in the same order as the questions.'.

```
0144                  'in the same order as the questions.')
```

Continuação da instrução na linha 143: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0145      return '\n'.join(parts)
```

Encerra esta função e devolve '\n'.join(parts) ao chamador; não imprime o resultado por si só.

'\n'.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: parts.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

36. src/models/promptcast.py

Adaptador HTTP do Qwen: configuração, schema de saída, parser, cliente JSON e classe QwenForecaster. forecasting_pipeline chama predict; post_json encapsula transporte, e parse_prediction valida sem corrigir artificialmente valores.

SHA-256: b222c85acd066aaed968da9b511fe19f49ee6f98e7dbc602ed1abda79792e9ee

Funções, contratos e pontos de conexão

```
L21: __post_init__(self)
```

Valida limites após a inicialização automática da dataclass; não é chamado manualmente no pipeline.

```
L26: prediction_schema(horizon: int) → dict
```

horizon: comprimento exigido do array. Retorna JSON Schema para objeto predictions, números não negativos e ausência de chaves extras.

```
L33: parse_prediction(text: str, horizon: int) → list[float]
```

text: string JSON do modelo; horizon: quantidade exata de valores. Retorna list[float] ou lança erro; não remove prose nem aceita resposta parcial.

```
L46: post_json(url: str, payload: dict, timeout: int) → dict
```

url: endpoint; payload: dict do pedido; timeout: segundos de espera. Retorna dict decodificado do envelope HTTP, não ainda a lista de previsões.

```
L54: __init__(self, config: OllamaConfig)
```

Construtor da classe: self recebe a instância e os demais argumentos inicializam configuração, backend ou estado. Confira o corpo abaixo para I/O e carregamento de modelo.

Chamadas do projeto: post_json → src/models/promptcast.py:46

```
L71: predict(self, history, horizon: int, *, prompt: str [obrigatório], future_covariates = None) →
     list[float]
```

history é tabela histórica; horizon é quantidade prevista; future_covariates é tabela sem alvo; prompt é texto já construído para Qwen. Retorna lista de floats; cada classe escolhe backend. O método pertence à classe adaptadora deste arquivo.

Chamadas do projeto: parse_prediction → src/models/promptcast.py:33; post_json → src/models/promptcast.py:46; prediction_schema → src/models/promptcast.py:26

APIs externas usadas neste arquivo

RuntimeError, ValueError, any, append, asdict, dataclass, encode, float, get, int, isinstance, json.dumps, json.load, json.loads, len, math.isfinite, min, range, set, str, time.sleep, type, urllib.request.Request, urllib.request.urlopen. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Qwen forecasting through Ollama, with strict output validation and audit data."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import math
```

Importa math. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import time
```

Importa time. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import urllib.error
```

Importa urllib.error. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import urllib.request
```

Importa urllib.request. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  from dataclasses import asdict, dataclass
```

Importa asdict, dataclass de dataclasses. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  @dataclass(frozen=True)
```

Aplica o decorador dataclass(frozen=True) à classe declarada em seguida; gera comportamento de dataclass.

dataclass — Decora a classe para gerar métodos a partir dos campos anotados. Argumentos usados: frozen: True.

```
0011  class OllamaConfig:
```

Declara a classe OllamaConfig. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0012      model: str = 'qwen3:1.7b'
```

Nome padrão legado. Configurações do experimento 4B substituem esse valor ao construir OllamaConfig; alterar apenas este default não muda JSON já fornecido.

```
0013      base_url: str = 'http://127.0.0.1:11434'
```

URL padrão legado, porta 11434. O JSON do 4B usa servidor dedicado local na porta 11435.

```
0014      temperature: float = 0
```

Temperatura padrão zero. É configuração de amostragem, não um normalizador dos dados; não garante igualdade bit a bit entre runtimes.

```
0015      num_ctx: int = 8192
```

Padrão do adaptador: 8192 tokens. As quatro configurações qwen_text e o benchmark atual sobrescrevem para 16384; a configuração histórica de 4B/15 dias usa 131072. Isso não representa dias de histórico.

```
0016      num_predict: int = 256
```

Máximo de tokens novos, contando pontuação e estrutura JSON, não quantidade de valores previstos.

```
0017      timeout: int = 600
```

Espera HTTP padrão de 600 segundos; configuração 4B sobrescreve para 7200.

```
0018      retries: int = 2
```

Número de tentativas adicionais, não total; retries=2 permite três chamadas no loop.

```
0019      seed: int = 42
```

Semente transmitida às opções de geração do Ollama.

```
0021      def __post_init__(self):
```

Gancho da dataclass: executado automaticamente depois da inicialização gerada, para rejeitar limites inválidos.

```
0022          if min(self.num_ctx, self.num_predict, self.timeout) < 1 or self.retries < 0:
```

Avalia a condição min(self.num_ctx, self.num_predict, self.timeout) < 1 or self.retries < 0. O bloco indentado seguinte só é executado quando ela é verdadeira.

min — Retorna menor valor. Argumentos usados: posicional 1: self.num_ctx; posicional 2: self.num_predict; posicional 3: self.timeout.

```
0023              raise ValueError('Invalid Ollama limits')
```

Interrompe o caminho atual lançando ValueError('Invalid Ollama limits'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Invalid Ollama limits'.

```
0026  def prediction_schema(horizon: int) -> dict:
```

Define prediction_schema(horizon: int) → dict. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0027      return {'type': 'object', 'properties': {'predictions': {
```

Abre schema de objeto JSON. properties descreve a chave predictions, sem inserir previsões reais no pedido.

```
0028          'type': 'array', 'items': {'type': 'number', 'minimum': 0},
```

Cada item deve ser número com limite inferior zero; aqui mínimo é uma restrição de saída, não clipping posterior.

```
0029          'minItems': horizon, 'maxItems': horizon}},
```

minItems e maxItems iguais a horizon impõem exatamente sete valores quando horizon=7.

```
0030          'required': ['predictions'], 'additionalProperties': False}
```

required torna predictions obrigatória; additionalProperties=False proíbe outras chaves no objeto.

```
0033  def parse_prediction(text: str, horizon: int) -> list[float]:
```

Define parse_prediction(text: str, horizon: int) → list[float]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0034      """Reject prose, strings, booleans, NaN, missing values and wrong horizons."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0035      parsed = json.loads(text)
```

Atribui a parsed o resultado de json.loads(text). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: text.

```
0036      if not isinstance(parsed, dict) or set(parsed) != {'predictions'}:
```

Avalia a condição not isinstance(parsed, dict) or set(parsed) != {'predictions'}. O bloco indentado seguinte só é executado quando ela é verdadeira.

isinstance — Verifica se um objeto é instância de um tipo ou tupla de tipos. Argumentos usados: posicional 1: parsed; posicional 2: dict.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: parsed.

```
0037          raise ValueError('Expected exactly the predictions key')
```

Interrompe o caminho atual lançando ValueError('Expected exactly the predictions key'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Expected exactly the predictions key'.

```
0038      values = parsed['predictions']
```

Atribui a values o resultado de parsed['predictions']. A expressão à direita é avaliada antes da atribuição.

```
0039      if not isinstance(values, list) or len(values) != horizon:
```

Avalia a condição not isinstance(values, list) or len(values) != horizon. O bloco indentado seguinte só é executado quando ela é verdadeira.

isinstance — Verifica se um objeto é instância de um tipo ou tupla de tipos. Argumentos usados: posicional 1: values; posicional 2: list.

len — Conta elementos. Argumentos usados: posicional 1: values.

```
0040          raise ValueError(f'Expected {horizon} predictions')
```

Interrompe o caminho atual lançando ValueError(f'Expected {horizon} predictions'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Expected {horizon} predictions'.

```
0041      if any(type(v) not in (int, float) or not math.isfinite(v) or v < 0 for v in values):
```

O teste type(v), diferente de isinstance(v,int), impede que true/false sejam aceitos como 1/0. O gerador rejeita também infinitos e negativos.

any — Reduz a existência de pelo menos um True. Argumentos usados: posicional 1: (type(v) not in (int, float) or not math.isfinite(v) or v < 0 for v in values).

type — Obtém a classe concreta do valor. Argumentos usados: posicional 1: v.

math.isfinite — Confere se um único número é finito. Argumentos usados: posicional 1: v.

```
0042          raise ValueError('Predictions must be finite, nonnegative numbers')
```

Interrompe o caminho atual lançando ValueError('Predictions must be finite, nonnegative numbers'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Predictions must be finite, nonnegative numbers'.

```
0043      return [float(v) for v in values]
```

Encerra esta função e devolve [float(v) for v in values] ao chamador; não imprime o resultado por si só.

float — Converte escalar para float Python. Argumentos usados: posicional 1: v.

```
0046  def post_json(url: str, payload: dict, timeout: int) -> dict:
```

Define post_json(url: str, payload: dict, timeout: int) → dict. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0047      request = urllib.request.Request(url, data=json.dumps(payload).encode('utf-8'),
```

Atribui a request o resultado de urllib.request.Request(url, data=json.dumps(payload).encode('utf-8'), headers={'Content-Type': 'application/json'}). A expressão à direita é avaliada antes da atribuição.

urllib.request.Request — Monta uma requisição HTTP. Argumentos usados: posicional 1: url; data: json.dumps(payload).encode('utf-8'); headers: {'Content-Type': 'application/json'}.

json.dumps(payload).encode — Codifica string em bytes. Argumentos usados: posicional 1: 'utf-8'.

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: payload.

```
0048                                      headers={'Content-Type': 'application/json'})
```

Define 'Content-Type': campo do registro/coleção em construção; recebe 'application/json'. Os valores são avaliados nesta janela/iteração.

```
0049      with urllib.request.urlopen(request, timeout=timeout) as response:
```

Abre o contexto gerenciado urllib.request.urlopen(request, timeout=timeout) como response. Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

urllib.request.urlopen — Abre conexão HTTP e devolve resposta como fluxo. Argumentos usados: posicional 1: request; timeout: timeout.

```
0050          return json.load(response)
```

Encerra esta função e devolve json.load(response) ao chamador; não imprime o resultado por si só.

json.load — Decodifica JSON lido de um arquivo/response. Argumentos usados: posicional 1: response.

```
0053  class QwenForecaster:
```

Declara a classe QwenForecaster. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0054      def __init__(self, config: OllamaConfig):
```

Define __init__(self, config: OllamaConfig). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0055          self.config = config
```

Atribui a self.config o resultado de config. A expressão à direita é avaliada antes da atribuição.

```
0056          self.last_audit = {}
```

Atribui a self.last_audit o resultado de {}. A expressão à direita é avaliada antes da atribuição.

```
0057          with urllib.request.urlopen(config.base_url + '/api/tags', timeout=10) as response:
```

Abre o contexto gerenciado urllib.request.urlopen(config.base_url + '/api/tags', timeout=10) como response. Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

urllib.request.urlopen — Abre conexão HTTP e devolve resposta como fluxo. Argumentos usados: posicional 1: config.base_url + '/api/tags'; timeout: 10.

```
0058              tags = json.load(response)
```

Atribui a tags o resultado de json.load(response). A expressão à direita é avaliada antes da atribuição.

json.load — Decodifica JSON lido de um arquivo/response. Argumentos usados: posicional 1: response.

```
0059          matches = [m for m in tags['models'] if m['name'] == config.model]
```

Filtra os modelos instalados por nome exato, incluindo tag/quantização. Não escolhe uma variante aproximada silenciosamente.

```
0060          if not matches:
```

Avalia a condição not matches. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0061              raise ValueError(f'Model {config.model} is not installed in Ollama')
```

Interrompe o caminho atual lançando ValueError(f'Model {config.model} is not installed in Ollama'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Model {config.model} is not installed in Ollama'.

```
0062          metadata = post_json(config.base_url + '/api/show', {'model': config.model}, 30)
```

Consulta metadados do checkpoint instalado; timeout posicional de 30 segundos só vale para esta consulta.

post_json — url: endpoint; payload: dict do pedido; timeout: segundos de espera. Retorna dict decodificado do envelope HTTP, não ainda a lista de previsões. Argumentos usados: posicional 1: config.base_url + '/api/show'; posicional 2: {'model': config.model}; posicional 3: 30.

```
0063          info = metadata['model_info']
```

Atribui a info o resultado de metadata['model_info']. A expressão à direita é avaliada antes da atribuição.

```
0064          architecture = info['general.architecture']
```

Obtém a arquitetura declarada e compõe a chave de metadados de contexto correspondente, em vez de fixar um nome de arquitetura.

```
0065          maximum_context = int(info[f'{architecture}.context_length'])
```

Obtém a arquitetura declarada e compõe a chave de metadados de contexto correspondente, em vez de fixar um nome de arquitetura.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: info[f'{architecture}.context_length'].

```
0066          if config.num_ctx > maximum_context:
```

Avalia a condição config.num_ctx > maximum_context. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0067              raise ValueError(f'Requested {config.num_ctx} tokens exceeds model limit
    {maximum_context}')
```

Interrompe o caminho atual lançando ValueError(f'Requested {config.num_ctx} tokens exceeds model limit {maximum_context}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Requested {config.num_ctx} tokens exceeds model limit {maximum_context}'.

```
0068          self.identity = {'model': config.model, 'digest': matches[0]['digest'],
```

Registra modelo, digest dos pesos, contexto declarado e configuração efetivamente construída. Este dict participa do hash de cache do pipeline.

```
0069                          'maximum_context_tokens': maximum_context, 'config': asdict(config)}
```

Registra modelo, digest dos pesos, contexto declarado e configuração efetivamente construída. Este dict participa do hash de cache do pipeline.

asdict — Converte dataclass em dicionário de seus campos. Argumentos usados: posicional 1: config.

```
0071      def predict(self, history, horizon: int, *, prompt: str, future_covariates=None) ->
    list[float]:
```

Mantém interface dos adaptadores: history e future_covariates são aceitos, mas este método usa o prompt já montado. horizon determina schema e parser; prompt deve ser nomeado após *.

```
0072          """Send only a prebuilt, target-free prompt; retain every attempted response."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0073          config = self.config
```

Atribui a config o resultado de self.config. A expressão à direita é avaliada antes da atribuição.

```
0074          # UTF-8 byte count is a conservative upper bound for byte-level tokenizers.
```

Comentário de implementação: UTF-8 byte count is a conservative upper bound for byte-level tokenizers.. Não é executado; as operações efetivas são as linhas seguintes.

```
0075          # Reserve space for the Ollama model template and the entire generated answer.
```

Comentário de implementação: Reserve space for the Ollama model template and the entire generated answer.. Não é executado; as operações efetivas são as linhas seguintes.

```
0076          if len(prompt.encode('utf-8')) + config.num_predict + 512 > config.num_ctx:
```

Compara bytes UTF-8 do prompt + reserva de geração + 512 bytes/tokens de margem com num_ctx. É limite conservador, não tokenizer exato. Um prompt legítimo pode ser rejeitado antes do limite real.

len — Conta elementos. Argumentos usados: posicional 1: prompt.encode('utf-8').

prompt.encode — Codifica string em bytes. Argumentos usados: posicional 1: 'utf-8'.

```
0077              raise ValueError('Prompt exceeds conservative context budget; increase num_ctx')
```

Interrompe o caminho atual lançando ValueError('Prompt exceeds conservative context budget; increase num_ctx'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Prompt exceeds conservative context budget; increase num_ctx'.

```
0078          payload = {'model': config.model, 'prompt': prompt, 'stream': False,
```

Monta corpo HTTP: nome do modelo, texto completo e stream=False para resposta única. Isso não envia ainda a requisição.

```
0079                      'think': False, 'keep_alive': '10m', 'truncate': False, 'format':
    prediction_schema(horizon),
```

think=False solicita geração sem modo de raciocínio; keep_alive='10m' controla retenção do modelo; truncate=False expressa intenção de não cortar; format recebe o schema. A auditoria do servidor continua necessária.

prediction_schema — horizon: comprimento exigido do array. Retorna JSON Schema para objeto predictions, números não negativos e ausência de chaves extras. Argumentos usados: posicional 1: horizon.

```
0080                      'options': {'temperature': config.temperature, 'seed': config.seed,
```

options agrupa configuração de amostragem e limites; são opções do Ollama, diferentes das opções do cliente urllib, como timeout.

```
0081                                 'num_ctx': config.num_ctx, 'num_predict': config.num_predict}}
```

options agrupa configuração de amostragem e limites; são opções do Ollama, diferentes das opções do cliente urllib, como timeout.

```
0082         self.last_audit = {'request': payload, 'attempts': []}
```

Reinicia auditoria da janela, guardando pedido exato e lista de tentativas. Não reutiliza uma resposta da janela anterior aqui.

```
0083         for attempt in range(config.retries + 1):
```

range(retries+1) inclui a tentativa inicial: com retries=1, attempt assume 0 e 1.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: config.retries + 1.

```
0084             audit = {'attempt': attempt + 1}
```

Cria dict mutável por tentativa e o anexa antes do HTTP. Quando response/error são adicionados depois, a lista aponta para o mesmo dict e preserva o registro.

```
0085             self.last_audit['attempts'].append(audit)
```

Cria dict mutável por tentativa e o anexa antes do HTTP. Quando response/error são adicionados depois, a lista aponta para o mesmo dict e preserva o registro.

self.last_audit['attempts'].append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: audit.

```
0086             try:
```

Inicia região protegida de execução. Exceções compatíveis serão tratadas pelo except correspondente; sucesso continua normalmente.

```
0087                 answer = post_json(config.base_url + '/api/generate', payload,
    config.timeout)
```

Atribui a answer o resultado de post_json(config.base_url + '/api/generate', payload, config.timeout). A expressão à direita é avaliada antes da atribuição.

post_json — url: endpoint; payload: dict do pedido; timeout: segundos de espera. Retorna dict decodificado do envelope HTTP, não ainda a lista de previsões. Argumentos usados: posicional 1: config.base_url + '/api/generate'; posicional 2: payload; posicional 3: config.timeout.

```
0088                 audit['response'] = answer
```

Atribui a audit['response'] o resultado de answer. A expressão à direita é avaliada antes da atribuição.

```
0089                 if not answer.get('done') or answer.get('done_reason') == 'length':
```

done ausente/False ou done_reason='length' rejeita resposta incompleta, mesmo que haja texto parseável.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'done'.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'done_reason'.

```
0090                     raise ValueError('Incomplete or token-limited Ollama response')
```

Interrompe o caminho atual lançando ValueError('Incomplete or token-limited Ollama response'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Incomplete or token-limited Ollama response'.

```
0091                 if answer.get('prompt_eval_count', 0) + answer.get('eval_count', 0) >=
    config.num_ctx:
```

Soma tokens de prompt e saída para rejeitar saturação. get(...,0) tolera campos ausentes, então esta checagem sozinha não certifica contadores válidos em toda versão.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'prompt_eval_count'; posicional 2: 0.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'eval_count'; posicional 2: 0.

```
0092                     raise ValueError('Context limit reached')
```

Interrompe o caminho atual lançando ValueError('Context limit reached'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Context limit reached'.

```
0093                 return parse_prediction(answer.get('response', ''), horizon)
```

Extrai apenas a string response do envelope e valida sete números. Não converte metadados do Ollama em previsão.

parse_prediction — text: string JSON do modelo; horizon: quantidade exata de valores. Retorna list[float] ou lança erro; não remove prose nem aceita resposta parcial. Argumentos usados: posicional 1: answer.get('response', ''); posicional 2: horizon.

answer.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'response'; posicional 2: ''.

```
0094             except (ValueError, urllib.error.URLError, TimeoutError) as exc:
```

Captura erros de validação, transporte e timeout; não captura toda exceção possível dentro deste método.

```
0095                 audit['error'] = str(exc)
```

Atribui a audit['error'] o resultado de str(exc). A expressão à direita é avaliada antes da atribuição.

str — Produz representação textual. Argumentos usados: posicional 1: exc.

```
0096                 if attempt == config.retries:
```

Avalia a condição attempt == config.retries. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0097                     raise
```

Relança a última exceção preservando seu tipo; o pipeline superior registrará status='error'.

```
0098                 time.sleep(min(2 ** attempt, 4))
```

Executa time.sleep(min(2 ** attempt, 4)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

time.sleep — Suspende o processo por uma duração. Argumentos usados: posicional 1: min(2 ** attempt, 4).

min — Retorna menor valor. Argumentos usados: posicional 1: 2 ** attempt; posicional 2: 4.

```
0099         raise RuntimeError('Unreachable retry state')
```

Guarda contra término inesperado do loop; caminhos normais retornam lista válida ou relançam erro antes daqui.

RuntimeError — Cria exceção para estado de execução inesperado. Argumentos usados: posicional 1: 'Unreachable retry state'.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

37. src/models/chronos.py

Adapter de Chronos2Pipeline. Importações pesadas ficam dentro do construtor, permitindo importar o pipeline em outros ambientes. Usa covariáveis passadas/futuras e devolve o quantil 0,5 como previsão pontual.

SHA-256: b87428a5ee2bc106c38203330df3866cfdc3a1b4b4e9ad7b5920a4907eff917c

Funções, contratos e pontos de conexão

```
L6: __init__(self, context: int, dataset: dict)
```

Construtor da classe: self recebe a instância e os demais argumentos inicializam configuração, backend ou estado. Confira o corpo abaixo para I/O e carregamento de modelo.

```
L18: predict(self, history, horizon: int, *, future_covariates [obrigatório], prompt = None) →
    list[float]
```

history é tabela histórica; horizon é quantidade prevista; future_covariates é tabela sem alvo; prompt é texto já construído para Qwen. Retorna lista de floats; cada classe escolhe backend. O método pertence à classe adaptadora deste arquivo.

Chamadas do projeto: covariate_columns → src/data/features/covariates.py:6

APIs externas usadas neste arquivo

ValueError, copy, from_pretrained, importlib.metadata.version, len, predict_df, str, to_numpy, tolist, torch.cuda.is_available, torch.manual_seed. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Chronos-2 with original historical/future covariates and growing context."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import importlib.metadata
```

Importa importlib.metadata. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  class ChronosForecaster:
```

Declara a classe ChronosForecaster. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0006      def __init__(self, context: int, dataset: dict):
```

Define __init__(self, context: int, dataset: dict). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0007          import torch
```

Importa torch. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008          from chronos import Chronos2Pipeline
```

Importa Chronos2Pipeline de chronos. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009          self.dataset = dataset
```

Atribui a self.dataset o resultado de dataset. A expressão à direita é avaliada antes da atribuição.

```
0010          torch.manual_seed(42)
```

Executa torch.manual_seed(42). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

torch.manual_seed — Define semente do gerador PyTorch. Argumentos usados: posicional 1: 42.

```
0011          self.pipeline = Chronos2Pipeline.from_pretrained(
```

Atribui a self.pipeline o resultado de Chronos2Pipeline.from_pretrained('amazon/chronos-2', device_map='cuda' if torch.cuda.is_available() else 'cpu', torch_dtype=torch.float32, local_files_only=True). A expressão à direita é avaliada antes da atribuição.

Chronos2Pipeline.from_pretrained — Carrega pesos/configuração de um checkpoint. Argumentos usados: posicional 1: 'amazon/chronos-2'; device_map: 'cuda' if torch.cuda.is_available() else 'cpu'; torch_dtype: torch.float32; local_files_only: True.

```
0012              'amazon/chronos-2', device_map='cuda' if torch.cuda.is_available() else 'cpu',
```

Continua os argumentos nomeados: device_map = 'cuda' if torch.cuda.is_available() else 'cpu'. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

torch.cuda.is_available — Consulta disponibilidade do backend CUDA. Argumentos usados: nenhum argumento explícito.

```
0013              torch_dtype=torch.float32, local_files_only=True)
```

Continua os argumentos nomeados: torch_dtype = torch.float32; local_files_only = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0014          self.identity = {'model': 'amazon/chronos-2', 'initial_context': context,
```

Atribui a self.identity o resultado de {'model': 'amazon/chronos-2', 'initial_context': context, 'version': importlib.metadata.version('chronos-forecasting')}. A expressão à direita é avaliada antes da atribuição.

```
0015                            'version': importlib.metadata.version('chronos-forecasting')}
```

Define 'version': campo do registro/coleção em construção; recebe importlib.metadata.version('chronos-forecasting'). Os valores são avaliados nesta janela/iteração.

importlib.metadata.version — Lê versão instalada de uma distribuição Python. Argumentos usados: posicional 1: 'chronos-forecasting'.

```
0016          self.last_audit = {}
```

Atribui a self.last_audit o resultado de {}. A expressão à direita é avaliada antes da atribuição.

```
0018      def predict(self, history, horizon: int, *, future_covariates, prompt=None) ->
    list[float]:
```

Define predict(self, history, horizon: int, *, future_covariates [obrigatório], prompt = None) → list[float]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0019          from src.data.features.covariates import covariate_columns
```

Importa covariate_columns de src.data.features.covariates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0020          if 'water_produced' in future_covariates:
```

Avalia a condição 'water_produced' in future_covariates. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0021              raise ValueError('Future targets cannot enter Chronos')
```

Interrompe o caminho atual lançando ValueError('Future targets cannot enter Chronos'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Future targets cannot enter Chronos'.

```
0022          columns = covariate_columns(self.dataset)
```

Atribui a columns o resultado de covariate_columns(self.dataset). A expressão à direita é avaliada antes da atribuição.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: self.dataset.

```
0023          frame = history[['timestamp', 'water_produced'] + columns].copy()
```

Atribui a frame o resultado de history[['timestamp', 'water_produced'] + columns].copy(). A expressão à direita é avaliada antes da atribuição.

history[['timestamp', 'water_produced'] + columns].copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0024          future = future_covariates[['timestamp'] + columns].copy()
```

Atribui a future o resultado de future_covariates[['timestamp'] + columns].copy(). A expressão à direita é avaliada antes da atribuição.

future_covariates[['timestamp'] + columns].copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0025          frame['unique_id'] = future['unique_id'] = 'water'
```

Atribui a frame['unique_id'], future['unique_id'] o resultado de 'water'. A expressão à direita é avaliada antes da atribuição.

```
0026          self.last_audit = {'history_rows': len(frame), 'context_length': len(frame),
```

Atribui a self.last_audit o resultado de {'history_rows': len(frame), 'context_length': len(frame), 'history_start': str(frame.timestamp.iloc[0]), 'history_end': str(frame.timestamp.iloc[-1]), 'future_columns': future.columns.tolist(), 'covariates': columns}. A expressão à direita é avaliada antes da atribuição.

len — Conta elementos. Argumentos usados: posicional 1: frame.

len — Conta elementos. Argumentos usados: posicional 1: frame.

```
0027                              'history_start': str(frame.timestamp.iloc[0]),
```

Define 'history_start': primeira data histórica efetivamente fornecida; recebe str(frame.timestamp.iloc[0]). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: frame.timestamp.iloc[0].

```
0028                              'history_end': str(frame.timestamp.iloc[-1]),
```

Define 'history_end': última data conhecida antes da origem; recebe str(frame.timestamp.iloc[-1]). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: frame.timestamp.iloc[-1].

```
0029                              'future_columns': future.columns.tolist(), 'covariates': columns}
```

Define 'future_columns': campo do registro/coleção em construção; recebe future.columns.tolist(); 'covariates': nomes das variáveis auxiliares; recebe columns. Os valores são avaliados nesta janela/iteração.

future.columns.tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

```
0030          result = self.pipeline.predict_df(
```

Atribui a result o resultado de self.pipeline.predict_df(frame, future_df=future, prediction_length=horizon, freq='D', batch_size=1, id_column='unique_id', timestamp_column='timestamp', target='water_produced', quantile_levels=[0.1, 0.5, 0.9], context_length=len(frame)). A expressão à direita é avaliada antes da atribuição.

self.pipeline.predict_df — Chronos: prevê uma tabela temporal com covariáveis. Argumentos usados: posicional 1: frame; future_df: future; prediction_length: horizon; freq: 'D'; batch_size: 1; id_column: 'unique_id'; timestamp_column: 'timestamp'; target: 'water_produced'; quantile_levels: [0.1, 0.5, 0.9]; context_length: len(frame).

```
0031              frame, future_df=future, prediction_length=horizon, freq='D', batch_size=1,
```

Continua os argumentos nomeados: future_df = future; prediction_length = horizon; freq = 'D'; batch_size = 1. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0032              id_column='unique_id', timestamp_column='timestamp', target='water_produced',
```

Continua os argumentos nomeados: id_column = 'unique_id'; timestamp_column = 'timestamp'; target = 'water_produced'. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0033              quantile_levels=[0.1, 0.5, 0.9], context_length=len(frame))
```

Continua os argumentos nomeados: quantile_levels = [0.1, 0.5, 0.9]; context_length = len(frame). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

len — Conta elementos. Argumentos usados: posicional 1: frame.

```
0034          return result['0.5'].to_numpy(dtype=float).tolist()
```

Encerra esta função e devolve result['0.5'].to_numpy(dtype=float).tolist() ao chamador; não imprime o resultado por si só.

result['0.5'].to_numpy(dtype=float).tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

result['0.5'].to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: dtype: float.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

38. src/models/timesfm.py

Adapter de TimesFM 2.5 com XReg. Prepara capacidade suficiente para todo o histórico a cada origem e devolve somente os primeiros horizon valores. Não corta a história ao contexto inicial.

SHA-256: 508356fc7f9b65ed2ef610fa1214374f15cc57dca933f767015cceac03892ff0

Funções, contratos e pontos de conexão

```
L8: __init__(self, context: int, dataset: dict)
```

Construtor da classe: self recebe a instância e os demais argumentos inicializam configuração, backend ou estado. Confira o corpo abaixo para I/O e carregamento de modelo.

```
L20: predict(self, history, horizon: int, *, future_covariates [obrigatório], prompt = None) →
    list[float]
```

history é tabela histórica; horizon é quantidade prevista; future_covariates é tabela sem alvo; prompt é texto já construído para Qwen. Retorna lista de floats; cada classe escolhe backend. O método pertence à classe adaptadora deste arquivo.

Chamadas do projeto: covariate_columns → src/data/features/covariates.py:6; timesfm_covariates → src/data/features/covariates.py:36

APIs externas usadas neste arquivo

compile, forecast_with_covariates, from_pretrained, importlib.metadata.version, len, np.asarray, str, timesfm.ForecastConfig, to_numpy, tolist, torch.manual_seed, torch.set_float32_matmul_precision. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Original TimesFM XReg + TimesFM path, with context growing at each origin."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import importlib.metadata
```

Importa importlib.metadata. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from src.data.features.covariates import timesfm_covariates, covariate_columns
```

Importa timesfm_covariates, covariate_columns de src.data.features.covariates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  class TimesFMForecaster:
```

Declara a classe TimesFMForecaster. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0008      def __init__(self, context: int, dataset: dict):
```

Define __init__(self, context: int, dataset: dict). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0009          import torch
```

Importa torch. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010          import timesfm
```

Importa timesfm. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011          self.dataset = dataset
```

Atribui a self.dataset o resultado de dataset. A expressão à direita é avaliada antes da atribuição.

```
0012          torch.manual_seed(42)
```

Executa torch.manual_seed(42). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

torch.manual_seed — Define semente do gerador PyTorch. Argumentos usados: posicional 1: 42.

```
0013          torch.set_float32_matmul_precision('high')
```

Executa torch.set_float32_matmul_precision('high'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

torch.set_float32_matmul_precision — Configura precisão permitida para multiplicações de matrizes float32. Argumentos usados: posicional 1: 'high'.

```
0014          self.model = timesfm.TimesFM_2p5_200M_torch.from_pretrained(
```

Atribui a self.model o resultado de timesfm.TimesFM_2p5_200M_torch.from_pretrained('google/timesfm-2.5-200m-pytorch', local_files_only=True). A expressão à direita é avaliada antes da atribuição.

timesfm.TimesFM_2p5_200M_torch.from_pretrained — Carrega pesos/configuração de um checkpoint. Argumentos usados: posicional 1: 'google/timesfm-2.5-200m-pytorch'; local_files_only: True.

```
0015              'google/timesfm-2.5-200m-pytorch', local_files_only=True)
```

Continua os argumentos nomeados: local_files_only = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0016          self.identity = {'model': 'google/timesfm-2.5-200m-pytorch', 'initial_context':
    context,
```

Atribui a self.identity o resultado de {'model': 'google/timesfm-2.5-200m-pytorch', 'initial_context': context, 'version': importlib.metadata.version('timesfm'), 'xreg_mode': 'xreg + timesfm'}. A expressão à direita é avaliada antes da atribuição.

```
0017                            'version': importlib.metadata.version('timesfm'), 'xreg_mode': 'xreg
     + timesfm'}
```

Define 'version': campo do registro/coleção em construção; recebe importlib.metadata.version('timesfm'); 'xreg_mode': campo do registro/coleção em construção; recebe 'xreg + timesfm'. Os valores são avaliados nesta janela/iteração.

importlib.metadata.version — Lê versão instalada de uma distribuição Python. Argumentos usados: posicional 1: 'timesfm'.

```
0018          self.last_audit = {}
```

Atribui a self.last_audit o resultado de {}. A expressão à direita é avaliada antes da atribuição.

```
0020      def predict(self, history, horizon: int, *, future_covariates, prompt=None) ->
    list[float]:
```

Define predict(self, history, horizon: int, *, future_covariates [obrigatório], prompt = None) → list[float]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0021          import timesfm
```

Importa timesfm. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0022          context = len(history)
```

Atribui a context o resultado de len(history). A expressão à direita é avaliada antes da atribuição.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0023          self.model.compile(timesfm.ForecastConfig(
```

Executa self.model.compile(timesfm.ForecastConfig(max_context=(context + 31) // 32 * 32, max_horizon=(horizon + 127) // 128 * 128, normalize_inputs=True, use_continuous_quantile_head=True, force_flip_invariance=True, infer_is_positive=True, fix_quantile_crossing=True, return_backcast=True)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.model.compile — Prepara a configuração de previsão do TimesFM. Argumentos usados: posicional 1: timesfm.ForecastConfig(max_context=(context + 31) // 32 * 32, max_horizon=(horizon + 127) // 128 * 128, normalize_inputs=True, use_continuous_quantile_head=True, force_flip_invariance=True, infer_is_positive=True, fix_quantile_crossing=True, return_backcast=True).

timesfm.ForecastConfig — Configura os limites e opções de inferência do TimesFM. Argumentos usados: max_context: (context + 31) // 32 * 32; max_horizon: (horizon + 127) // 128 * 128; normalize_inputs: True; use_continuous_quantile_head: True; force_flip_invariance: True; infer_is_positive: True; fix_quantile_crossing: True; return_backcast: True.

```
0024              max_context=((context + 31) // 32) * 32,
```

Continua os argumentos nomeados: max_context = (context + 31) // 32 * 32. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0025              max_horizon=((horizon + 127) // 128) * 128,
```

Continua os argumentos nomeados: max_horizon = (horizon + 127) // 128 * 128. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0026              normalize_inputs=True, use_continuous_quantile_head=True,
```

Continua os argumentos nomeados: normalize_inputs = True; use_continuous_quantile_head = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0027              force_flip_invariance=True, infer_is_positive=True,
```

Continua os argumentos nomeados: force_flip_invariance = True; infer_is_positive = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0028              fix_quantile_crossing=True, return_backcast=True))
```

Continua os argumentos nomeados: fix_quantile_crossing = True; return_backcast = True. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0029          numerical, categorical = timesfm_covariates(history, future_covariates, self.dataset)
```

Atribui a (numerical, categorical) o resultado de timesfm_covariates(history, future_covariates, self.dataset). A expressão à direita é avaliada antes da atribuição.

timesfm_covariates — history/future: tabelas separadas; dataset: listas numéricas/categóricas. Retorna dois dicts com listas de séries cobrindo passado e futuro, sem demanda futura. Argumentos usados: posicional 1: history; posicional 2: future_covariates; posicional 3: self.dataset.

```
0030          self.last_audit = {'history_rows': context,
```

Atribui a self.last_audit o resultado de {'history_rows': context, 'history_start': str(history.timestamp.iloc[0]), 'history_end': str(history.timestamp.iloc[-1]), 'max_context': (context + 31) // 32 * 32, 'covariates': covariate_columns(self.dataset), 'future_rows': len(future_covariates), 'xreg_mode': 'xreg + timesfm'}. A expressão à direita é avaliada antes da atribuição.

```
0031                              'history_start': str(history.timestamp.iloc[0]),
```

Define 'history_start': primeira data histórica efetivamente fornecida; recebe str(history.timestamp.iloc[0]). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: history.timestamp.iloc[0].

```
0032                              'history_end': str(history.timestamp.iloc[-1]),
```

Define 'history_end': última data conhecida antes da origem; recebe str(history.timestamp.iloc[-1]). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: history.timestamp.iloc[-1].

```
0033                             'max_context': ((context+31)//32)*32,
```

Define 'max_context': campo do registro/coleção em construção; recebe (context + 31) // 32 * 32. Os valores são avaliados nesta janela/iteração.

```
0034                             'covariates': covariate_columns(self.dataset),
```

Define 'covariates': nomes das variáveis auxiliares; recebe covariate_columns(self.dataset). Os valores são avaliados nesta janela/iteração.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: self.dataset.

```
0035                             'future_rows': len(future_covariates), 'xreg_mode': 'xreg +
    timesfm'}
```

Define 'future_rows': campo do registro/coleção em construção; recebe len(future_covariates); 'xreg_mode': campo do registro/coleção em construção; recebe 'xreg + timesfm'. Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: future_covariates.

```
0036         point, _ = self.model.forecast_with_covariates(
```

Atribui a (point, _) o resultado de self.model.forecast_with_covariates(inputs=[history.water_produced.to_numpy(dtype=float)], dynamic_numerical_covariates=numerical, dynamic_categorical_covariates=categorical, xreg_mode='xreg + timesfm'). A expressão à direita é avaliada antes da atribuição.

self.model.forecast_with_covariates — Executa regressão auxiliar e TimesFM com variáveis exógenas. Argumentos usados: inputs: [history.water_produced.to_numpy(dtype=float)]; dynamic_numerical_covariates: numerical; dynamic_categorical_covariates: categorical; xreg_mode: 'xreg + timesfm'.

```
0037             inputs=[history.water_produced.to_numpy(dtype=float)],
```

Continua os argumentos nomeados: inputs = [history.water_produced.to_numpy(dtype=float)]; dtype = float. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

history.water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: dtype: float.

```
0038             dynamic_numerical_covariates=numerical,
```

Continua os argumentos nomeados: dynamic_numerical_covariates = numerical. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0039             dynamic_categorical_covariates=categorical,
```

Continua os argumentos nomeados: dynamic_categorical_covariates = categorical. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0040             xreg_mode='xreg + timesfm')
```

Continua os argumentos nomeados: xreg_mode = 'xreg + timesfm'. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0041         return np.asarray(point[0], dtype=float)[:horizon].tolist()
```

Encerra esta função e devolve np.asarray(point[0], dtype=float)[:horizon].tolist() ao chamador; não imprime o resultado por si só.

np.asarray(point[0], dtype=float)[:horizon].tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: point[0]; dtype: float.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

39. src/evaluation/metrics.py

Função numérica sem I/O. Recebe vetores reais e previstos já alinhados e retorna dicionário de métricas. É chamada pelos comparadores, testes e auditoria.

SHA-256: f4141db5d243c7b5e939064ef2f4c022545b6d99fc6846ce36a6aeee4875d2a2

Funções, contratos e pontos de conexão

```
L5: deterministic_metrics(ytrue, yhat) → dict
```

ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R² e contagem de zeros excluídos do MAPE.

APIs externas usadas neste arquivo

ValueError, all, any, float, int, np.abs, np.asarray, np.divide, np.isfinite, np.mean, np.sqrt, np.sum, np.zeros_like, sum. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Explicit metric definitions, with no independent dropping of true/predicted rows."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  def deterministic_metrics(ytrue, yhat) -> dict:
```

Define deterministic_metrics(ytrue, yhat) → dict. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0006      actual, predicted = np.asarray(ytrue, dtype=float), np.asarray(yhat, dtype=float)
```

Converte ambos os vetores para floats sem remover qualquer linha. Mesma posição deve representar a mesma data e lead.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: ytrue; dtype: float.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: yhat; dtype: float.

```
0007      if actual.shape != predicted.shape or actual.size == 0:
```

Avalia a condição actual.shape != predicted.shape or actual.size == 0. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0008          raise ValueError('Aligned nonempty targets and predictions are required')
```

Interrompe o caminho atual lançando ValueError('Aligned nonempty targets and predictions are required'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Aligned nonempty targets and predictions are required'.

```
0009      if not np.isfinite(actual).all() or not np.isfinite(predicted).all():
```

Avalia a condição not np.isfinite(actual).all() or not np.isfinite(predicted).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.isfinite(actual).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite(predicted).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: actual.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: predicted.

```
0010          raise ValueError('Nonfinite values are not silently removed')
```

Interrompe o caminho atual lançando ValueError('Nonfinite values are not silently removed'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Nonfinite values are not silently removed'.

```
0011      error = predicted - actual
```

Convenção: previsão menos observado. O sinal desaparece em MAE/RMSE, mas é útil para viés na auditoria.

```
0012      nonzero = actual != 0
```

Máscara apenas para MAPE; os zeros continuam incluídos em MAE/RMSE/sMAPE/R².

```
0013      denom = np.abs(actual) + np.abs(predicted)
```

Atribui a denom o resultado de np.abs(actual) + np.abs(predicted). A expressão à direita é avaliada antes da atribuição.

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: actual.

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: predicted.

```
0014      smape = np.divide(2 * np.abs(error), denom, out=np.zeros_like(error), where=denom != 0)
```

Numerador 2*abs(erro), denominador abs(real)+abs(previsto). Onde ambos são zero, out inicializado com zero conserva contribuição zero.

np.divide — Divide vetores com máscara e destino explícitos. Argumentos usados: posicional 1: 2 * np.abs(error); posicional 2: denom; out: np.zeros_like(error); where: denom != 0.

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: error.

np.zeros_like — Cria zeros com formato e tipo semelhantes ao array. Argumentos usados: posicional 1: error.

```
0015      total = float(np.sum((actual - np.mean(actual)) ** 2))
```

Soma dos quadrados em relação à média real; é denominador de R², não variância já dividida por N.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.sum((actual - np.mean(actual)) ** 2).

np.sum — Soma elementos. Argumentos usados: posicional 1: (actual - np.mean(actual)) ** 2.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: actual.

```
0016      return {'mae': float(np.mean(np.abs(error))), 'rmse': float(np.sqrt(np.mean(error **
    2))),
```

MAE é média do módulo; RMSE é raiz da média dos quadrados em TODOS os pares fornecidos. Não é média de RMSEs por janela.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.mean(np.abs(error)).

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.sqrt(np.mean(error ** 2)).

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: np.abs(error).

np.sqrt — Raiz quadrada elemento a elemento ou escalar. Argumentos usados: posicional 1: np.mean(error ** 2).

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: error.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: error ** 2.

```
0017              'mape_percent': float(np.mean(np.abs(error[nonzero] / actual[nonzero])) * 100) if
     nonzero.any() else None,
```

MAPE considera apenas alvos não zero, multiplica por 100 e retorna None se nenhum alvo é elegível.

nonzero.any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.mean(np.abs(error[nonzero] / actual[nonzero])) * 100.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: np.abs(error[nonzero] / actual[nonzero]).

np.abs — Módulo elemento a elemento. Argumentos usados: posicional 1: error[nonzero] / actual[nonzero].

```
0018             'smape_percent': float(np.mean(smape) * 100),
```

sMAPE é apresentado em porcentagem; nesta definição com fator dois pode chegar a 200%.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.mean(smape) * 100.

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: smape.

```
0019             'r2': 1 - float(np.sum(error ** 2)) / total if total else None,
```

$R^2 = 1 - SSE/SST$. Se os alvos forem constantes, total=0 e retorna None. Valores negativos são possíveis e não são erro do parser.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.sum(error ** 2).

np.sum — Soma elementos. Argumentos usados: posicional 1: error ** 2.

```
0020             'mape_zero_targets_excluded': int((~nonzero).sum())}
```

Conta zeros excluídos do MAPE para não esconder redução do conjunto dessa métrica.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: (~nonzero).sum().

(~nonzero).sum — Soma elementos de iterável. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

40. src/pipelines/compare_models.py

Compara execuções completas do benchmark Qwen/Chronos/TimesFM. Valida contextos fixos do Qwen e rotula corretamente sete ou quinze dias.

SHA-256: 3fa92592c861fda7882d86936543424733deb37258c1cbddc1cc7b89f01d7bf1

Funções, contratos e pontos de conexão

```
L10: compare(output: Path)
```

output contém benchmark completo de três modelos. Verifica mesmas origens/alvos/procedência, contexto Qwen7/15 e histórias crescentes dos baselines; grava métricas.

Chamadas do projeto: deterministic_metrics → src/evaluation/metrics.py:5; qwen_history_days → src/pipelines/forecasting_pipeline.py:44

```
L57: main()
```

Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada.

Chamadas do projeto: compare → src/pipelines/compare_models.py:10

APIs externas usadas neste arquivo

ValueError, add_argument, any, append, argparse.ArgumentParser, dict, drop_duplicates, duplicated, equals, float, groupby, int, join, json.loads, len, max, mean, min, nunique, parse_args, pd.DataFrame, pd.read_csv, print, range, read_text, reset_index, sort_values, sum, to_csv, to_string, tolist. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Refuse incomplete or mismatched comparisons; report aggregate and per-lead metrics."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import argparse
```

Importa argparse. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  from src.evaluation.metrics import deterministic_metrics
```

Importa deterministic_metrics de src.evaluation.metrics. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  from src.pipelines.forecasting_pipeline import ROOT, qwen_history_days
```

Importa ROOT, qwen_history_days de src.pipelines.forecasting_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  def compare(output: Path):
```

Define compare(output: Path). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0011      summaries, per_lead = [], []
```

Atribui a (summaries, per_lead) o resultado de ([], []). A expressão à direita é avaliada antes da atribuição.

```
0012      for dataset in ('joinville', 'guaratuba'):
```

Percorre ('joinville', 'guaratuba'); em cada iteração associa o item a dataset. O corpo indentado aplica a operação a esse item/grupo.

```
0013          reference = None
```

Atribui a reference o resultado de None. A expressão à direita é avaliada antes da atribuição.

```
0014          reference_run = None
```

Atribui a reference_run o resultado de None. A expressão à direita é avaliada antes da atribuição.

```
0015          for model in ('qwen', 'chronos', 'timesfm'):
```

Percorre ('qwen', 'chronos', 'timesfm'); em cada iteração associa o item a model. O corpo indentado aplica a operação a esse item/grupo.

```
0016              folder = output / dataset / model
```

Atribui a folder o resultado de output / dataset / model. A expressão à direita é avaliada antes da atribuição.

```
0017              status = json.loads((folder / 'status.json').read_text())
```

Atribui a status o resultado de json.loads((folder / 'status.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'status.json').read_text().

(folder / 'status.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0018              run = json.loads((folder / 'run.json').read_text())
```

Atribui a run o resultado de json.loads((folder / 'run.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'run.json').read_text().

(folder / 'run.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0019              if status['failures'] or status['successful_windows'] !=
    status['expected_windows'] or run['limit']:
```

Avalia a condição status['failures'] or status['successful_windows'] != status['expected_windows'] or run['limit']. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0020                  raise ValueError(f'Incomplete benchmark: {dataset}/{model}')
```

Interrompe o caminho atual lançando ValueError(f'Incomplete benchmark: {dataset}/{model}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Incomplete benchmark: {dataset}/{model}'.

```
0021              frame = pd.read_csv(folder / 'predictions.csv').sort_values(['origin',
    'lead']).reset_index(drop=True)
```

Atribui a frame o resultado de pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index(drop=True). A expressão à direita é avaliada antes da atribuição.

pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

pd.read_csv(folder / 'predictions.csv').sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: ['origin', 'lead'].

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: folder / 'predictions.csv'.

```
0022              keys = frame[['origin', 'timestamp', 'lead', 'y_true', 'initial_context',
    'horizon']]
```

Atribui a keys o resultado de frame[['origin', 'timestamp', 'lead', 'y_true', 'initial_context', 'horizon']]. A expressão à direita é avaliada antes da atribuição.

```
0023             origins = frame.drop_duplicates('origin')
```

Atribui a origins o resultado de frame.drop_duplicates('origin'). A expressão à direita é avaliada antes da atribuição.

frame.drop_duplicates — Retém uma linha por chave repetida. Argumentos usados: posicional 1: 'origin'.

```
0024             initial = int(frame.initial_context.iloc[0])
```

Atribui a initial o resultado de int(frame.initial_context.iloc[0]). A expressão à direita é avaliada antes da atribuição.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.initial_context.iloc[0].

```
0025             expected_contexts = ([qwen_history_days(run)] * len(origins) if model == 'qwen'
```

Atribui a expected_contexts o resultado de [qwen_history_days(run)] * len(origins) if model == 'qwen' else [initial + 7 * i for i in range(len(origins))]. A expressão à direita é avaliada antes da atribuição.

len — Conta elementos. Argumentos usados: posicional 1: origins.

qwen_history_days — config precisa conter inteiro exato 7 ou 15. Ausência, None, strings, bool, floats e outros comprimentos são rejeitados antes de carregar dados/modelo. Argumentos usados: posicional 1: run.

```
0026                                 else [initial + 7*i for i in range(len(origins))])
```

Continuação da instrução na linha 25: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: len(origins).

len — Conta elementos. Argumentos usados: posicional 1: origins.

```
0027             if origins.context.tolist() != expected_contexts:
```

Avalia a condição origins.context.tolist() != expected_contexts. O bloco indentado seguinte só é executado quando ela é verdadeira.

origins.context.tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

```
0028                 raise ValueError(f'Incorrect history lengths: {dataset}/{model}')
```

Interrompe o caminho atual lançando ValueError(f'Incorrect history lengths: {dataset}/{model}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Incorrect history lengths: {dataset}/{model}'.

```
0029             if model != 'qwen' and origins.history_start.nunique() != 1:
```

Avalia a condição model != 'qwen' and origins.history_start.nunique() != 1. O bloco indentado seguinte só é executado quando ela é verdadeira.

origins.history_start.nunique — Conta valores distintos. Argumentos usados: nenhum argumento explícito.

```
0030                 raise ValueError('Expanding history must always start on the first date')
```

Interrompe o caminho atual lançando ValueError('Expanding history must always start on the first date'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Expanding history must always start on the first date'.

```
0031             if frame.duplicated(['origin', 'lead']).any() or len(frame) !=
    status['expected_windows'] * 7:
```

Avalia a condição frame.duplicated(['origin', 'lead']).any() or len(frame) != status['expected_windows'] * 7. O bloco indentado seguinte só é executado quando ela é verdadeira.

frame.duplicated(['origin', 'lead']).any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

len — Conta elementos. Argumentos usados: posicional 1: frame.

frame.duplicated — Marca repetições. Argumentos usados: posicional 1: ['origin', 'lead'].

```
0032                 raise ValueError('Duplicate or missing forecasts')
```

Interrompe o caminho atual lançando ValueError('Duplicate or missing forecasts'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Duplicate or missing forecasts'.

```
0033             comparable = {key: run[key] for key in ('config', 'dataset_sha256',
    'source_sha256', 'n_windows')}
```

Atribui a comparable o resultado de {key: run[key] for key in ('config', 'dataset_sha256', 'source_sha256', 'n_windows')}. A expressão à direita é avaliada antes da atribuição.

```
0034             if reference is not None and (not keys.equals(reference) or comparable !=
    reference_run):
```

Avalia a condição reference is not None and (not keys.equals(reference) or comparable != reference_run). O bloco indentado seguinte só é executado quando ela é verdadeira.

keys.equals — Compara conteúdo e estrutura do receptor com outro objeto pandas. Argumentos usados: posicional 1: reference.

```
0035                 raise ValueError(f'Incomparable evaluation for {dataset}/{model}')
```

Interrompe o caminho atual lançando ValueError(f'Incomparable evaluation for {dataset}/{model}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Incomparable evaluation for {dataset}/{model}'.

```
0036             reference, reference_run = keys, comparable
```

Atribui a (reference, reference_run) o resultado de (keys, comparable). A expressão à direita é avaliada antes da atribuição.

```
0037             common = {'dataset': dataset, 'model': model, 'model_id': run['model']['model'],
```

Atribui a common o resultado de {'dataset': dataset, 'model': model, 'model_id': run['model']['model'], 'initial_context_days': int(frame.initial_context.iloc[0]), 'context_days_first': int(frame.context.iloc[0]), 'context_days_last': int(frame.context.iloc[-1]), 'history_policy': f'last {qwen_history_days(run)} days' if model == 'qwen' else 'expanding', 'covariates': '|'.join(run['covariates']), 'horizon_days': 7, 'step_days': 7, 'strategy': run['config']['strategy'] if model == 'qwen' else 'pretrained', 'n_windows': status['expected_windows'], 'n_predictions': len(frame), 'test_start': frame.timestamp.min(), 'test_end': frame.timestamp.max()}. A expressão à direita é avaliada antes da atribuição.

```
0038                     'initial_context_days': int(frame.initial_context.iloc[0]),
```

Define 'initial_context_days': campo do registro/coleção em construção; recebe int(frame.initial_context.iloc[0]). Os valores são avaliados nesta janela/iteração.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.initial_context.iloc[0].

```
0039                     'context_days_first': int(frame.context.iloc[0]),
```

Define 'context_days_first': campo do registro/coleção em construção; recebe int(frame.context.iloc[0]). Os valores são avaliados nesta janela/iteração.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.context.iloc[0].

```
0040                     'context_days_last': int(frame.context.iloc[-1]),
```

Define 'context_days_last': campo do registro/coleção em construção; recebe int(frame.context.iloc[-1]). Os valores são avaliados nesta janela/iteração.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.context.iloc[-1].

```
0041                     'history_policy': f'last {qwen_history_days(run)} days' if model ==
    'qwen' else 'expanding',
```

Define 'history_policy': campo do registro/coleção em construção; recebe f'last {qwen_history_days(run)} days' if model == 'qwen' else 'expanding'. Os valores são avaliados nesta janela/iteração.

qwen_history_days — config precisa conter inteiro exato 7 ou 15. Ausência, None, strings, bool, floats e outros comprimentos são rejeitados antes de carregar dados/modelo. Argumentos usados: posicional 1: run.

```
0042                        'covariates': '|'.join(run['covariates']), 'horizon_days': 7,
    'step_days': 7,
```

Define 'covariates': nomes das variáveis auxiliares; recebe '|'.join(run['covariates']); 'horizon_days': campo do registro/coleção em construção; recebe 7; 'step_days': campo do registro/coleção em construção; recebe 7. Os valores são avaliados nesta janela/iteração.

'|'.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: run['covariates'].

```
0043                        'strategy': run['config']['strategy'] if model == 'qwen' else
    'pretrained',
```

Define 'strategy': campo do registro/coleção em construção; recebe run['config']['strategy'] if model == 'qwen' else 'pretrained'. Os valores são avaliados nesta janela/iteração.

```
0044                        'n_windows': status['expected_windows'], 'n_predictions': len(frame),
```

Define 'n_windows': campo do registro/coleção em construção; recebe status['expected_windows']; 'n_predictions': campo do registro/coleção em construção; recebe len(frame). Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: frame.

```
0045                        'test_start': frame.timestamp.min(), 'test_end': frame.timestamp.max()}
```

Define 'test_start': campo do registro/coleção em construção; recebe frame.timestamp.min(); 'test_end': campo do registro/coleção em construção; recebe frame.timestamp.max(). Os valores são avaliados nesta janela/iteração.

frame.timestamp.min — Retorna menor valor. Argumentos usados: nenhum argumento explícito.

frame.timestamp.max — Retorna maior valor. Argumentos usados: nenhum argumento explícito.

```
0046              timing = frame.drop_duplicates('origin').elapsed_seconds_window
```

Atribui a timing o resultado de frame.drop_duplicates('origin').elapsed_seconds_window. A expressão à direita é avaliada antes da atribuição.

frame.drop_duplicates — Retém uma linha por chave repetida. Argumentos usados: posicional 1: 'origin'.

```
0047              summaries.append(dict(common, **deterministic_metrics(frame.y_true,
    frame.y_pred),
```

Executa summaries.append(dict(common, **deterministic_metrics(frame.y_true, frame.y_pred), inference_seconds=float(timing.sum()), mean_window_seconds=float(timing.mean()))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

summaries.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(common, **deterministic_metrics(frame.y_true, frame.y_pred), inference_seconds=float(timing.sum()), mean_window_seconds=float(timing.mean())).

dict — Cria dicionário. Argumentos usados: posicional 1: common; **desempacotamento: deterministic_metrics(frame.y_true, frame.y_pred); inference_seconds: float(timing.sum()); mean_window_seconds: float(timing.mean()).

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R^2 e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: frame.y_true; posicional 2: frame.y_pred.

```
0048                                    inference_seconds=float(timing.sum()),
    mean_window_seconds=float(timing.mean())))
```

Continua os argumentos nomeados: inference_seconds = float(timing.sum()); mean_window_seconds = float(timing.mean()). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

float — Converte escalar para float Python. Argumentos usados: posicional 1: timing.sum().

float — Converte escalar para float Python. Argumentos usados: posicional 1: timing.mean().

timing.sum — Soma elementos de iterável. Argumentos usados: nenhum argumento explícito.

timing.mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

```
0049              for lead, subset in frame.groupby('lead'):
```

Percorre frame.groupby('lead'); em cada iteração associa o item a (lead, subset). O corpo indentado aplica a operação a esse item/grupo.

frame.groupby — Cria grupos sem calcular métricas por si só. Argumentos usados: posicional 1: 'lead'.

```
0050                  per_lead.append(dict(dataset=dataset, model=model, lead=int(lead),
```

Executa per_lead.append(dict(dataset=dataset, model=model, lead=int(lead), **deterministic_metrics(subset.y_true, subset.y_pred))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

per_lead.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(dataset=dataset, model=model, lead=int(lead), **deterministic_metrics(subset.y_true, subset.y_pred)).

dict — Cria dicionário. Argumentos usados: dataset: dataset; model: model; lead: int(lead); **desempacotamento: deterministic_metrics(subset.y_true, subset.y_pred).

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: lead.

```
0051                                      **deterministic_metrics(subset.y_true, subset.y_pred)))
```

Continua os argumentos nomeados: ** = deterministic_metrics(subset.y_true, subset.y_pred). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R² e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: subset.y_true; posicional 2: subset.y_pred.

```
0052      pd.DataFrame(summaries).to_csv(output / 'metrics_comparison.csv', index=False)
```

Executa pd.DataFrame(summaries).to_csv(output / 'metrics_comparison.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(summaries).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: output / 'metrics_comparison.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: summaries.

```
0053      pd.DataFrame(per_lead).to_csv(output / 'metrics_by_horizon.csv', index=False)
```

Executa pd.DataFrame(per_lead).to_csv(output / 'metrics_by_horizon.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(per_lead).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: output / 'metrics_by_horizon.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: per_lead.

```
0054      print(pd.DataFrame(summaries)[['dataset', 'model', 'mae', 'rmse', 'mape_percent',
    'r2']].to_string(index=False))
```

Executa print(pd.DataFrame(summaries)[['dataset', 'model', 'mae', 'rmse', 'mape_percent', 'r2']].to_string(index=False)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: pd.DataFrame(summaries)[['dataset', 'model', 'mae', 'rmse', 'mape_percent', 'r2']].to_string(index=False).

pd.DataFrame(summaries)[['dataset', 'model', 'mae', 'rmse', 'mape_percent', 'r2']].to_string — Formata tabela como texto para terminal/log. Argumentos usados: index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: summaries.

```
0057  def main():
```

Define main(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0058      parser = argparse.ArgumentParser(description=__doc__)
```

Atribui a parser o resultado de argparse.ArgumentParser(description=__doc__). A expressão à direita é avaliada antes da atribuição.

argparse.ArgumentParser — Cria o parser da interface de linha de comando. Argumentos usados: description: __doc__.

```
0059      parser.add_argument('--output', type=Path, default=ROOT / 'experiments/benchmark')
```

Executa parser.add_argument('--output', type=Path, default=ROOT / 'experiments/benchmark'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--output'; type: Path; default: ROOT / 'experiments/benchmark'.

```
0060      compare(parser.parse_args().output)
```

Executa compare(parser.parse_args().output). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

compare — output contém benchmark completo de três modelos. Verifica mesmas origens/alvos/procedência, contexto Qwen7/15 e histórias crescentes dos baselines; grava métricas. Argumentos usados: posicional 1: parser.parse_args().output.

parser.parse_args — Interpreta argumentos do processo e retorna Namespace. Argumentos usados: nenhum argumento explícito.

```
0063  if __name__ == '__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0064      main()
```

Executa main(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

main — Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

41. src/training/train.py

Ponto auxiliar para uma extensão de treinamento: oferece slices cronológicos treino/validação. Não é chamado pela inferência atual nem treina pesos.

SHA-256: bdff53809e5067f833bbec7698ea9b0be09af475a4f6a893f5465c905e04df30

Funções, contratos e pontos de conexão

```
L5: training_partition(first_test_origin: int, validation_fraction: float = 0.2)
```

first_test_origin: tamanho de prefixo de treino; validation_fraction=0.2: proporção final de validação. Retorna os slices de validation_split; não faz fit.

Chamadas do projeto: validation_split → src/evaluation/cross_validation.py:32

Código comentado, linha a linha

```
0001  """Explicit experiment policy: pretrained inference only, with no fit on test data."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  from src.evaluation.cross_validation import validation_split
```

Importa validation_split de src.evaluation.cross_validation. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  def training_partition(first_test_origin: int, validation_fraction: float = 0.2):
```

Define training_partition(first_test_origin: int, validation_fraction: float = 0.2). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0006      """Provide chronological train/validation slices for a future training extension.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0008      This benchmark does not optimize weights or hyperparameters. Callers must
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0009      slice the dataset at first_test_origin before fitting any future estimator.
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0010      """
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0011      return validation_split(first_test_origin, validation_fraction)
```

Encerra esta função e devolve validation_split(first_test_origin, validation_fraction) ao chamador; não imprime o resultado por si só.

validation_split — training_size: comprimento do prefixo anterior ao teste; fraction=0.2: parte final reservada à validação. Retorna dois slices sem embaralhar. Argumentos usados: posicional 1: first_test_origin; posicional 2: validation_fraction.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

42. src/visualization/qwen_text_results.py

Produz seis gráficos das oito avaliações textuais concluídas: quatro pares observado/previsto e duas comparações de RMSE. Usa todos os sete leads.

SHA-256: 5c074dd1a15fbabbed4089ea68fec38c19ea51ff17348fee975205f092df9f79

Funções, contratos e pontos de conexão

```
L11: main()
```

Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada.

Chamadas do projeto: summarize → src/pipelines/qwen_text_experiment.py:49

APIs externas usadas neste arquivo

bar, bar_label, copy, grid, itertuples, legend, matplotlib.use, max, mdates.DateFormatter, mkdir, pd.read_csv, plot, plt.close, plt.rcParams.update, plt.subplots, print, query, savefig, set_major_formatter, set_title, set_ylabel, set_ylim, suptitle, title, zip. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Plot all seven forecast leads from completed text-dataset Qwen evaluations."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import matplotlib
```

Importa matplotlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  matplotlib.use('Agg')
```

Executa matplotlib.use('Agg'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

matplotlib.use — Seleciona backend de renderização. Argumentos usados: posicional 1: 'Agg'.

```
0004  import matplotlib.dates as mdates
```

Importa matplotlib.dates como mdates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import matplotlib.pyplot as plt
```

Importa matplotlib.pyplot como plt. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  from src.pipelines.qwen_text_experiment import OUTPUT, RUNS, summarize
```

Importa OUTPUT, RUNS, summarize de src.pipelines.qwen_text_experiment. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  def main():
```

Define main(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0012      summarize()
```

Executa summarize(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

summarize — Exige as oito avaliações completas, contextos corretos, sete leads e alvos iguais. Escreve CSVs simples por cidade/execução, métricas agregadas e por lead. Não altera previsões. Argumentos usados: nenhum argumento explícito.

```
0013      destination = OUTPUT / 'plots'
```

Atribui a destination o resultado de OUTPUT / 'plots'. A expressão à direita é avaliada antes da atribuição.

```
0014      destination.mkdir(exist_ok=True)
```

Executa destination.mkdir(exist_ok=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

destination.mkdir — Cria diretório. Argumentos usados: exist_ok: True.

```
0015      metrics = pd.read_csv(OUTPUT / 'metrics.csv')
```

Atribui a metrics o resultado de pd.read_csv(OUTPUT / 'metrics.csv'). A expressão à direita é avaliada antes da atribuição.

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: OUTPUT / 'metrics.csv'.

```
0016      plt.rcParams.update({'font.size': 12, 'axes.spines.top': False, 'axes.spines.right':
    False})
```

Executa plt.rcParams.update({'font.size': 12, 'axes.spines.top': False, 'axes.spines.right': False}). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

plt.rcParams.update — Define padrões visuais da sessão Matplotlib. Argumentos usados: posicional 1: {'font.size': 12, 'axes.spines.top': False, 'axes.spines.right': False}.

```
0017      for city in ('joinville', 'guaratuba'):
```

Percorre ('joinville', 'guaratuba'); em cada iteração associa o item a city. O corpo indentado aplica a operação a esse item/grupo.

```
0018          for slug in ('1_7b', '4b'):
```

Percorre ('1_7b', '4b'); em cada iteração associa o item a slug. O corpo indentado aplica a operação a esse item/grupo.

```
0019              label = 'Qwen3 1.7B' if slug == '1_7b' else 'Qwen3 4B'
```

Atribui a label o resultado de 'Qwen3 1.7B' if slug == '1_7b' else 'Qwen3 4B'. A expressão à direita é avaliada antes da atribuição.

```
0020              fig, axes = plt.subplots(2, 1, figsize=(16, 8), sharex=True,
    constrained_layout=True)
```

Atribui a (fig, axes) o resultado de plt.subplots(2, 1, figsize=(16, 8), sharex=True, constrained_layout=True). A expressão à direita é avaliada antes da atribuição.

plt.subplots — Cria figura e eixos Matplotlib. Argumentos usados: posicional 1: 2; posicional 2: 1; figsize: (16, 8); sharex: True; constrained_layout: True.

```
0021              for ax, days in zip(axes, (7, 15)):
```

Percorre zip(axes, (7, 15)); em cada iteração associa o item a (ax, days). O corpo indentado aplica a operação a esse item/grupo.

zip — Emparelha iteráveis pela posição. Argumentos usados: posicional 1: axes; posicional 2: (7, 15).

```
0022                  frame = pd.read_csv(OUTPUT / f'{slug}_{days}' / city /
    'qwen/predictions.csv', parse_dates=['timestamp'])
```

Atribui a frame o resultado de pd.read_csv(OUTPUT / f'{slug}_{days}' / city / 'qwen/predictions.csv', parse_dates=['timestamp']). A expressão à direita é avaliada antes da atribuição.

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: OUTPUT / f'{slug}_{days}' / city / 'qwen/predictions.csv'; parse_dates: ['timestamp'].

```
0023                  score = metrics.query('dataset == @city and slug == @slug and context ==
    @days').iloc[0]
```

Atribui a score o resultado de metrics.query('dataset == @city and slug == @slug and context == @days').iloc[0]. A expressão à direita é avaliada antes da atribuição.

metrics.query — Filtra linhas por expressão pandas. Argumentos usados: posicional 1: 'dataset == @city and slug == @slug and context == @days'.

```
0024                 ax.plot(frame.timestamp, frame.y_true, color='#818181', linewidth=1.3,
    label='Real')
```

Executa ax.plot(frame.timestamp, frame.y_true, color='#818181', linewidth=1.3, label='Real'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.plot — Desenha sequência de pontos/linhas no eixo. Argumentos usados: posicional 1: frame.timestamp; posicional 2: frame.y_true; color: '#818181'; linewidth: 1.3; label: 'Real'.

```
0025                 ax.plot(frame.timestamp, frame.y_pred, color='#2478b5', linewidth=1.2,
    label='Previsto')
```

Executa ax.plot(frame.timestamp, frame.y_pred, color='#2478b5', linewidth=1.2, label='Previsto'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.plot — Desenha sequência de pontos/linhas no eixo. Argumentos usados: posicional 1: frame.timestamp; posicional 2: frame.y_pred; color: '#2478b5'; linewidth: 1.2; label: 'Previsto'.

```
0026                 ax.set_title(f'Contexto: {days} dias | RMSE: {score.rmse:,.2f} | R²:
    {score.r2:.3f}', loc='left')
```

Executa ax.set_title(f'Contexto: {days} dias | RMSE: {score.rmse:,.2f} | R²: {score.r2:.3f}', loc='left'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.set_title — Define título de eixo. Argumentos usados: posicional 1: f'Contexto: {days} dias | RMSE: {score.rmse:,.2f} | R²: {score.r2:.3f}'; loc: 'left'.

```
0027                 ax.set_ylabel('Produção de água (m³)')
```

Executa ax.set_ylabel('Produção de água (m³)'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.set_ylabel — Define unidade/rótulo vertical. Argumentos usados: posicional 1: 'Produção de água (m³)'.

```
0028                 ax.grid(alpha=.2)
```

Executa ax.grid(alpha=0.2). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.grid — Exibe linhas de referência. Argumentos usados: alpha: 0.2.

```
0029                 ax.legend(loc='upper right', ncol=2)
```

Executa ax.legend(loc='upper right', ncol=2). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.legend — Mostra identificação das séries. Argumentos usados: loc: 'upper right'; ncol: 2.

```
0030             axes[-1].xaxis.set_major_formatter(mdates.DateFormatter('%m/%Y'))
```

Executa axes[-1].xaxis.set_major_formatter(mdates.DateFormatter('%m/%Y')). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

axes[-1].xaxis.set_major_formatter — Define formatação dos rótulos principais. Argumentos usados: posicional 1: mdates.DateFormatter('%m/%Y').

mdates.DateFormatter — Formata rótulos temporais do eixo. Argumentos usados: posicional 1: '%m/%Y'.

```
0031             fig.suptitle(f'{city.title()} — {label} | Dataset textual e prompts em linguagem
    natural', fontsize=18)
```

Executa fig.suptitle(f'{city.title()} — {label} | Dataset textual e prompts em linguagem natural', fontsize=18). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

fig.suptitle — Define título geral da figura. Argumentos usados: posicional 1: f'{city.title()} — {label} | Dataset textual e prompts em linguagem natural'; fontsize: 18.

city.title — Converte texto para capitalização de título. Argumentos usados: nenhum argumento explícito.

```
0032             fig.savefig(destination / f'{city}_{slug}.png', dpi=150)
```

Executa fig.savefig(destination / f'{city}_{slug}.png', dpi=150). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

fig.savefig — Exporta uma figura para arquivo. Argumentos usados: posicional 1: destination / f'{city}_{slug}.png'; dpi: 150.

```
0033             plt.close(fig)
```

Executa plt.close(fig). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

plt.close — Libera recursos da figura. Argumentos usados: posicional 1: fig.

```
0034         score = metrics.query('dataset == @city').copy()
```

Atribui a score o resultado de metrics.query('dataset == @city').copy(). A expressão à direita é avaliada antes da atribuição.

metrics.query('dataset == @city').copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

metrics.query — Filtra linhas por expressão pandas. Argumentos usados: posicional 1: 'dataset == @city'.

```
0035         labels = [f'{row.model}\n{row.context} dias' for row in score.itertuples()]
```

Atribui a labels o resultado de [f'{row.model}\n{row.context} dias' for row in score.itertuples()]. A expressão à direita é avaliada antes da atribuição.

score.itertuples — Itera linhas como tuplas nomeadas. Argumentos usados: nenhum argumento explícito.

```
0036         fig, ax = plt.subplots(figsize=(16, 8), constrained_layout=True)
```

Atribui a (fig, ax) o resultado de plt.subplots(figsize=(16, 8), constrained_layout=True). A expressão à direita é avaliada antes da atribuição.

plt.subplots — Cria figura e eixos Matplotlib. Argumentos usados: figsize: (16, 8); constrained_layout: True.

```
0037         bars = ax.bar(labels, score.rmse, color=['#2478b5', '#75b5df', '#db8438', '#f3bd89'])
```

Atribui a bars o resultado de ax.bar(labels, score.rmse, color=['#2478b5', '#75b5df', '#db8438', '#f3bd89']). A expressão à direita é avaliada antes da atribuição.

ax.bar — Desenha barras de comparação. Argumentos usados: posicional 1: labels; posicional 2: score.rmse; color: ['#2478b5', '#75b5df', '#db8438', '#f3bd89'].

```
0038         ax.bar_label(bars, labels=[f'{value:,.2f}' for value in score.rmse], padding=6,
    fontsize=15)
```

Executa ax.bar_label(bars, labels=[f'{value:,.2f}' for value in score.rmse], padding=6, fontsize=15). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.bar_label — Escreve valores sobre as barras. Argumentos usados: posicional 1: bars; labels: [f'{value:,.2f}' for value in score.rmse]; padding: 6; fontsize: 15.

```
0039         ax.set_ylim(0, score.rmse.max()*1.2)
```

Executa ax.set_ylim(0, score.rmse.max() * 1.2). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.set_ylim — Define limites verticais do eixo. Argumentos usados: posicional 1: 0; posicional 2: score.rmse.max() * 1.2.

score.rmse.max — Retorna maior valor. Argumentos usados: nenhum argumento explícito.

```
0040         ax.set_ylabel('RMSE (m³) — menor é melhor')
```

Executa ax.set_ylabel('RMSE (m³) — menor é melhor'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.set_ylabel — Define unidade/rótulo vertical. Argumentos usados: posicional 1: 'RMSE (m³) — menor é melhor'.

```
0041         ax.set_title(f'{city.title()} — Qwen | Comparação dos contextos fixos', fontsize=21)
```

Executa ax.set_title(f'{city.title()} — Qwen | Comparação dos contextos fixos', fontsize=21). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.set_title — Define título de eixo. Argumentos usados: posicional 1: f'{city.title()} — Qwen | Comparação dos contextos fixos'; fontsize: 21.

city.title — Converte texto para capitalização de título. Argumentos usados: nenhum argumento explícito.

```
0042          ax.grid(axis='y', alpha=.2)
```

Executa ax.grid(axis='y', alpha=0.2). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.grid — Exibe linhas de referência. Argumentos usados: axis: 'y'; alpha: 0.2.

```
0043          fig.savefig(destination / f'{city}_rmse.png', dpi=150)
```

Executa fig.savefig(destination / f'{city}_rmse.png', dpi=150). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

fig.savefig — Exporta uma figura para arquivo. Argumentos usados: posicional 1: destination / f'{city}_rmse.png'; dpi: 150.

```
0044          plt.close(fig)
```

Executa plt.close(fig). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

plt.close — Libera recursos da figura. Argumentos usados: posicional 1: fig.

```
0045      print(f'Saved six figures in {destination}')
```

Executa print(f'Saved six figures in {destination}'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: f'Saved six figures in {destination}'.

```
0048  if __name__ == '__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0049      main()
```

Executa main(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

main — Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

43. src/visualization/forecast_comparison.py

Renderiza os resultados históricos preservados dos baselines e dos Qwen de quinze dias. É distinto dos novos gráficos do experimento textual.

SHA-256: 04f9333ebc4efdf4a972649787b83eb94b33936095fca760c3f64552a5c433ba

Funções, contratos e pontos de conexão

```
L20: main()
```

Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada.

Chamadas do projeto: plots.linear_graph → src/visualization/plots.py:24

APIs externas usadas neste arquivo

Path, ValueError, all, any, append, apply, dict, duplicated, groupby, hashlib.sha256, hexdigest, int, json.loads, len, list, matplotlib.use, mkdir, pd.DataFrame, pd.read_csv, pd.testing.assert_frame_equal, print, range, read_bytes, read_text, relative_to, rename, reset_index, resolve, sort_values, sorted, str, title, to_csv. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Render completed forecasts using the visualization style copied from Siaqua."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import hashlib
```

Importa hashlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import matplotlib
```

Importa matplotlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  matplotlib.use('Agg')
```

Executa matplotlib.use('Agg'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

matplotlib.use — Seleciona backend de renderização. Argumentos usados: posicional 1: 'Agg'.

```
0007  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  from . import plots
```

Importa plots de .. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  ROOT = Path(__file__).resolve().parents[2]
```

Atribui a ROOT o resultado de Path(__file__).resolve().parents[2]. A expressão à direita é avaliada antes da atribuição.

Path(__file__).resolve — Resolve caminho absoluto e links simbólicos. Argumentos usados: nenhum argumento explícito.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: __file__.

```
0011  OUTPUT = ROOT / 'experiments/forecast_plots'
```

Atribui a OUTPUT o resultado de ROOT / 'experiments/forecast_plots'. A expressão à direita é avaliada antes da atribuição.

```
0012  MODELS = [
```

Atribui a MODELS o resultado de [('chronos2', 'Chronos2', 'experiments/benchmark', 'chronos', 'expansivo'), ('timesfm25', 'TimesFM2.5', 'experiments/benchmark', 'timesfm', 'expansivo'), ('qwen17b_15', 'Qwen3 1.7B', 'experiments/benchmark', 'qwen', '15 dias'), ('qwen4b_15', 'Qwen3 4B', 'experiments/qwen_15_days', 'qwen', '15 dias')]. A expressão à direita é avaliada antes da atribuição.

```
0013      ('chronos2', 'Chronos2', 'experiments/benchmark', 'chronos', 'expansivo'),
```

Continuação da instrução na linha 12: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0014      ('timesfm25', 'TimesFM2.5', 'experiments/benchmark', 'timesfm', 'expansivo'),
```

Continuação da instrução na linha 12: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0015      ('qwen17b_15', 'Qwen3 1.7B', 'experiments/benchmark', 'qwen', '15 dias'),
```

Continuação da instrução na linha 12: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0016      ('qwen4b_15', 'Qwen3 4B', 'experiments/qwen_15_days', 'qwen', '15 dias'),
```

Continuação da instrução na linha 12: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0017  ]
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0020  def main():
```

Define main(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0021      OUTPUT.mkdir(parents=True, exist_ok=True)
```

Executa OUTPUT.mkdir(parents=True, exist_ok=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

OUTPUT.mkdir — Cria diretório. Argumentos usados: parents: True; exist_ok: True.

```
0022      # Reuse the original saver with a destination inside this experiment.
```

Comentário de implementação: Reuse the original saver with a destination inside this experiment.. Não é executado; as operações efetivas são as linhas seguintes.

```
0023      plots.OUTPUT_MODELS_FOLDER = OUTPUT
```

Atribui a plots.OUTPUT_MODELS_FOLDER o resultado de OUTPUT. A expressão à direita é avaliada antes da atribuição.

```
0024      records = []
```

Atribui a records o resultado de []. A expressão à direita é avaliada antes da atribuição.

```
0025      for dataset in ('joinville', 'guaratuba'):
```

Percorre ('joinville', 'guaratuba'); em cada iteração associa o item a dataset. O corpo indentado aplica a operação a esse item/grupo.

```
0026          reference = None
```

Atribui a reference o resultado de None. A expressão à direita é avaliada antes da atribuição.

```
0027         for slug, label, root, model, policy in MODELS:
```

Percorre MODELS; em cada iteração associa o item a (slug, label, root, model, policy). O corpo indentado aplica a operação a esse item/grupo.

```
0028             source = ROOT / root / dataset / model
```

Atribui a source o resultado de ROOT / root / dataset / model. A expressão à direita é avaliada antes da atribuição.

```
0029             status = json.loads((source / 'status.json').read_text())
```

Atribui a status o resultado de json.loads((source / 'status.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (source / 'status.json').read_text().

(source / 'status.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0030             if status['failures'] or status['successful_windows'] !=
    status['expected_windows']:
```

Avalia a condição status['failures'] or status['successful_windows'] != status['expected_windows']. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0031                 raise ValueError(f'Incomplete predictions: {source}')
```

Interrompe o caminho atual lançando ValueError(f'Incomplete predictions: {source}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Incomplete predictions: {source}'.

```
0032             csv = source / 'predictions.csv'
```

Atribui a csv o resultado de source / 'predictions.csv'. A expressão à direita é avaliada antes da atribuição.

```
0033             frame = pd.read_csv(csv).sort_values(['timestamp',
    'lead']).reset_index(drop=True)
```

Atribui a frame o resultado de pd.read_csv(csv).sort_values(['timestamp', 'lead']).reset_index(drop=True). A expressão à direita é avaliada antes da atribuição.

pd.read_csv(csv).sort_values(['timestamp', 'lead']).reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

pd.read_csv(csv).sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: ['timestamp', 'lead'].

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: csv.

```
0034             if frame.timestamp.duplicated().any() or len(frame) != status['expected_windows']
     * 7:
```

Avalia a condição frame.timestamp.duplicated().any() or len(frame) != status['expected_windows'] * 7. O bloco indentado seguinte só é executado quando ela é verdadeira.

frame.timestamp.duplicated().any — Reduz a existência de pelo menos um True. Argumentos usados: nenhum argumento explícito.

len — Conta elementos. Argumentos usados: posicional 1: frame.

frame.timestamp.duplicated — Marca repetições. Argumentos usados: nenhum argumento explícito.

```
0035                 raise ValueError('Duplicate dates or incomplete forecast horizon')
```

Interrompe o caminho atual lançando ValueError('Duplicate dates or incomplete forecast horizon'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Duplicate dates or incomplete forecast horizon'.

```
0036             if not frame.groupby('origin').lead.apply(lambda x: sorted(x) ==
    list(range(1,8))).all():
```

Avalia a condição not frame.groupby('origin').lead.apply(lambda x: sorted(x) == list(range(1, 8))).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

frame.groupby('origin').lead.apply(lambda x: sorted(x) == list(range(1, 8))).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

frame.groupby('origin').lead.apply — Aplica uma função por grupo/objeto. Argumentos usados: posicional 1: lambda x: sorted(x) == list(range(1, 8)).

frame.groupby — Cria grupos sem calcular métricas por si só. Argumentos usados: posicional 1: 'origin'.

sorted — Devolve uma lista ordenada. Argumentos usados: posicional 1: x.

list — Materializa um iterável em lista. Argumentos usados: posicional 1: range(1, 8).

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 1; posicional 2: 8.

```
0037                 raise ValueError('Missing forecast leads')
```

Interrompe o caminho atual lançando ValueError('Missing forecast leads'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Missing forecast leads'.

```
0038             keys = frame[['timestamp', 'y_true']]
```

Atribui a keys o resultado de frame[['timestamp', 'y_true']]. A expressão à direita é avaliada antes da atribuição.

```
0039             if reference is not None:
```

Avalia a condição reference is not None. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0040                 pd.testing.assert_frame_equal(reference, keys)
```

Executa pd.testing.assert_frame_equal(reference, keys). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_frame_equal — Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem. Argumentos usados: posicional 1: reference; posicional 2: keys.

```
0041             reference = keys
```

Atribui a reference o resultado de keys. A expressão à direita é avaliada antes da atribuição.

```
0042             plot_frame = frame[['timestamp', 'y_true', 'y_pred']].rename(
```

Atribui a plot_frame o resultado de frame[['timestamp', 'y_true', 'y_pred']].rename(columns={'y_true': 'Real', 'y_pred': 'Predito'}). A expressão à direita é avaliada antes da atribuição.

frame[['timestamp', 'y_true', 'y_pred']].rename — Renomeia colunas de um DataFrame. Argumentos usados: columns: {'y_true': 'Real', 'y_pred': 'Predito'}.

```
0043                 columns={'y_true': 'Real', 'y_pred': 'Predito'})
```

Define 'y_true': demanda observada usada somente na avaliação; recebe 'Real'; 'y_pred': valor retornado pelo modelo para a data correspondente; recebe 'Predito'. Os valores são avaliados nesta janela/iteração.

```
0044             name = f'{dataset}_{slug}'
```

Atribui a name o resultado de f'{dataset}_{slug}'. A expressão à direita é avaliada antes da atribuição.

```
0045             title = f'{dataset.title()} — {label} | contexto {policy}'
```

Atribui a title o resultado de f'{dataset.title()} — {label} | contexto {policy}'. A expressão à direita é avaliada antes da atribuição.

dataset.title — Converte texto para capitalização de título. Argumentos usados: nenhum argumento explícito.

```
0046             plots.linear_graph(plot_frame, timestamp_col='timestamp',
    highlight_col='Predito',
```

Executa plots.linear_graph(plot_frame, timestamp_col='timestamp', highlight_col='Predito', compare_cols=['Real'], output_type='output_models', model_name=name, title=title, filename='real_vs_predito.png'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

plots.linear_graph — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: plot_frame; timestamp_col: 'timestamp'; highlight_col: 'Predito'; compare_cols: ['Real']; output_type: 'output_models'; model_name: name; title: title; filename: 'real_vs_predito.png'.

```
0047                 compare_cols=['Real'], output_type='output_models', model_name=name,
```

Continua os argumentos nomeados: compare_cols = ['Real']; output_type = 'output_models'; model_name = name. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0048                 title=title, filename='real_vs_predito.png')
```

Continua os argumentos nomeados: title = title; filename = 'real_vs_predito.png'. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0049             destination = OUTPUT / f'output_{name}' / f'{name}_graph' / 'real_vs_predito.png'
```

Atribui a destination o resultado de OUTPUT / f'output_{name}' / f'{name}_graph' / 'real_vs_predito.png'. A expressão à direita é avaliada antes da atribuição.

```
0050             records.append(dict(dataset=dataset,model=label,context=policy,title=title,
```

Executa records.append(dict(dataset=dataset, model=label, context=policy, title=title, n_predictions=len(frame), first_date=frame.timestamp.iloc[0], last_date=frame.timestamp.iloc[-1], context_first=int(frame.context.iloc[0]), context_last=int(frame.context.iloc[-1]), predictions_csv=str(csv.relative_to(ROOT)), source_sha256=hashlib.sha256(csv.read_bytes()).hexdigest(), image=str(destination.relative_to(ROOT)))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

records.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(dataset=dataset, model=label, context=policy, title=title, n_predictions=len(frame), first_date=frame.timestamp.iloc[0], last_date=frame.timestamp.iloc[-1], context_first=int(frame.context.iloc[0]), context_last=int(frame.context.iloc[-1]), predictions_csv=str(csv.relative_to(ROOT)), source_sha256=hashlib.sha256(csv.read_bytes()).hexdigest(), image=str(destination.relative_to(ROOT))).

dict — Cria dicionário. Argumentos usados: dataset: dataset; model: label; context: policy; title: title; n_predictions: len(frame); first_date: frame.timestamp.iloc[0]; last_date: frame.timestamp.iloc[-1]; context_first: int(frame.context.iloc[0]); context_last: int(frame.context.iloc[-1]); predictions_csv: str(csv.relative_to(ROOT)); source_sha256: hashlib.sha256(csv.read_bytes()).hexdigest(); image: str(destination.relative_to(ROOT)).

```
0051
    n_predictions=len(frame),first_date=frame.timestamp.iloc[0],last_date=frame.timestamp.iloc[-1],
```

Continua os argumentos nomeados: n_predictions = len(frame); first_date = frame.timestamp.iloc[0]; last_date = frame.timestamp.iloc[-1]. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

len — Conta elementos. Argumentos usados: posicional 1: frame.

```
0052
    context_first=int(frame.context.iloc[0]),context_last=int(frame.context.iloc[-1]),
```

Continua os argumentos nomeados: context_first = int(frame.context.iloc[0]); context_last = int(frame.context.iloc[-1]). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.context.iloc[0].

int — Converte para inteiro, truncando parte fracionária numérica. Argumentos usados: posicional 1: frame.context.iloc[-1].

```
0053                 predictions_csv=str(csv.relative_to(ROOT)),source_sha256=hashlib.sha256(csv.r
    ead_bytes()).hexdigest(),
```

Continua os argumentos nomeados: predictions_csv = str(csv.relative_to(ROOT)); source_sha256 = hashlib.sha256(csv.read_bytes()).hexdigest(). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

str — Produz representação textual. Argumentos usados: posicional 1: csv.relative_to(ROOT).

hashlib.sha256(csv.read_bytes()).hexdigest — Retorna a representação hexadecimal do digest acumulado. Argumentos usados: nenhum argumento explícito.

csv.relative_to — Calcula caminho relativo a um ancestral de um Path. Argumentos usados: posicional 1: ROOT.

hashlib.sha256 — Cria calculador de digest SHA-256. Argumentos usados: posicional 1: csv.read_bytes().

csv.read_bytes — Lê bytes integrais do arquivo. Argumentos usados: nenhum argumento explícito.

```
0054                 image=str(destination.relative_to(ROOT))))
```

Continua os argumentos nomeados: image = str(destination.relative_to(ROOT)). Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

str — Produz representação textual. Argumentos usados: posicional 1: destination.relative_to(ROOT).

destination.relative_to — Calcula caminho relativo a um ancestral de um Path. Argumentos usados: posicional 1: ROOT.

```
0055      pd.DataFrame(records).to_csv(OUTPUT / 'manifest.csv', index=False)
```

Executa pd.DataFrame(records).to_csv(OUTPUT / 'manifest.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(records).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: OUTPUT / 'manifest.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: records.

```
0056      print(f'{len(records)} plots generated in {OUTPUT}')
```

Executa print(f'{len(records)} plots generated in {OUTPUT}'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: f'{len(records)} plots generated in {OUTPUT}'.

len — Conta elementos. Argumentos usados: posicional 1: records.

```
0059  if __name__ == '__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0060      main()
```

Executa main(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

main — Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

44. src/visualization/plots.py

Funções de desenho e exportação usadas pela visualização histórica, mantendo o estilo copiado de Siaqua.

SHA-256: 0d454a2a246ed68f309eecdb976a83b450db76b3de7b4ae553d4a037a4ee18f4

Funções, contratos e pontos de conexão

```
L12: _resolve_save_folder(output_type: str, model_name: Optional[str]) → Optional[Path]
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

```
L24: linear_graph(df: pd.DataFrame, timestamp_col: str, highlight_col: str, output_type:
    Optional[str] = None, compare_cols: Sequence[str] = (), model_name: Optional[str] = None,
    title: Optional[str] = None, filename: str = 'line_graph.png')
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

Chamadas do projeto: _resolve_save_folder → src/visualization/plots.py:12

```
L80: error_bar_graph(df: pd.DataFrame, timestamp_col: str, highlight_col: str, output_type:
    Optional[str] = None, compare_cols: Sequence[str] = (), model_name: Optional[str] = None,
    title: Optional[str] = None, filename: str = 'line_graph.png')
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

APIs externas usadas neste arquivo

Path, ValueError, copy, enumerate, grid, join, legend, len, mkdir, pd.to_datetime, plot, plt.close, plt.show, plt.style.use, plt.subplots, plt.tight_layout, resolve, savefig, set_color, set_title, set_visible, tick_params. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0002  import matplotlib.pyplot as plt
```

Importa matplotlib.pyplot como plt. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from typing import Sequence, Optional
```

Importa Sequence, Optional de typing. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  from .colors import COLORS, NEUTRALS, PALETTES
```

Importa COLORS, NEUTRALS, PALETTES de .colors. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  PROJECT_ROOT = Path(__file__).resolve().parents[2]
```

Atribui a PROJECT_ROOT o resultado de Path(__file__).resolve().parents[2]. A expressão à direita é avaliada antes da atribuição.

Path(__file__).resolve — Resolve caminho absoluto e links simbólicos. Argumentos usados: nenhum argumento explícito.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: __file__.

```
0008  EDA_GRAPH_FOLDER = PROJECT_ROOT / "eda_graphs"
```

Atribui a EDA_GRAPH_FOLDER o resultado de PROJECT_ROOT / 'eda_graphs'. A expressão à direita é avaliada antes da atribuição.

```
0009  OUTPUT_MODELS_FOLDER = PROJECT_ROOT / "output_models"
```

Atribui a OUTPUT_MODELS_FOLDER o resultado de PROJECT_ROOT / 'output_models'. A expressão à direita é avaliada antes da atribuição.

```
0012  def _resolve_save_folder(output_type: str, model_name: Optional[str]) -> Optional[Path]:
```

Define _resolve_save_folder(output_type: str, model_name: Optional[str]) → Optional[Path]. O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0013      if output_type == "eda":
```

Avalia a condição output_type == 'eda'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0014          return EDA_GRAPH_FOLDER
```

Encerra esta função e devolve EDA_GRAPH_FOLDER ao chamador; não imprime o resultado por si só.

```
0016      if output_type == "output_models":
```

Avalia a condição output_type == 'output_models'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0017          if not model_name:
```

Avalia a condição not model_name. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0018              raise ValueError("model_name é obrigatório quando output_type='output_models'")
```

Interrompe o caminho atual lançando ValueError("model_name é obrigatório quando output_type='output_models'"). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: "model_name é obrigatório quando output_type='output_models'".

```
0019          return OUTPUT_MODELS_FOLDER / f"output_{model_name}" / f"{model_name}_graph"
```

Encerra esta função e devolve OUTPUT_MODELS_FOLDER / f'output_{model_name}' / f'{model_name}_graph' ao chamador; não imprime o resultado por si só.

```
0021      raise ValueError("wrong output_type, set 'eda' or 'output_models'")
```

Interrompe o caminho atual lançando ValueError("wrong output_type, set 'eda' or 'output_models'"). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: "wrong output_type, set 'eda' or 'output_models'".

```
0024  def linear_graph(
```

Define linear_graph(df: pd.DataFrame, timestamp_col: str, highlight_col: str, output_type: Optional[str] = None, compare_cols: Sequence[str] = (), model_name: Optional[str] = None, title: Optional[str] = None, filename: str = 'line_graph.png'). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0025      df: pd.DataFrame,
```

Continuação da instrução na linha 24: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0026      timestamp_col: str,
```

Continuação da instrução na linha 24: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0027      highlight_col: str,
```

Continuação da instrução na linha 24: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0028      output_type: Optional[str] = None,
```

Continuação da instrução na linha 24: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0029      compare_cols: Sequence[str] = (),
```

Continuação da instrução na linha 24: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0030      model_name: Optional[str] = None,
```

Continuação da instrução na linha 24: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0031      title: Optional[str] = None,
```

Continuação da instrução na linha 24: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0032      filename: str = "line_graph.png",
```

Continuação da instrução na linha 24: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0033  ):
```

Continuação da instrução na linha 24: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0034      df = df.copy()
```

Atribui a df o resultado de df.copy(). A expressão à direita é avaliada antes da atribuição.

df.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0035      df[timestamp_col] = pd.to_datetime(df[timestamp_col])
```

Atribui a df[timestamp_col] o resultado de pd.to_datetime(df[timestamp_col]). A expressão à direita é avaliada antes da atribuição.

pd.to_datetime — Converte valores em datas pandas. Argumentos usados: posicional 1: df[timestamp_col].

```
0037      fig, ax = plt.subplots(figsize=(12, 6))
```

Atribui a (fig, ax) o resultado de plt.subplots(figsize=(12, 6)). A expressão à direita é avaliada antes da atribuição.

plt.subplots — Cria figura e eixos Matplotlib. Argumentos usados: figsize: (12, 6).

```
0038      plt.style.use("default")
```

Executa plt.style.use('default'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

plt.style.use — Aplica um estilo visual Matplotlib. Argumentos usados: posicional 1: 'default'.

```
0040      gray_tones = [NEUTRALS["medium"], NEUTRALS["light"], NEUTRALS["lighter"]]
```

Atribui a gray_tones o resultado de [NEUTRALS['medium'], NEUTRALS['light'], NEUTRALS['lighter']]. A expressão à direita é avaliada antes da atribuição.

```
0041      for i, col in enumerate(compare_cols):
```

Percorre enumerate(compare_cols); em cada iteração associa o item a (i, col). O corpo indentado aplica a operação a esse item/grupo.

enumerate — Percorre iterável com contador. Argumentos usados: posicional 1: compare_cols.

```
0042          ax.plot(
```

Executa ax.plot(df[timestamp_col], df[col], color=gray_tones[i % len(gray_tones)], alpha=0.8, linewidth=0.6, label=col, zorder=1). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.plot — Desenha sequência de pontos/linhas no eixo. Argumentos usados: posicional 1: df[timestamp_col]; posicional 2: df[col]; color: gray_tones[i % len(gray_tones)]; alpha: 0.8; linewidth: 0.6; label: col; zorder: 1.

```
0043              df[timestamp_col], df[col],
```

Continuação da instrução na linha 42: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0044              color=gray_tones[i % len(gray_tones)],
```

Continua os argumentos nomeados: color = gray_tones[i % len(gray_tones)]. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

len — Conta elementos. Argumentos usados: posicional 1: gray_tones.

```
0045              alpha=0.8, linewidth=0.6, label=col, zorder=1,
```

Continua os argumentos nomeados: alpha = 0.8; linewidth = 0.6; label = col; zorder = 1. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0046          )
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0048      highlight_color = PALETTES["highlight"]["highlight"]
```

Atribui a highlight_color o resultado de PALETTES['highlight']['highlight']. A expressão à direita é avaliada antes da atribuição.

```
0049      ax.plot(
```

Executa ax.plot(df[timestamp_col], df[highlight_col], color=highlight_color, linewidth=0.8, alpha=0.95, label=highlight_col, zorder=3). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.plot — Desenha sequência de pontos/linhas no eixo. Argumentos usados: posicional 1: df[timestamp_col]; posicional 2: df[highlight_col]; color: highlight_color; linewidth: 0.8; alpha: 0.95; label: highlight_col; zorder: 3.

```
0050          df[timestamp_col], df[highlight_col],
```

Continuação da instrução na linha 49: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0051          color=highlight_color, linewidth=0.8, alpha=0.95,
```

Continua os argumentos nomeados: color = highlight_color; linewidth = 0.8; alpha = 0.95. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0052          label=highlight_col, zorder=3,
```

Continua os argumentos nomeados: label = highlight_col; zorder = 3. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0053      )
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0055      ax.set_title(
```

Executa ax.set_title(title or f"{highlight_col} x {', '.join(compare_cols)}", fontsize=16, loc='center', pad=10, color=NEUTRALS['dark']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.set_title — Define título de eixo. Argumentos usados: posicional 1: title or f"{highlight_col} x {', '.join(compare_cols)}"; fontsize: 16; loc: 'center'; pad: 10; color: NEUTRALS['dark'].

```
0056          title or f"{highlight_col} x {', '.join(compare_cols)}",
```

Continuação da instrução na linha 55: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

', '.join — Une textos com o separador que recebe o método. Argumentos usados: posicional 1: compare_cols.

```
0057          fontsize=16, loc="center", pad=10, color=NEUTRALS["dark"],
```

Continua os argumentos nomeados: fontsize = 16; loc = 'center'; pad = 10; color = NEUTRALS['dark']. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0058      )
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0060      ax.spines["top"].set_visible(False)
```

Executa ax.spines['top'].set_visible(False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.spines['top'].set_visible — Mostra ou oculta uma borda do eixo. Argumentos usados: posicional 1: False.

```
0061      ax.spines["right"].set_visible(False)
```

Executa ax.spines['right'].set_visible(False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.spines['right'].set_visible — Mostra ou oculta uma borda do eixo. Argumentos usados: posicional 1: False.

```
0062      ax.spines["left"].set_color(NEUTRALS["lighter"])
```

Executa ax.spines['left'].set_color(NEUTRALS['lighter']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.spines['left'].set_color — Define cor de uma borda do eixo. Argumentos usados: posicional 1: NEUTRALS['lighter'].

```
0063      ax.spines["bottom"].set_color(NEUTRALS["lighter"])
```

Executa ax.spines['bottom'].set_color(NEUTRALS['lighter']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.spines['bottom'].set_color — Define cor de uma borda do eixo. Argumentos usados: posicional 1: NEUTRALS['lighter'].

```
0064      ax.tick_params(axis="both", colors=NEUTRALS["medium"])
```

Executa ax.tick_params(axis='both', colors=NEUTRALS['medium']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.tick_params — Ajusta estilo dos marcadores e rótulos dos eixos. Argumentos usados: axis: 'both'; colors: NEUTRALS['medium'].

```
0065      ax.grid(axis="y", linestyle="--", alpha=0.3, color=NEUTRALS["lighter"])
```

Executa ax.grid(axis='y', linestyle='--', alpha=0.3, color=NEUTRALS['lighter']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.grid — Exibe linhas de referência. Argumentos usados: axis: 'y'; linestyle: '--'; alpha: 0.3; color: NEUTRALS['lighter'].

```
0067      ax.legend(frameon=False, loc="upper left", fontsize=9)
```

Executa ax.legend(frameon=False, loc='upper left', fontsize=9). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.legend — Mostra identificação das séries. Argumentos usados: frameon: False; loc: 'upper left'; fontsize: 9.

```
0069      plt.tight_layout()
```

Executa plt.tight_layout(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

plt.tight_layout — Ajusta espaçamento entre eixos e rótulos. Argumentos usados: nenhum argumento explícito.

```
0071      if output_type is None:
```

Avalia a condição output_type is None. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0072          plt.show()
```

Executa plt.show(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

plt.show — Solicita exibição da figura no backend ativo. Argumentos usados: nenhum argumento explícito.

```
0073          return None
```

Encerra esta função e devolve None ao chamador; não imprime o resultado por si só.

```
0075      save_folder = _resolve_save_folder(output_type, model_name)
```

Atribui a save_folder o resultado de _resolve_save_folder(output_type, model_name). A expressão à direita é avaliada antes da atribuição.

_resolve_save_folder — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: output_type; posicional 2: model_name.

```
0076      save_folder.mkdir(parents=True, exist_ok=True)
```

Executa save_folder.mkdir(parents=True, exist_ok=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

save_folder.mkdir — Cria diretório. Argumentos usados: parents: True; exist_ok: True.

```
0077      fig.savefig(save_folder / filename, dpi=150)
```

Executa fig.savefig(save_folder / filename, dpi=150). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

fig.savefig — Exporta uma figura para arquivo. Argumentos usados: posicional 1: save_folder / filename; dpi: 150.

```
0078      plt.close(fig)
```

Executa plt.close(fig). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

plt.close — Libera recursos da figura. Argumentos usados: posicional 1: fig.

```
0080  def error_bar_graph(
```

Define error_bar_graph(df: pd.DataFrame, timestamp_col: str, highlight_col: str, output_type: Optional[str] = None, compare_cols: Sequence[str] = (), model_name: Optional[str] = None, title: Optional[str] = None, filename: str = 'line_graph.png'). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0081      df: pd.DataFrame,
```

Continuação da instrução na linha 80: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0082      timestamp_col: str,
```

Continuação da instrução na linha 80: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0083      highlight_col: str,
```

Continuação da instrução na linha 80: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0084      output_type: Optional[str] = None,
```

Continuação da instrução na linha 80: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0085      compare_cols: Sequence[str] = (),
```

Continuação da instrução na linha 80: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0086      model_name: Optional[str] = None,
```

Continuação da instrução na linha 80: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0087      title: Optional[str] = None,
```

Continuação da instrução na linha 80: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0088      filename: str = "line_graph.png",
```

Continuação da instrução na linha 80: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0089  ):
```

Continuação da instrução na linha 80: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0090      return None
```

Encerra esta função e devolve None ao chamador; não imprime o resultado por si só.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

45. src/visualization/colors.py

Paleta e seleção de cores das funções de visualização histórica.

SHA-256: 2329eb07dab897ca296c56e621fc413dfbdf9c63775c8341bcb0a06aecc1ab8a

Código comentado, linha a linha

```
0001  NEUTRALS = {
```

Atribui a NEUTRALS o resultado de {'dark': '#4A4A4A', 'medium': '#7A7A7A', 'light': '#A6A6A6', 'lighter': '#D9D9D9'}. A expressão à direita é avaliada antes da atribuição.

```
0002      "dark": "#4A4A4A",
```

Define 'dark': campo do registro/coleção em construção; recebe '#4A4A4A'. Os valores são avaliados nesta janela/iteração.

```
0003      "medium": "#7A7A7A",
```

Define 'medium': campo do registro/coleção em construção; recebe '#7A7A7A'. Os valores são avaliados nesta janela/iteração.

```
0004      "light": "#A6A6A6",
```

Define 'light': campo do registro/coleção em construção; recebe '#A6A6A6'. Os valores são avaliados nesta janela/iteração.

```
0005      "lighter": "#D9D9D9",
```

Define 'lighter': campo do registro/coleção em construção; recebe '#D9D9D9'. Os valores são avaliados nesta janela/iteração.

```
0006  }
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0008  COLORS = {
```

Atribui a COLORS o resultado de {'blue': '#3B6EA8', 'orange': '#D77A3F', 'teal': '#4F8A8B', 'purple': '#7A628A', 'brown': '#8C6A4A', 'olive': '#6B705C', 'terracotta': '#B85C38'}. A expressão à direita é avaliada antes da atribuição.

```
0009      "blue": "#3B6EA8",
```

Define 'blue': campo do registro/coleção em construção; recebe '#3B6EA8'. Os valores são avaliados nesta janela/iteração.

```
0010      "orange": "#D77A3F",
```

Define 'orange': campo do registro/coleção em construção; recebe '#D77A3F'. Os valores são avaliados nesta janela/iteração.

```
0011      "teal": "#4F8A8B",
```

Define 'teal': campo do registro/coleção em construção; recebe '#4F8A8B'. Os valores são avaliados nesta janela/iteração.

```
0012      "purple": "#7A628A",
```

Define 'purple': campo do registro/coleção em construção; recebe '#7A628A'. Os valores são avaliados nesta janela/iteração.

```
0013      "brown": "#8C6A4A",
```

Define 'brown': campo do registro/coleção em construção; recebe '#8C6A4A'. Os valores são avaliados nesta janela/iteração.

```
0014      "olive": "#6B705C",
```

Define 'olive': campo do registro/coleção em construção; recebe '#6B705C'. Os valores são avaliados nesta janela/iteração.

```
0015      "terracotta": "#B85C38",
```

Define 'terracotta': campo do registro/coleção em construção; recebe '#B85C38'. Os valores são avaliados nesta janela/iteração.

```
0016  }
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

```
0018  PALETTES = {
```

Atribui a PALETTES o resultado de {'default': [COLORS['blue'], COLORS['orange'], COLORS['purple'], COLORS['teal'], COLORS['brown']], 'earth': ['#6B4F3A', '#8C6A4A', '#B08968', '#B85C38', '#6B705C'], 'cool': [COLORS['blue'], COLORS['teal'], COLORS['purple'], '#536878'], 'highlight': {'background': NEUTRALS['light'], 'highlight': COLORS['blue']}}. A expressão à direita é avaliada antes da atribuição.

```
0019      "default": [
```

Define 'default': campo do registro/coleção em construção; recebe [COLORS['blue'], COLORS['orange'], COLORS['purple'], COLORS['teal'], COLORS['brown']]. Os valores são avaliados nesta janela/iteração.

```
0020          COLORS["blue"],
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0021          COLORS["orange"],
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0022          COLORS["purple"],
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0023          COLORS["teal"],
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0024          COLORS["brown"],
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0025      ],
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0027      "earth": [
```

Define 'earth': campo do registro/coleção em construção; recebe ['#6B4F3A', '#8C6A4A', '#B08968', '#B85C38', '#6B705C']. Os valores são avaliados nesta janela/iteração.

```
0028          "#6B4F3A",
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0029          "#8C6A4A",
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0030          "#B08968",
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0031          "#B85C38",
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0032          "#6B705C",
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0033      ],
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0035      "cool": [
```

Define 'cool': campo do registro/coleção em construção; recebe [COLORS['blue'], COLORS['teal'], COLORS['purple'], '#536878']. Os valores são avaliados nesta janela/iteração.

```
0036          COLORS["blue"],
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0037          COLORS["teal"],
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0038          COLORS["purple"],
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0039          "#536878",
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0040      ],
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0042      "highlight": {
```

Define 'highlight': campo do registro/coleção em construção; recebe {'background': NEUTRALS['light'], 'highlight': COLORS['blue']}. Os valores são avaliados nesta janela/iteração.

```
0043          "background": NEUTRALS["light"],
```

Define 'background': campo do registro/coleção em construção; recebe NEUTRALS['light']. Os valores são avaliados nesta janela/iteração.

```
0044          "highlight": COLORS["blue"],
```

Define 'highlight': campo do registro/coleção em construção; recebe COLORS['blue']. Os valores são avaliados nesta janela/iteração.

```
0045      },
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0046  }
```

Fecha a coleção/chamada iniciada acima. Não inicia uma nova transformação de dados; a indentação seguinte determina a continuidade do bloco.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

46. tests/test_calendar_prompts.py

Testes de regressão. São executados por unittest discover, não durante a inferência.

SHA-256: 14551c5cdad1a4978f50182b4e6e878bb74c46155bd7e27e31ddcb23b027f86b

Funções, contratos e pontos de conexão

```
L12: frame(self)
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

```
L23: test_date_words_preserve_year_and_leap_day(self)
```

Caso de teste: date words preserve year and leap day. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: date_to_text → src/data/preprocessing/calendar_text.py:79

```
L28: test_conversion_preserves_measurements_and_source(self)
```

Caso de teste: conversion preserves measurements and source. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: calendar_to_text → src/data/preprocessing/calendar_text.py:98

```
L42: test_seasons_decode_supplied_encoding_at_boundaries(self)
```

Caso de teste: seasons decode supplied encoding at boundaries. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: calendar_to_text → src/data/preprocessing/calendar_text.py:98

```
L50: test_prompt_narrates_all_covariates_and_questions(self)
```

Caso de teste: prompt narrates all covariates and questions. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:96

```
L67: test_future_targets_cannot_change_prompt(self)
```

Caso de teste: future targets cannot change prompt. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:96

```
L69: prompt(data)
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:96

APIs externas usadas neste arquivo

all, any, assertEqual, assertFalse, assertIn, assertNotIn, assertRaisesRegex, assertTrue, copy, count, drop, endswith, eq, np.arange, np.cos, np.sin, pd.DataFrame, pd.date_range, pd.testing.assert_frame_equal, pd.to_datetime, tolist. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Calendar semantics, natural language alignment, and target isolation."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import unittest
```

Importa unittest. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  from src.data.preprocessing.calendar_text import calendar_to_text, date_to_text
```

Importa calendar_to_text, date_to_text de src.data.preprocessing.calendar_text. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  from src.data.preprocessing.generate_prompts import build_prompt
```

Importa build_prompt de src.data.preprocessing.generate_prompts. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  class CalendarPromptTests(unittest.TestCase):
```

Declara a classe CalendarPromptTests(unittest.TestCase). Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0012      def frame(self):
```

Define frame(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0013          dates = pd.date_range('2016-01-01', periods=10)
```

Atribui a dates o resultado de pd.date_range('2016-01-01', periods=10). A expressão à direita é avaliada antes da atribuição.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2016-01-01'; periods: 10.

```
0014          phase = dates.dayofweek * 2 * np.pi / 7
```

Atribui a phase o resultado de dates.dayofweek * 2 * np.pi / 7. A expressão à direita é avaliada antes da atribuição.

```
0015          return pd.DataFrame({
```

Encerra esta função e devolve pd.DataFrame({'timestamp': dates, 'water_produced': np.arange(10.0) + 100, 'temperature_mean': np.arange(10.0) + 20, 'day': dates.day, 'dayofweek_sin': np.sin(phase), 'dayofweek_cos': np.cos(phase), 'season_sin': 0.0, 'season_cos': 1.0, 'is_holiday': [1] + [0] * 9, 'is_school_recess_pr': 1}) ao chamador; não imprime o resultado por si só.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': dates, 'water_produced': np.arange(10.0) + 100, 'temperature_mean': np.arange(10.0) + 20, 'day': dates.day, 'dayofweek_sin': np.sin(phase), 'dayofweek_cos': np.cos(phase), 'season_sin': 0.0, 'season_cos': 1.0, 'is_holiday': [1] + [0] * 9, 'is_school_recess_pr': 1}.

```
0016              'timestamp': dates, 'water_produced': np.arange(10.) + 100,
```

Define 'timestamp': campo do registro/coleção em construção; recebe dates; 'water_produced': campo do registro/coleção em construção; recebe np.arange(10.0) + 100. Os valores são avaliados nesta janela/iteração.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 10.0.

```
0017              'temperature_mean': np.arange(10.) + 20, 'day': dates.day,
```

Define 'temperature_mean': campo do registro/coleção em construção; recebe np.arange(10.0) + 20; 'day': campo do registro/coleção em construção; recebe dates.day. Os valores são avaliados nesta janela/iteração.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 10.0.

```
0018              'dayofweek_sin': np.sin(phase), 'dayofweek_cos': np.cos(phase),
```

Define 'dayofweek_sin': campo do registro/coleção em construção; recebe np.sin(phase); 'dayofweek_cos': campo do registro/coleção em construção; recebe np.cos(phase). Os valores são avaliados nesta janela/iteração.

np.sin — Calcula seno em radianos. Argumentos usados: posicional 1: phase.

np.cos — Calcula cosseno em radianos. Argumentos usados: posicional 1: phase.

```
0019              'season_sin': 0., 'season_cos': 1., 'is_holiday': [1] + [0]*9,
```

Define 'season_sin': campo do registro/coleção em construção; recebe 0.0; 'season_cos': campo do registro/coleção em construção; recebe 1.0; 'is_holiday': campo do registro/coleção em construção; recebe [1] + [0] * 9. Os valores são avaliados nesta janela/iteração.

```
0020              'is_school_recess_pr': 1,
```

Define 'is_school_recess_pr': campo do registro/coleção em construção; recebe 1. Os valores são avaliados nesta janela/iteração.

```
0021          })
```

Continuação da instrução na linha 15: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0023      def test_date_words_preserve_year_and_leap_day(self):
```

Define test_date_words_preserve_year_and_leap_day(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0024          self.assertEqual(date_to_text('2016-01-01'), 'two thousand and sixteen, day one of
    January')
```

Executa self.assertEqual(date_to_text('2016-01-01'), 'two thousand and sixteen, day one of January'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: date_to_text('2016-01-01'); posicional 2: 'two thousand and sixteen, day one of January'.

date_to_text — value é uma data ou sua representação textual canônica. Retorna ano, dia e mês em inglês sem mudar o ano original. Argumentos usados: posicional 1: '2016-01-01'.

```
0025          self.assertEqual(date_to_text('2026-01-01'), 'two thousand and twenty-six, day one of
     January')
```

Executa self.assertEqual(date_to_text('2026-01-01'), 'two thousand and twenty-six, day one of January'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: date_to_text('2026-01-01'); posicional 2: 'two thousand and twenty-six, day one of January'.

date_to_text — value é uma data ou sua representação textual canônica. Retorna ano, dia e mês em inglês sem mudar o ano original. Argumentos usados: posicional 1: '2026-01-01'.

```
0026          self.assertEqual(date_to_text('2020-02-29'), 'two thousand and twenty, day twenty-
    nine of February')
```

Executa self.assertEqual(date_to_text('2020-02-29'), 'two thousand and twenty, day twenty-nine of February'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: date_to_text('2020-02-29'); posicional 2: 'two thousand and twenty, day twenty-nine of February'.

date_to_text — value é uma data ou sua representação textual canônica. Retorna ano, dia e mês em inglês sem mudar o ano original. Argumentos usados: posicional 1: '2020-02-29'.

```
0028      def test_conversion_preserves_measurements_and_source(self):
```

Define test_conversion_preserves_measurements_and_source(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0029          source = self.frame()
```

Atribui a source o resultado de self.frame(). A expressão à direita é avaliada antes da atribuição.

self.frame — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: nenhum argumento explícito.

```
0030          before = source.copy(deep=True)
```

Atribui a before o resultado de source.copy(deep=True). A expressão à direita é avaliada antes da atribuição.

source.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: deep: True.

```
0031          result = calendar_to_text(source)
```

Atribui a result o resultado de calendar_to_text(source). A expressão à direita é avaliada antes da atribuição.

calendar_to_text — frame pode conter calendário numérico ou já convertido. Devolve cópia com calendário inglês, mescla pares cíclicos e preserva demanda/meteorologia. Não ajusta estatísticas nem altera o argumento. Argumentos usados: posicional 1: source.

```
0032          pd.testing.assert_frame_equal(source, before)
```

Executa pd.testing.assert_frame_equal(source, before). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_frame_equal — Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem. Argumentos usados: posicional 1: source; posicional 2: before.

```
0033          pd.testing.assert_frame_equal(result[['water_produced', 'temperature_mean']],
```

Executa pd.testing.assert_frame_equal(result[['water_produced', 'temperature_mean']], source[['water_produced', 'temperature_mean']]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_frame_equal — Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem. Argumentos usados: posicional 1: result[['water_produced', 'temperature_mean']]; posicional 2: source[['water_produced', 'temperature_mean']].

```
0034                                          source[['water_produced', 'temperature_mean']])
```

Continuação da instrução na linha 33: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0035          self.assertEqual(result.dayofweek.iloc[:3].tolist(), ['Friday', 'Saturday',
    'Sunday'])
```

Executa self.assertEqual(result.dayofweek.iloc[:3].tolist(), ['Friday', 'Saturday', 'Sunday']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: result.dayofweek.iloc[:3].tolist(); posicional 2: ['Friday', 'Saturday', 'Sunday'].

result.dayofweek.iloc[:3].tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

```
0036          self.assertTrue(result.season.eq('summer').all())
```

Executa self.assertTrue(result.season.eq('summer').all()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: result.season.eq('summer').all().

result.season.eq('summer').all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

result.season.eq — Compara igualdade elemento a elemento. Argumentos usados: posicional 1: 'summer'.

```
0037          self.assertEqual(result.day.iloc[0], 'one')
```

Executa self.assertEqual(result.day.iloc[0], 'one'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: result.day.iloc[0]; posicional 2: 'one'.

```
0038          self.assertEqual(result.is_holiday.iloc[:2].tolist(), ['a public holiday', 'not a
    public holiday'])
```

Executa self.assertEqual(result.is_holiday.iloc[:2].tolist(), ['a public holiday', 'not a public holiday']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: result.is_holiday.iloc[:2].tolist(); posicional 2: ['a public holiday', 'not a public holiday'].

result.is_holiday.iloc[:2].tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

```
0039          self.assertTrue(result.is_school_recess_pr.eq('during the Paraná school
    recess').all())
```

Executa self.assertTrue(result.is_school_recess_pr.eq('during the Paraná school recess').all()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: result.is_school_recess_pr.eq('during the Paraná school recess').all().

result.is_school_recess_pr.eq('during the Paraná school recess').all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

result.is_school_recess_pr.eq — Compara igualdade elemento a elemento. Argumentos usados: posicional 1: 'during the Paraná school recess'.

```
0040          self.assertFalse(any(c.endswith(('_sin', '_cos')) for c in result))
```

Executa self.assertFalse(any((c.endswith(('_sin', '_cos')) for c in result))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertFalse — Asserção de condição falsa. Argumentos usados: posicional 1: any((c.endswith(('_sin', '_cos')) for c in result)).

any — Reduz a existência de pelo menos um True. Argumentos usados: posicional 1: (c.endswith(('_sin', '_cos')) for c in result).

c.endswith — Verifica sufixo de string. Argumentos usados: posicional 1: ('_sin', '_cos').

```
0042      def test_seasons_decode_supplied_encoding_at_boundaries(self):
```

Define test_seasons_decode_supplied_encoding_at_boundaries(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0043          frame = pd.DataFrame({'timestamp': pd.to_datetime(['2016-03-19', '2016-03-20',
```

Atribui a frame o resultado de pd.DataFrame({'timestamp': pd.to_datetime(['2016-03-19', '2016-03-20', '2016-06-20', '2016-09-22']), 'season_sin': [0.0, 1.0, 0.0, -1.0], 'season_cos': [1.0, 0.0, -1.0, 0.0]}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.to_datetime(['2016-03-19', '2016-03-20', '2016-06-20', '2016-09-22']), 'season_sin': [0.0, 1.0, 0.0, -1.0], 'season_cos': [1.0, 0.0, -1.0, 0.0]}.

pd.to_datetime — Converte valores em datas pandas. Argumentos usados: posicional 1: ['2016-03-19', '2016-03-20', '2016-06-20', '2016-09-22'].

```
0044                                '2016-06-20', '2016-09-22']),
```

Continuação da instrução na linha 43: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0045                                'season_sin': [0., 1., 0., -1.], 'season_cos': [1., 0., -1.,
    0.]})
```

Define 'season_sin': campo do registro/coleção em construção; recebe [0.0, 1.0, 0.0, -1.0]; 'season_cos': campo do registro/coleção em construção; recebe [1.0, 0.0, -1.0, 0.0]. Os valores são avaliados nesta janela/iteração.

```
0046          self.assertEqual(calendar_to_text(frame).season.tolist(), ['summer', 'autumn',
    'winter', 'spring'])
```

Executa self.assertEqual(calendar_to_text(frame).season.tolist(), ['summer', 'autumn', 'winter', 'spring']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: calendar_to_text(frame).season.tolist(); posicional 2: ['summer', 'autumn', 'winter', 'spring'].

calendar_to_text(frame).season.tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

calendar_to_text — frame pode conter calendário numérico ou já convertido. Devolve cópia com calendário inglês, mescla pares cíclicos e preserva demanda/meteorologia. Não ajusta estatísticas nem altera o argumento. Argumentos usados: posicional 1: frame.

```
0047         with self.assertRaisesRegex(ValueError, 'Both season'):
```

Abre o contexto gerenciado self.assertRaisesRegex(ValueError, 'Both season'). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaisesRegex — Exige exceção e mensagem compatível com expressão regular. Argumentos usados: posicional 1: ValueError; posicional 2: 'Both season'.

```
0048             calendar_to_text(frame.drop(columns='season_cos'))
```

Executa calendar_to_text(frame.drop(columns='season_cos')). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

calendar_to_text — frame pode conter calendário numérico ou já convertido. Devolve cópia com calendário inglês, mescla pares cíclicos e preserva demanda/meteorologia. Não ajusta estatísticas nem altera o argumento. Argumentos usados: posicional 1: frame.drop(columns='season_cos').

frame.drop — Retorna tabela sem as linhas/colunas indicadas. Argumentos usados: columns: 'season_cos'.

```
0050     def test_prompt_narrates_all_covariates_and_questions(self):
```

Define test_prompt_narrates_all_covariates_and_questions(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0051         frame = self.frame()
```

Atribui a frame o resultado de self.frame(). A expressão à direita é avaliada antes da atribuição.

self.frame — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: nenhum argumento explícito.

```
0052         prompt = build_prompt(frame.iloc[:3], 7,
```

Atribui a prompt o resultado de build_prompt(frame.iloc[:3], 7, future_covariates=frame.iloc[3:].drop(columns='water_produced'), region='Guaratuba'). A expressão à direita é avaliada antes da atribuição.

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: frame.iloc[:3]; posicional 2: 7; future_covariates: frame.iloc[3:].drop(columns='water_produced'); region: 'Guaratuba'.

```
0053
    future_covariates=frame.iloc[3:].drop(columns='water_produced'), region='Guaratuba')
```

Continua os argumentos nomeados: future_covariates = frame.iloc[3:].drop(columns='water_produced'); region = 'Guaratuba'; columns = 'water_produced'. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

frame.iloc[3:].drop — Retorna tabela sem as linhas/colunas indicadas. Argumentos usados: columns: 'water_produced'.

```
0054         self.assertIn('From two thousand and sixteen, day one of January to '
```

Executa self.assertIn('From two thousand and sixteen, day one of January to two thousand and sixteen, day three of January, the average temperature of region Guaratuba was 20; 21; 22 degrees Celsius on each day, respectively.', prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIn — Exige item presente em container. Argumentos usados: posicional 1: 'From two thousand and sixteen, day one of January to two thousand and sixteen, day three of January, the average temperature of region Guaratuba was 20; 21; 22 degrees Celsius on each day, respectively.'; posicional 2: prompt.

```
0055                      'two thousand and sixteen, day three of January, the average
    temperature '
```

Continuação da instrução na linha 54: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0056                      'of region Guaratuba was 20; 21; 22 degrees Celsius on each day,
    respectively.', prompt)
```

Continuação da instrução na linha 54: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0057          self.assertIn('100.00; 101.00; 102.00', prompt)
```

Executa self.assertIn('100.00; 101.00; 102.00', prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIn — Exige item presente em container. Argumentos usados: posicional 1: '100.00; 101.00; 102.00'; posicional 2: prompt.

```
0058          self.assertIn('Friday; Saturday; Sunday', prompt)
```

Executa self.assertIn('Friday; Saturday; Sunday', prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIn — Exige item presente em container. Argumentos usados: posicional 1: 'Friday; Saturday; Sunday'; posicional 2: prompt.

```
0059          self.assertIn('December through March', prompt)
```

Executa self.assertIn('December through March', prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIn — Exige item presente em container. Argumentos usados: posicional 1: 'December through March'; posicional 2: prompt.

```
0060          self.assertIn('Brazilian holidays and school recesses', prompt)
```

Executa self.assertIn('Brazilian holidays and school recesses', prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIn — Exige item presente em container. Argumentos usados: posicional 1: 'Brazilian holidays and school recesses'; posicional 2: prompt.

```
0061          self.assertEqual(prompt.count('What is the water_produced going to be on'), 7)
```

Executa self.assertEqual(prompt.count('What is the water_produced going to be on'), 7). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: prompt.count('What is the water_produced going to be on'); posicional 2: 7.

prompt.count — Conta ocorrências na sequência. Argumentos usados: posicional 1: 'What is the water_produced going to be on'.

```
0062          self.assertNotIn('CSV', prompt)
```

Executa self.assertNotIn('CSV', prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: 'CSV'; posicional 2: prompt.

```
0063          self.assertNotIn('2016-01-01', prompt)
```

Executa self.assertNotIn('2016-01-01', prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: '2016-01-01'; posicional 2: prompt.

```
0064          self.assertNotIn('season_sin', prompt)
```

Executa self.assertNotIn('season_sin', prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: 'season_sin'; posicional 2: prompt.

```
0065          self.assertNotIn('tourist city', build_prompt(frame.iloc[:3], 1, region='Joinville'))
```

Executa self.assertNotIn('tourist city', build_prompt(frame.iloc[:3], 1, region='Joinville')). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: 'tourist city'; posicional 2: build_prompt(frame.iloc[:3], 1, region='Joinville').

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: frame.iloc[:3]; posicional 2: 1; region: 'Joinville'.

```
0067      def test_future_targets_cannot_change_prompt(self):
```

Define test_future_targets_cannot_change_prompt(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0068          frame = self.frame()
```

Atribui a frame o resultado de self.frame(). A expressão à direita é avaliada antes da atribuição.

self.frame — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: nenhum argumento explícito.

```
0069          def prompt(data):
```

Define prompt(data). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0070              return build_prompt(data.iloc[:3], 7,
```

Encerra esta função e devolve build_prompt(data.iloc[:3], 7, future_covariates=data.iloc[3:].drop(columns='water_produced')) ao chamador; não imprime o resultado por si só.

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: data.iloc[:3]; posicional 2: 7; future_covariates: data.iloc[3:].drop(columns='water_produced').

```
0071
    future_covariates=data.iloc[3:].drop(columns='water_produced'))
```

Continua os argumentos nomeados: future_covariates = data.iloc[3:].drop(columns='water_produced'); columns = 'water_produced'. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

data.iloc[3:].drop — Retorna tabela sem as linhas/colunas indicadas. Argumentos usados: columns: 'water_produced'.

```
0072          original = prompt(frame)
```

Atribui a original o resultado de prompt(frame). A expressão à direita é avaliada antes da atribuição.

prompt — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: frame.

```
0073          frame.loc[3:, 'water_produced'] = 99999999
```

Atribui a frame.loc[3:, 'water_produced'] o resultado de 99999999. A expressão à direita é avaliada antes da atribuição.

```
0074          self.assertEqual(original, prompt(frame))
```

Executa self.assertEqual(original, prompt(frame)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: original; posicional 2: prompt(frame).

prompt — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: frame.

```
0075          with self.assertRaisesRegex(ValueError, 'Future target'):
```

Abre o contexto gerenciado self.assertRaisesRegex(ValueError, 'Future target'). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaisesRegex — Exige exceção e mensagem compatível com expressão regular. Argumentos usados: posicional 1: ValueError; posicional 2: 'Future target'.

```
0076              build_prompt(frame.iloc[:3], 7, future_covariates=frame.iloc[3:])
```

Executa build_prompt(frame.iloc[:3], 7, future_covariates=frame.iloc[3:]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: frame.iloc[:3]; posicional 2: 7; future_covariates: frame.iloc[3:].

```
0077         with self.assertRaisesRegex(ValueError, 'dates must match'):
```

Abre o contexto gerenciado self.assertRaisesRegex(ValueError, 'dates must match'). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaisesRegex — Exige exceção e mensagem compatível com expressão regular. Argumentos usados: posicional 1: ValueError; posicional 2: 'dates must match'.

```
0078             build_prompt(frame.iloc[:3], 7,
    future_covariates=frame.iloc[2:9].drop(columns='water_produced'))
```

Executa build_prompt(frame.iloc[:3], 7, future_covariates=frame.iloc[2:9].drop(columns='water_produced')). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: frame.iloc[:3]; posicional 2: 7; future_covariates: frame.iloc[2:9].drop(columns='water_produced').

frame.iloc[2:9].drop — Retorna tabela sem as linhas/colunas indicadas. Argumentos usados: columns: 'water_produced'.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

47. tests/test_forecasting.py

Testes de regressão. São executados por unittest discover, não durante a inferência.

SHA-256: d8531e299d642f79da82402c83d5088ec06af73a60613a8a8d852d90fa9473da

Funções, contratos e pontos de conexão

`L21: test_real_datasets_shared_protocol(self)`

Caso de teste: real datasets shared protocol. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: prepare_dataset → src/pipelines/data_pipeline.py:8

`L33: test_future_target_mutation_does_not_change_prompt(self)`

Caso de teste: future target mutation does not change prompt. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:96

`L42: test_demonstrations_remain_before_frozen_test(self)`

Caso de teste: demonstrations remain before frozen test. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: select_demonstrations → src/data/preprocessing/generate_prompts.py:17

`L49: test_missing_day_is_filled_from_past_only(self)`

Caso de teste: missing day is filled from past only. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: daily_grid → src/data/preprocessing/resampling.py:5; fill_past_only → src/data/preprocessing/missing_values.py:5

`L55: test_imputed_test_days_not_scored(self)`

Caso de teste: imputed test days not scored. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_windows → src/evaluation/cross_validation.py:20

`L62: test_covariates_and_history_grow_without_future_targets(self)`

Caso de teste: covariates and history grow without future targets. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:96; calendar_to_text → src/data/preprocessing/calendar_text.py:98; covariate_columns → src/data/features/covariates.py:6; model_inputs → src/data/features/covariates.py:25; prepare_dataset → src/pipelines/data_pipeline.py:8; timesfm_covariates → src/data/features/covariates.py:36; validate_covariates → src/data/features/covariates.py:13

`L90: test_future_target_column_is_rejected(self)`

Caso de teste: future target column is rejected. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:96

`L96: test_chronos_receives_entire_growing_history_and_covariates(self)`

Caso de teste: chronos receives entire growing history and covariates. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: adapter.predict → src/models/promptcast.py:71; adapter.predict → src/models/chronos.py:18; adapter.predict → src/models/timesfm.py:20; adapter.predict → src/models/classical.py:31; adapter.predict → src/models/classical.py:51; covariate_columns → src/data/features/covariates.py:6; model_inputs → src/data/features/covariates.py:25; prepare_dataset → src/pipelines/data_pipeline.py:8

`L115: test_invalid_model_responses(self)`

Caso de teste: invalid model responses. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: parse_prediction → src/models/promptcast.py:33

```
L124: test_retry_and_raw_response_audit(self)
```

Caso de teste: retry and raw response audit. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: OllamaConfig → src/models/promptcast.py:11; model.predict → src/models/promptcast.py:71; model.predict → src/models/chronos.py:18; model.predict → src/models/timesfm.py:20; model.predict → src/models/classical.py:31; model.predict → src/models/classical.py:51

```
L134: test_context_budget_rejects_silent_truncation(self)
```

Caso de teste: context budget rejects silent truncation. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: OllamaConfig → src/models/promptcast.py:11; model.predict → src/models/promptcast.py:71; model.predict → src/models/chronos.py:18; model.predict → src/models/timesfm.py:20; model.predict → src/models/classical.py:31; model.predict → src/models/classical.py:51

```
L140: test_metric_units(self)
```

Caso de teste: metric units. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: deterministic_metrics → src/evaluation/metrics.py:5

```
L150: test_validation_uses_training_prefix(self)
```

Caso de teste: validation uses training prefix. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: validation_split → src/evaluation/cross_validation.py:32

```
L154: test_duplicate_dates_rejected(self)
```

Caso de teste: duplicate dates rejected. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: validate_series → src/data/validation/schema.py:6

APIs externas usadas neste arquivo

Mock, OllamaConfig, all, assertAlmostEqual, assertEqual, assertFalse, assertGreaterEqual, assertIn, assertLess, assertNotIn, assertRaises, assertRaisesRegex, assertTrue, copy, enumerate, float, get, issubset, json.loads, len, list, max, min, np.arange, object.__new__, patch, pd.DataFrame, pd.date_range, pd.testing.assert_frame_equal, pd.to_datetime, predict, read_text, replace, set, subTest, unittest.main, values, zip. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Leakage, evaluation and malformed-output regression tests; no model downloads."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import unittest
```

Importa unittest. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from unittest.mock import patch
```

Importa patch de unittest.mock. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  from src.data.preprocessing.generate_prompts import LABELS, build_prompt,
    select_demonstrations
```

Importa LABELS, build_prompt, select_demonstrations de src.data.preprocessing.generate_prompts. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  from src.data.preprocessing.calendar_text import calendar_to_text
```

Importa calendar_to_text de src.data.preprocessing.calendar_text. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  from src.data.preprocessing.missing_values import fill_past_only
```

Importa fill_past_only de src.data.preprocessing.missing_values. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.data.preprocessing.resampling import daily_grid
```

Importa daily_grid de src.data.preprocessing.resampling. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  from src.data.validation.schema import validate_series
```

Importa validate_series de src.data.validation.schema. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0012  from src.evaluation.cross_validation import build_windows, validation_split
```

Importa build_windows, validation_split de src.evaluation.cross_validation. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0013  from src.evaluation.metrics import deterministic_metrics
```

Importa deterministic_metrics de src.evaluation.metrics. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0014  from src.models.promptcast import OllamaConfig, QwenForecaster, parse_prediction
```

Importa OllamaConfig, QwenForecaster, parse_prediction de src.models.promptcast. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0015  from src.pipelines.data_pipeline import prepare_dataset
```

Importa prepare_dataset de src.pipelines.data_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0016  from src.pipelines.forecasting_pipeline import ROOT
```

Importa ROOT de src.pipelines.forecasting_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0017  from src.data.features.covariates import model_inputs, covariate_columns, timesfm_covariates,
     validate_covariates
```

Importa model_inputs, covariate_columns, timesfm_covariates, validate_covariates de src.data.features.covariates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0020  class ForecastTests(unittest.TestCase):
```

Declara a classe ForecastTests(unittest.TestCase). Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0021      def test_real_datasets_shared_protocol(self):
```

Define test_real_datasets_shared_protocol(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0022          for name, context in [('joinville_processed_1day',365),
    ('guaratuba_processed_1daysin',730)]:
```

Percorre [('joinville_processed_1day', 365), ('guaratuba_processed_1daysin', 730)]; em cada iteração associa o item a (name, context). O corpo indentado aplica a operação a esse item/grupo.

```
0023              df, windows = prepare_dataset(ROOT / f'data/processed/{name}.csv', context)
```

Atribui a (df, windows) o resultado de prepare_dataset(ROOT / f'data/processed/{name}.csv', context). A expressão à direita é avaliada antes da atribuição.

prepare_dataset — path aponta o CSV; context define a primeira origem; horizon/step definem grade de avaliação; textual_calendar escolhe interpretação de datas em inglês. Retorna tabela preparada e lista de ForecastWindow. Argumentos usados: posicional 1: ROOT / f'data/processed/{name}.csv'; posicional 2: context.

```
0024              for window in windows:
```

Percorre windows; em cada iteração associa o item a window. O corpo indentado aplica a operação a esse item/grupo.

```
0025                  self.assertEqual(window.context_start, 0)
```

Executa self.assertEqual(window.context_start, 0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: window.context_start; posicional 2: 0.

```
0026                  self.assertGreaterEqual(window.origin, context)
```

Executa self.assertGreaterEqual(window.origin, context). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertGreaterEqual — Exige first >= second. Argumentos usados: posicional 1: window.origin; posicional 2: context.

```
0027                  self.assertEqual(window.history.stop, window.target.start)
```

Executa self.assertEqual(window.history.stop, window.target.start). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: window.history.stop; posicional 2: window.target.start.

```
0028                  self.assertEqual(window.horizon, 7)
```

Executa self.assertEqual(window.horizon, 7). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: window.horizon; posicional 2: 7.

```
0029                  self.assertTrue(df.iloc[window.target].observed.all())
```

Executa self.assertTrue(df.iloc[window.target].observed.all()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: df.iloc[window.target].observed.all().

df.iloc[window.target].observed.all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

```
0030                  self.assertLess(df.iloc[window.history].timestamp.max(),
    df.iloc[window.target].timestamp.min())
```

Executa self.assertLess(df.iloc[window.history].timestamp.max(), df.iloc[window.target].timestamp.min()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertLess — Exige first < second. Argumentos usados: posicional 1: df.iloc[window.history].timestamp.max(); posicional 2: df.iloc[window.target].timestamp.min().

df.iloc[window.history].timestamp.max — Retorna maior valor. Argumentos usados: nenhum argumento explícito.

df.iloc[window.target].timestamp.min — Retorna menor valor. Argumentos usados: nenhum argumento explícito.

```
0031              self.assertTrue(all(b.origin-a.origin == 7 for a,b in zip(windows,windows[1:])))
```

Executa self.assertTrue(all((b.origin - a.origin == 7 for a, b in zip(windows, windows[1:])))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: all((b.origin - a.origin == 7 for a, b in zip(windows, windows[1:]))).

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (b.origin - a.origin == 7 for a, b in zip(windows, windows[1:])).

zip — Emparelha iteráveis pela posição. Argumentos usados: posicional 1: windows; posicional 2: windows[1:].

```
0033      def test_future_target_mutation_does_not_change_prompt(self):
```

Define test_future_target_mutation_does_not_change_prompt(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0034          df = pd.DataFrame({'timestamp': pd.date_range('2020-01-01',periods=30),
```

Atribui a df o resultado de pd.DataFrame({'timestamp': pd.date_range('2020-01-01', periods=30), 'water_produced': np.arange(30, dtype=float)}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.date_range('2020-01-01', periods=30), 'water_produced': np.arange(30, dtype=float)}.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-01'; periods: 30.

```
0035                                'water_produced': np.arange(30,dtype=float)})
```

Define 'water_produced': campo do registro/coleção em construção; recebe np.arange(30, dtype=float). Os valores são avaliados nesta janela/iteração.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 30; dtype: float.

```
0036          before = build_prompt(df.iloc[:20],7)
```

Atribui a before o resultado de build_prompt(df.iloc[:20], 7). A expressão à direita é avaliada antes da atribuição.

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: df.iloc[:20]; posicional 2: 7.

```
0037          df.loc[20:, 'water_produced'] = 987654321
```

Atribui a df.loc[20:, 'water_produced'] o resultado de 987654321. A expressão à direita é avaliada antes da atribuição.

```
0038          self.assertEqual(before,build_prompt(df.iloc[:20],7))
```

Executa self.assertEqual(before, build_prompt(df.iloc[:20], 7)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: before; posicional 2: build_prompt(df.iloc[:20], 7).

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: df.iloc[:20]; posicional 2: 7.

```
0039          self.assertNotIn('987654321',before)
```

Executa self.assertNotIn('987654321', before). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: '987654321'; posicional 2: before.

```
0040          self.assertIn('two thousand and twenty, day twenty-one of January',before)
```

Executa self.assertIn('two thousand and twenty, day twenty-one of January', before). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIn — Exige item presente em container. Argumentos usados: posicional 1: 'two thousand and twenty, day twenty-one of January'; posicional 2: before.

```
0042      def test_demonstrations_remain_before_frozen_test(self):
```

Define test_demonstrations_remain_before_frozen_test(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0043          for strategy in ['few_random','few_similar']:
```

Percorre ['few_random', 'few_similar']; em cada iteração associa o item a strategy. O corpo indentado aplica a operação a esse item/grupo.

```
0044              examples = select_demonstrations(np.arange(80.), np.arange(20.), 7,
    strategy=strategy)
```

Atribui a examples o resultado de select_demonstrations(np.arange(80.0), np.arange(20.0), 7, strategy=strategy). A expressão à direita é avaliada antes da atribuição.

select_demonstrations — training_values: prefixo congelado anterior ao primeiro teste; history: demanda histórica corrente; horizon: tamanho de alvo dos exemplos; strategy: zero/few_random/few_similar; count=3: quantidade; seed=42: sorteio. Retorna lista de Demonstration. Argumentos usados: posicional 1: np.arange(80.0); posicional 2: np.arange(20.0); posicional 3: 7; strategy: strategy.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 80.0.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 20.0.

```
0045              self.assertEqual(len(examples),3)
```

Executa self.assertEqual(len(examples), 3). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(examples); posicional 2: 3.

len — Conta elementos. Argumentos usados: posicional 1: examples.

```
0046              self.assertTrue(all(e.target_end <= 80 for e in examples))
```

Executa self.assertTrue(all((e.target_end <= 80 for e in examples))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: all((e.target_end <= 80 for e in examples)).

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (e.target_end <= 80 for e in examples).

```
0047              self.assertTrue(all(max(e.target)<80 for e in examples))
```

Executa self.assertTrue(all((max(e.target) < 80 for e in examples))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: all((max(e.target) < 80 for e in examples)).

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (max(e.target) < 80 for e in examples).

max — Retorna maior valor. Argumentos usados: posicional 1: e.target.

```
0049      def test_missing_day_is_filled_from_past_only(self):
```

Define test_missing_day_is_filled_from_past_only(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0050          df = pd.DataFrame({'timestamp':pd.to_datetime(['2020-01-01','2020-01-03']),
    'water_produced':[10.,999.]})
```

Atribui a df o resultado de pd.DataFrame({'timestamp': pd.to_datetime(['2020-01-01', '2020-01-03']), 'water_produced': [10.0, 999.0]}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.to_datetime(['2020-01-01', '2020-01-03']), 'water_produced': [10.0, 999.0]}.

pd.to_datetime — Converte valores em datas pandas. Argumentos usados: posicional 1: ['2020-01-01', '2020-01-03'].

```
0051          result = fill_past_only(daily_grid(df))
```

Atribui a result o resultado de fill_past_only(daily_grid(df)). A expressão à direita é avaliada antes da atribuição.

fill_past_only — df: grade com possíveis NaN. Retorno: cópia preenchida com passado. Não reconstrói atributos de calendário, limitação conhecida. Argumentos usados: posicional 1: daily_grid(df).

daily_grid — df: tabela com timestamp. Retorno: grade diária com observed. Dias ausentes são expostos, não removidos. Argumentos usados: posicional 1: df.

```
0052          self.assertEqual(result.water_produced.iloc[1],10.)
```

Executa self.assertEqual(result.water_produced.iloc[1], 10.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: result.water_produced.iloc[1]; posicional 2: 10.0.

```
0053          self.assertFalse(result.observed.iloc[1])
```

Executa self.assertFalse(result.observed.iloc[1]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertFalse — Asserção de condição falsa. Argumentos usados: posicional 1: result.observed.iloc[1].

```
0055      def test_imputed_test_days_not_scored(self):
```

Define test_imputed_test_days_not_scored(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0056          df = pd.DataFrame({'timestamp':pd.date_range('2020-01-01',periods=30),
```

Atribui a df o resultado de pd.DataFrame({'timestamp': pd.date_range('2020-01-01', periods=30), 'water_produced': np.arange(30.0), 'observed': True}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.date_range('2020-01-01', periods=30), 'water_produced': np.arange(30.0), 'observed': True}.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-01'; periods: 30.

```
0057                                'water_produced':np.arange(30.), 'observed':True})
```

Define 'water_produced': campo do registro/coleção em construção; recebe np.arange(30.0); 'observed': campo do registro/coleção em construção; recebe True. Os valores são avaliados nesta janela/iteração.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 30.0.

```
0058          df.loc[12,'observed'] = False
```

Atribui a df.loc[12, 'observed'] o resultado de False. A expressão à direita é avaliada antes da atribuição.

```
0059          windows=build_windows(df,10,horizon=7,step=7)
```

Atribui a windows o resultado de build_windows(df, 10, horizon=7, step=7). A expressão à direita é avaliada antes da atribuição.

build_windows — df: tabela preparada; context: primeira origem; horizon=7: quantidade prevista; step=7: avanço entre origens. Retorna lista ForecastWindow, excluindo janelas com alvo imputado. Argumentos usados: posicional 1: df; posicional 2: 10; horizon: 7; step: 7.

```
0060          self.assertTrue(all(not(w.origin<=12<w.origin+7) for w in windows))
```

Executa self.assertTrue(all((not w.origin <= 12 < w.origin + 7 for w in windows))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: all((not w.origin <= 12 < w.origin + 7 for w in windows)).

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (not w.origin <= 12 < w.origin + 7 for w in windows).

```
0062      def test_covariates_and_history_grow_without_future_targets(self):
```

Define test_covariates_and_history_grow_without_future_targets(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0063          config = json.loads((ROOT / 'configs/benchmark.json').read_text())
```

Atribui a config o resultado de json.loads((ROOT / 'configs/benchmark.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (ROOT / 'configs/benchmark.json').read_text().

(ROOT / 'configs/benchmark.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0064         for dataset in config['datasets'].values():
```

Percorre config['datasets'].values(); em cada iteração associa o item a dataset. O corpo indentado aplica a operação a esse item/grupo.

config['datasets'].values — Percorre valores do dicionário. Argumentos usados: nenhum argumento explícito.

```
0065             df, windows = prepare_dataset(ROOT / dataset['path'], dataset['context'])
```

Atribui a (df, windows) o resultado de prepare_dataset(ROOT / dataset['path'], dataset['context']). A expressão à direita é avaliada antes da atribuição.

prepare_dataset — path aponta o CSV; context define a primeira origem; horizon/step definem grade de avaliação; textual_calendar escolhe interpretação de datas em inglês. Retorna tabela preparada e lista de ForecastWindow. Argumentos usados: posicional 1: ROOT / dataset['path']; posicional 2: dataset['context'].

```
0066             validate_covariates(df, dataset)
```

Executa validate_covariates(df, dataset). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

validate_covariates — df: tabela preparada; dataset: contrato de colunas. Retorna None; falha em ausentes/NaN/números não finitos. Argumentos usados: posicional 1: df; posicional 2: dataset.

```
0067             self.assertEqual(windows[0].origin, dataset['context'])
```

Executa self.assertEqual(windows[0].origin, dataset['context']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: windows[0].origin; posicional 2: dataset['context'].

```
0068             columns = covariate_columns(dataset)
```

Atribui a columns o resultado de covariate_columns(dataset). A expressão à direita é avaliada antes da atribuição.

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0069             for offset, window in enumerate(windows[:3]):
```

Percorre enumerate(windows[:3]); em cada iteração associa o item a (offset, window). O corpo indentado aplica a operação a esse item/grupo.

enumerate — Percorre iterável com contador. Argumentos usados: posicional 1: windows[:3].

```
0070                 history, future = model_inputs(df, window, dataset)
```

Atribui a (history, future) o resultado de model_inputs(df, window, dataset). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: df; posicional 2: window; posicional 3: dataset.

```
0071                 self.assertEqual(len(history), dataset['context'] + offset*7)
```

Executa self.assertEqual(len(history), dataset['context'] + offset * 7). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(history); posicional 2: dataset['context'] + offset * 7.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0072                 self.assertEqual(history.timestamp.iloc[0], df.timestamp.iloc[0])
```

Executa self.assertEqual(history.timestamp.iloc[0], df.timestamp.iloc[0]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: history.timestamp.iloc[0]; posicional 2: df.timestamp.iloc[0].

```
0073                 self.assertEqual(list(future.columns), ['timestamp'] + columns)
```

Executa self.assertEqual(list(future.columns), ['timestamp'] + columns). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: list(future.columns); posicional 2: ['timestamp'] + columns.

list — Materializa um iterável em lista. Argumentos usados: posicional 1: future.columns.

```
0074                 self.assertNotIn('water_produced', future)
```

Executa self.assertNotIn('water_produced', future). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: 'water_produced'; posicional 2: future.

```
0075                 numerical, categorical = timesfm_covariates(history, future, dataset)
```

Atribui a (numerical, categorical) o resultado de timesfm_covariates(history, future, dataset). A expressão à direita é avaliada antes da atribuição.

timesfm_covariates — history/future: tabelas separadas; dataset: listas numéricas/categóricas. Retorna dois dicts com listas de séries cobrindo passado e futuro, sem demanda futura. Argumentos usados: posicional 1: history; posicional 2: future; posicional 3: dataset.

```
0076                 self.assertEqual(set(numerical) | set(categorical), set(columns))
```

Executa self.assertEqual(set(numerical) | set(categorical), set(columns)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: set(numerical) | set(categorical); posicional 2: set(columns).

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: columns.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: numerical.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: categorical.

```
0077                 for values in {**numerical, **categorical}.values():
```

Percorre {**numerical, **categorical}.values(); em cada iteração associa o item a values. O corpo indentado aplica a operação a esse item/grupo.

{**numerical, **categorical}.values — Percorre valores do dicionário. Argumentos usados: nenhum argumento explícito.

```
0078                     self.assertEqual(len(values[0]), len(history)+7)
```

Executa self.assertEqual(len(values[0]), len(history) + 7). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(values[0]); posicional 2: len(history) + 7.

len — Conta elementos. Argumentos usados: posicional 1: values[0].

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0079                 short, same_future = model_inputs(df, window, dataset, history_days=15)
```

Atribui a (short, same_future) o resultado de model_inputs(df, window, dataset, history_days=15). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: df; posicional 2: window; posicional 3: dataset; history_days: 15.

```
0080                 self.assertEqual(len(short), 15)
```

Executa self.assertEqual(len(short), 15). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(short); posicional 2: 15.

len — Conta elementos. Argumentos usados: posicional 1: short.

```
0081                 pd.testing.assert_frame_equal(future, same_future)
```

Executa pd.testing.assert_frame_equal(future, same_future). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_frame_equal — Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem. Argumentos usados: posicional 1: future; posicional 2: same_future.

```
0082                 prompt = build_prompt(short, 7, future_covariates=future)
```

Atribui a prompt o resultado de build_prompt(short, 7, future_covariates=future). A expressão à direita é avaliada antes da atribuição.

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: short; posicional 2: 7; future_covariates: future.

```
0083                 self.assertTrue(all(LABELS.get(c, c.replace("_", " ")) in prompt
```

Executa self.assertTrue(all((LABELS.get(c, c.replace('_', ' ')) in prompt for c in calendar_to_text(short).columns if c != 'timestamp'))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: all((LABELS.get(c, c.replace('_', ' ')) in prompt for c in calendar_to_text(short).columns if c != 'timestamp')).

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (LABELS.get(c, c.replace('_', ' ')) in prompt for c in calendar_to_text(short).columns if c != 'timestamp').

LABELS.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: c; posicional 2: c.replace('_', ' ').

c.replace — Substitui o destino pelo arquivo representado pelo Path. Argumentos usados: posicional 1: '_'; posicional 2: ' '.

```
0084                                     for c in calendar_to_text(short).columns if c !=
    "timestamp"))
```

Cláusula de compreensão: percorre os valores indicados para construir a coleção da instrução anterior, mantendo a ordem desse percurso.

calendar_to_text — frame pode conter calendário numérico ou já convertido. Devolve cópia com calendário inglês, mescla pares cíclicos e preserva demanda/meteorologia. Não ajusta estatísticas nem altera o argumento. Argumentos usados: posicional 1: short.

```
0085                 poisoned = df.copy()
```

Atribui a poisoned o resultado de df.copy(). A expressão à direita é avaliada antes da atribuição.

df.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0086                 poisoned.loc[window.origin:, 'water_produced'] = 987654321
```

Atribui a poisoned.loc[window.origin:, 'water_produced'] o resultado de 987654321. A expressão à direita é avaliada antes da atribuição.

```
0087                 other_history, other_future = model_inputs(poisoned, window, dataset,
    history_days=15)
```

Atribui a (other_history, other_future) o resultado de model_inputs(poisoned, window, dataset, history_days=15). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: poisoned; posicional 2: window; posicional 3: dataset; history_days: 15.

```
0088                 self.assertEqual(prompt, build_prompt(other_history, 7,
    future_covariates=other_future))
```

Executa self.assertEqual(prompt, build_prompt(other_history, 7, future_covariates=other_future)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: prompt; posicional 2: build_prompt(other_history, 7, future_covariates=other_future).

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: other_history; posicional 2: 7; future_covariates: other_future.

```
0090      def test_future_target_column_is_rejected(self):
```

Define test_future_target_column_is_rejected(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0091          history = pd.DataFrame({'timestamp': pd.date_range('2020-01-01', periods=3),
    'water_produced':[1.,2.,3.]})
```

Atribui a history o resultado de pd.DataFrame({'timestamp': pd.date_range('2020-01-01', periods=3), 'water_produced': [1.0, 2.0, 3.0]}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.date_range('2020-01-01', periods=3), 'water_produced': [1.0, 2.0, 3.0]}.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-01'; periods: 3.

```
0092          future = pd.DataFrame({'timestamp':pd.date_range('2020-01-04',periods=7),
    'water_produced':99.})
```

Atribui a future o resultado de pd.DataFrame({'timestamp': pd.date_range('2020-01-04', periods=7), 'water_produced': 99.0}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.date_range('2020-01-04', periods=7), 'water_produced': 99.0}.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-04'; periods: 7.

```
0093          with self.assertRaisesRegex(ValueError, 'Future target'):
```

Abre o contexto gerenciado self.assertRaisesRegex(ValueError, 'Future target'). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaisesRegex — Exige exceção e mensagem compatível com expressão regular. Argumentos usados: posicional 1: ValueError; posicional 2: 'Future target'.

```
0094              build_prompt(history, 7, future_covariates=future)
```

Executa build_prompt(history, 7, future_covariates=future). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: history; posicional 2: 7; future_covariates: future.

```
0096      def test_chronos_receives_entire_growing_history_and_covariates(self):
```

Define test_chronos_receives_entire_growing_history_and_covariates(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0097          from src.models.chronos import ChronosForecaster
```

Importa ChronosForecaster de src.models.chronos. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0098          from unittest.mock import Mock
```

Importa Mock de unittest.mock. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0099          config = json.loads((ROOT / 'configs/benchmark.json').read_text())
```

Atribui a config o resultado de json.loads((ROOT / 'configs/benchmark.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (ROOT / 'configs/benchmark.json').read_text().

(ROOT / 'configs/benchmark.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0100          dataset = config['datasets']['joinville']
```

Atribui a dataset o resultado de config['datasets']['joinville']. A expressão à direita é avaliada antes da atribuição.

```
0101          df, windows = prepare_dataset(ROOT / dataset['path'], dataset['context'])
```

Atribui a (df, windows) o resultado de prepare_dataset(ROOT / dataset['path'], dataset['context']). A expressão à direita é avaliada antes da atribuição.

prepare_dataset — path aponta o CSV; context define a primeira origem; horizon/step definem grade de avaliação; textual_calendar escolhe interpretação de datas em inglês. Retorna tabela preparada e lista de ForecastWindow. Argumentos usados: posicional 1: ROOT / dataset['path']; posicional 2: dataset['context'].

```
0102          adapter = object.__new__(ChronosForecaster)
```

Atribui a adapter o resultado de object.__new__(ChronosForecaster). A expressão à direita é avaliada antes da atribuição.

object.__new__ — Cria instância sem executar seu __init__. Argumentos usados: posicional 1: ChronosForecaster.

```
0103          adapter.dataset = dataset
```

Atribui a adapter.dataset o resultado de dataset. A expressão à direita é avaliada antes da atribuição.

```
0104          adapter.pipeline = Mock()
```

Atribui a adapter.pipeline o resultado de Mock(). A expressão à direita é avaliada antes da atribuição.

Mock — Cria objeto simulado para registrar chamadas. Argumentos usados: nenhum argumento explícito.

```
0105          adapter.pipeline.predict_df.return_value = pd.DataFrame({'0.5': [1.]*7})
```

Atribui a adapter.pipeline.predict_df.return_value o resultado de pd.DataFrame({'0.5': [1.0] * 7}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'0.5': [1.0] * 7}.

```
0106          for window in windows[:3]:
```

Percorre windows[:3]; em cada iteração associa o item a window. O corpo indentado aplica a operação a esse item/grupo.

```
0107              history, future = model_inputs(df, window, dataset)
```

Atribui a (history, future) o resultado de model_inputs(df, window, dataset). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: df; posicional 2: window; posicional 3: dataset.

```
0108              adapter.predict(history, 7, future_covariates=future)
```

Executa adapter.predict(history, 7, future_covariates=future). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

adapter.predict — Interface comum dos adaptadores do projeto. Argumentos usados: posicional 1: history; posicional 2: 7; future_covariates: future.

```
0109              call = adapter.pipeline.predict_df.call_args
```

Atribui a call o resultado de adapter.pipeline.predict_df.call_args. A expressão à direita é avaliada antes da atribuição.

```
0110              self.assertEqual(call.kwargs['context_length'], window.origin)
```

Executa self.assertEqual(call.kwargs['context_length'], window.origin). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: call.kwargs['context_length']; posicional 2: window.origin.

```
0111              self.assertEqual(len(call.args[0]), window.origin)
```

Executa self.assertEqual(len(call.args[0]), window.origin). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(call.args[0]); posicional 2: window.origin.

len — Conta elementos. Argumentos usados: posicional 1: call.args[0].

```
0112              self.assertNotIn('water_produced', call.kwargs['future_df'])
```

Executa self.assertNotIn('water_produced', call.kwargs['future_df']). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: 'water_produced'; posicional 2: call.kwargs['future_df'].

```
0113
    self.assertTrue(set(covariate_columns(dataset)).issubset(call.kwargs['future_df']))
```

Executa self.assertTrue(set(covariate_columns(dataset)).issubset(call.kwargs['future_df'])). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: set(covariate_columns(dataset)).issubset(call.kwargs['future_df']).

set(covariate_columns(dataset)).issubset — Verifica se todos os elementos estão contidos em outro conjunto/iterável. Argumentos usados: posicional 1: call.kwargs['future_df'].

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: covariate_columns(dataset).

covariate_columns — dataset: bloco de configuração de uma cidade. Retorna nomes numéricos seguidos dos categóricos, após rejeitar duplicatas e nomes reservados. Argumentos usados: posicional 1: dataset.

```
0115      def test_invalid_model_responses(self):
```

Define test_invalid_model_responses(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0116          for text in ['{"predictions":[true]}','{"predictions":["1"]}',
```

Percorre ['{"predictions":[true]}', '{"predictions":["1"]}', '{"predictions":[NaN]}', '{"predictions":[-1]}', '{"predictions":[1,2]}', '[1]', 'answer: 1', '{"predictions":[1],"extra":0}']; em cada iteração associa o item a text. O corpo indentado aplica a operação a esse item/grupo.

```
0117                        '{"predictions":[NaN]}','{"predictions":[-1]}',
```

Continuação da instrução na linha 116: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0118                        '{"predictions":[1,2]}','[1]','answer: 1',
```

Continuação da instrução na linha 116: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0119                        '{"predictions":[1],"extra":0}']:
```

Continuação da instrução na linha 116: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0120              with self.subTest(text=text), self.assertRaises((ValueError,TypeError)):
```

Abre o contexto gerenciado self.subTest(text=text), self.assertRaises((ValueError, TypeError)). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.subTest — Identifica um caso dentro de um loop de teste. Argumentos usados: text: text.

self.assertRaises — Context manager que exige uma exceção. Argumentos usados: posicional 1: (ValueError, TypeError).

```
0121                  parse_prediction(text,1)
```

Executa parse_prediction(text, 1). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parse_prediction — text: string JSON do modelo; horizon: quantidade exata de valores. Retorna list[float] ou lança erro; não remove prose nem aceita resposta parcial. Argumentos usados: posicional 1: text; posicional 2: 1.

```
0122          self.assertEqual(parse_prediction('{"predictions":[1.5]}',1),[1.5])
```

Executa self.assertEqual(parse_prediction('{"predictions":[1.5]}', 1), [1.5]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: parse_prediction('{"predictions":[1.5]}', 1); posicional 2: [1.5].

parse_prediction — text: string JSON do modelo; horizon: quantidade exata de valores. Retorna list[float] ou lança erro; não remove prose nem aceita resposta parcial. Argumentos usados: posicional 1: '{"predictions":[1.5]}'; posicional 2: 1.

```
0124      def test_retry_and_raw_response_audit(self):
```

Define test_retry_and_raw_response_audit(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0125          model = object.__new__(QwenForecaster)
```

Atribui a model o resultado de object.__new__(QwenForecaster). A expressão à direita é avaliada antes da atribuição.

object.__new__ — Cria instância sem executar seu __init__. Argumentos usados: posicional 1: QwenForecaster.

```
0126          model.config = OllamaConfig(retries=1)
```

Atribui a model.config o resultado de OllamaConfig(retries=1). A expressão à direita é avaliada antes da atribuição.

OllamaConfig — Constrói configuração tipada do cliente Qwen. Argumentos usados: retries: 1.

```
0127          model.last_audit = {}
```

Atribui a model.last_audit o resultado de {}. A expressão à direita é avaliada antes da atribuição.

```
0128          answers=[{'done':True,'response':'{}'},
    {'done':True,'response':'{"predictions":[2]}'}]
```

Atribui a answers o resultado de [{'done': True, 'response': '{}'}, {'done': True, 'response': '{"predictions":[2]}'}]. A expressão à direita é avaliada antes da atribuição.

```
0129          with patch('src.models.promptcast.post_json',side_effect=answers),
    patch('src.models.promptcast.time.sleep'):
```

Abre o contexto gerenciado patch('src.models.promptcast.post_json', side_effect=answers), patch('src.models.promptcast.time.sleep'). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

patch — Substitui temporariamente uma referência durante o bloco with. Argumentos usados: posicional 1: 'src.models.promptcast.post_json'; side_effect: answers.

patch — Substitui temporariamente uma referência durante o bloco with. Argumentos usados: posicional 1: 'src.models.promptcast.time.sleep'.

```
0130              self.assertEqual(model.predict(None,1,prompt='history'),[2.])
```

Executa self.assertEqual(model.predict(None, 1, prompt='history'), [2.0]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: model.predict(None, 1, prompt='history'); posicional 2: [2.0].

model.predict — Interface comum dos adaptadores do projeto. Argumentos usados: posicional 1: None; posicional 2: 1; prompt: 'history'.

```
0131          self.assertEqual(len(model.last_audit['attempts']),2)
```

Executa self.assertEqual(len(model.last_audit['attempts']), 2). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(model.last_audit['attempts']); posicional 2: 2.

len — Conta elementos. Argumentos usados: posicional 1: model.last_audit['attempts'].

```
0132          self.assertIn('error',model.last_audit['attempts'][0])
```

Executa self.assertIn('error', model.last_audit['attempts'][0]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIn — Exige item presente em container. Argumentos usados: posicional 1: 'error'; posicional 2: model.last_audit['attempts'][0].

```
0134      def test_context_budget_rejects_silent_truncation(self):
```

Define test_context_budget_rejects_silent_truncation(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0135          model=object.__new__(QwenForecaster)
```

Atribui a model o resultado de object.__new__(QwenForecaster). A expressão à direita é avaliada antes da atribuição.

object.__new__ — Cria instância sem executar seu __init__. Argumentos usados: posicional 1: QwenForecaster.

```
0136          model.config=OllamaConfig(num_ctx=800)
```

Atribui a model.config o resultado de OllamaConfig(num_ctx=800). A expressão à direita é avaliada antes da atribuição.

OllamaConfig — Constrói configuração tipada do cliente Qwen. Argumentos usados: num_ctx: 800.

```
0137          with self.assertRaises(ValueError):
```

Abre o contexto gerenciado self.assertRaises(ValueError). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaises — Context manager que exige uma exceção. Argumentos usados: posicional 1: ValueError.

```
0138              model.predict(None,7,prompt='x'*1000)
```

Executa model.predict(None, 7, prompt='x' * 1000). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

model.predict — Interface comum dos adaptadores do projeto. Argumentos usados: posicional 1: None; posicional 2: 7; prompt: 'x' * 1000.

```
0140      def test_metric_units(self):
```

Define test_metric_units(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0141          metrics=deterministic_metrics([10.,20.],[12.,18.])
```

Atribui a metrics o resultado de deterministic_metrics([10.0, 20.0], [12.0, 18.0]). A expressão à direita é avaliada antes da atribuição.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R² e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: [10.0, 20.0]; posicional 2: [12.0, 18.0].

```
0142          self.assertEqual(metrics['mae'],2.)
```

Executa self.assertEqual(metrics['mae'], 2.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: metrics['mae']; posicional 2: 2.0.

```
0143          self.assertEqual(metrics['rmse'],2.)
```

Executa self.assertEqual(metrics['rmse'], 2.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: metrics['rmse']; posicional 2: 2.0.

```
0144          self.assertAlmostEqual(metrics['mape_percent'],15.)
```

Executa self.assertAlmostEqual(metrics['mape_percent'], 15.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertAlmostEqual — Compara números com aproximação decimal. Argumentos usados: posicional 1: metrics['mape_percent']; posicional 2: 15.0.

```
0145          self.assertAlmostEqual(metrics['r2'],0.84)
```

Executa self.assertAlmostEqual(metrics['r2'], 0.84). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertAlmostEqual — Compara números com aproximação decimal. Argumentos usados: posicional 1: metrics['r2']; posicional 2: 0.84.

```
0146
    self.assertEqual(deterministic_metrics([0.,10.],[0.,12.])['mape_zero_targets_excluded'],1)
```

Executa self.assertEqual(deterministic_metrics([0.0, 10.0], [0.0, 12.0])['mape_zero_targets_excluded'], 1). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: deterministic_metrics([0.0, 10.0], [0.0, 12.0])['mape_zero_targets_excluded']; posicional 2: 1.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R² e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: [0.0, 10.0]; posicional 2: [0.0, 12.0].

```
0147         with self.assertRaises(ValueError):
```

Abre o contexto gerenciado self.assertRaises(ValueError). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaises — Context manager que exige uma exceção. Argumentos usados: posicional 1: ValueError.

```
0148             deterministic_metrics([1.,2.],[1.,float('nan')])
```

Executa deterministic_metrics([1.0, 2.0], [1.0, float('nan')]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R² e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: [1.0, 2.0]; posicional 2: [1.0, float('nan')].

float — Converte escalar para float Python. Argumentos usados: posicional 1: 'nan'.

```
0150     def test_validation_uses_training_prefix(self):
```

Define test_validation_uses_training_prefix(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0151         train,validation=validation_split(100)
```

Atribui a (train, validation) o resultado de validation_split(100). A expressão à direita é avaliada antes da atribuição.

validation_split — training_size: comprimento do prefixo anterior ao teste; fraction=0.2: parte final reservada à validação. Retorna dois slices sem embaralhar. Argumentos usados: posicional 1: 100.

```
0152
    self.assertEqual((train.start,train.stop,validation.start,validation.stop),(0,80,80,100))
```

Executa self.assertEqual((train.start, train.stop, validation.start, validation.stop), (0, 80, 80, 100)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: (train.start, train.stop, validation.start, validation.stop); posicional 2: (0, 80, 80, 100).

```
0154     def test_duplicate_dates_rejected(self):
```

Define test_duplicate_dates_rejected(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0155         with self.assertRaises(ValueError):
```

Abre o contexto gerenciado self.assertRaises(ValueError). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaises — Context manager que exige uma exceção. Argumentos usados: posicional 1: ValueError.

```
0156
    validate_series(pd.DataFrame({'timestamp':['2020-01-01']*2,'water_produced':[1,2]}))
```

Executa validate_series(pd.DataFrame({'timestamp': ['2020-01-01'] * 2, 'water_produced': [1, 2]})). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

validate_series — df: tabela lida. Retorno: cópia ordenada. Rejeita esquema, datas e demanda inválidos antes de qualquer modelo. Argumentos usados: posicional 1: pd.DataFrame({'timestamp': ['2020-01-01'] * 2, 'water_produced': [1, 2]}).

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': ['2020-01-01'] * 2, 'water_produced': [1, 2]}.

```
0159  if __name__ == '__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0160      unittest.main()
```

Executa unittest.main(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

unittest.main — Executa testes quando o arquivo é ponto de entrada. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

48. tests/test_qwen_fixed_context.py

Testes de regressão. São executados por unittest discover, não durante a inferência.

SHA-256: 9a4df35cc00ebe64eff1dcab089fc708719ae2c27000e4f5990080ec9c72ce19

Funções, contratos e pontos de conexão

```
L15: test_invalid_context_rejected_before_loading_data_or_model(self)
```

Caso de teste: invalid context rejected before loading data or model. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: run_dataset → src/pipelines/forecasting_pipeline.py:52

```
L25: test_fixed_context_preserves_evaluation_dates_and_target_isolation(self)
```

Caso de teste: fixed context preserves evaluation dates and target isolation. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: ForecastWindow → src/evaluation/cross_validation.py:6; model_inputs → src/data/features/covariates.py:25; qwen_history_days → src/pipelines/forecasting_pipeline.py:44

APIs externas usadas neste arquivo

ForecastWindow, Path, assertEqual, assertLess, assertNotIn, assertRaisesRegex, assert_not_called, len, max, min, np.arange, patch, pd.DataFrame, pd.date_range, pd.testing.assert_frame_equal, subTest. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """LLM history stays fixed while evaluation origins advance."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import unittest
```

Importa unittest. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from unittest.mock import patch
```

Importa patch de unittest.mock. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  from src.data.features.covariates import model_inputs
```

Importa model_inputs de src.data.features.covariates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.evaluation.cross_validation import ForecastWindow
```

Importa ForecastWindow de src.evaluation.cross_validation. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  from src.pipelines.forecasting_pipeline import qwen_history_days, run_dataset
```

Importa qwen_history_days, run_dataset de src.pipelines.forecasting_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0014  class QwenFixedContextTests(unittest.TestCase):
```

Declara a classe QwenFixedContextTests(unittest.TestCase). Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0015      def test_invalid_context_rejected_before_loading_data_or_model(self):
```

Define test_invalid_context_rejected_before_loading_data_or_model(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0016          for config in [{}, *({'qwen_history_days': value} for value in
```

Percorre [{}, *({'qwen_history_days': value} for value in (None, 0, 30, 365, 730, True, 7.0, '15'))]; em cada iteração associa o item a config. O corpo indentado aplica a operação a esse item/grupo.

```
0017                                (None, 0, 30, 365, 730, True, 7.0, '15'))]:
```

Continuação da instrução na linha 16: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0018              with self.subTest(config=config),
    patch('src.pipelines.forecasting_pipeline.prepare_dataset') as prepare, \
```

Abre o contexto gerenciado self.subTest(config=config), patch('src.pipelines.forecasting_pipeline.prepare_dataset') como prepare, patch('src.pipelines.forecasting_pipeline.create_model') como create. Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.subTest — Identifica um caso dentro de um loop de teste. Argumentos usados: config: config.

patch — Substitui temporariamente uma referência durante o bloco with. Argumentos usados: posicional 1: 'src.pipelines.forecasting_pipeline.prepare_dataset'.

```
0019                      patch('src.pipelines.forecasting_pipeline.create_model') as create:
```

Continuação da instrução na linha 18: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

patch — Substitui temporariamente uma referência durante o bloco with. Argumentos usados: posicional 1: 'src.pipelines.forecasting_pipeline.create_model'.

```
0020                  with self.assertRaisesRegex(ValueError, 'must be 7 or 15'):
```

Abre o contexto gerenciado self.assertRaisesRegex(ValueError, 'must be 7 or 15'). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

self.assertRaisesRegex — Exige exceção e mensagem compatível com expressão regular. Argumentos usados: posicional 1: ValueError; posicional 2: 'must be 7 or 15'.

```
0021                      run_dataset('joinville', {}, config, 'qwen', Path('unused'))
```

Executa run_dataset('joinville', {}, config, 'qwen', Path('unused')). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

run_dataset — name, dataset e config identificam cidade e parâmetros; model_name seleciona adaptador; output define pasta; limit só limita testes de fumaça. Valida contexto, executa janelas, salva previsões e status; retorna número de falhas. Argumentos usados: posicional 1: 'joinville'; posicional 2: {}; posicional 3: config; posicional 4: 'qwen'; posicional 5: Path('unused').

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: 'unused'.

```
0022                  prepare.assert_not_called()
```

Executa prepare.assert_not_called(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

prepare.assert_not_called — Verifica que um mock não foi chamado. Argumentos usados: nenhum argumento explícito.

```
0023                  create.assert_not_called()
```

Executa create.assert_not_called(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

create.assert_not_called — Verifica que um mock não foi chamado. Argumentos usados: nenhum argumento explícito.

```
0025     def test_fixed_context_preserves_evaluation_dates_and_target_isolation(self):
```

Define test_fixed_context_preserves_evaluation_dates_and_target_isolation(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0026         frame = pd.DataFrame({'timestamp': pd.date_range('2020-01-01', periods=70),
```

Atribui a frame o resultado de pd.DataFrame({'timestamp': pd.date_range('2020-01-01', periods=70), 'water_produced': np.arange(70.0)}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': pd.date_range('2020-01-01', periods=70), 'water_produced': np.arange(70.0)}.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-01'; periods: 70.

```
0027                                'water_produced': np.arange(70.)})
```

Define 'water_produced': campo do registro/coleção em construção; recebe np.arange(70.0). Os valores são avaliados nesta janela/iteração.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 70.0.

```
0028         dataset = {'numerical_covariates': [], 'categorical_covariates': []}
```

Atribui a dataset o resultado de {'numerical_covariates': [], 'categorical_covariates': []}. A expressão à direita é avaliada antes da atribuição.

```
0029         for origin in (30, 37, 44):
```

Percorre (30, 37, 44); em cada iteração associa o item a origin. O corpo indentado aplica a operação a esse item/grupo.

```
0030             window = ForecastWindow(0, origin, 7)
```

Atribui a window o resultado de ForecastWindow(0, origin, 7). A expressão à direita é avaliada antes da atribuição.

ForecastWindow — Constrói dataclass de uma origem temporal. Argumentos usados: posicional 1: 0; posicional 2: origin; posicional 3: 7.

```
0031             baseline, expected_future = model_inputs(frame, window, dataset)
```

Atribui a (baseline, expected_future) o resultado de model_inputs(frame, window, dataset). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: frame; posicional 2: window; posicional 3: dataset.

```
0032             self.assertEqual(len(baseline), origin)
```

Executa self.assertEqual(len(baseline), origin). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(baseline); posicional 2: origin.

len — Conta elementos. Argumentos usados: posicional 1: baseline.

```
0033             for days in (7, 15):
```

Percorre (7, 15); em cada iteração associa o item a days. O corpo indentado aplica a operação a esse item/grupo.

```
0034                 validated = qwen_history_days({'qwen_history_days': days})
```

Atribui a validated o resultado de qwen_history_days({'qwen_history_days': days}). A expressão à direita é avaliada antes da atribuição.

qwen_history_days — config precisa conter inteiro exato 7 ou 15. Ausência, None, strings, bool, floats e outros comprimentos são rejeitados antes de carregar dados/modelo. Argumentos usados: posicional 1: {'qwen_history_days': days}.

```
0035                 history, future = model_inputs(frame, window, dataset,
    history_days=validated)
```

Atribui a (history, future) o resultado de model_inputs(frame, window, dataset, history_days=validated). A expressão à direita é avaliada antes da atribuição.

model_inputs — df: série completa; window: ForecastWindow; dataset: contrato; history_days: None para expansivo ou inteiro para últimos N dias. Retorna (history, future), excluindo alvo do futuro. Argumentos usados: posicional 1: frame; posicional 2: window; posicional 3: dataset; history_days: validated.

```
0036                 self.assertEqual(len(history), days)
```

Executa self.assertEqual(len(history), days). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(history); posicional 2: days.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0037                 self.assertEqual(history.timestamp.iloc[0], frame.timestamp.iloc[origin-
    days])
```

Executa self.assertEqual(history.timestamp.iloc[0], frame.timestamp.iloc[origin - days]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: history.timestamp.iloc[0]; posicional 2: frame.timestamp.iloc[origin - days].

```
0038                 self.assertLess(history.timestamp.max(), future.timestamp.min())
```

Executa self.assertLess(history.timestamp.max(), future.timestamp.min()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertLess — Exige first < second. Argumentos usados: posicional 1: history.timestamp.max(); posicional 2: future.timestamp.min().

history.timestamp.max — Retorna maior valor. Argumentos usados: nenhum argumento explícito.

future.timestamp.min — Retorna menor valor. Argumentos usados: nenhum argumento explícito.

```
0039                 self.assertNotIn('water_produced', future)
```

Executa self.assertNotIn('water_produced', future). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: 'water_produced'; posicional 2: future.

```
0040                 pd.testing.assert_frame_equal(future, expected_future)
```

Executa pd.testing.assert_frame_equal(future, expected_future). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_frame_equal — Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem. Argumentos usados: posicional 1: future; posicional 2: expected_future.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

49. tests/test_seven_day_metrics.py

Testes de regressão. São executados por unittest discover, não durante a inferência.

SHA-256: fff364c2855fb53d1cbdec0bb8e08dea740b2b6c3df90c5e8cfaed79e43b8c63

Funções, contratos e pontos de conexão

```
L14: test_comparison_includes_errors_after_day_two(self)
```

Caso de teste: comparison includes errors after day two. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: compare → src/pipelines/compare_models.py:10

APIs externas usadas neste arquivo

Path, assertEqual, contextlib.redirect_stdout, io.StringIO, json.dumps, len, mkdir, np.sqrt, np.testing.assert_allclose, pd.DataFrame, pd.date_range, pd.read_csv, range, set, strftime, tempfile.TemporaryDirectory, to_csv, write_text. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Regression test: days 3–7 must contribute to the actual comparison CSV."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import contextlib
```

Importa contextlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import io
```

Importa io. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import tempfile
```

Importa tempfile. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  import unittest
```

Importa unittest. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.pipelines.compare_models import compare
```

Importa compare de src.pipelines.compare_models. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0013  class SevenDayMetricsTests(unittest.TestCase):
```

Declara a classe SevenDayMetricsTests(unittest.TestCase). Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0014      def test_comparison_includes_errors_after_day_two(self):
```

Define test_comparison_includes_errors_after_day_two(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0015          with tempfile.TemporaryDirectory() as temporary:
```

Abre o contexto gerenciado tempfile.TemporaryDirectory() como temporary. Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

tempfile.TemporaryDirectory — Cria diretório temporário gerenciado. Argumentos usados: nenhum argumento explícito.

```
0016              root = Path(temporary)
```

Atribui a root o resultado de Path(temporary). A expressão à direita é avaliada antes da atribuição.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: temporary.

```
0017              for dataset, initial in [('joinville', 365), ('guaratuba', 730)]:
```

Percorre [('joinville', 365), ('guaratuba', 730)]; em cada iteração associa o item a (dataset, initial). O corpo indentado aplica a operação a esse item/grupo.

```
0018                  for model in ['qwen', 'chronos', 'timesfm']:
```

Percorre ['qwen', 'chronos', 'timesfm']; em cada iteração associa o item a model. O corpo indentado aplica a operação a esse item/grupo.

```
0019                      folder = root / dataset / model
```

Atribui a folder o resultado de root / dataset / model. A expressão à direita é avaliada antes da atribuição.

```
0020                      folder.mkdir(parents=True)
```

Executa folder.mkdir(parents=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

folder.mkdir — Cria diretório. Argumentos usados: parents: True.

```
0021                      run = {'limit': 0, 'qwen_history_days': 15,
```

Atribui a run o resultado de {'limit': 0, 'qwen_history_days': 15, 'config': {'strategy': 'zero'}, 'dataset_sha256': dataset, 'source_sha256': 'same-code', 'n_windows': 1, 'model': {'model': model}, 'covariates': ['temperature']}. A expressão à direita é avaliada antes da atribuição.

```
0022                              'config': {'strategy': 'zero'}, 'dataset_sha256': dataset,
```

Define 'config': campo do registro/coleção em construção; recebe {'strategy': 'zero'}; 'dataset_sha256': hash dos bytes do CSV de entrada; recebe dataset; 'strategy': campo do registro/coleção em construção; recebe 'zero'. Os valores são avaliados nesta janela/iteração.

```
0023                              'source_sha256': 'same-code', 'n_windows': 1,
```

Define 'source_sha256': hash de todos os .py sob src; recebe 'same-code'; 'n_windows': campo do registro/coleção em construção; recebe 1. Os valores são avaliados nesta janela/iteração.

```
0024                              'model': {'model': model}, 'covariates': ['temperature']}
```

Define 'model': identidade do checkpoint/adaptador; recebe {'model': model}; 'covariates': nomes das variáveis auxiliares; recebe ['temperature']; 'model': identidade do checkpoint/adaptador; recebe model. Os valores são avaliados nesta janela/iteração.

```
0025                      status = {'failures': [], 'successful_windows': 1, 'expected_windows': 1}
```

Atribui a status o resultado de {'failures': [], 'successful_windows': 1, 'expected_windows': 1}. A expressão à direita é avaliada antes da atribuição.

```
0026                      (folder/'run.json').write_text(json.dumps(run))
```

Executa (folder / 'run.json').write_text(json.dumps(run)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

(folder / 'run.json').write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: json.dumps(run).

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: run.

```
0027                     (folder/'status.json').write_text(json.dumps(status))
```

Executa (folder / 'status.json').write_text(json.dumps(status)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

(folder / 'status.json').write_text — Grava uma string em arquivo e retorna a quantidade de caracteres escrita. Argumentos usados: posicional 1: json.dumps(status).

json.dumps — Serializa objeto Python em string JSON. Argumentos usados: posicional 1: status.

```
0028                     pd.DataFrame({
```

Executa pd.DataFrame({'origin': ['2020-01-01'] * 7, 'timestamp': pd.date_range('2020-01-01', periods=7).strftime('%Y-%m-%d'), 'lead': range(1, 8), 'y_true': [10.0, 20.0, 30.0, 40.0, 50.0, 60.0, 70.0], 'y_pred': [10.0, 20.0, 40.0, 50.0, 60.0, 70.0, 80.0], 'initial_context': [initial] * 7, 'context': [15 if model == 'qwen' else initial] * 7, 'history_start': ['2018-01-01'] * 7, 'horizon': [7] * 7, 'elapsed_seconds_window': [1.0] * 7}).to_csv(folder / 'predictions.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame({'origin': ['2020-01-01'] * 7, 'timestamp': pd.date_range('2020-01-01', periods=7).strftime('%Y-%m-%d'), 'lead': range(1, 8), 'y_true': [10.0, 20.0, 30.0, 40.0, 50.0, 60.0, 70.0], 'y_pred': [10.0, 20.0, 40.0, 50.0, 60.0, 70.0, 80.0], 'initial_context': [initial] * 7, 'context': [15 if model == 'qwen' else initial] * 7, 'history_start': ['2018-01-01'] * 7, 'horizon': [7] * 7, 'elapsed_seconds_window': [1.0] * 7}).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: folder / 'predictions.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'origin': ['2020-01-01'] * 7, 'timestamp': pd.date_range('2020-01-01', periods=7).strftime('%Y-%m-%d'), 'lead': range(1, 8), 'y_true': [10.0, 20.0, 30.0, 40.0, 50.0, 60.0, 70.0], 'y_pred': [10.0, 20.0, 40.0, 50.0, 60.0, 70.0, 80.0], 'initial_context': [initial] * 7, 'context': [15 if model == 'qwen' else initial] * 7, 'history_start': ['2018-01-01'] * 7, 'horizon': [7] * 7, 'elapsed_seconds_window': [1.0] * 7}.

```
0029                         'origin': ['2020-01-01']*7,
```

Define 'origin': data de início do horizonte previsto; recebe ['2020-01-01'] * 7. Os valores são avaliados nesta janela/iteração.

```
0030                         'timestamp': pd.date_range('2020-01-01',
    periods=7).strftime('%Y-%m-%d'),
```

Define 'timestamp': campo do registro/coleção em construção; recebe pd.date_range('2020-01-01', periods=7).strftime('%Y-%m-%d'). Os valores são avaliados nesta janela/iteração.

pd.date_range('2020-01-01', periods=7).strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2020-01-01'; periods: 7.

```
0031                         'lead': range(1,8), 'y_true': [10.,20.,30.,40.,50.,60.,70.],
```

Define 'lead': posição dentro do horizonte: 1 a 7 no Qwen ou 1 a 30 nos baselines clássicos; recebe range(1, 8); 'y_true': demanda observada usada somente na avaliação; recebe [10.0, 20.0, 30.0, 40.0, 50.0, 60.0, 70.0]. Os valores são avaliados nesta janela/iteração.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 1; posicional 2: 8.

```
0032                         'y_pred': [10.,20.,40.,50.,60.,70.,80.],
```

Define 'y_pred': valor retornado pelo modelo para a data correspondente; recebe [10.0, 20.0, 40.0, 50.0, 60.0, 70.0, 80.0]. Os valores são avaliados nesta janela/iteração.

```
0033                         'initial_context': [initial]*7,
```

Define 'initial_context': origem inicial do protocolo de avaliação; recebe [initial] * 7. Os valores são avaliados nesta janela/iteração.

```
0034                         'context': [15 if model == 'qwen' else initial]*7,
```

Define 'context': número de linhas históricas efetivamente fornecidas; recebe [15 if model == 'qwen' else initial] * 7. Os valores são avaliados nesta janela/iteração.

```
0035                         'history_start': ['2018-01-01']*7,
```

Define 'history_start': primeira data histórica efetivamente fornecida; recebe ['2018-01-01'] * 7. Os valores são avaliados nesta janela/iteração.

```
0036                         'horizon': [7]*7, 'elapsed_seconds_window': [1.]*7
```

Define 'horizon': campo do registro/coleção em construção; recebe [7] * 7; 'elapsed_seconds_window': duração da janela, repetida em seus sete registros; recebe [1.0] * 7. Os valores são avaliados nesta janela/iteração.

```
0037                     }).to_csv(folder/'predictions.csv', index=False)
```

Continua os argumentos nomeados: index = False. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0038             with contextlib.redirect_stdout(io.StringIO()):
```

Abre o contexto gerenciado contextlib.redirect_stdout(io.StringIO()). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

contextlib.redirect_stdout — Redireciona print temporariamente. Argumentos usados: posicional 1: io.StringIO().

io.StringIO — Buffer de texto em memória com interface de arquivo. Argumentos usados: nenhum argumento explícito.

```
0039                 compare(root)
```

Executa compare(root). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

compare — output contém benchmark completo de três modelos. Verifica mesmas origens/alvos/procedência, contexto Qwen7/15 e histórias crescentes dos baselines; grava métricas. Argumentos usados: posicional 1: root.

```
0040             aggregate = pd.read_csv(root/'metrics_comparison.csv')
```

Atribui a aggregate o resultado de pd.read_csv(root / 'metrics_comparison.csv'). A expressão à direita é avaliada antes da atribuição.

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: root / 'metrics_comparison.csv'.

```
0041             by_day = pd.read_csv(root/'metrics_by_horizon.csv')
```

Atribui a by_day o resultado de pd.read_csv(root / 'metrics_by_horizon.csv'). A expressão à direita é avaliada antes da atribuição.

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: root / 'metrics_by_horizon.csv'.

```
0042             self.assertEqual(len(aggregate), 6)
```

Executa self.assertEqual(len(aggregate), 6). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(aggregate); posicional 2: 6.

len — Conta elementos. Argumentos usados: posicional 1: aggregate.

```
0043             self.assertEqual(len(by_day), 42)
```

Executa self.assertEqual(len(by_day), 42). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(by_day); posicional 2: 42.

len — Conta elementos. Argumentos usados: posicional 1: by_day.

```
0044             np.testing.assert_allclose(aggregate.rmse, np.sqrt(500/7))
```

Executa np.testing.assert_allclose(aggregate.rmse, np.sqrt(500 / 7)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

np.testing.assert_allclose — Asserção numérica vetorial tolerante. Argumentos usados: posicional 1: aggregate.rmse; posicional 2: np.sqrt(500 / 7).

np.sqrt — Raiz quadrada elemento a elemento ou escalar. Argumentos usados: posicional 1: 500 / 7.

```
0045             np.testing.assert_allclose(by_day.loc[by_day.lead <= 2, 'rmse'], 0.)
```

Executa np.testing.assert_allclose(by_day.loc[by_day.lead <= 2, 'rmse'], 0.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

np.testing.assert_allclose — Asserção numérica vetorial tolerante. Argumentos usados: posicional 1: by_day.loc[by_day.lead <= 2, 'rmse']; posicional 2: 0.0.

```
0046              np.testing.assert_allclose(by_day.loc[by_day.lead >= 3, 'rmse'], 10.)
```

Executa np.testing.assert_allclose(by_day.loc[by_day.lead >= 3, 'rmse'], 10.0). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

np.testing.assert_allclose — Asserção numérica vetorial tolerante. Argumentos usados: posicional 1: by_day.loc[by_day.lead >= 3, 'rmse']; posicional 2: 10.0.

```
0047              self.assertEqual(set(by_day.lead), set(range(1,8)))
```

Executa self.assertEqual(set(by_day.lead), set(range(1, 8))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: set(by_day.lead); posicional 2: set(range(1, 8)).

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: by_day.lead.

set — Cria conjunto de elementos únicos. Argumentos usados: posicional 1: range(1, 8).

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: 1; posicional 2: 8.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

50. tests/test_text_dataset_input.py

Testes de regressão. São executados por unittest discover, não durante a inferência.

SHA-256: 227240a66a7ee45875cedbf2797e8b2087e50ad4d8b8c88bbe848fe41fd58ed5

Funções, contratos e pontos de conexão

```
L15: test_text_roundtrip_and_prompt_target_isolation(self)
```

Caso de teste: text roundtrip and prompt target isolation. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: build_prompt → src/data/preprocessing/generate_prompts.py:96; calendar_to_text → src/data/preprocessing/calendar_text.py:98; read_series → src/data/ingestion/files.py:8

```
L36: test_date_parser_preserves_year(self)
```

Caso de teste: date parser preserves year. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: date_from_text → src/data/preprocessing/calendar_text.py:70; date_to_text → src/data/preprocessing/calendar_text.py:79

APIs externas usadas neste arquivo

Path, assertEqual, assertIn, assertNotIn, drop, np.arange, pd.DataFrame, pd.Timestamp, pd.date_range, pd.testing.assert_frame_equal, pd.testing.assert_series_equal, tempfile.TemporaryDirectory, to_csv. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Direct textual CSV ingestion must preserve calendar meaning and split boundaries."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import tempfile
```

Importa tempfile. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import unittest
```

Importa unittest. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  from src.data.ingestion.files import read_series
```

Importa read_series de src.data.ingestion.files. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.data.preprocessing.calendar_text import calendar_to_text, date_from_text,
    date_to_text
```

Importa calendar_to_text, date_from_text, date_to_text de src.data.preprocessing.calendar_text. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  from src.data.preprocessing.generate_prompts import build_prompt
```

Importa build_prompt de src.data.preprocessing.generate_prompts. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0014  class TextDatasetTests(unittest.TestCase):
```

Declara a classe TextDatasetTests(unittest.TestCase). Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0015      def test_text_roundtrip_and_prompt_target_isolation(self):
```

Define test_text_roundtrip_and_prompt_target_isolation(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0016          dates = pd.date_range('2016-12-29', periods=10)
```

Atribui a dates o resultado de pd.date_range('2016-12-29', periods=10). A expressão à direita é avaliada antes da atribuição.

pd.date_range — Produz uma sequência ordenada de timestamps. Argumentos usados: posicional 1: '2016-12-29'; periods: 10.

```
0017          original = pd.DataFrame({'timestamp': dates, 'water_produced': np.arange(10.)+100,
```

Atribui a original o resultado de pd.DataFrame({'timestamp': dates, 'water_produced': np.arange(10.0) + 100, 'day': dates.day, 'is_holiday': [False] * 10}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'timestamp': dates, 'water_produced': np.arange(10.0) + 100, 'day': dates.day, 'is_holiday': [False] * 10}.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 10.0.

```
0018                                    'day': dates.day, 'is_holiday': [False]*10})
```

Define 'day': campo do registro/coleção em construção; recebe dates.day; 'is_holiday': campo do registro/coleção em construção; recebe [False] * 10. Os valores são avaliados nesta janela/iteração.

```
0019          words = calendar_to_text(original)
```

Atribui a words o resultado de calendar_to_text(original). A expressão à direita é avaliada antes da atribuição.

calendar_to_text — frame pode conter calendário numérico ou já convertido. Devolve cópia com calendário inglês, mescla pares cíclicos e preserva demanda/meteorologia. Não ajusta estatísticas nem altera o argumento. Argumentos usados: posicional 1: original.

```
0020          pd.testing.assert_frame_equal(words, calendar_to_text(words))
```

Executa pd.testing.assert_frame_equal(words, calendar_to_text(words)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_frame_equal — Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem. Argumentos usados: posicional 1: words; posicional 2: calendar_to_text(words).

calendar_to_text — frame pode conter calendário numérico ou já convertido. Devolve cópia com calendário inglês, mescla pares cíclicos e preserva demanda/meteorologia. Não ajusta estatísticas nem altera o argumento. Argumentos usados: posicional 1: words.

```
0021          with tempfile.TemporaryDirectory() as directory:
```

Abre o contexto gerenciado tempfile.TemporaryDirectory() como directory. Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

tempfile.TemporaryDirectory — Cria diretório temporário gerenciado. Argumentos usados: nenhum argumento explícito.

```
0022              path = Path(directory) / 'text.csv'
```

Atribui a path o resultado de Path(directory) / 'text.csv'. A expressão à direita é avaliada antes da atribuição.

Path — Representa caminho de arquivo com operações portáveis. Argumentos usados: posicional 1: directory.

```
0023              words.to_csv(path, index=False)
```

Executa words.to_csv(path, index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

words.to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: path; index: False.

```
0024             frame = read_series(path, textual_calendar=True)
```

Atribui a frame o resultado de read_series(path, textual_calendar=True). A expressão à direita é avaliada antes da atribuição.

read_series — path é o CSV; textual_calendar=False lê datas usuais; True aplica date_from_text à coluna timestamp. Retorna DataFrame validado e ordenado. Calendário categórico continua textual. Argumentos usados: posicional 1: path; textual_calendar: True.

```
0025         pd.testing.assert_series_equal(frame.timestamp, original.timestamp)
```

Executa pd.testing.assert_series_equal(frame.timestamp, original.timestamp). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_series_equal — Exige igualdade de duas Series pandas. Argumentos usados: posicional 1: frame.timestamp; posicional 2: original.timestamp.

```
0026         self.assertEqual(frame.day.iloc[0], 'twenty-nine')
```

Executa self.assertEqual(frame.day.iloc[0], 'twenty-nine'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: frame.day.iloc[0]; posicional 2: 'twenty-nine'.

```
0027         future = frame.iloc[3:].drop(columns='water_produced')
```

Atribui a future o resultado de frame.iloc[3:].drop(columns='water_produced'). A expressão à direita é avaliada antes da atribuição.

frame.iloc[3:].drop — Retorna tabela sem as linhas/colunas indicadas. Argumentos usados: columns: 'water_produced'.

```
0028         prompt = build_prompt(frame.iloc[:3], 7, future_covariates=future,
    region='Guaratuba')
```

Atribui a prompt o resultado de build_prompt(frame.iloc[:3], 7, future_covariates=future, region='Guaratuba'). A expressão à direita é avaliada antes da atribuição.

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: frame.iloc[:3]; posicional 2: 7; future_covariates: future; region: 'Guaratuba'.

```
0029         self.assertIn('twenty-nine; thirty; thirty-one', prompt)
```

Executa self.assertIn('twenty-nine; thirty; thirty-one', prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIn — Exige item presente em container. Argumentos usados: posicional 1: 'twenty-nine; thirty; thirty-one'; posicional 2: prompt.

```
0030         self.assertIn('not a public holiday', prompt)
```

Executa self.assertIn('not a public holiday', prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertIn — Exige item presente em container. Argumentos usados: posicional 1: 'not a public holiday'; posicional 2: prompt.

```
0031         self.assertNotIn('2016-12-29', prompt)
```

Executa self.assertNotIn('2016-12-29', prompt). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertNotIn — Exige item ausente de container. Argumentos usados: posicional 1: '2016-12-29'; posicional 2: prompt.

```
0032         frame.loc[3:, 'water_produced'] = 999999
```

Atribui a frame.loc[3:, 'water_produced'] o resultado de 999999. A expressão à direita é avaliada antes da atribuição.

```
0033         self.assertEqual(prompt, build_prompt(frame.iloc[:3], 7,
```

Executa self.assertEqual(prompt, build_prompt(frame.iloc[:3], 7, future_covariates=frame.iloc[3:].drop(columns='water_produced'), region='Guaratuba')). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: prompt; posicional 2: build_prompt(frame.iloc[:3], 7, future_covariates=frame.iloc[3:].drop(columns='water_produced'), region='Guaratuba').

build_prompt — history deve conter dias consecutivos e demanda finita. horizon deve ser positivo; future_covariates não pode conter water_produced e precisa coincidir com o horizonte. region aplica contexto de Guaratuba. Retorna uma string narrativa com uma pergunta por dia e schema JSON. Argumentos usados: posicional 1: frame.iloc[:3]; posicional 2: 7; future_covariates: frame.iloc[3:].drop(columns='water_produced'); region: 'Guaratuba'.

```
0034                          future_covariates=frame.iloc[3:].drop(columns='water_produced'),
    region='Guaratuba'))
```

Continua os argumentos nomeados: future_covariates = frame.iloc[3:].drop(columns='water_produced'); region = 'Guaratuba'; columns = 'water_produced'. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

frame.iloc[3:].drop — Retorna tabela sem as linhas/colunas indicadas. Argumentos usados: columns: 'water_produced'.

```
0036      def test_date_parser_preserves_year(self):
```

Define test_date_parser_preserves_year(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0037          for date in ('2016-01-01', '2026-01-01', '2020-02-29'):
```

Percorre ('2016-01-01', '2026-01-01', '2020-02-29'); em cada iteração associa o item a date. O corpo indentado aplica a operação a esse item/grupo.

```
0038              self.assertEqual(date_from_text(date_to_text(date)), pd.Timestamp(date))
```

Executa self.assertEqual(date_from_text(date_to_text(date)), pd.Timestamp(date)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: date_from_text(date_to_text(date)); posicional 2: pd.Timestamp(date).

date_from_text — text é a data no formato exato do exportador. Valida com fullmatch, converte ano/dia e retorna Timestamp; utilizado somente na organização temporal dos inputs. Argumentos usados: posicional 1: date_to_text(date).

pd.Timestamp — Cria instante temporal pandas. Argumentos usados: posicional 1: date.

date_to_text — value é uma data ou sua representação textual canônica. Retorna ano, dia e mês em inglês sem mudar o ano original. Argumentos usados: posicional 1: date.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

51. configs/classical_baselines.json

Define séries, protocolo e grades dos três baselines univariados.

SHA-256: 4dbeb6d02cdc3f61c41d5f381b44e068860fa5578498da8dbcc5b951f0ec2557

Código comentado, linha a linha

```
0001  {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0002    "datasets": {
```

Mapeamento dos dois datasets; cada bloco fornece caminho, primeira origem e contrato de covariáveis. JSON é lido por json.loads; não executa chamadas por si só.

```
0003      "joinville": {
```

Bloco da série de Joinville. Contexto inicial de avaliação: 365 dias; total atual 54 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0004        "path": "data/processed/joinville_processed_1day.csv"
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0005      },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0006      "guaratuba": {
```

Bloco da série de Guaratuba. Contexto inicial de avaliação: 730 dias; total atual 104 janelas completas. JSON é lido por json.loads; não executa chamadas por si só.

```
0007        "path": "data/processed/guaratuba_processed_1daysin.csv"
```

Caminho relativo à raiz do projeto. Os arquivos qwen_text apontam os CSVs dentro de data/processed/calendar_text. JSON é lido por json.loads; não executa chamadas por si só.

```
0008      }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0009    },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0010    "initial_training": 355,
```

Treino inicial de seleção: 355 dias, exclusivo dos baselines clássicos. JSON é lido por json.loads; não executa chamadas por si só.

```
0011    "validation_days": 30,
```

Bloco cronológico de 30 dias para seleção de parâmetros. JSON é lido por json.loads; não executa chamadas por si só.

```
0012    "test_days": 30,
```

Horizonte de teste de 30 dias, sem atualização com verdade interna. JSON é lido por json.loads; não executa chamadas por si só.

```
0013    "step_days": 30,
```

Avanço 30, obrigatoriamente igual ao teste neste pipeline. JSON é lido por json.loads; não executa chamadas por si só.

```
0014    "refit_train_validation": true,
```

Exige reajuste final no prefixo treino+validação anterior ao teste. JSON é lido por json.loads; não executa chamadas por si só.

```
0015    "inputs": "univariate historical water_produced; no future weather",
```

Campo declarativo do arquivo JSON. JSON é lido por json.loads; não executa chamadas por si só.

```
0016    "candidates": {
```

Grade explícita selecionada exclusivamente pela validação de cada fold. JSON é lido por json.loads; não executa chamadas por si só.

```
0017      "linear_regression": [
```

Campo declarativo do arquivo JSON. JSON é lido por json.loads; não executa chamadas por si só.

```
0018        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0019          "lags": 7
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0020        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0021        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0022          "lags": 15
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0023        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0024        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0025          "lags": 30
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0026        }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0027      ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0028      "knn": [
```

Campo declarativo do arquivo JSON. JSON é lido por json.loads; não executa chamadas por si só.

```
0029        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0030          "lags": 7,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0031          "neighbors": 3,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0032          "weights": "uniform"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0033        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0034        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0035          "lags": 7,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0036          "neighbors": 3,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0037          "weights": "distance"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0038        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0039        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0040          "lags": 7,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0041          "neighbors": 5,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0042          "weights": "uniform"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0043        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0044        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0045          "lags": 7,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0046          "neighbors": 5,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0047          "weights": "distance"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0048        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0049        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0050          "lags": 7,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0051          "neighbors": 10,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0052          "weights": "uniform"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0053        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0054        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0055          "lags": 7,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0056          "neighbors": 10,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0057          "weights": "distance"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0058        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0059        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0060          "lags": 15,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0061          "neighbors": 3,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0062          "weights": "uniform"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0063        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0064        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0065          "lags": 15,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0066          "neighbors": 3,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0067          "weights": "distance"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0068        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0069        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0070          "lags": 15,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0071          "neighbors": 5,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0072          "weights": "uniform"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0073        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0074        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0075          "lags": 15,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0076          "neighbors": 5,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0077          "weights": "distance"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0078        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0079        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0080          "lags": 15,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0081          "neighbors": 10,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0082          "weights": "uniform"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0083        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0084        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0085          "lags": 15,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0086          "neighbors": 10,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0087          "weights": "distance"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0088        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0089        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0090          "lags": 30,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0091          "neighbors": 3,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0092          "weights": "uniform"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0093        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0094        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0095          "lags": 30,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0096          "neighbors": 3,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0097          "weights": "distance"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0098        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0099        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0100          "lags": 30,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0101          "neighbors": 5,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0102          "weights": "uniform"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0103        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0104        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0105          "lags": 30,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0106          "neighbors": 5,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0107          "weights": "distance"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0108        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0109        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0110          "lags": 30,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0111          "neighbors": 10,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0112          "weights": "uniform"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0113        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0114        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0115          "lags": 30,
```

Número de demandas passadas por exemplo supervisionado. JSON é lido por json.loads; não executa chamadas por si só.

```
0116          "neighbors": 10,
```

Número de vizinhos do KNN. JSON é lido por json.loads; não executa chamadas por si só.

```
0117          "weights": "distance"
```

uniform atribui pesos iguais; distance pondera por distância. JSON é lido por json.loads; não executa chamadas por si só.

```
0118        }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0119     ],
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0120     "arima": [
```

Campo declarativo do arquivo JSON. JSON é lido por json.loads; não executa chamadas por si só.

```
0121       {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0122         "order": [
```

Ordem (p,d,q) do ARIMA não sazonal. JSON é lido por json.loads; não executa chamadas por si só.

```
0123           1,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0124           0,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0125           0
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0126         ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0127       },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0128       {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0129         "order": [
```

Ordem (p,d,q) do ARIMA não sazonal. JSON é lido por json.loads; não executa chamadas por si só.

```
0130           2,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0131           0,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0132           0
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0133         ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0134       },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0135       {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0136         "order": [
```

Ordem (p,d,q) do ARIMA não sazonal. JSON é lido por json.loads; não executa chamadas por si só.

```
0137           1,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0138           0,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0139           1
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0140         ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0141       },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0142       {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0143         "order": [
```

Ordem (p,d,q) do ARIMA não sazonal. JSON é lido por json.loads; não executa chamadas por si só.

```
0144           2,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0145           0,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0146           1
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0147         ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0148       },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0149        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0150          "order": [
```

Ordem (p,d,q) do ARIMA não sazonal. JSON é lido por json.loads; não executa chamadas por si só.

```
0151            1,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0152            1,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0153            0
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0154          ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0155        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0156        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0157          "order": [
```

Ordem (p,d,q) do ARIMA não sazonal. JSON é lido por json.loads; não executa chamadas por si só.

```
0158            2,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0159            1,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0160            0
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0161          ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0162        },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0163        {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0164         "order": [
```

Ordem (p,d,q) do ARIMA não sazonal. JSON é lido por json.loads; não executa chamadas por si só.

```
0165           0,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0166           1,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0167           1
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0168         ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0169       },
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0170       {
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0171         "order": [
```

Ordem (p,d,q) do ARIMA não sazonal. JSON é lido por json.loads; não executa chamadas por si só.

```
0172           1,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0173           1,
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0174           1
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0175         ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0176       }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0177     ]
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0178   }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

```
0179  }
```

Delimitador JSON: abre/fecha um objeto ou lista e organiza os campos do bloco. As vírgulas separam elementos; null é ausência de valor JSON; qwen_history_days não aceita esse valor.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

52. src/data/features/autoregressive.py

Constrói atrasos anteriores ao alvo e previsão recursiva sem observações do horizonte.

SHA-256: adb2b0e234b2614da64e7c7f84d9e62cf944dca61f466d7f3d9813ae0e73a8ea

Funções, contratos e pontos de conexão

```
L5: lagged_samples(values, lags: int)
```

values é o histórico autorizado; lags é o número de atrasos. Devolve X com lags colunas e y com o próximo valor, sem atributos posteriores ao alvo.

```
L13: recursive_forecast(estimator, history, horizon: int, lags: int)
```

Recebe estimador ajustado, histórico permitido, horizonte e atrasos. Acrescenta apenas previsões durante o horizonte e devolve um vetor finito.

Chamadas do projeto: estimator.predict → src/models/promptcast.py:71; estimator.predict → src/models/chronos.py:18; estimator.predict → src/models/timesfm.py:20; estimator.predict → src/models/classical.py:31; estimator.predict → src/models/classical.py:51

APIs externas usadas neste arquivo

ValueError, all, append, copy, float, len, list, np.asarray, np.isfinite, np.lib.stride_tricks.sliding_window_view, predict, range, reshape. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Lag features derived exclusively from observations preceding each target."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  def lagged_samples(values, lags: int):
```

Define lagged_samples(values, lags: int). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0006      values = np.asarray(values, dtype=float)
```

Atribui a values o resultado de np.asarray(values, dtype=float). A expressão à direita é avaliada antes da atribuição.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: values; dtype: float.

```
0007      if lags < 1 or len(values) <= lags or not np.isfinite(values).all():
```

Avalia a condição lags < 1 or len(values) <= lags or (not np.isfinite(values).all()). O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: values.

np.isfinite(values).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: values.

```
0008          raise ValueError('Finite history longer than a positive lag count is required')
```

Interrompe o caminho atual lançando ValueError('Finite history longer than a positive lag count is required'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Finite history longer than a positive lag count is required'.

```
0009      windows = np.lib.stride_tricks.sliding_window_view(values, lags + 1)
```

Atribui a windows o resultado de np.lib.stride_tricks.sliding_window_view(values, lags + 1). A expressão à direita é avaliada antes da atribuição.

np.lib.stride_tricks.sliding_window_view — Cria janelas consecutivas sobre um array. Argumentos usados: posicional 1: values; posicional 2: lags + 1.

```
0010      return windows[:, :-1].copy(), windows[:, -1].copy()
```

Encerra esta função e devolve (windows[:, :-1].copy(), windows[:, -1].copy()) ao chamador; não imprime o resultado por si só.

windows[:, :-1].copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

windows[:, -1].copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0013  def recursive_forecast(estimator, history, horizon: int, lags: int):
```

Define recursive_forecast(estimator, history, horizon: int, lags: int). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0014      """Append predictions, never real values from inside the forecast horizon."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0015      history = list(np.asarray(history, dtype=float))
```

Atribui a history o resultado de list(np.asarray(history, dtype=float)). A expressão à direita é avaliada antes da atribuição.

list — Materializa um iterável em lista. Argumentos usados: posicional 1: np.asarray(history, dtype=float).

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: history; dtype: float.

```
0016      if horizon < 1 or len(history) < lags:
```

Avalia a condição horizon < 1 or len(history) < lags. O bloco indentado seguinte só é executado quando ela é verdadeira.

len — Conta elementos. Argumentos usados: posicional 1: history.

```
0017          raise ValueError('Insufficient history or invalid horizon')
```

Interrompe o caminho atual lançando ValueError('Insufficient history or invalid horizon'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Insufficient history or invalid horizon'.

```
0018      predictions = []
```

Atribui a predictions o resultado de []. A expressão à direita é avaliada antes da atribuição.

```
0019      for _ in range(horizon):
```

Percorre range(horizon); em cada iteração associa o item a _. O corpo indentado aplica a operação a esse item/grupo.

range — Gera inteiros com limite final exclusivo. Argumentos usados: posicional 1: horizon.

```
0020          value = float(estimator.predict(np.asarray(history[-lags:]).reshape(1, -1))[0])
```

Atribui a value o resultado de float(estimator.predict(np.asarray(history[-lags:]).reshape(1, -1))[0]). A expressão à direita é avaliada antes da atribuição.

float — Converte escalar para float Python. Argumentos usados: posicional 1: estimator.predict(np.asarray(history[-lags:]).reshape(1, -1))[0].

estimator.predict — Interface comum dos adaptadores do projeto. Argumentos usados: posicional 1: np.asarray(history[-lags:]).reshape(1, -1).

np.asarray(history[-lags:]).reshape — Muda a forma de um array. Argumentos usados: posicional 1: 1; posicional 2: -1.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: history[-lags:].

```
0021          if not np.isfinite(value):
```

Avalia a condição not np.isfinite(value). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: value.

```
0022              raise ValueError('Nonfinite recursive forecast')
```

Interrompe o caminho atual lançando ValueError('Nonfinite recursive forecast'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Nonfinite recursive forecast'.

```
0023          predictions.append(value)
```

Executa predictions.append(value). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

predictions.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: value.

```
0024          history.append(value)
```

Executa history.append(value). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

history.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: value.

```
0025      return np.asarray(predictions)
```

Encerra esta função e devolve np.asarray(predictions) ao chamador; não imprime o resultado por si só.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: predictions.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

53. src/models/classical.py

Adapta regressão linear, KNN e ARIMA ao contrato fit(history)/predict(horizon).

SHA-256: 20b137bb97dea2f6649093127e1694ec9d8cdc9df9eb28ec157c13bc8bb16bd4

Funções, contratos e pontos de conexão

```
L15: __init__(self, name: str, *, lags: int [obrigatório], neighbors = 5, weights = 'uniform')
```

Construtor da classe: self recebe a instância e os demais argumentos inicializam configuração, backend ou estado. Confira o corpo abaixo para I/O e carregamento de modelo.

```
L25: fit(self, history)
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

Chamadas do projeto: lagged_samples → src/data/features/autoregressive.py:5

```
L31: predict(self, horizon: int)
```

Recebe apenas horizon: número de passos futuros. Usa o estado ajustado; não recebe demanda real futura. Retorna vetor na escala da demanda.

Chamadas do projeto: recursive_forecast → src/data/features/autoregressive.py:13

```
L36: __init__(self, order)
```

Construtor da classe: self recebe a instância e os demais argumentos inicializam configuração, backend ou estado. Confira o corpo abaixo para I/O e carregamento de modelo.

```
L39: fit(self, history)
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

```
L51: predict(self, horizon: int)
```

Recebe apenas horizon: número de passos futuros. Usa o estado ajustado; não recebe demanda real futura. Retorna vetor na escala da demanda.

```
L58: create_classical_model(name, parameters)
```

Cria o adaptador ARIMA ou LagRegressor conforme name e os parâmetros selecionados.

Chamadas do projeto: ARIMAForecaster → src/models/classical.py:35; LagRegressor → src/models/classical.py:14

APIs externas usadas neste arquivo

ARIMA, KNeighborsRegressor, LinearRegression, StandardScaler, ValueError, all, copy, float, make_pipeline, max, mean, np.asarray, np.isfinite, std, tuple, warnings.catch_warnings, warnings.simplefilter. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Univariate classical baselines with one explicit fit/predict contract."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import warnings
```

Importa warnings. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  from sklearn.linear_model import LinearRegression
```

Importa LinearRegression de sklearn.linear_model. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  from sklearn.neighbors import KNeighborsRegressor
```

Importa KNeighborsRegressor de sklearn.neighbors. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  from sklearn.pipeline import make_pipeline
```

Importa make_pipeline de sklearn.pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  from sklearn.preprocessing import StandardScaler
```

Importa StandardScaler de sklearn.preprocessing. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  from statsmodels.tsa.arima.model import ARIMA
```

Importa ARIMA de statsmodels.tsa.arima.model. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  from statsmodels.tools.sm_exceptions import ConvergenceWarning
```

Importa ConvergenceWarning de statsmodels.tools.sm_exceptions. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  from src.data.features.autoregressive import lagged_samples, recursive_forecast
```

Importa lagged_samples, recursive_forecast de src.data.features.autoregressive. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0014  class LagRegressor:
```

Declara a classe LagRegressor. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0015      def __init__(self, name: str, *, lags: int, neighbors=5, weights='uniform'):
```

Define __init__(self, name: str, *, lags: int [obrigatório], neighbors = 5, weights = 'uniform'). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0016          self.lags = lags
```

Atribui a self.lags o resultado de lags. A expressão à direita é avaliada antes da atribuição.

```
0017          if name == 'linear_regression':
```

Avalia a condição name == 'linear_regression'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0018              regressor = LinearRegression()
```

Atribui a regressor o resultado de LinearRegression(). A expressão à direita é avaliada antes da atribuição.

LinearRegression — Regressão linear por mínimos quadrados. Argumentos usados: nenhum argumento explícito.

```
0019          elif name == 'knn':
```

Avalia a condição name == 'knn'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0020              regressor = KNeighborsRegressor(n_neighbors=neighbors, weights=weights, n_jobs=1)
```

Atribui a regressor o resultado de KNeighborsRegressor(n_neighbors=neighbors, weights=weights, n_jobs=1). A expressão à direita é avaliada antes da atribuição.

KNeighborsRegressor — Regressor por vizinhos mais próximos. Argumentos usados: n_neighbors: neighbors; weights: weights; n_jobs: 1.

```
0021          else:
```

Ramo alternativo da condição imediatamente anterior: executa quando o teste anterior não foi satisfeito.

```
0022              raise ValueError(f'Unknown lag regressor: {name}')
```

Interrompe o caminho atual lançando ValueError(f'Unknown lag regressor: {name}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'Unknown lag regressor: {name}'.

```
0023          self.estimator = make_pipeline(StandardScaler(), regressor)
```

Atribui a self.estimator o resultado de make_pipeline(StandardScaler(), regressor). A expressão à direita é avaliada antes da atribuição.

make_pipeline — Encadeia transformações e estimador. Argumentos usados: posicional 1: StandardScaler(); posicional 2: regressor.

StandardScaler — Padroniza atributos por média e desvio do ajuste. Argumentos usados: nenhum argumento explícito.

```
0025      def fit(self, history):
```

Define fit(self, history). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0026          self.history = np.asarray(history, dtype=float).copy()
```

Atribui a self.history o resultado de np.asarray(history, dtype=float).copy(). A expressão à direita é avaliada antes da atribuição.

np.asarray(history, dtype=float).copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: history; dtype: float.

```
0027          features, targets = lagged_samples(self.history, self.lags)
```

Atribui a (features, targets) o resultado de lagged_samples(self.history, self.lags). A expressão à direita é avaliada antes da atribuição.

lagged_samples — values é o histórico autorizado; lags é o número de atrasos. Devolve X com lags colunas e y com o próximo valor, sem atributos posteriores ao alvo. Argumentos usados: posicional 1: self.history; posicional 2: self.lags.

```
0028          self.estimator.fit(features, targets)
```

Executa self.estimator.fit(features, targets). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.estimator.fit — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: features; posicional 2: targets.

```
0029          return self
```

Encerra esta função e devolve self ao chamador; não imprime o resultado por si só.

```
0031      def predict(self, horizon: int):
```

Define predict(self, horizon: int). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0032          return recursive_forecast(self.estimator, self.history, horizon, self.lags)
```

Encerra esta função e devolve recursive_forecast(self.estimator, self.history, horizon, self.lags) ao chamador; não imprime o resultado por si só.

recursive_forecast — Recebe estimador ajustado, histórico permitido, horizonte e atrasos. Acrescenta apenas previsões durante o horizonte e devolve um vetor finito. Argumentos usados: posicional 1: self.estimator; posicional 2: self.history; posicional 3: horizon; posicional 4: self.lags.

```
0035  class ARIMAForecaster:
```

Declara a classe ARIMAForecaster. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0036      def __init__(self, order):
```

Define __init__(self, order). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0037          self.order = tuple(order)
```

Atribui a self.order o resultado de tuple(order). A expressão à direita é avaliada antes da atribuição.

tuple — Materializa sequência imutável. Argumentos usados: posicional 1: order.

```
0039      def fit(self, history):
```

Define fit(self, history). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0040          values = np.asarray(history, dtype=float)
```

Atribui a values o resultado de np.asarray(history, dtype=float). A expressão à direita é avaliada antes da atribuição.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: history; dtype: float.

```
0041          if not np.isfinite(values).all():
```

Avalia a condição not np.isfinite(values).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.isfinite(values).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: values.

```
0042              raise ValueError('Nonfinite ARIMA history')
```

Interrompe o caminho atual lançando ValueError('Nonfinite ARIMA history'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Nonfinite ARIMA history'.

```
0043          self.mean = float(values.mean())
```

Atribui a self.mean o resultado de float(values.mean()). A expressão à direita é avaliada antes da atribuição.

float — Converte escalar para float Python. Argumentos usados: posicional 1: values.mean().

values.mean — Calcula média dos valores do objeto receptor. Argumentos usados: nenhum argumento explícito.

```
0044          self.scale = max(float(values.std()), 1e-8)
```

Atribui a self.scale o resultado de max(float(values.std()), 1e-08). A expressão à direita é avaliada antes da atribuição.

max — Retorna maior valor. Argumentos usados: posicional 1: float(values.std()); posicional 2: 1e-08.

float — Converte escalar para float Python. Argumentos usados: posicional 1: values.std().

values.std — Desvio-padrão do receptor quando usado como método. Argumentos usados: nenhum argumento explícito.

```
0045          scaled = (values - self.mean) / self.scale
```

Atribui a scaled o resultado de (values - self.mean) / self.scale. A expressão à direita é avaliada antes da atribuição.

```
0046          with warnings.catch_warnings():
```

Abre o contexto gerenciado warnings.catch_warnings(). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

warnings.catch_warnings — Isola temporariamente a política de avisos. Argumentos usados: nenhum argumento explícito.

```
0047              warnings.simplefilter('error', ConvergenceWarning)
```

Executa warnings.simplefilter('error', ConvergenceWarning). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

warnings.simplefilter — Define tratamento de uma categoria de avisos. Argumentos usados: posicional 1: 'error'; posicional 2: ConvergenceWarning.

```
0048              self.result = ARIMA(scaled, order=self.order).fit(method_kwargs={'maxiter': 200})
```

Atribui a self.result o resultado de ARIMA(scaled, order=self.order).fit(method_kwargs={'maxiter': 200}). A expressão à direita é avaliada antes da atribuição.

ARIMA(scaled, order=self.order).fit — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: method_kwargs: {'maxiter': 200}.

ARIMA — Constrói o modelo autorregressivo integrado de médias móveis. Argumentos usados: posicional 1: scaled; order: self.order.

```
0049          return self
```

Encerra esta função e devolve self ao chamador; não imprime o resultado por si só.

```
0051      def predict(self, horizon: int):
```

Define predict(self, horizon: int). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0052          values = np.asarray(self.result.forecast(steps=horizon))*self.scale + self.mean
```

Atribui a values o resultado de np.asarray(self.result.forecast(steps=horizon)) * self.scale + self.mean. A expressão à direita é avaliada antes da atribuição.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: self.result.forecast(steps=horizon).

self.result.forecast — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: steps: horizon.

```
0053          if not np.isfinite(values).all():
```

Avalia a condição not np.isfinite(values).all(). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.isfinite(values).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: values.

```
0054              raise ValueError('Nonfinite ARIMA predictions')
```

Interrompe o caminho atual lançando ValueError('Nonfinite ARIMA predictions'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Nonfinite ARIMA predictions'.

```
0055          return values
```

Encerra esta função e devolve values ao chamador; não imprime o resultado por si só.

```
0058  def create_classical_model(name, parameters):
```

Define create_classical_model(name, parameters). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0059      if name == 'arima':
```

Avalia a condição name == 'arima'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0060          return ARIMAForecaster(**parameters)
```

Encerra esta função e devolve ARIMAForecaster(**parameters) ao chamador; não imprime o resultado por si só.

ARIMAForecaster — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: **desempacotamento: parameters.

```
0061      return LagRegressor(name, **parameters)
```

Encerra esta função e devolve LagRegressor(name, **parameters) ao chamador; não imprime o resultado por si só.

LagRegressor — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: name; **desempacotamento: parameters.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

54. src/training/classical.py

Seleciona parâmetros pelo RMSE de validação e reajusta somente o prefixo anterior ao teste.

SHA-256: fd49843a03486f3cb80c8e39428b2db29174b4a23c5a84d81b7fee41bf7d40b2

Funções, contratos e pontos de conexão

```
L6: select_and_forecast(name, training, validation, horizon, candidates)
```

Recebe nome, treino, validação, horizonte e candidatos, nunca alvos de teste. Seleciona pelo RMSE de validação, reajusta treino+validação e devolve previsões e auditoria.

Chamadas do projeto: create_classical_model → src/models/classical.py:58; create_classical_model(name, best).fit → src/models/classical.py:25; create_classical_model(name, best).fit → src/models/classical.py:39; create_classical_model(name, parameters).fit → src/models/classical.py:25; create_classical_model(name, parameters).fit → src/models/classical.py:39; estimator.predict → src/models/promptcast.py:71; estimator.predict → src/models/chronos.py:18; estimator.predict → src/models/timesfm.py:20; estimator.predict → src/models/classical.py:31; estimator.predict → src/models/classical.py:51; final_model.predict → src/models/promptcast.py:71; final_model.predict → src/models/chronos.py:18; final_model.predict → src/models/timesfm.py:20; final_model.predict → src/models/classical.py:31; final_model.predict → src/models/classical.py:51

APIs externas usadas neste arquivo

ValueError, append, float, len, np.asarray, np.concatenate, np.isfinite, np.mean, np.sqrt, predict, type, update. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Select parameters on validation, then refit only the pre-test prefix."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from src.models.classical import create_classical_model
```

Importa create_classical_model de src.models.classical. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  def select_and_forecast(name, training, validation, horizon, candidates):
```

Define select_and_forecast(name, training, validation, horizon, candidates). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0007      """No test targets are accepted by this interface, including during selection."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0008      train = np.asarray(training, dtype=float)
```

Atribui a train o resultado de np.asarray(training, dtype=float). A expressão à direita é avaliada antes da atribuição.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: training; dtype: float.

```
0009      valid = np.asarray(validation, dtype=float)
```

Atribui a valid o resultado de np.asarray(validation, dtype=float). A expressão à direita é avaliada antes da atribuição.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: validation; dtype: float.

```
0010      trials, best, best_score = [], None, np.inf
```

Atribui a (trials, best, best_score) o resultado de ([], None, np.inf). A expressão à direita é avaliada antes da atribuição.

```
0011      for parameters in candidates:
```

Percorre candidates; em cada iteração associa o item a parameters. O corpo indentado aplica a operação a esse item/grupo.

```
0012          record = {'parameters': parameters}
```

Atribui a record o resultado de {'parameters': parameters}. A expressão à direita é avaliada antes da atribuição.

```
0013          try:
```

Inicia região protegida de execução. Exceções compatíveis serão tratadas pelo except correspondente; sucesso continua normalmente.

```
0014              estimator = create_classical_model(name, parameters).fit(train)
```

Atribui a estimator o resultado de create_classical_model(name, parameters).fit(train). A expressão à direita é avaliada antes da atribuição.

create_classical_model(name, parameters).fit — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: train.

create_classical_model — Cria o adaptador ARIMA ou LagRegressor conforme name e os parâmetros selecionados. Argumentos usados: posicional 1: name; posicional 2: parameters.

```
0015              predictions = estimator.predict(len(valid))
```

Atribui a predictions o resultado de estimator.predict(len(valid)). A expressão à direita é avaliada antes da atribuição.

estimator.predict — Interface comum dos adaptadores do projeto. Argumentos usados: posicional 1: len(valid).

len — Conta elementos. Argumentos usados: posicional 1: valid.

```
0016              rmse = float(np.sqrt(np.mean((predictions-valid)**2)))
```

Atribui a rmse o resultado de float(np.sqrt(np.mean((predictions - valid) ** 2))). A expressão à direita é avaliada antes da atribuição.

float — Converte escalar para float Python. Argumentos usados: posicional 1: np.sqrt(np.mean((predictions - valid) ** 2)).

np.sqrt — Raiz quadrada elemento a elemento ou escalar. Argumentos usados: posicional 1: np.mean((predictions - valid) ** 2).

np.mean — Calcula média aritmética. Argumentos usados: posicional 1: (predictions - valid) ** 2.

```
0017              if not np.isfinite(rmse):
```

Avalia a condição not np.isfinite(rmse). O bloco indentado seguinte só é executado quando ela é verdadeira.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: rmse.

```
0018                  raise ValueError('Nonfinite validation score')
```

Interrompe o caminho atual lançando ValueError('Nonfinite validation score'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Nonfinite validation score'.

```
0019              record.update(status='ok', validation_rmse=rmse)
```

Executa record.update(status='ok', validation_rmse=rmse). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

record.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: status: 'ok'; validation_rmse: rmse.

```
0020              if rmse < best_score:
```

Avalia a condição rmse < best_score. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0021                 best, best_score = parameters, rmse
```

Atribui a (best, best_score) o resultado de (parameters, rmse). A expressão à direita é avaliada antes da atribuição.

```
0022         except (ValueError, np.linalg.LinAlgError, RuntimeError, Warning) as exc:
```

Trata as exceções indicadas na linha e vincula a falha a exc quando declarado. O registro permite diagnosticar a tentativa sem inventar previsões.

```
0023             record.update(status='error', error=f'{type(exc).__name__}: {exc}')
```

Executa record.update(status='error', error=f'{type(exc).__name__}: {exc}'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

record.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: status: 'error'; error: f'{type(exc).__name__}: {exc}'.

type — Obtém a classe concreta do valor. Argumentos usados: posicional 1: exc.

```
0024         trials.append(record)
```

Executa trials.append(record). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

trials.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: record.

```
0025     if best is None:
```

Avalia a condição best is None. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0026         raise ValueError(f'All {name} candidates failed: {trials}')
```

Interrompe o caminho atual lançando ValueError(f'All {name} candidates failed: {trials}'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: f'All {name} candidates failed: {trials}'.

```
0027     pre_test = np.concatenate([train, valid])
```

Atribui a pre_test o resultado de np.concatenate([train, valid]). A expressão à direita é avaliada antes da atribuição.

np.concatenate — Concatena arrays. Argumentos usados: posicional 1: [train, valid].

```
0028     final_model = create_classical_model(name, best).fit(pre_test)
```

Atribui a final_model o resultado de create_classical_model(name, best).fit(pre_test). A expressão à direita é avaliada antes da atribuição.

create_classical_model(name, best).fit — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: pre_test.

create_classical_model — Cria o adaptador ARIMA ou LagRegressor conforme name e os parâmetros selecionados. Argumentos usados: posicional 1: name; posicional 2: best.

```
0029     predictions = final_model.predict(horizon)
```

Atribui a predictions o resultado de final_model.predict(horizon). A expressão à direita é avaliada antes da atribuição.

final_model.predict — Interface comum dos adaptadores do projeto. Argumentos usados: posicional 1: horizon.

```
0030     return predictions, {'selected': best, 'validation_rmse': best_score,
```

Encerra esta função e devolve (predictions, {'selected': best, 'validation_rmse': best_score, 'selection_training_size': len(train), 'validation_size': len(valid), 'final_fit_size': len(pre_test), 'trials': trials}) ao chamador; não imprime o resultado por si só.

```
0031                          'selection_training_size': len(train), 'validation_size':
    len(valid),
```

Define 'selection_training_size': campo do registro/coleção em construção; recebe len(train); 'validation_size': campo do registro/coleção em construção; recebe len(valid). Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: train.

len — Conta elementos. Argumentos usados: posicional 1: valid.

```
0032                            'final_fit_size': len(pre_test), 'trials': trials}
```

Define 'final_fit_size': campo do registro/coleção em construção; recebe len(pre_test); 'trials': campo do registro/coleção em construção; recebe trials. Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: pre_test.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

55. src/pipelines/classical_baselines.py

Executa o protocolo exclusivo 355/30/30, salva procedência, auditorias e previsões.

SHA-256: 74f8b5235266f30a8b94c50ce90c32c8f9b34eeea2012e4f296c041cb4b982ee

Funções, contratos e pontos de conexão

```
L21: run(config, output = OUTPUT)
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

Chamadas do projeto: daily_grid → src/data/preprocessing/resampling.py:5; deterministic_metrics → src/evaluation/metrics.py:5; expanding_validation_folds → src/evaluation/cross_validation.py:60; fill_past_only → src/data/preprocessing/missing_values.py:5; fingerprint → src/pipelines/forecasting_pipeline.py:18; read_series → src/data/ingestion/files.py:8; select_and_forecast → src/training/classical.py:6; source_fingerprint → src/pipelines/forecasting_pipeline.py:22; write_json → src/data/ingestion/files.py:16

```
L94: main()
```

Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada.

Chamadas do projeto: run → src/pipelines/classical_baselines.py:21

APIs externas usadas neste arquivo

RuntimeError, ValueError, add_argument, append, argparse.ArgumentParser, asdict, date, dict, enumerate, exists, get, hashlib.sha256, hexdigest, items, itertuples, json.loads, len, parse_args, pd.DataFrame, print, read_bytes, read_text, str, strftime, time.monotonic, to_csv, to_numpy, tolist, type, update, zip. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Execute expanding 355/30/30 baseline folds with auditable pre-test fitting."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import argparse
```

Importa argparse. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from dataclasses import asdict
```

Importa asdict de dataclasses. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import hashlib
```

Importa hashlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  import time
```

Importa time. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.data.ingestion.files import read_series, write_json
```

Importa read_series, write_json de src.data.ingestion.files. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0011  from src.data.preprocessing.resampling import daily_grid
```

Importa daily_grid de src.data.preprocessing.resampling. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0012  from src.data.preprocessing.missing_values import fill_past_only
```

Importa fill_past_only de src.data.preprocessing.missing_values. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0013  from src.evaluation.cross_validation import expanding_validation_folds
```

Importa expanding_validation_folds de src.evaluation.cross_validation. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0014  from src.evaluation.metrics import deterministic_metrics
```

Importa deterministic_metrics de src.evaluation.metrics. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0015  from src.pipelines.forecasting_pipeline import ROOT, fingerprint, source_fingerprint
```

Importa ROOT, fingerprint, source_fingerprint de src.pipelines.forecasting_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0016  from src.training.classical import select_and_forecast
```

Importa select_and_forecast de src.training.classical. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0018  OUTPUT = ROOT / 'experiments/classical_baselines'
```

Atribui a OUTPUT o resultado de ROOT / 'experiments/classical_baselines'. A expressão à direita é avaliada antes da atribuição.

```
0021  def run(config, output=OUTPUT):
```

Define run(config, output = OUTPUT). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0022      summaries = []
```

Atribui a summaries o resultado de []. A expressão à direita é avaliada antes da atribuição.

```
0023      source_hash = source_fingerprint()
```

Atribui a source_hash o resultado de source_fingerprint(). A expressão à direita é avaliada antes da atribuição.

source_fingerprint — Sem parâmetros. Retorna digest incremental de nomes e bytes de todos os .py de src, em ordem fixa. Argumentos usados: nenhum argumento explícito.

```
0024      for city, spec in config['datasets'].items():
```

Percorre config['datasets'].items(); em cada iteração associa o item a (city, spec). O corpo indentado aplica a operação a esse item/grupo.

config['datasets'].items — Percorre pares (chave, valor) do dicionário. Argumentos usados: nenhum argumento explícito.

```
0025          path = ROOT / spec['path']
```

Atribui a path o resultado de ROOT / spec['path']. A expressão à direita é avaliada antes da atribuição.

```
0026          frame = fill_past_only(daily_grid(read_series(path)))
```

Atribui a frame o resultado de fill_past_only(daily_grid(read_series(path))). A expressão à direita é avaliada antes da atribuição.

fill_past_only — df: grade com possíveis NaN. Retorno: cópia preenchida com passado. Não reconstrói atributos de calendário, limitação conhecida. Argumentos usados: posicional 1: daily_grid(read_series(path)).

daily_grid — df: tabela com timestamp. Retorno: grade diária com observed. Dias ausentes são expostos, não removidos. Argumentos usados: posicional 1: read_series(path).

read_series — path é o CSV; textual_calendar=False lê datas usuais; True aplica date_from_text à coluna timestamp. Retorna DataFrame validado e ordenado. Calendário categórico continua textual. Argumentos usados: posicional 1: path.

```
0027          folds = expanding_validation_folds(frame, config['initial_training'],
```

Atribui a folds o resultado de expanding_validation_folds(frame, config['initial_training'], config['validation_days'], config['test_days']). A expressão à direita é avaliada antes da atribuição.

expanding_validation_folds — Gera treino expansivo desde zero, validação móvel e teste completo, com avanço igual ao tamanho do teste. Rejeita alvos ausentes nos blocos pontuados. Argumentos usados: posicional 1: frame; posicional 2: config['initial_training']; posicional 3: config['validation_days']; posicional 4: config['test_days'].

```
0028                                            config['validation_days'], config['test_days'])
```

Continuação da instrução na linha 27: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0029          for name, candidates in config['candidates'].items():
```

Percorre config['candidates'].items(); em cada iteração associa o item a (name, candidates). O corpo indentado aplica a operação a esse item/grupo.

config['candidates'].items — Percorre pares (chave, valor) do dicionário. Argumentos usados: nenhum argumento explícito.

```
0030              folder = output / city / name
```

Atribui a folder o resultado de output / city / name. A expressão à direita é avaliada antes da atribuição.

```
0031              provenance = {'dataset': city, 'model': name, 'config': config,
```

Atribui a provenance o resultado de {'dataset': city, 'model': name, 'config': config, 'dataset_sha256': hashlib.sha256(path.read_bytes()).hexdigest(), 'source_sha256': source_hash, 'folds': [asdict(f) for f in folds], 'unused_tail_days': len(frame) - folds[-1].test_end, 'protocol': 'expanding selection training; trailing validation; refit pre-test prefix', 'no_future_covariates': True}. A expressão à direita é avaliada antes da atribuição.

```
0032                              'dataset_sha256': hashlib.sha256(path.read_bytes()).hexdigest(),
```

Define 'dataset_sha256': hash dos bytes do CSV de entrada; recebe hashlib.sha256(path.read_bytes()).hexdigest(). Os valores são avaliados nesta janela/iteração.

hashlib.sha256(path.read_bytes()).hexdigest — Retorna a representação hexadecimal do digest acumulado. Argumentos usados: nenhum argumento explícito.

hashlib.sha256 — Cria calculador de digest SHA-256. Argumentos usados: posicional 1: path.read_bytes().

path.read_bytes — Lê bytes integrais do arquivo. Argumentos usados: nenhum argumento explícito.

```
0033                              'source_sha256': source_hash, 'folds': [asdict(f) for f in folds],
```

Define 'source_sha256': hash de todos os .py sob src; recebe source_hash; 'folds': campo do registro/coleção em construção; recebe [asdict(f) for f in folds]. Os valores são avaliados nesta janela/iteração.

asdict — Converte dataclass em dicionário de seus campos. Argumentos usados: posicional 1: f.

```
0034                              'unused_tail_days': len(frame)-folds[-1].test_end,
```

Define 'unused_tail_days': campo do registro/coleção em construção; recebe len(frame) - folds[-1].test_end. Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: frame.

```
0035                              'protocol': 'expanding selection training; trailing validation;
    refit pre-test prefix',
```

Define 'protocol': campo do registro/coleção em construção; recebe 'expanding selection training; trailing validation; refit pre-test prefix'. Os valores são avaliados nesta janela/iteração.

```
0036                              'no_future_covariates': True}
```

Define 'no_future_covariates': campo do registro/coleção em construção; recebe True. Os valores são avaliados nesta janela/iteração.

```
0037             run_id = fingerprint(provenance)
```

Atribui a run_id o resultado de fingerprint(provenance). A expressão à direita é avaliada antes da atribuição.

fingerprint — value: objeto serializável. Retorna SHA-256 hexadecimal do JSON com chaves ordenadas. Argumentos usados: posicional 1: provenance.

```
0038             write_json(folder/'run.json', dict(provenance, run_id=run_id))
```

Executa write_json(folder / 'run.json', dict(provenance, run_id=run_id)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

write_json — path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino. Argumentos usados: posicional 1: folder / 'run.json'; posicional 2: dict(provenance, run_id=run_id).

dict — Cria dicionário. Argumentos usados: posicional 1: provenance; run_id: run_id.

```
0039             rows, failures = [], []
```

Atribui a (rows, failures) o resultado de ([], []). A expressão à direita é avaliada antes da atribuição.

```
0040             for fold in folds:
```

Percorre folds; em cada iteração associa o item a fold. O corpo indentado aplica a operação a esse item/grupo.

```
0041                 origin = frame.timestamp.iloc[fold.validation_end].strftime('%Y-%m-%d')
```

Atribui a origin o resultado de frame.timestamp.iloc[fold.validation_end].strftime('%Y-%m-%d'). A expressão à direita é avaliada antes da atribuição.

frame.timestamp.iloc[fold.validation_end].strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0042                 record_path = folder/'folds'/f'{origin}.json'
```

Atribui a record_path o resultado de folder / 'folds' / f'{origin}.json'. A expressão à direita é avaliada antes da atribuição.

```
0043                 request_id = fingerprint({'run_id': run_id, 'fold': asdict(fold)})
```

Atribui a request_id o resultado de fingerprint({'run_id': run_id, 'fold': asdict(fold)}). A expressão à direita é avaliada antes da atribuição.

fingerprint — value: objeto serializável. Retorna SHA-256 hexadecimal do JSON com chaves ordenadas. Argumentos usados: posicional 1: {'run_id': run_id, 'fold': asdict(fold)}.

asdict — Converte dataclass em dicionário de seus campos. Argumentos usados: posicional 1: fold.

```
0044                 record = json.loads(record_path.read_text()) if record_path.exists() else {}
```

Atribui a record o resultado de json.loads(record_path.read_text()) if record_path.exists() else {}. A expressão à direita é avaliada antes da atribuição.

record_path.exists — Verifica existência do caminho. Argumentos usados: nenhum argumento explícito.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: record_path.read_text().

record_path.read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0045                 if record.get('request_id') != request_id or record.get('status') != 'ok':
```

Avalia a condição record.get('request_id') != request_id or record.get('status') != 'ok'. O bloco indentado seguinte só é executado quando ela é verdadeira.

record.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'request_id'.

record.get — Busca chave de dicionário com padrão se ausente. Argumentos usados: posicional 1: 'status'.

```
0046                     start = time.monotonic()
```

Atribui a start o resultado de time.monotonic(). A expressão à direita é avaliada antes da atribuição.

time.monotonic — Relógio crescente para medir duração. Argumentos usados: nenhum argumento explícito.

```
0047                     record = {'request_id': request_id, 'fold': asdict(fold), 'origin':
    origin,
```

Atribui a record o resultado de {'request_id': request_id, 'fold': asdict(fold), 'origin': origin, 'training_end': str(frame.timestamp.iloc[fold.train_end - 1].date()), 'validation_end': str(frame.timestamp.iloc[fold.validation_end - 1].date()), 'test_end': str(frame.timestamp.iloc[fold.test_end - 1].date())}. A expressão à direita é avaliada antes da atribuição.

asdict — Converte dataclass em dicionário de seus campos. Argumentos usados: posicional 1: fold.

```
0048                                 'training_end':
    str(frame.timestamp.iloc[fold.train_end-1].date()),
```

Define 'training_end': campo do registro/coleção em construção; recebe str(frame.timestamp.iloc[fold.train_end - 1].date()). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: frame.timestamp.iloc[fold.train_end - 1].date().

frame.timestamp.iloc[fold.train_end - 1].date — Extrai a parte civil da data de um Timestamp. Argumentos usados: nenhum argumento explícito.

```
0049                                 'validation_end':
    str(frame.timestamp.iloc[fold.validation_end-1].date()),
```

Define 'validation_end': campo do registro/coleção em construção; recebe str(frame.timestamp.iloc[fold.validation_end - 1].date()). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: frame.timestamp.iloc[fold.validation_end - 1].date().

frame.timestamp.iloc[fold.validation_end - 1].date — Extrai a parte civil da data de um Timestamp. Argumentos usados: nenhum argumento explícito.

```
0050                                 'test_end': str(frame.timestamp.iloc[fold.test_end-1].date())}
```

Define 'test_end': campo do registro/coleção em construção; recebe str(frame.timestamp.iloc[fold.test_end - 1].date()). Os valores são avaliados nesta janela/iteração.

str — Produz representação textual. Argumentos usados: posicional 1: frame.timestamp.iloc[fold.test_end - 1].date().

frame.timestamp.iloc[fold.test_end - 1].date — Extrai a parte civil da data de um Timestamp. Argumentos usados: nenhum argumento explícito.

```
0051                     try:
```

Inicia região protegida de execução. Exceções compatíveis serão tratadas pelo except correspondente; sucesso continua normalmente.

```
0052                         # Only these two pre-test arrays cross the fitting boundary.
```

Comentário de implementação: Only these two pre-test arrays cross the fitting boundary.. Não é executado; as operações efetivas são as linhas seguintes.

```
0053                         predictions, audit = select_and_forecast(name,
```

Atribui a (predictions, audit) o resultado de select_and_forecast(name, frame.iloc[fold.training].water_produced.to_numpy(), frame.iloc[fold.validation].water_produced.to_numpy(), config['test_days'], candidates). A expressão à direita é avaliada antes da atribuição.

select_and_forecast — Recebe nome, treino, validação, horizonte e candidatos, nunca alvos de teste. Seleciona pelo RMSE de validação, reajusta treino+validação e devolve previsões e auditoria. Argumentos usados: posicional 1: name; posicional 2: frame.iloc[fold.training].water_produced.to_numpy(); posicional 3: frame.iloc[fold.validation].water_produced.to_numpy(); posicional 4: config['test_days']; posicional 5: candidates.

```
0054                             frame.iloc[fold.training].water_produced.to_numpy(),
```

Continuação da instrução na linha 53: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

frame.iloc[fold.training].water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

```
0055                             frame.iloc[fold.validation].water_produced.to_numpy(),
```

Continuação da instrução na linha 53: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

frame.iloc[fold.validation].water_produced.to_numpy — Converte Series/DataFrame em array NumPy. Argumentos usados: nenhum argumento explícito.

```
0056                                config['test_days'], candidates)
```

Continuação da instrução na linha 53: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0057                            record.update(status='ok', predictions=predictions.tolist(),
    audit=audit)
```

Executa record.update(status='ok', predictions=predictions.tolist(), audit=audit). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

record.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: status: 'ok'; predictions: predictions.tolist(); audit: audit.

predictions.tolist — Converte array/índice/Series em lista Python. Argumentos usados: nenhum argumento explícito.

```
0058                        except Exception as exc:
```

Trata as exceções indicadas na linha e vincula a falha a exc quando declarado. O registro permite diagnosticar a tentativa sem inventar previsões.

```
0059                            record.update(status='error', error=f'{type(exc).__name__}: {exc}')
```

Executa record.update(status='error', error=f'{type(exc).__name__}: {exc}'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

record.update — Acrescenta conteúdo ao objeto mutável. Argumentos usados: status: 'error'; error: f'{type(exc).__name__}: {exc}'.

type — Obtém a classe concreta do valor. Argumentos usados: posicional 1: exc.

```
0060                        record['elapsed_seconds'] = time.monotonic()-start
```

Atribui a record['elapsed_seconds'] o resultado de time.monotonic() - start. A expressão à direita é avaliada antes da atribuição.

time.monotonic — Relógio crescente para medir duração. Argumentos usados: nenhum argumento explícito.

```
0061                        write_json(record_path, record)
```

Executa write_json(record_path, record). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

write_json — path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino. Argumentos usados: posicional 1: record_path; posicional 2: record.

```
0062                    print(f'{city}/{name} fold {fold.number}/{len(folds)} {origin}:
    {record["status"]}', flush=True)
```

Executa print(f"{city}/{name} fold {fold.number}/{len(folds)} {origin}: {record['status']}", flush=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

print — Escreve representação textual na saída padrão. Argumentos usados: posicional 1: f"{city}/{name} fold {fold.number}/{len(folds)} {origin}: {record['status']}"; flush: True.

len — Conta elementos. Argumentos usados: posicional 1: folds.

```
0063                    if record['status'] != 'ok':
```

Avalia a condição record['status'] != 'ok'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0064                        failures.append({'origin': origin, 'error': record['error']})
```

Executa failures.append({'origin': origin, 'error': record['error']}). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

failures.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: {'origin': origin, 'error': record['error']}.

```
0065                        continue
```

Avança imediatamente à próxima iteração; esta janela com falha não recebe linhas de previsão.

```
0066                 # Ground truth is read only after predictions have been produced.
```

Comentário de implementação: Ground truth is read only after predictions have been produced.. Não é executado; as operações efetivas são as linhas seguintes.

```
0067                 truth = frame.iloc[fold.test]
```

Atribui a truth o resultado de frame.iloc[fold.test]. A expressão à direita é avaliada antes da atribuição.

```
0068                 for lead, (observation, predicted) in enumerate(zip(truth.itertuples(),
    record['predictions']), 1):
```

Percorre enumerate(zip(truth.itertuples(), record['predictions']), 1); em cada iteração associa o item a (lead, (observation, predicted)). O corpo indentado aplica a operação a esse item/grupo.

enumerate — Percorre iterável com contador. Argumentos usados: posicional 1: zip(truth.itertuples(), record['predictions']); posicional 2: 1.

zip — Emparelha iteráveis pela posição. Argumentos usados: posicional 1: truth.itertuples(); posicional 2: record['predictions'].

truth.itertuples — Itera linhas como tuplas nomeadas. Argumentos usados: nenhum argumento explícito.

```
0069                     rows.append({'dataset': city, 'model': name, 'fold': fold.number,
    'origin': origin,
```

Executa rows.append({'dataset': city, 'model': name, 'fold': fold.number, 'origin': origin, 'timestamp': observation.timestamp.strftime('%Y-%m-%d'), 'lead': lead, 'y_true': observation.water_produced, 'y_pred': predicted, 'initial_context': config['initial_training'], 'training_days': fold.train_end, 'validation_days': config['validation_days'], 'fit_days': fold.validation_end, 'horizon': config['test_days'], 'run_id': run_id}). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

rows.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: {'dataset': city, 'model': name, 'fold': fold.number, 'origin': origin, 'timestamp': observation.timestamp.strftime('%Y-%m-%d'), 'lead': lead, 'y_true': observation.water_produced, 'y_pred': predicted, 'initial_context': config['initial_training'], 'training_days': fold.train_end, 'validation_days': config['validation_days'], 'fit_days': fold.validation_end, 'horizon': config['test_days'], 'run_id': run_id}.

```
0070                                 'timestamp': observation.timestamp.strftime('%Y-%m-%d'),
    'lead': lead,
```

Define 'timestamp': campo do registro/coleção em construção; recebe observation.timestamp.strftime('%Y-%m-%d'); 'lead': posição dentro do horizonte: 1 a 7 no Qwen ou 1 a 30 nos baselines clássicos; recebe lead. Os valores são avaliados nesta janela/iteração.

observation.timestamp.strftime — Formata datas como texto. Argumentos usados: posicional 1: '%Y-%m-%d'.

```
0071                                 'y_true': observation.water_produced, 'y_pred': predicted,
```

Define 'y_true': demanda observada usada somente na avaliação; recebe observation.water_produced; 'y_pred': valor retornado pelo modelo para a data correspondente; recebe predicted. Os valores são avaliados nesta janela/iteração.

```
0072                                 'initial_context': config['initial_training'],
    'training_days': fold.train_end,
```

Define 'initial_context': origem inicial do protocolo de avaliação; recebe config['initial_training']; 'training_days': campo do registro/coleção em construção; recebe fold.train_end. Os valores são avaliados nesta janela/iteração.

```
0073                                 'validation_days': config['validation_days'], 'fit_days':
    fold.validation_end,
```

Define 'validation_days': campo do registro/coleção em construção; recebe config['validation_days']; 'fit_days': campo do registro/coleção em construção; recebe fold.validation_end. Os valores são avaliados nesta janela/iteração.

```
0074                                 'horizon': config['test_days'], 'run_id': run_id})
```

Define 'horizon': campo do registro/coleção em construção; recebe config['test_days']; 'run_id': identificador derivado da procedência da execução; recebe run_id. Os valores são avaliados nesta janela/iteração.

```
0075             pd.DataFrame(rows).to_csv(folder/'predictions.csv', index=False)
```

Executa pd.DataFrame(rows).to_csv(folder / 'predictions.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(rows).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: folder / 'predictions.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: rows.

```
0076             write_json(folder/'status.json', {'expected_folds': len(folds),
```

Executa write_json(folder / 'status.json', {'expected_folds': len(folds), 'successful_folds': len(rows) // config['test_days'], 'failures': failures}). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

write_json — path: destino; value: dict serializável. Sem retorno útil. Cria pais, escreve .tmp e substitui destino. Argumentos usados: posicional 1: folder / 'status.json'; posicional 2: {'expected_folds': len(folds), 'successful_folds': len(rows) // config['test_days'], 'failures': failures}.

len — Conta elementos. Argumentos usados: posicional 1: folds.

```
0077                         'successful_folds': len(rows)//config['test_days'], 'failures':
    failures})
```

Define 'successful_folds': campo do registro/coleção em construção; recebe len(rows) // config['test_days']; 'failures': lista de janelas com falha; recebe failures. Os valores são avaliados nesta janela/iteração.

len — Conta elementos. Argumentos usados: posicional 1: rows.

```
0078             if failures:
```

Avalia a condição failures. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0079                 raise RuntimeError(f'Incomplete {city}/{name}: {failures}')
```

Interrompe o caminho atual lançando RuntimeError(f'Incomplete {city}/{name}: {failures}'). O chamador precisa capturar a exceção ou ela encerra a execução.

RuntimeError — Cria exceção para estado de execução inesperado. Argumentos usados: posicional 1: f'Incomplete {city}/{name}: {failures}'.

```
0080             predictions = pd.DataFrame(rows)
```

Atribui a predictions o resultado de pd.DataFrame(rows). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: rows.

```
0081             scores = deterministic_metrics(predictions.y_true, predictions.y_pred)
```

Atribui a scores o resultado de deterministic_metrics(predictions.y_true, predictions.y_pred). A expressão à direita é avaliada antes da atribuição.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R^2 e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: predictions.y_true; posicional 2: predictions.y_pred.

```
0082             simple = {'modelo': name, 'contexto_inicial': config['initial_training'],
```

Atribui a simple o resultado de {'modelo': name, 'contexto_inicial': config['initial_training'], 'RMSE': scores['rmse'], 'R2': scores['r2'], 'MAE': scores['mae'], 'MAPE': scores['mape_percent']}. A expressão à direita é avaliada antes da atribuição.

```
0083                         'RMSE': scores['rmse'], 'R2': scores['r2'], 'MAE': scores['mae'],
    'MAPE': scores['mape_percent']}
```

Define 'RMSE': campo do registro/coleção em construção; recebe scores['rmse']; 'R2': campo do registro/coleção em construção; recebe scores['r2']; 'MAE': campo do registro/coleção em construção; recebe scores['mae']; 'MAPE': campo do registro/coleção em construção; recebe scores['mape_percent']. Os valores são avaliados nesta janela/iteração.

```
0084             pd.DataFrame([simple]).to_csv(folder/'comparison.csv', index=False)
```

Executa pd.DataFrame([simple]).to_csv(folder / 'comparison.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame([simple]).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: folder / 'comparison.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: [simple].

```
0085              summaries.append(dict(dataset=city, model=name, folds=len(folds),
    predictions=len(rows), **scores))
```

Executa summaries.append(dict(dataset=city, model=name, folds=len(folds), predictions=len(rows), **scores)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

summaries.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(dataset=city, model=name, folds=len(folds), predictions=len(rows), **scores).

dict — Cria dicionário. Argumentos usados: dataset: city; model: name; folds: len(folds); predictions: len(rows); **desempacotamento: scores.

len — Conta elementos. Argumentos usados: posicional 1: folds.

len — Conta elementos. Argumentos usados: posicional 1: rows.

```
0086          city_rows = [row for row in summaries if row['dataset']==city]
```

Atribui a city_rows o resultado de [row for row in summaries if row['dataset'] == city]. A expressão à direita é avaliada antes da atribuição.

```
0087          pd.DataFrame([{'modelo': row['model'], 'contexto_inicial':
    config['initial_training'],
```

Executa pd.DataFrame([{'modelo': row['model'], 'contexto_inicial': config['initial_training'], 'RMSE': row['rmse'], 'R2': row['r2'], 'MAE': row['mae'], 'MAPE': row['mape_percent']} for row in city_rows]).to_csv(output / f'comparison_{city}.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame([{'modelo': row['model'], 'contexto_inicial': config['initial_training'], 'RMSE': row['rmse'], 'R2': row['r2'], 'MAE': row['mae'], 'MAPE': row['mape_percent']} for row in city_rows]).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: output / f'comparison_{city}.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: [{'modelo': row['model'], 'contexto_inicial': config['initial_training'], 'RMSE': row['rmse'], 'R2': row['r2'], 'MAE': row['mae'], 'MAPE': row['mape_percent']} for row in city_rows].

```
0088                          'RMSE': row['rmse'], 'R2': row['r2'], 'MAE': row['mae'], 'MAPE':
    row['mape_percent']}
```

Define 'RMSE': campo do registro/coleção em construção; recebe row['rmse']; 'R2': campo do registro/coleção em construção; recebe row['r2']; 'MAE': campo do registro/coleção em construção; recebe row['mae']; 'MAPE': campo do registro/coleção em construção; recebe row['mape_percent']. Os valores são avaliados nesta janela/iteração.

```
0089                          for row in city_rows]).to_csv(output/f'comparison_{city}.csv',
    index=False)
```

Continua os argumentos nomeados: index = False. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0090      pd.DataFrame(summaries).to_csv(output/'metrics.csv', index=False)
```

Executa pd.DataFrame(summaries).to_csv(output / 'metrics.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(summaries).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: output / 'metrics.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: summaries.

```
0091      return pd.DataFrame(summaries)
```

Encerra esta função e devolve pd.DataFrame(summaries) ao chamador; não imprime o resultado por si só.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: summaries.

```
0094  def main():
```

Define main(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0095      parser = argparse.ArgumentParser(description=__doc__)
```

Atribui a parser o resultado de argparse.ArgumentParser(description=__doc__). A expressão à direita é avaliada antes da atribuição.

argparse.ArgumentParser — Cria o parser da interface de linha de comando. Argumentos usados: description: __doc__.

```
0096      parser.add_argument('--config', type=Path,
    default=ROOT/'configs/classical_baselines.json')
```

Executa parser.add_argument('--config', type=Path, default=ROOT / 'configs/classical_baselines.json'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--config'; type: Path; default: ROOT / 'configs/classical_baselines.json'.

```
0097      parser.add_argument('--output', type=Path, default=OUTPUT)
```

Executa parser.add_argument('--output', type=Path, default=OUTPUT). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

parser.add_argument — Registra uma opção do CLI. Argumentos usados: posicional 1: '--output'; type: Path; default: OUTPUT.

```
0098      args = parser.parse_args()
```

Atribui a args o resultado de parser.parse_args(). A expressão à direita é avaliada antes da atribuição.

parser.parse_args — Interpreta argumentos do processo e retorna Namespace. Argumentos usados: nenhum argumento explícito.

```
0099      config = json.loads(args.config.read_text())
```

Atribui a config o resultado de json.loads(args.config.read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: args.config.read_text().

args.config.read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0100      if config['step_days'] != config['test_days'] or config['refit_train_validation'] is not
    True:
```

Avalia a condição config['step_days'] != config['test_days'] or config['refit_train_validation'] is not True. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0101          raise ValueError('This protocol requires nonoverlapping tests and pre-test
    refitting')
```

Interrompe o caminho atual lançando ValueError('This protocol requires nonoverlapping tests and pre-test refitting'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'This protocol requires nonoverlapping tests and pre-test refitting'.

```
0102      run(config, args.output)
```

Executa run(config, args.output). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

run — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: config; posicional 2: args.output.

```
0105  if __name__ == '__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0106      main()
```

Executa main(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

main — Entrada CLI do módulo correspondente. Lê argumentos ou coordena a rotina documentada no início do arquivo. Os efeitos variam: conversão, execução, métricas ou gráficos; não são a mesma função compartilhada. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

56. src/visualization/comparison_results.py

Compara os prompts Qwen históricos e gera gráficos separados dos baselines.

SHA-256: 733048922276619623e0d5a8d8a7c5200732befb978f48bdccb5045de1c2c0aa

Funções, contratos e pontos de conexão

```
L16: prompt_comparison()
```

Confere completude, mesmos alvos e checkpoint; calcula métricas e redução histórica de RMSE e gera quatro gráficos de contexto 15.

Chamadas do projeto: deterministic_metrics → src/evaluation/metrics.py:5

```
L71: baseline_plots()
```

Lê os seis resultados clássicos, exige mesmos alvos entre modelos e desenha três painéis por cidade.

APIs externas usadas neste arquivo

ValueError, append, dict, grid, json.loads, legend, len, matplotlib.use, mdates.DateFormatter, mkdir, nunique, pd.DataFrame, pd.read_csv, pd.testing.assert_frame_equal, pd.to_datetime, plot, plt.close, plt.subplots, query, read_text, reset_index, savefig, set_major_formatter, set_title, set_ylabel, sort_values, suptitle, title, to_csv, zip. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Retain historical numeric Qwen outputs beside narrative prompts and new baselines."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import json
```

Importa json. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from pathlib import Path
```

Importa Path de pathlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import matplotlib
```

Importa matplotlib. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  matplotlib.use('Agg')
```

Executa matplotlib.use('Agg'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

matplotlib.use — Seleciona backend de renderização. Argumentos usados: posicional 1: 'Agg'.

```
0006  import matplotlib.pyplot as plt
```

Importa matplotlib.pyplot como plt. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  import matplotlib.dates as mdates
```

Importa matplotlib.dates como mdates. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  from src.evaluation.metrics import deterministic_metrics
```

Importa deterministic_metrics de src.evaluation.metrics. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0010  from src.pipelines.forecasting_pipeline import ROOT
```

Importa ROOT de src.pipelines.forecasting_pipeline. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0012  OUTPUT = ROOT/'experiments/prompt_comparison'
```

Atribui a OUTPUT o resultado de ROOT / 'experiments/prompt_comparison'. A expressão à direita é avaliada antes da atribuição.

```
0013  LABELS = {'linear_regression':'Regressão linear', 'knn':'KNN', 'arima':'ARIMA'}
```

Atribui a LABELS o resultado de {'linear_regression': 'Regressão linear', 'knn': 'KNN', 'arima': 'ARIMA'}. A expressão à direita é avaliada antes da atribuição.

```
0016  def prompt_comparison():
```

Define prompt_comparison(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0017      OUTPUT.mkdir(parents=True, exist_ok=True)
```

Executa OUTPUT.mkdir(parents=True, exist_ok=True). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

OUTPUT.mkdir — Cria diretório. Argumentos usados: parents: True; exist_ok: True.

```
0018      rows, changes = [], []
```

Atribui a (rows, changes) o resultado de ([], []). A expressão à direita é avaliada antes da atribuição.

```
0019      for city in ('joinville','guaratuba'):
```

Percorre ('joinville', 'guaratuba'); em cada iteração associa o item a city. O corpo indentado aplica a operação a esse item/grupo.

```
0020          for slug, label, old_root in [('1_7b','Qwen3 1.7B','experiments/benchmark'),
```

Percorre [('1_7b', 'Qwen3 1.7B', 'experiments/benchmark'), ('4b', 'Qwen3 4B', 'experiments/qwen_15_days')]; em cada iteração associa o item a (slug, label, old_root). O corpo indentado aplica a operação a esse item/grupo.

```
0021                                      ('4b','Qwen3 4B','experiments/qwen_15_days')]:
```

Continuação da instrução na linha 20: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0022              old_folder = ROOT/old_root/city/'qwen'
```

Atribui a old_folder o resultado de ROOT / old_root / city / 'qwen'. A expressão à direita é avaliada antes da atribuição.

```
0023              old_run = json.loads((old_folder/'run.json').read_text())
```

Atribui a old_run o resultado de json.loads((old_folder / 'run.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (old_folder / 'run.json').read_text().

(old_folder / 'run.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0024              old_status = json.loads((old_folder/'status.json').read_text())
```

Atribui a old_status o resultado de json.loads((old_folder / 'status.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (old_folder / 'status.json').read_text().

(old_folder / 'status.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0025              if old_status['failures'] or
    old_status['successful_windows']!=old_status['expected_windows']:
```

Avalia a condição old_status['failures'] or old_status['successful_windows'] != old_status['expected_windows']. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0026                 raise ValueError('Incomplete historical Qwen evaluation')
```

Interrompe o caminho atual lançando ValueError('Incomplete historical Qwen evaluation'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Incomplete historical Qwen evaluation'.

```
0027             old =
    pd.read_csv(old_folder/'predictions.csv').sort_values(['origin','lead']).reset_index(drop=True)
```

Atribui a old o resultado de pd.read_csv(old_folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index(drop=True). A expressão à direita é avaliada antes da atribuição.

pd.read_csv(old_folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

pd.read_csv(old_folder / 'predictions.csv').sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: ['origin', 'lead'].

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: old_folder / 'predictions.csv'.

```
0028             old_score = deterministic_metrics(old.y_true,old.y_pred)
```

Atribui a old_score o resultado de deterministic_metrics(old.y_true, old.y_pred). A expressão à direita é avaliada antes da atribuição.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R^2 e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: old.y_true; posicional 2: old.y_pred.

```
0029             rows.append(dict(dataset=city,model=label,slug=slug,format='numérico/tabular
    anterior',context=15,
```

Executa rows.append(dict(dataset=city, model=label, slug=slug, format='numérico/tabular anterior', context=15, windows=old.origin.nunique(), predictions=len(old), **old_score)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

rows.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(dataset=city, model=label, slug=slug, format='numérico/tabular anterior', context=15, windows=old.origin.nunique(), predictions=len(old), **old_score).

dict — Cria dicionário. Argumentos usados: dataset: city; model: label; slug: slug; format: 'numérico/tabular anterior'; context: 15; windows: old.origin.nunique(); predictions: len(old); **desempacotamento: old_score.

```
0030                                 windows=old.origin.nunique(),predictions=len(old),**old_score))
```

Continua os argumentos nomeados: windows = old.origin.nunique(); predictions = len(old); ** = old_score. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

old.origin.nunique — Conta valores distintos. Argumentos usados: nenhum argumento explícito.

len — Conta elementos. Argumentos usados: posicional 1: old.

```
0031             for days in (7,15):
```

Percorre (7, 15); em cada iteração associa o item a days. O corpo indentado aplica a operação a esse item/grupo.

```
0032                 folder = ROOT/'experiments/qwen_text'/f'{slug}_{days}'/city/'qwen'
```

Atribui a folder o resultado de ROOT / 'experiments/qwen_text' / f'{slug}_{days}' / city / 'qwen'. A expressão à direita é avaliada antes da atribuição.

```
0033                 new =
    pd.read_csv(folder/'predictions.csv').sort_values(['origin','lead']).reset_index(drop=True)
```

Atribui a new o resultado de pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index(drop=True). A expressão à direita é avaliada antes da atribuição.

pd.read_csv(folder / 'predictions.csv').sort_values(['origin', 'lead']).reset_index — Transforma o índice atual em coluna ou o descarta e cria índice sequencial. Argumentos usados: drop: True.

pd.read_csv(folder / 'predictions.csv').sort_values — Ordena linhas pelos valores de colunas. Argumentos usados: posicional 1: ['origin', 'lead'].

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: folder / 'predictions.csv'.

```
0034                 new_run = json.loads((folder/'run.json').read_text())
```

Atribui a new_run o resultado de json.loads((folder / 'run.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'run.json').read_text().

(folder / 'run.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0035                 status = json.loads((folder/'status.json').read_text())
```

Atribui a status o resultado de json.loads((folder / 'status.json').read_text()). A expressão à direita é avaliada antes da atribuição.

json.loads — Decodifica uma string JSON em objetos Python. Argumentos usados: posicional 1: (folder / 'status.json').read_text().

(folder / 'status.json').read_text — Lê todo o arquivo como string. Argumentos usados: nenhum argumento explícito.

```
0036                 if status['failures'] or
    status['successful_windows']!=status['expected_windows']:
```

Avalia a condição status['failures'] or status['successful_windows'] != status['expected_windows']. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0037                     raise ValueError('Incomplete narrative Qwen evaluation')
```

Interrompe o caminho atual lançando ValueError('Incomplete narrative Qwen evaluation'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Incomplete narrative Qwen evaluation'.

```
0038                 keys = ['origin','timestamp','lead','y_true']
```

Atribui a keys o resultado de ['origin', 'timestamp', 'lead', 'y_true']. A expressão à direita é avaliada antes da atribuição.

```
0039                 pd.testing.assert_frame_equal(old[keys],new[keys])
```

Executa pd.testing.assert_frame_equal(old[keys], new[keys]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_frame_equal — Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem. Argumentos usados: posicional 1: old[keys]; posicional 2: new[keys].

```
0040                 if old_run['model']['digest'] != new_run['model']['digest']:
```

Avalia a condição old_run['model']['digest'] != new_run['model']['digest']. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0041                     raise ValueError('Historical and narrative checkpoints differ')
```

Interrompe o caminho atual lançando ValueError('Historical and narrative checkpoints differ'). O chamador precisa capturar a exceção ou ela encerra a execução.

ValueError — Cria exceção para valor/formato inválido. Argumentos usados: posicional 1: 'Historical and narrative checkpoints differ'.

```
0042                 scores = deterministic_metrics(new.y_true,new.y_pred)
```

Atribui a scores o resultado de deterministic_metrics(new.y_true, new.y_pred). A expressão à direita é avaliada antes da atribuição.

deterministic_metrics — ytrue/yhat: vetores de mesmo formato, não vazios e finitos. Retorna MAE, RMSE, MAPE%, sMAPE%, R^2 e contagem de zeros excluídos do MAPE. Argumentos usados: posicional 1: new.y_true; posicional 2: new.y_pred.

```
0043                 rows.append(dict(dataset=city,model=label,slug=slug,format='linguagem
    natural',context=days,
```

Executa rows.append(dict(dataset=city, model=label, slug=slug, format='linguagem natural', context=days, windows=new.origin.nunique(), predictions=len(new), **scores)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

rows.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(dataset=city, model=label, slug=slug, format='linguagem natural', context=days, windows=new.origin.nunique(), predictions=len(new), **scores).

dict — Cria dicionário. Argumentos usados: dataset: city; model: label; slug: slug; format: 'linguagem natural'; context: days; windows: new.origin.nunique(); predictions: len(new); **desempacotamento: scores.

```
0044                                    windows=new.origin.nunique(),predictions=len(new),**scores))
```

Continua os argumentos nomeados: windows = new.origin.nunique(); predictions = len(new); ** = scores. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

new.origin.nunique — Conta valores distintos. Argumentos usados: nenhum argumento explícito.

len — Conta elementos. Argumentos usados: posicional 1: new.

```
0045                if days==15:
```

Avalia a condição days == 15. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0046                    changes.append(dict(dataset=city,model=label,context=15,
```

Executa changes.append(dict(dataset=city, model=label, context=15, old_num_ctx=old_run['config']['qwen']['num_ctx'], new_num_ctx=new_run['config']['qwen']['num_ctx'], old_rmse=old_score['rmse'], new_rmse=scores['rmse'], rmse_reduction_percent=100 * (old_score['rmse'] - scores['rmse']) / old_score['rmse'], interpretation='Historical comparison; token budgets and prompt contents also differ')). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

changes.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: dict(dataset=city, model=label, context=15, old_num_ctx=old_run['config']['qwen']['num_ctx'], new_num_ctx=new_run['config']['qwen']['num_ctx'], old_rmse=old_score['rmse'], new_rmse=scores['rmse'], rmse_reduction_percent=100 * (old_score['rmse'] - scores['rmse']) / old_score['rmse'], interpretation='Historical comparison; token budgets and prompt contents also differ').

dict — Cria dicionário. Argumentos usados: dataset: city; model: label; context: 15; old_num_ctx: old_run['config']['qwen']['num_ctx']; new_num_ctx: new_run['config']['qwen']['num_ctx']; old_rmse: old_score['rmse']; new_rmse: scores['rmse']; rmse_reduction_percent: 100 * (old_score['rmse'] - scores['rmse']) / old_score['rmse']; interpretation: 'Historical comparison; token budgets and prompt contents also differ'.

```
0047
    old_num_ctx=old_run['config']['qwen']['num_ctx'],new_num_ctx=new_run['config']['qwen']['num_ctx
    '],
```

Continua os argumentos nomeados: old_num_ctx = old_run['config']['qwen']['num_ctx']; new_num_ctx = new_run['config']['qwen']['num_ctx']. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0048                        old_rmse=old_score['rmse'],new_rmse=scores['rmse'],
```

Continua os argumentos nomeados: old_rmse = old_score['rmse']; new_rmse = scores['rmse']. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0049
    rmse_reduction_percent=100*(old_score['rmse']-scores['rmse'])/old_score['rmse'],
```

Continua os argumentos nomeados: rmse_reduction_percent = 100 * (old_score['rmse'] - scores['rmse']) / old_score['rmse']. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0050                        interpretation='Historical comparison; token budgets and prompt
    contents also differ'))
```

Continua os argumentos nomeados: interpretation = 'Historical comparison; token budgets and prompt contents also differ'. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0051                    fig,axes =
    plt.subplots(2,1,figsize=(16,8),sharex=True,constrained_layout=True)
```

Atribui a (fig, axes) o resultado de plt.subplots(2, 1, figsize=(16, 8), sharex=True, constrained_layout=True). A expressão à direita é avaliada antes da atribuição.

plt.subplots — Cria figura e eixos Matplotlib. Argumentos usados: posicional 1: 2; posicional 2: 1; figsize: (16, 8); sharex: True; constrained_layout: True.

```
0052                     for ax,frame,title,score in [(axes[0],old,'Calendário numérico / prompt
    anterior',old_score),
```

Percorre [(axes[0], old, 'Calendário numérico / prompt anterior', old_score), (axes[1], new, 'Calendário textual / linguagem natural', scores)]; em cada iteração associa o item a (ax, frame, title, score). O corpo indentado aplica a operação a esse item/grupo.

```
0053                                              (axes[1],new,'Calendário textual / linguagem
    natural',scores)]:
```

Continuação da instrução na linha 52: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0054                         dates=pd.to_datetime(frame.timestamp)
```

Atribui a dates o resultado de pd.to_datetime(frame.timestamp). A expressão à direita é avaliada antes da atribuição.

pd.to_datetime — Converte valores em datas pandas. Argumentos usados: posicional 1: frame.timestamp.

```
0055                         ax.plot(dates,frame.y_true,color='#777777',lw=1.25,label='Real')
```

Executa ax.plot(dates, frame.y_true, color='#777777', lw=1.25, label='Real'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.plot — Desenha sequência de pontos/linhas no eixo. Argumentos usados: posicional 1: dates; posicional 2: frame.y_true; color: '#777777'; lw: 1.25; label: 'Real'.

```
0056                         ax.plot(dates,frame.y_pred,color='#2478b5',lw=1.1,label='Previsto')
```

Executa ax.plot(dates, frame.y_pred, color='#2478b5', lw=1.1, label='Previsto'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.plot — Desenha sequência de pontos/linhas no eixo. Argumentos usados: posicional 1: dates; posicional 2: frame.y_pred; color: '#2478b5'; lw: 1.1; label: 'Previsto'.

```
0057                         ax.set_title(f'{title} | RMSE {score["rmse"]:,.2f} | R²
    {score["r2"]:.3f}',loc='left')
```

Executa ax.set_title(f"{title} | RMSE {score['rmse']:,.2f} | R² {score['r2']:.3f}", loc='left'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.set_title — Define título de eixo. Argumentos usados: posicional 1: f"{title} | RMSE {score['rmse']:,.2f} | R² {score['r2']:.3f}"; loc: 'left'.

```
0058                         ax.set_ylabel('Produção
    (m³)');ax.grid(alpha=.2);ax.legend(ncol=2,loc='upper right')
```

Executa ax.set_ylabel('Produção (m³)'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição. Executa ax.grid(alpha=0.2). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição. Executa ax.legend(ncol=2, loc='upper right'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.set_ylabel — Define unidade/rótulo vertical. Argumentos usados: posicional 1: 'Produção (m³)'.

ax.grid — Exibe linhas de referência. Argumentos usados: alpha: 0.2.

ax.legend — Mostra identificação das séries. Argumentos usados: ncol: 2; loc: 'upper right'.

```
0059                     axes[1].xaxis.set_major_formatter(mdates.DateFormatter('%m/%Y'))
```

Executa axes[1].xaxis.set_major_formatter(mdates.DateFormatter('%m/%Y')). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

axes[1].xaxis.set_major_formatter — Define formatação dos rótulos principais. Argumentos usados: posicional 1: mdates.DateFormatter('%m/%Y').

mdates.DateFormatter — Formata rótulos temporais do eixo. Argumentos usados: posicional 1: '%m/%Y'.

```
0060                      fig.suptitle(f'{city.title()} — {label} | Comparação com contexto de 15
    dias',fontsize=19)
```

Executa fig.suptitle(f'{city.title()} — {label} | Comparação com contexto de 15 dias', fontsize=19). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

fig.suptitle — Define título geral da figura. Argumentos usados: posicional 1: f'{city.title()} — {label} | Comparação com contexto de 15 dias'; fontsize: 19.

city.title — Converte texto para capitalização de título. Argumentos usados: nenhum argumento explícito.

```
0061                      fig.savefig(OUTPUT/f'{city}_{slug}_comparison.png',dpi=150)
```

Executa fig.savefig(OUTPUT / f'{city}_{slug}_comparison.png', dpi=150). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

fig.savefig — Exporta uma figura para arquivo. Argumentos usados: posicional 1: OUTPUT / f'{city}_{slug}_comparison.png'; dpi: 150.

```
0062                      plt.close(fig)
```

Executa plt.close(fig). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

plt.close — Libera recursos da figura. Argumentos usados: posicional 1: fig.

```
0063          selected=[row for row in rows if row['dataset']==city]
```

Atribui a selected o resultado de [row for row in rows if row['dataset'] == city]. A expressão à direita é avaliada antes da atribuição.

```
0064          pd.DataFrame([{'modelo':f'{r["model"]} —
    {r["format"]}','contexto_inicial':r['context'],
```

Executa pd.DataFrame([{'modelo': f"{r['model']} — {r['format']}", 'contexto_inicial': r['context'], 'RMSE': r['rmse'], 'R2': r['r2'], 'MAE': r['mae'], 'MAPE': r['mape_percent']} for r in selected]).to_csv(OUTPUT / f'comparison_{city}.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame([{'modelo': f"{r['model']} — {r['format']}", 'contexto_inicial': r['context'], 'RMSE': r['rmse'], 'R2': r['r2'], 'MAE': r['mae'], 'MAPE': r['mape_percent']} for r in selected]).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: OUTPUT / f'comparison_{city}.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: [{'modelo': f"{r['model']} — {r['format']}", 'contexto_inicial': r['context'], 'RMSE': r['rmse'], 'R2': r['r2'], 'MAE': r['mae'], 'MAPE': r['mape_percent']} for r in selected].

```
0065                            'RMSE':r['rmse'],'R2':r['r2'],'MAE':r['mae'],'MAPE':r['mape_percent']}
```

Define 'RMSE': campo do registro/coleção em construção; recebe r['rmse']; 'R2': campo do registro/coleção em construção; recebe r['r2']; 'MAE': campo do registro/coleção em construção; recebe r['mae']; 'MAPE': campo do registro/coleção em construção; recebe r['mape_percent']. Os valores são avaliados nesta janela/iteração.

```
0066                          for r in selected]).to_csv(OUTPUT/f'comparison_{city}.csv',index=False)
```

Continua os argumentos nomeados: index = False. Esses valores pertencem à chamada iniciada acima; consulte seu contrato de parâmetros.

```
0067      pd.DataFrame(rows).to_csv(OUTPUT/'metrics.csv',index=False)
```

Executa pd.DataFrame(rows).to_csv(OUTPUT / 'metrics.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(rows).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: OUTPUT / 'metrics.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: rows.

```
0068      pd.DataFrame(changes).to_csv(OUTPUT/'prompt_change.csv',index=False)
```

Executa pd.DataFrame(changes).to_csv(OUTPUT / 'prompt_change.csv', index=False). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.DataFrame(changes).to_csv — Serializa DataFrame como CSV. Argumentos usados: posicional 1: OUTPUT / 'prompt_change.csv'; index: False.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: changes.

```
0071  def baseline_plots():
```

Define baseline_plots(). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0072      root=ROOT/'experiments/classical_baselines'
```

Atribui a root o resultado de ROOT / 'experiments/classical_baselines'. A expressão à direita é avaliada antes da atribuição.

```
0073      metrics=pd.read_csv(root/'metrics.csv')
```

Atribui a metrics o resultado de pd.read_csv(root / 'metrics.csv'). A expressão à direita é avaliada antes da atribuição.

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: root / 'metrics.csv'.

```
0074      for city in ('joinville','guaratuba'):
```

Percorre ('joinville', 'guaratuba'); em cada iteração associa o item a city. O corpo indentado aplica a operação a esse item/grupo.

```
0075          fig,axes=plt.subplots(3,1,figsize=(16,9),sharex=True,constrained_layout=True)
```

Atribui a (fig, axes) o resultado de plt.subplots(3, 1, figsize=(16, 9), sharex=True, constrained_layout=True). A expressão à direita é avaliada antes da atribuição.

plt.subplots — Cria figura e eixos Matplotlib. Argumentos usados: posicional 1: 3; posicional 2: 1; figsize: (16, 9); sharex: True; constrained_layout: True.

```
0076          reference=None
```

Atribui a reference o resultado de None. A expressão à direita é avaliada antes da atribuição.

```
0077          for ax,name in zip(axes,LABELS):
```

Percorre zip(axes, LABELS); em cada iteração associa o item a (ax, name). O corpo indentado aplica a operação a esse item/grupo.

zip — Emparelha iteráveis pela posição. Argumentos usados: posicional 1: axes; posicional 2: LABELS.

```
0078              frame=pd.read_csv(root/city/name/'predictions.csv')
```

Atribui a frame o resultado de pd.read_csv(root / city / name / 'predictions.csv'). A expressão à direita é avaliada antes da atribuição.

pd.read_csv — Lê CSV em DataFrame. Argumentos usados: posicional 1: root / city / name / 'predictions.csv'.

```
0079              keys=frame[['origin','timestamp','lead','y_true']]
```

Atribui a keys o resultado de frame[['origin', 'timestamp', 'lead', 'y_true']]. A expressão à direita é avaliada antes da atribuição.

```
0080              if reference is not None:pd.testing.assert_frame_equal(reference,keys)
```

Avalia a condição reference is not None. O bloco indentado seguinte só é executado quando ela é verdadeira. Executa pd.testing.assert_frame_equal(reference, keys). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

pd.testing.assert_frame_equal — Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem. Argumentos usados: posicional 1: reference; posicional 2: keys.

```
0081              reference=keys
```

Atribui a reference o resultado de keys. A expressão à direita é avaliada antes da atribuição.

```
0082              m=metrics.query('dataset == @city and model == @name').iloc[0]
```

Atribui a m o resultado de metrics.query('dataset == @city and model == @name').iloc[0]. A expressão à direita é avaliada antes da atribuição.

metrics.query — Filtra linhas por expressão pandas. Argumentos usados: posicional 1: 'dataset == @city and model == @name'.

```
0083              dates=pd.to_datetime(frame.timestamp)
```

Atribui a dates o resultado de pd.to_datetime(frame.timestamp). A expressão à direita é avaliada antes da atribuição.

pd.to_datetime — Converte valores em datas pandas. Argumentos usados: posicional 1: frame.timestamp.

```
0084              ax.plot(dates,frame.y_true,color='#777777',lw=1.1,label='Real')
```

Executa ax.plot(dates, frame.y_true, color='#777777', lw=1.1, label='Real'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.plot — Desenha sequência de pontos/linhas no eixo. Argumentos usados: posicional 1: dates; posicional 2: frame.y_true; color: '#777777'; lw: 1.1; label: 'Real'.

```
0085              ax.plot(dates,frame.y_pred,color='#2478b5',lw=1.1,label='Previsto')
```

Executa ax.plot(dates, frame.y_pred, color='#2478b5', lw=1.1, label='Previsto'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.plot — Desenha sequência de pontos/linhas no eixo. Argumentos usados: posicional 1: dates; posicional 2: frame.y_pred; color: '#2478b5'; lw: 1.1; label: 'Previsto'.

```
0086              ax.set_title(f'{LABELS[name]} | RMSE {m.rmse:,.2f} | R² {m.r2:.3f}',loc='left')
```

Executa ax.set_title(f'{LABELS[name]} | RMSE {m.rmse:,.2f} | R² {m.r2:.3f}', loc='left'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.set_title — Define título de eixo. Argumentos usados: posicional 1: f'{LABELS[name]} | RMSE {m.rmse:,.2f} | R² {m.r2:.3f}'; loc: 'left'.

```
0087              ax.set_ylabel('Produção (m³)');ax.grid(alpha=.2);ax.legend(ncol=2,loc='upper
    right')
```

Executa ax.set_ylabel('Produção (m³)'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição. Executa ax.grid(alpha=0.2). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição. Executa ax.legend(ncol=2, loc='upper right'). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

ax.set_ylabel — Define unidade/rótulo vertical. Argumentos usados: posicional 1: 'Produção (m³)'.

ax.grid — Exibe linhas de referência. Argumentos usados: alpha: 0.2.

ax.legend — Mostra identificação das séries. Argumentos usados: ncol: 2; loc: 'upper right'.

```
0088          axes[-1].xaxis.set_major_formatter(mdates.DateFormatter('%m/%Y'))
```

Executa axes[-1].xaxis.set_major_formatter(mdates.DateFormatter('%m/%Y')). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

axes[-1].xaxis.set_major_formatter — Define formatação dos rótulos principais. Argumentos usados: posicional 1: mdates.DateFormatter('%m/%Y').

mdates.DateFormatter — Formata rótulos temporais do eixo. Argumentos usados: posicional 1: '%m/%Y'.

```
0089          fig.suptitle(f'{city.title()} — Baselines | Treino inicial 355 + validação 30 + teste
     30 dias',fontsize=18)
```

Executa fig.suptitle(f'{city.title()} — Baselines | Treino inicial 355 + validação 30 + teste 30 dias', fontsize=18). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

fig.suptitle — Define título geral da figura. Argumentos usados: posicional 1: f'{city.title()} — Baselines | Treino inicial 355 + validação 30 + teste 30 dias'; fontsize: 18.

city.title — Converte texto para capitalização de título. Argumentos usados: nenhum argumento explícito.

```
0090          fig.savefig(OUTPUT/f'{city}_baselines.png',dpi=150)
```

Executa fig.savefig(OUTPUT / f'{city}_baselines.png', dpi=150). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

fig.savefig — Exporta uma figura para arquivo. Argumentos usados: posicional 1: OUTPUT / f'{city}_baselines.png'; dpi: 150.

```
0091          plt.close(fig)
```

Executa plt.close(fig). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

plt.close — Libera recursos da figura. Argumentos usados: posicional 1: fig.

```
0094  if __name__=='__main__':
```

Avalia a condição __name__ == '__main__'. O bloco indentado seguinte só é executado quando ela é verdadeira.

```
0095      prompt_comparison()
```

Executa prompt_comparison(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

prompt_comparison — Confere completude, mesmos alvos e checkpoint; calcula métricas e redução histórica de RMSE e gera quatro gráficos de contexto 15. Argumentos usados: nenhum argumento explícito.

```
0096      baseline_plots()
```

Executa baseline_plots(). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

baseline_plots — Lê os seis resultados clássicos, exige mesmos alvos entre modelos e desenha três painéis por cidade. Argumentos usados: nenhum argumento explícito.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

57. tests/test_classical_baselines.py

Verifica isolamento temporal, seleção, escala e previsão recursiva.

SHA-256: e095a31e8dd9306d2fdbe67d0984be57157c68336af1d18a0beb8315102d69ed

Funções, contratos e pontos de conexão

```
L13: test_folds_expand_by_thirty_without_overlap(self)
```

Caso de teste: folds expand by thirty without overlap. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: expanding_validation_folds → src/evaluation/cross_validation.py:60

```
L22: test_features_and_scaler_only_use_training_history(self)
```

Caso de teste: features and scaler only use training history. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: LagRegressor → src/models/classical.py:14; LagRegressor('knn', lags=7, neighbors=3).fit → src/models/classical.py:25; LagRegressor('knn', lags=7, neighbors=3).fit → src/models/classical.py:39; lagged_samples → src/data/features/autoregressive.py:5

```
L30: test_selection_and_final_refit_do_not_receive_test_targets(self)
```

Caso de teste: selection and final refit do not receive test targets. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: select_and_forecast → src/training/classical.py:6

```
L33: fit(self, values)
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

```
L35: predict(self, horizon)
```

history é tabela histórica; horizon é quantidade prevista; future_covariates é tabela sem alvo; prompt é texto já construído para Qwen. Retorna lista de floats; cada classe escolhe backend. O método pertence à classe adaptadora deste arquivo.

```
L38: forecast(values)
```

Rotina auxiliar deste arquivo; as entradas são as da assinatura e os retornos/efeitos estão detalhados nas linhas abaixo.

Chamadas do projeto: select_and_forecast → src/training/classical.py:6

```
L47: test_real_lag_models_recursively_forecast_a_fixed_block(self)
```

Caso de teste: real lag models recursively forecast a fixed block. self é a instância de unittest.TestCase; as asserções descritas abaixo definem o contrato exigido.

Chamadas do projeto: select_and_forecast → src/training/classical.py:6

APIs externas usadas neste arquivo

all, append, assertEqual, assertTrue, copy, len, max, mean, np.arange, np.asarray, np.isfinite, np.repeat, np.sin, np.testing.assert_allclose, np.testing.assert_array_equal, patch, pd.DataFrame, zip. O catálogo final explica finalidade e parâmetros; junto a cada chamada abaixo constam os argumentos concretos.

Código comentado, linha a linha

```
0001  """Temporal boundaries, training-only preprocessing and recursive test isolation."""
```

Documentação textual do módulo/função. Descreve o contrato; não é instrução de cálculo ou acesso ao dataset.

```
0002  import unittest
```

Importa unittest. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0003  from unittest.mock import patch
```

Importa patch de unittest.mock. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0004  import numpy as np
```

Importa numpy como np. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0005  import pandas as pd
```

Importa pandas como pd. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0006  from src.data.features.autoregressive import lagged_samples
```

Importa lagged_samples de src.data.features.autoregressive. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0007  from src.evaluation.cross_validation import expanding_validation_folds
```

Importa expanding_validation_folds de src.evaluation.cross_validation. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0008  from src.models.classical import LagRegressor
```

Importa LagRegressor de src.models.classical. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0009  from src.training.classical import select_and_forecast
```

Importa select_and_forecast de src.training.classical. Disponibiliza os nomes para as chamadas deste módulo; não executa a previsão.

```
0012  class ClassicalBaselineTests(unittest.TestCase):
```

Declara a classe ClassicalBaselineTests(unittest.TestCase). Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0013      def test_folds_expand_by_thirty_without_overlap(self):
```

Define test_folds_expand_by_thirty_without_overlap(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0014          frame = pd.DataFrame({'observed': [True]*500})
```

Atribui a frame o resultado de pd.DataFrame({'observed': [True] * 500}). A expressão à direita é avaliada antes da atribuição.

pd.DataFrame — Constrói uma tabela a partir de registros ou colunas. Argumentos usados: posicional 1: {'observed': [True] * 500}.

```
0015          folds = expanding_validation_folds(frame)
```

Atribui a folds o resultado de expanding_validation_folds(frame). A expressão à direita é avaliada antes da atribuição.

expanding_validation_folds — Gera treino expansivo desde zero, validação móvel e teste completo, com avanço igual ao tamanho do teste. Rejeita alvos ausentes nos blocos pontuados. Argumentos usados: posicional 1: frame.

```
0016          self.assertEqual([(f.train_end,f.validation_end,f.test_end) for f in folds],
```

Executa self.assertEqual([(f.train_end, f.validation_end, f.test_end) for f in folds], [(355, 385, 415), (385, 415, 445), (415, 445, 475)]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: [(f.train_end, f.validation_end, f.test_end) for f in folds]; posicional 2: [(355, 385, 415), (385, 415, 445), (415, 445, 475)].

```
0017                           [(355,385,415),(385,415,445),(415,445,475)])
```

Continuação da instrução na linha 16: fornece os valores/expressões mostrados no código, na mesma chamada ou coleção.

```
0018          for previous, current in zip(folds, folds[1:]):
```

Percorre zip(folds, folds[1:]); em cada iteração associa o item a (previous, current). O corpo indentado aplica a operação a esse item/grupo.

zip — Emparelha iteráveis pela posição. Argumentos usados: posicional 1: folds; posicional 2: folds[1:].

```
0019              self.assertEqual(previous.test_end,current.validation_end)
```

Executa self.assertEqual(previous.test_end, current.validation_end). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: previous.test_end; posicional 2: current.validation_end.

```
0020              self.assertEqual(current.train_end-previous.train_end,30)
```

Executa self.assertEqual(current.train_end - previous.train_end, 30). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: current.train_end - previous.train_end; posicional 2: 30.

```
0022      def test_features_and_scaler_only_use_training_history(self):
```

Define test_features_and_scaler_only_use_training_history(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0023          values = np.arange(60.,dtype=float)
```

Atribui a values o resultado de np.arange(60.0, dtype=float). A expressão à direita é avaliada antes da atribuição.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 60.0; dtype: float.

```
0024          features, targets = lagged_samples(values[:40],7)
```

Atribui a (features, targets) o resultado de lagged_samples(values[:40], 7). A expressão à direita é avaliada antes da atribuição.

lagged_samples — values é o histórico autorizado; lags é o número de atrasos. Devolve X com lags colunas e y com o próximo valor, sem atributos posteriores ao alvo. Argumentos usados: posicional 1: values[:40]; posicional 2: 7.

```
0025          np.testing.assert_array_equal(features[-1],values[32:39])
```

Executa np.testing.assert_array_equal(features[-1], values[32:39]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

np.testing.assert_array_equal — Exige igualdade exata entre arrays. Argumentos usados: posicional 1: features[-1]; posicional 2: values[32:39].

```
0026          self.assertEqual(targets[-1],39)
```

Executa self.assertEqual(targets[-1], 39). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: targets[-1]; posicional 2: 39.

```
0027          model=LagRegressor('knn',lags=7,neighbors=3).fit(values[:40])
```

Atribui a model o resultado de LagRegressor('knn', lags=7, neighbors=3).fit(values[:40]). A expressão à direita é avaliada antes da atribuição.

LagRegressor('knn', lags=7, neighbors=3).fit — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: values[:40].

LagRegressor — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: 'knn'; lags: 7; neighbors: 3.

```
0028          np.testing.assert_allclose(model.estimator.named_steps['standardscaler'].mean_,featur
    es.mean(axis=0))
```

Executa np.testing.assert_allclose(model.estimator.named_steps['standardscaler'].mean_, features.mean(axis=0)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

np.testing.assert_allclose — Asserção numérica vetorial tolerante. Argumentos usados: posicional 1: model.estimator.named_steps['standardscaler'].mean_; posicional 2: features.mean(axis=0).

features.mean — Calcula média dos valores do objeto receptor. Argumentos usados: axis: 0.

```
0030     def test_selection_and_final_refit_do_not_receive_test_targets(self):
```

Define test_selection_and_final_refit_do_not_receive_test_targets(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0031         seen=[]
```

Atribui a seen o resultado de []. A expressão à direita é avaliada antes da atribuição.

```
0032         class Fake:
```

Declara a classe Fake. Seus métodos compartilham estado por self; a declaração não cria uma previsão.

```
0033             def fit(self, values):
```

Define fit(self, values). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0034                 self.values=np.asarray(values);seen.append(self.values.copy());return self
```

Atribui a self.values o resultado de np.asarray(values). A expressão à direita é avaliada antes da atribuição. Executa seen.append(self.values.copy()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição. Encerra esta função e devolve self ao chamador; não imprime o resultado por si só.

np.asarray — Representa dados como ndarray, evitando cópia quando possível. Argumentos usados: posicional 1: values.

seen.append — Adiciona um elemento ao final da lista. Argumentos usados: posicional 1: self.values.copy().

self.values.copy — Copia uma tabela/dicionário para não alterar diretamente o objeto anterior. Argumentos usados: nenhum argumento explícito.

```
0035             def predict(self,horizon):
```

Define predict(self, horizon). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0036                 return np.repeat(self.values[-1],horizon)
```

Encerra esta função e devolve np.repeat(self.values[-1], horizon) ao chamador; não imprime o resultado por si só.

np.repeat — Repete valores em um array. Argumentos usados: posicional 1: self.values[-1]; posicional 2: horizon.

```
0037         full=np.arange(445.,dtype=float)
```

Atribui a full o resultado de np.arange(445.0, dtype=float). A expressão à direita é avaliada antes da atribuição.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 445.0; dtype: float.

```
0038         def forecast(values):
```

Define forecast(values). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0039             with patch('src.training.classical.create_classical_model',side_effect=lambda
    *args:Fake()):
```

Abre o contexto gerenciado patch('src.training.classical.create_classical_model', side_effect=lambda *args: Fake()). Ao sair, libera o recurso ou restaura a simulação/redirecionamento, inclusive se ocorrer erro.

patch — Substitui temporariamente uma referência durante o bloco with. Argumentos usados: posicional 1: 'src.training.classical.create_classical_model'; side_effect: lambda *args: Fake().

Fake — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: nenhum argumento explícito.

```
0040                 return
    select_and_forecast('knn',values[:355],values[355:385],30,[{'lags':7}])[0]
```

Encerra esta função e devolve select_and_forecast('knn', values[:355], values[355:385], 30, [{'lags': 7}])[0] ao chamador; não imprime o resultado por si só.

select_and_forecast — Recebe nome, treino, validação, horizonte e candidatos, nunca alvos de teste. Seleciona pelo RMSE de validação, reajusta treino+validação e devolve previsões e auditoria. Argumentos usados: posicional 1: 'knn'; posicional 2: values[:355]; posicional 3: values[355:385]; posicional 4: 30; posicional 5: [{'lags': 7}].

```
0041         original=forecast(full)
```

Atribui a original o resultado de forecast(full). A expressão à direita é avaliada antes da atribuição.

forecast — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: full.

```
0042         full[385:]=999999
```

Atribui a full[385:] o resultado de 999999. A expressão à direita é avaliada antes da atribuição.

```
0043         np.testing.assert_array_equal(original,forecast(full))
```

Executa np.testing.assert_array_equal(original, forecast(full)). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

np.testing.assert_array_equal — Exige igualdade exata entre arrays. Argumentos usados: posicional 1: original; posicional 2: forecast(full).

forecast — Chamada externa; os argumentos concretos e o contexto da instrução são apresentados abaixo. Argumentos usados: posicional 1: full.

```
0044         self.assertEqual([len(x) for x in seen],[355,385,355,385])
```

Executa self.assertEqual([len(x) for x in seen], [355, 385, 355, 385]). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: [len(x) for x in seen]; posicional 2: [355, 385, 355, 385].

len — Conta elementos. Argumentos usados: posicional 1: x.

```
0045         self.assertTrue(all(x.max()<385 for x in seen))
```

Executa self.assertTrue(all((x.max() < 385 for x in seen))). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: all((x.max() < 385 for x in seen)).

all — Exige que todos os elementos sejam True. Argumentos usados: posicional 1: (x.max() < 385 for x in seen).

x.max — Retorna maior valor. Argumentos usados: nenhum argumento explícito.

```
0047     def test_real_lag_models_recursively_forecast_a_fixed_block(self):
```

Define test_real_lag_models_recursively_forecast_a_fixed_block(self). O corpo só executa quando essa função/método é chamado. Seu contrato e chamadores estão na abertura deste arquivo.

```
0048         series=100+np.sin(np.arange(90.)/3)
```

Atribui a series o resultado de 100 + np.sin(np.arange(90.0) / 3). A expressão à direita é avaliada antes da atribuição.

np.sin — Calcula seno em radianos. Argumentos usados: posicional 1: np.arange(90.0) / 3.

np.arange — Gera progressão de valores com fim exclusivo. Argumentos usados: posicional 1: 90.0.

```
0049         for name,params in
    [('linear_regression',{'lags':7}),('knn',{'lags':7,'neighbors':3})]:
```

Percorre [('linear_regression', {'lags': 7}), ('knn', {'lags': 7, 'neighbors': 3})]; em cada iteração associa o item a (name, params). O corpo indentado aplica a operação a esse item/grupo.

```
0050             predictions,audit=select_and_forecast(name,series[:50],series[50:60],30,[params])
```

Atribui a (predictions, audit) o resultado de select_and_forecast(name, series[:50], series[50:60], 30, [params]). A expressão à direita é avaliada antes da atribuição.

select_and_forecast — Recebe nome, treino, validação, horizonte e candidatos, nunca alvos de teste. Seleciona pelo RMSE de validação, reajusta treino+validação e devolve previsões e auditoria. Argumentos usados: posicional 1: name; posicional 2: series[:50]; posicional 3: series[50:60]; posicional 4: 30; posicional 5: [params].

```
0051             self.assertEqual(len(predictions),30)
```

Executa self.assertEqual(len(predictions), 30). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: len(predictions); posicional 2: 30.

len — Conta elementos. Argumentos usados: posicional 1: predictions.

```
0052             self.assertTrue(np.isfinite(predictions).all())
```

Executa self.assertTrue(np.isfinite(predictions).all()). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertTrue — Asserção de condição verdadeira. Argumentos usados: posicional 1: np.isfinite(predictions).all().

np.isfinite(predictions).all — Exige que todos os elementos sejam True. Argumentos usados: nenhum argumento explícito.

np.isfinite — Testa finitude elemento a elemento. Argumentos usados: posicional 1: predictions.

```
0053             self.assertEqual(audit['final_fit_size'],60)
```

Executa self.assertEqual(audit['final_fit_size'], 60). O resultado não é atribuído; o efeito pode ser gravar arquivo, modificar lista, imprimir ou validar uma condição.

self.assertEqual — Asserção de igualdade do unittest. Argumentos usados: posicional 1: audit['final_fit_size']; posicional 2: 60.

Linhas em branco são separadores sem execução; seus números foram preservados nos saltos da listagem. As linhas de código e comentários acima seguem o arquivo integral, sem omissão de trechos.

Catálogo de funções e métodos de biblioteca

Esta referência explica as APIs efetivamente chamadas. Em cada ocorrência na listagem, os argumentos concretos foram mostrados, inclusive os posicionais. Aqui estão seu significado e os parâmetros relevantes. Métodos homônimos (como update de dict e de hash) dependem do objeto receptor. O catálogo não deve ser lido como lista de todos os parâmetros opcionais que a biblioteca admite.

ARIMA

Constrói o modelo autorregressivo integrado de médias móveis.

Parâmetros e uso neste projeto: endog é a demanda normalizada do prefixo permitido; order=(p,d,q) define autorregressão, diferenciação e médias móveis. Não configura sazonalidade.

ChronosForecaster

Constrói adaptador Chronos2 com pesos locais.

Parâmetros e uso neste projeto: context é contexto inicial registrado; dataset fornece nomes de covariáveis. predict depois recebe história crescente de verdade.

Demonstration

Constrói dataclass de exemplo few-shot.

Parâmetros e uso neste projeto: history e target são tuplas de números; target_end é posição final exclusiva dentro do prefixo autorizado.

ForecastWindow

Constrói dataclass de uma origem temporal.

Parâmetros e uso neste projeto: context_start=0, origin=posição do primeiro alvo, horizon=sete dias. history/target são properties que devolvem slices.

KNeighborsRegressor

Regressor por vizinhos mais próximos.

Parâmetros e uso neste projeto: n_neighbors escolhe quantos exemplos históricos usar; weights é uniform ou distance; n_jobs=1 limita paralelismo.

LinearRegression

Regressão linear por mínimos quadrados.

Parâmetros e uso neste projeto: Sem argumentos explícitos; usa intercepto padrão. Ajusta coeficientes nos atrasos padronizados.

Mock

Cria objeto simulado para registrar chamadas.

Parâmetros e uso neste projeto: Sem argumentos neste teste. pipeline.predict_df.return_value é atribuído ao DataFrame artificial que predict receberá.

OllamaConfig

Constrói configuração tipada do cliente Qwen.

Parâmetros e uso neste projeto: Campos do JSON qwen e seed são argumentos nomeados. Os defaults legados só valem para campos omitidos; __post_init__ valida limites.

Path

Representa caminho de arquivo com operações portáveis.

Parâmetros e uso neste projeto: path posicional é string ou Path. O operador / combina diretórios, não faz divisão numérica nesse contexto.

QwenForecaster

Constrói adaptador Qwen e consulta servidor local.

Parâmetros e uso neste projeto: config é OllamaConfig. Valida modelo/contexto e cria identity; não chama geração até predict.

RuntimeError

Cria exceção para estado de execução inesperado.

Parâmetros e uso neste projeto: Mensagem sinaliza caminho que deveria ser inalcançável depois das tentativas.

StandardScaler

Padroniza atributos por média e desvio do ajuste.

Parâmetros e uso neste projeto: Sem argumentos explícitos; é ajustado dentro da Pipeline apenas nas linhas de atributos do histórico autorizado.

SystemExit

Solicita encerramento do processo Python.

Parâmetros e uso neste projeto: code=0 indica sucesso; 1 sinaliza falhas para scripts shell. Usada com raise no CLI.

TimesFMForecaster

Constrói adaptador TimesFM2.5 local.

Parâmetros e uso neste projeto: context é contexto inicial registrado; dataset define covariáveis. Previsão usa XReg + TimesFM.

ValueError

Cria exceção para valor/formato inválido.

Parâmetros e uso neste projeto: A string explica a violação: datas, dimensões, covariáveis, JSON ou contexto. Só é lançada quando precedida por raise ou usada por teste de exceção.

add_argument

Registra uma opção do CLI.

Parâmetros e uso neste projeto: Primeiro argumento é '--model' etc.; choices restringe valores; default define ausência; type=Path/int converte; help descreve a opção no --help. type=Path não verifica se o arquivo existe.

all

Exige que todos os elementos sejam True.

Parâmetros e uso neste projeto: Series/ndarray.all() reduz a máscara; builtin all(iterável) pode receber um gerador. Num conjunto vazio o resultado é True: outras verificações precisam excluir entradas vazias.

any

Reduz a existência de pelo menos um True.

Parâmetros e uso neste projeto: Series/ndarray.any() sem eixo reduz seus valores; DataFrame.any() reduz por coluna e o segundo .any() reduz as colunas. O builtin any(iterável) percorre um gerador de condições e para no primeiro True.

append

Adiciona um elemento ao final da lista.

Parâmetros e uso neste projeto: item pode ser dicionário de previsão, auditoria ou bloco textual. Modifica a lista e retorna None.

apply

Aplica uma função por grupo/objeto.

Parâmetros e uso neste projeto: No grupo de leads, func=lambda x: sorted(x)==list(range(1,8)) exige uma ocorrência de cada lead; .all() exige isso em toda janela.

argparse.ArgumentParser

Cria o parser da interface de linha de comando.

Parâmetros e uso neste projeto: description=__doc__ utiliza a docstring do módulo como texto de ajuda; parse_args vai interpretar os argumentos depois.

asdict

Converte dataclass em dicionário de seus campos.

Parâmetros e uso neste projeto: obj=config ou window. É usado na procedência e request_id; não converte uma classe qualquer sem dataclass.

asfreq

Reindexa a série para uma frequência temporal regular.

Parâmetros e uso neste projeto: freq='D' cria posições diárias entre começo e fim; dias ausentes ganham NaN. Não soma nem calcula média como resample(...).mean().

assertAlmostEqual

Compara números com aproximação decimal.

Parâmetros e uso neste projeto: first/second são métricas e valores esperados; tolerância padrão do unittest se não informada.

assertEqual

Asserção de igualdade do unittest.

Parâmetros e uso neste projeto: first/second são valores comparados; testa contratos discretos, como tamanho, datas ou listas de previsões.

assertFalse

Asserção de condição falsa.

Parâmetros e uso neste projeto: expr verifica, por exemplo, que um dia imputado não está marcado como observado.

assertGreaterEqual

Exige first >= second.

Parâmetros e uso neste projeto: Usada para origem nunca anteceder o contexto inicial.

assertIn

Exige item presente em container.

Parâmetros e uso neste projeto: Verifica datas ou chave de erro na auditoria de uma retentativa.

assertLess

Exige first < second.

Parâmetros e uso neste projeto: Compara a maior data histórica com a menor data de teste para detectar sobreposição.

assertNotIn

Exige item ausente de container.

Parâmetros e uso neste projeto: Usada para coluna-alvo não estar em future, e sentinela futura não estar no prompt.

assertRaises

Context manager que exige uma exceção.

Parâmetros e uso neste projeto: expected_exception é classe ou tupla de classes; o trecho dentro de with precisa falhar de modo esperado.

assertRaisesRegex

Exige exceção e mensagem compatível com expressão regular.

Parâmetros e uso neste projeto: expected_exception e expected_regex distinguem a razão de falha, como mudança no modelo da comparação.

assertTrue

Asserção de condição verdadeira.

Parâmetros e uso neste projeto: expr é uma condição já calculada; falha se falsa. Não corrige a condição.

assert_not_called

Verifica que um mock não foi chamado.

Parâmetros e uso neste projeto: Os testes exigem rejeitar contexto Qwen inválido antes de leitura de dados ou criação do modelo.

astype

Converte o tipo dos valores.

Parâmetros e uso neste projeto: dtype=str transforma categorias em texto para TimesFM. Não faz one-hot encoding; a biblioteca recebe as categorias para processar.

bar

Desenha barras de comparação.

Parâmetros e uso neste projeto: Rótulos identificam checkpoint/contexto e alturas são RMSEs calculados.

bar_label

Escreve valores sobre as barras.

Parâmetros e uso neste projeto: labels contém RMSE formatado; padding afasta texto da barra.

capitalize

Capitaliza a primeira letra do texto.

Parâmetros e uso neste projeto: Converte o identificador de cidade em nome usado no prompt.

casefold

Normaliza caixa de texto para comparação.

Parâmetros e uso neste projeto: Permite identificar Guaratuba independentemente das letras maiúsculas/minúsculas.

choice

Sorteia elementos do gerador NumPy.

Parâmetros e uso neste projeto: a=origins, size=count e replace=False selecionam origens diferentes; não sorteia do período de teste.

compile

Prepara a configuração de previsão do TimesFM.

Parâmetros e uso neste projeto: forecast_config é ForecastConfig. O nome não significa treinamento ou alteração dos pesos. É chamado novamente por origem para capacidade compatível com história crescente.

contextlib.redirect_stdout

Redireciona print temporariamente.

Parâmetros e uso neste projeto: new_target=io.StringIO() captura saída em memória, para testes não poluírem o terminal.

copy

Copia uma tabela/dicionário para não alterar diretamente o objeto anterior.

Parâmetros e uso neste projeto: DataFrame.copy() usa deep=True por padrão; para objetos Python dentro de células não significa cópia recursiva de tudo. dict.copy é superficial. Não recebe argumentos nas chamadas deste projeto.

count

Conta ocorrências na sequência.

Parâmetros e uso neste projeto: No teste, conta perguntas para exigir uma por dia do horizonte.

dataclass

Decora a classe para gerar métodos a partir dos campos anotados.

Parâmetros e uso neste projeto: frozen=True impede reatribuição comum após construção. Não executa treinamento, nem valida sozinho os tipos anotados.

date

Extrai a parte civil da data de um Timestamp.

Parâmetros e uso neste projeto: Sem argumentos. O retorno é date, que a f-string representa como YYYY-MM-DD, sem horário.

dict

Cria dicionário.

Parâmetros e uso neste projeto: Pode receber mapeamento posicional, campos nomeados e **outro_dict. dict(config) é cópia superficial; pop remove só a chave nessa cópia.

divmod

Devolve quociente inteiro e resto.

Parâmetros e uso neste projeto: Usado com bases 10, 100 ou 1000 na escrita de números; preserva o valor original.

drop

Retorna tabela sem as linhas/colunas indicadas.

Parâmetros e uso neste projeto: columns='water_produced' nos testes remove o alvo do futuro; inplace padrão False mantém o original intacto.

drop_duplicates

Retém uma linha por chave repetida.

Parâmetros e uso neste projeto: subset='origin' retém o primeiro registro de cada janela. Serve para não somar sete vezes a duração, repetida nas sete linhas do CSV.

duplicated

Marca repetições.

Parâmetros e uso neste projeto: Em DataFrame, subset=['origin','lead'] define a chave; em Series de datas não recebe subset. keep padrão 'first' preserva a primeira ocorrência como não duplicada.

encode

Codifica string em bytes.

Parâmetros e uso neste projeto: encoding='utf-8' no HTTP e orçamento do prompt; sem argumento, str.encode usa UTF-8. Bytes são usados por SHA-256 e corpo HTTP.

endswith

Verifica sufixo de string.

Parâmetros e uso neste projeto: A tupla ('_sin','_cos') identifica componentes cíclicos; retorna bool.

enumerate

Percorre iterável com contador.

Parâmetros e uso neste projeto: iterable e start=1 nos leads/janelas; o contador é separado do índice temporal do dataset.

eq

Compara igualdade elemento a elemento.

Parâmetros e uso neste projeto: other é uma Series compatível ou escalar, como 50 no teste da redução de RMSE; retorna máscara, que .all() agrega.

equals

Compara conteúdo e estrutura do receptor com outro objeto pandas.

Parâmetros e uso neste projeto: other é DataFrame ou DatetimeIndex compatível. Retorna bool; datas/ordem precisam coincidir. Não alinha ou reordena automaticamente para fazer as tabelas parecerem iguais.

error

Em ArgumentParser, mostra mensagem e encerra com erro de CLI.

Parâmetros e uso neste projeto: message é o motivo, por exemplo limite negativo. Não é o erro numérico de uma previsão.

exists

Verifica existência do caminho.

Parâmetros e uso neste projeto: Sem argumentos. Define se vale tentar ler cache já gravado.

extend

Adiciona cada elemento de um iterável à lista.

Parâmetros e uso neste projeto: iterable é lista de blocos de prompt. Diferente de append, não cria uma lista aninhada.

ffill

Propaga o último valor válido anterior para lacunas subsequentes.

Parâmetros e uso neste projeto: Sem argumentos. Não usa uma observação futura, mas não reconstrói calendário: copiar is_weekend/day para uma nova data é incorreto.

float

Converte escalar para float Python.

Parâmetros e uso neste projeto: x é valor numérico ou NumPy. Facilita serialização em JSON; não normaliza nem muda a unidade.

forecast_with_covariates

Executa regressão auxiliar e TimesFM com variáveis exógenas.

Parâmetros e uso neste projeto: inputs é lista de séries-alvo históricas; dynamic_numerical_covariates e dynamic_categorical_covariates são dicts com listas por série cobrindo passado+futuro; xreg_mode='xreg + timesfm' combina regressão e previsão residual. Retorna pontos e saídas adicionais; o segundo retorno é ignorado aqui.

from_pretrained

Carrega pesos/configuração de um checkpoint.

Parâmetros e uso neste projeto: Primeiro argumento é ID do modelo; local_files_only=True exige cache local. Em Chronos, device_map seleciona dispositivo e torch_dtype=torch.float32 força pesos nessa

precisão. Em TimesFM usa a classe própria TimesFM_2p5_200M_torch.

get

Busca chave de dicionário com padrão se ausente.

Parâmetros e uso neste projeto: key é done/response/status; default é 0 ou '' quando informado. Evita KeyError, mas padrão 0 nos contadores não prova que servidor os forneceu.

grid

Exibe linhas de referência.

Parâmetros e uso neste projeto: alpha define transparência e axis pode limitar ao eixo y.

groupby

Cria grupos sem calcular métricas por si só.

Parâmetros e uso neste projeto: by='lead' separa antecedências; by='origin' separa janelas; sort=False preserva ordem encontrada na auditoria. É necessário iterar, apply ou agregar depois.

hashlib.sha256

Cria calculador de digest SHA-256.

Parâmetros e uso neste projeto: data opcional são bytes, não texto. Na versão incremental inicia vazio e recebe update; no hash de configuração recebe JSON codificado.

hexdigest

Retorna a representação hexadecimal do digest acumulado.

Parâmetros e uso neste projeto: Sem argumentos. É texto para comparar identidades; não criptografa conteúdo para confidencialidade.

importlib.metadata.version

Lê versão instalada de uma distribuição Python.

Parâmetros e uso neste projeto: distribution_name é 'chronos-forecasting' ou 'timesfm'; não é necessariamente o mesmo nome do import. Pode falhar se a distribuição não estiver instalada.

index

Localiza a posição de um valor numa sequência.

Parâmetros e uso neste projeto: MONTHS.index recebe o nome do mês; o resultado começa em zero e soma-se um para construir Timestamp.

int

Converte para inteiro, truncando parte fracionária numérica.

Parâmetros e uso neste projeto: x é contexto, tamanho da validação ou contagem; int(training_size*fraction) arredonda para baixo para positivos.

io.StringIO

Buffer de texto em memória com interface de arquivo.

Parâmetros e uso neste projeto: initial_value=table permite pd.read_csv sem arquivo físico; vazio serve para capturar prints.

isinstance

Verifica se um objeto é instância de um tipo ou tupla de tipos.

Parâmetros e uso neste projeto: parsed,dict e values,list validam formato do JSON. Para números o código prefere type(v) para excluir bool.

isna

Retorna máscara booleana de valores ausentes.

Parâmetros e uso neste projeto: Sem argumentos. Em DataFrame a máscara tem o mesmo formato; em Series tem uma entrada por linha.

issubset

Verifica se todos os elementos estão contidos em outro conjunto/iterável.

Parâmetros e uso neste projeto: other são colunas do DataFrame; valida presença do contrato sem exigir que não existam colunas extras.

items

Percorre pares (chave, valor) do dicionário.

Parâmetros e uso neste projeto: Sem argumentos. main obtém name,dataset da configuração.

iterrows

Itera pares (índice, Series da linha).

Parâmetros e uso neste projeto: Sem argumentos. O pipeline ignora o índice com _ e usa observation.timestamp/water_produced para registrar cada lead.

itertuples

Itera linhas como tuplas nomeadas.

Parâmetros e uso neste projeto: As métricas fornecem row.model, row.context e row.rmse para rótulos/gráficos; a ordem acompanha a tabela.

join

Une textos com o separador que recebe o método.

Parâmetros e uso neste projeto: '\\n'.join(parts) organiza prompt; ','.join(...) organiza datas/valores; '|'.join(...) registra nomes de covariáveis. Todos os elementos devem ser strings.

json.dumps

Serializa objeto Python em string JSON.

Parâmetros e uso neste projeto: obj=value; sort_keys=True estabiliza hashes; ensure_ascii=False preserva acentos; indent=2 melhora leitura; allow_nan=False rejeita NaN/inf na persistência. Nem todos esses argumentos aparecem em toda chamada.

json.load

Decodifica JSON lido de um arquivo/response.

Parâmetros e uso neste projeto: fp é o fluxo retornado por urlopen. Diferente de loads, não recebe diretamente a string do modelo.

json.loads

Decodifica uma string JSON em objetos Python.

Parâmetros e uso neste projeto: s=text; JSON null vira None, true vira True, arrays viram listas. Pode aceitar NaN no parser padrão, por isso parse_prediction testa math.isfinite depois.

legend

Mostra identificação das séries.

Parâmetros e uso neste projeto: loc e ncol controlam posição e colunas de Real/Previsto.

len

Conta elementos.

Parâmetros e uso neste projeto: Objeto pode ser histórico, previsões ou texto/bytes. len(prompt) não é contagem de tokens; len(history) conta linhas.

list

Materializa um iterável em lista.

Parâmetros e uso neste projeto: iterável pode ser colunas, range ou tupla; pode também representar tipo em isinstance.

lower

Converte letras para minúsculas.

Parâmetros e uso neste projeto: Normaliza indicadores booleanos recebidos como texto; os rótulos válidos são reconhecidos antes.

make_pipeline

Encadeia transformações e estimador.

Parâmetros e uso neste projeto: Recebe StandardScaler e regressor em ordem; fit aprende a escala antes de ajustar o regressor e predict reutiliza essa escala.

map

Aplica uma função a cada valor da Series.

Parâmetros e uso neste projeto: Na conversão aplica date_to_text, number_to_words ou uma função de indicadores; na ingestão textual aplica date_from_text somente a timestamp.

math.isfinite

Confere se um único número é finito.

Parâmetros e uso neste projeto: x=v; combina-se com checagem de tipo e sinal para rejeitar NaN, inf e valores negativos.

matplotlib.use

Seleciona backend de renderização.

Parâmetros e uso neste projeto: Agg permite exportar PNG sem janela gráfica; precisa ser definido antes de importar pyplot.

max

Retorna maior valor.

Parâmetros e uso neste projeto: max(context_start,origin-history_days) impede começo negativo; max(std,1e-8) evita divisão por zero; método de Series reduz valores.

mdates.DateFormatter

Formata rótulos temporais do eixo.

Parâmetros e uso neste projeto: %m/%Y apresenta mês/ano no gráfico; não modifica timestamps dos dados.

mean

Calcula média dos valores do objeto receptor.

Parâmetros e uso neste projeto: Sem argumentos nas Series de duração/demanda e arrays de erro; o receptor já foi filtrado para a janela ou subconjunto correto.

min

Retorna menor valor.

Parâmetros e uso neste projeto: Recebe limites para validar positividade, tempos de espera ou timestamps. Método Series.min opera sobre a coluna.

mkdir

Cria diretório.

Parâmetros e uso neste projeto: parents=True cria pais ausentes; exist_ok=True aceita diretório já existente. Sem exist_ok, colisão pode lançar FileExistsError.

normalize

Zera o horário de cada timestamp, preservando sua data.

Parâmetros e uso neste projeto: Series.dt.normalize() não recebe argumentos aqui. A comparação com a coluna original rejeita horários diferentes de meia-noite.

notna

Retorna True onde o valor não é ausente.

Parâmetros e uso neste projeto: Sem argumentos. Na grade diária, define observed antes de preencher water_produced, preservando quais alvos eram reais.

np.abs

Módulo elemento a elemento.

Parâmetros e uso neste projeto: x pode ser erro ou alvo; remove o sinal para MAE/MAPE/sMAPE, mas o viés auditado mantém o sinal.

np.allclose

Verifica proximidade numérica elemento a elemento.

Parâmetros e uso neste projeto: As componentes fornecidas são comparadas a sin/cos dos índices recuperados; atol=1e-6 tolera precisão da representação.

np.arange

Gera progressão de valores com fim exclusivo.

Parâmetros e uso neste projeto: stop único produz 0...stop-1; start,stop definem origens válidas. dtype=float nos testes força floats; passo padrão é 1.

np.arctan2

Calcula ângulo a partir de seno e cosseno.

Parâmetros e uso neste projeto: Primeiro argumento é seno, segundo cosseno; preserva o quadrante e permite recuperar a posição no ciclo.

np.argsort

Retorna índices que ordenariam os valores.

Parâmetros e uso neste projeto: a=distances; kind='stable' preserva ordem de empates. [:count] seleciona os exemplos mais semelhantes.

np.asarray

Representa dados como ndarray, evitando cópia quando possível.

Parâmetros e uso neste projeto: a é o vetor/lista recebido; dtype=float converte para ponto flutuante. Não valida finitude sozinho.

np.concatenate

Concatena arrays.

Parâmetros e uso neste projeto: [train,valid] forma o prefixo autorizado para reajuste; nenhum alvo do teste é concatenado.

np.cos

Calcula cosseno em radianos.

Parâmetros e uso neste projeto: Recebe ângulos do ciclo reconstruído para validar a categoria.

np.divide

Divide vetores com máscara e destino explícitos.

Parâmetros e uso neste projeto: x1=2*abs(error), x2=denom, out=zeros_like(error), where=denom!=0. Posições de denominador zero mantêm zero; evita NaN no sMAPE quando real e previsto são zero.

np.isfinite

Testa finitude elemento a elemento.

Parâmetros e uso neste projeto: x é array numérico; retorna False para NaN e infinitos. .all() transforma em condição única.

np.lib.stride_tricks.sliding_window_view

Cria janelas consecutivas sobre um array.

Parâmetros e uso neste projeto: values é a série histórica; lags+1 inclui atributos anteriores e alvo seguinte. O código copia X e y antes de retorná-los.

np.mean

Calcula média aritmética.

Parâmetros e uso neste projeto: a é o vetor; axis omitido considera todos os elementos. Em métricas deve receber erros já alinhados, nunca listas filtradas separadamente.

np.mod

Calcula resto elemento a elemento.

Parâmetros e uso neste projeto: O divisor 2*pi normaliza ângulos para um ciclo completo; não ajusta parâmetros a partir de teste.

np.random.default_rng

Cria um gerador pseudoaleatório local.

Parâmetros e uso neste projeto: seed=42 torna a seleção de demonstrações repetível sem depender do estado global de np.random.

np.repeat

Repete valores em um array.

Parâmetros e uso neste projeto: Usado no estimador simulado dos testes para produzir previsões de comprimento determinado.

np.rint

Arredonda valores ao inteiro mais próximo.

Parâmetros e uso neste projeto: A posição angular multiplicada pelo número de categorias produz índices de dia/estação.

np.sin

Calcula seno em radianos.

Parâmetros e uso neste projeto: Recebe ângulos do ciclo reconstruído para validar a categoria.

np.sqrt

Raiz quadrada elemento a elemento ou escalar.

Parâmetros e uso neste projeto: x é MSE; produz RMSE nas mesmas unidades do alvo.

np.std

Calcula desvio-padrão.

Parâmetros e uso neste projeto: a é janela histórica; ddof padrão 0. É usado para padronizar forma na seleção por similaridade, não para transformar o alvo de teste.

np.sum

Soma elementos.

Parâmetros e uso neste projeto: a é erro ao quadrado ou desvios do alvo; axis omitido soma tudo. Resultados NumPy são convertidos por float para JSON.

np.testing.assert_allclose

Asserção numérica vetorial tolerante.

Parâmetros e uso neste projeto: actual e desired são resultados e referência; usa tolerâncias padrão se não especificadas. Lança AssertionError e é usada para testar métricas e fidelidade do prompt.

np.testing.assert_array_equal

Exige igualdade exata entre arrays.

Parâmetros e uso neste projeto: Compara resultados nos testes; divergência de forma ou valores falha a asserção.

np.zeros_like

Cria zeros com formato e tipo semelhantes ao array.

Parâmetros e uso neste projeto: a=error fornece o formato do destino de divide. Não altera error.

nunique

Conta valores distintos.

Parâmetros e uso neste projeto: Sem argumentos. history_start.nunique()==1 verifica que todas as janelas começam na mesma data; origin.nunique() conta janelas.

object.__new__

Cria instância sem executar seu __init__.

Parâmetros e uso neste projeto: cls é QwenForecaster ou ChronosForecaster. Exclusivo dos testes para evitar servidor/download/GPU; é preciso preencher atributos manualmente depois.

parse_args

Interpreta argumentos do processo e retorna Namespace.

Parâmetros e uso neste projeto: Sem args explícitos, lê sys.argv[1:]. main acessa args.model/config/dataset/output/limit.

patch

Substitui temporariamente uma referência durante o bloco with.

Parâmetros e uso neste projeto: target é nome completo no módulo consumidor; side_effect=answers entrega respostas sequenciais; patch(time.sleep) evita esperas nos testes. Não altera permanentemente o arquivo-fonte.

pd.DataFrame

Constrói uma tabela a partir de registros ou colunas.

Parâmetros e uso neste projeto: data posicional pode ser lista de dicts (linhas) ou dict de listas (colunas); os comprimentos devem ser compatíveis. Colunas escalares em testes são propagadas pelo índice definido pelas demais.

pd.DatetimeIndex

Constrói um índice temporal para comparação.

Parâmetros e uso neste projeto: data é a coluna timestamp futura; é comparada com dates para exigir as mesmas datas e ordem.

pd.Timedelta

Representa uma duração temporal.

Parâmetros e uso neste projeto: days=1 adiciona um dia à última observação; essa expressão desloca a primeira data prevista para o dia seguinte.

pd.Timestamp

Cria instante temporal pandas.

Parâmetros e uso neste projeto: A ingestão textual fornece year, month e day inteiros; valores calendáricos inválidos lançam erro. Não usa a demanda para descobrir a data.

pd.concat

Concatena tabelas.

Parâmetros e uso neste projeto: objs=[história, futuro] e axis padrão 0 empilham linhas; ignore_index=True renumera o índice. Aqui só concatena covariáveis, nunca alvos futuros.

pd.date_range

Produz uma sequência ordenada de timestamps.

Parâmetros e uso neste projeto: start é a primeira data; periods é a quantidade, não a data final; freq='D' incrementa um dia por elemento.

pd.read_csv

Lê CSV em DataFrame.

Parâmetros e uso neste projeto: path aponta o arquivo; na ingestão textual, timestamp é interpretado por date_from_text; no gráfico parse_dates converte datas ISO dos CSVs de previsões.

pd.testing.assert_frame_equal

Verifica igualdade detalhada entre dois DataFrames e lança AssertionError se diferirem.

Parâmetros e uso neste projeto: left/right são tabelas com colunas de alinhamento ou covariáveis; o código usa padrões da biblioteca para checar também índices e tipos. É uma asserção, não retorna uma tabela corrigida.

pd.testing.assert_series_equal

Exige igualdade de duas Series pandas.

Parâmetros e uso neste projeto: Compara timestamps originais e recuperados do CSV textual; erro interrompe o teste.

pd.to_datetime

Converte valores em datas pandas.

Parâmetros e uso neste projeto: arg é a coluna/lista de datas; errors='raise' interrompe se algo não puder ser interpretado. Não inventa datas para entradas inválidas.

pd.to_numeric

Converte texto/objetos em números.

Parâmetros e uso neste projeto: arg é a Series; errors='raise' rejeita texto não numérico, em vez de convertê-lo silenciosamente para NaN.

plot

Desenha sequência de pontos/linhas no eixo.

Parâmetros e uso neste projeto: x recebe timestamps e y recebe y_true ou y_pred; cor distingue observado e previsto; nenhum filtro de erro é aplicado.

plt.close

Libera recursos da figura.

Parâmetros e uso neste projeto: fig identifica a figura concluída, evitando acumular memória entre exportações.

plt.rcParams.update

Define padrões visuais da sessão Matplotlib.

Parâmetros e uso neste projeto: Dicionário ajusta fonte e bordas; não muda números ou seleção de pontos.

plt.show

Solicita exibição da figura no backend ativo.

Parâmetros e uso neste projeto: Na rotina histórica é opcional; exportação para PNG usa savefig e independe de janela gráfica.

plt.style.use

Aplica um estilo visual Matplotlib.

Parâmetros e uso neste projeto: O estilo altera parâmetros de apresentação, não dados.

plt.subplots

Cria figura e eixos Matplotlib.

Parâmetros e uso neste projeto: 2,1 cria dois painéis; sharex=True compartilha datas; figsize define polegadas; constrained_layout ajusta margens.

plt.tight_layout

Ajusta espaçamento entre eixos e rótulos.

Parâmetros e uso neste projeto: Sem argumentos nas chamadas listadas; previne sobreposição na figura histórica.

predict

Interface comum dos adaptadores do projeto.

Parâmetros e uso neste projeto: history é tabela histórica; horizon é quantidade prevista; future_covariates é tabela sem alvo; prompt é texto já construído para Qwen. Retorna lista de floats; cada classe escolhe backend.

predict_df

Chronos: prevê uma tabela temporal com covariáveis.

Parâmetros e uso neste projeto: frame é histórico; future_df é tabela futura sem alvo; prediction_length=horizon; freq='D'; batch_size=1; id_column, timestamp_column e target identificam colunas; quantile_levels=[0.1,0.5,0.9]; context_length=len(frame) pede todo o histórico. Retorno é DataFrame; coluna '0.5' é mediana.

print

Escreve representação textual na saída padrão.

Parâmetros e uso neste projeto: flush=True força envio imediato ao log de progresso; index=False pertence a to_string, não a print.

query

Filtra linhas por expressão pandas.

Parâmetros e uso neste projeto: Expressões sobre dataset, slug e context selecionam métricas da figura. @city, @slug e @days referem-se às variáveis do escopo.

range

Gera inteiros com limite final exclusivo.

Parâmetros e uso neste projeto: start, stop, step definem origens; range(retries+1) inclui tentativa inicial e retentativas; range(1,8) gera sete leads.

re.fullmatch

Exige correspondência da string inteira com expressão regular.

Parâmetros e uso neste projeto: O padrão possui três grupos: ano escrito, dia escrito e nome do mês; sem correspondência, retorna None.

read_bytes

Lê bytes integrais do arquivo.

Parâmetros e uso neste projeto: Sem argumentos. O hash do dataset usa bytes originais, incluindo quebras de linha.

read_text

Lê todo o arquivo como string.

Parâmetros e uso neste projeto: Sem encoding explícito, usa a configuração padrão do ambiente. JSON e prompts esperam texto compatível; não retorna dict sem json.loads.

relative_to

Calcula caminho relativo a um ancestral de um Path.

Parâmetros e uso neste projeto: other=ROOT; o resultado entra no hash de fonte para considerar nome/local do arquivo sem depender do prefixo absoluto da máquina.

rename

Renomeia colunas de um DataFrame.

Parâmetros e uso neste projeto: columns mapeia y_true para Real e y_pred para Predito, sem mudar os valores das previsões.

rename_axis

Nomeia o eixo do índice.

Parâmetros e uso neste projeto: mapper='timestamp' permite que reset_index restaure uma coluna com esse nome.

replace

Substitui o destino pelo arquivo representado pelo Path.

Parâmetros e uso neste projeto: target=path final. No write_json, fonte e destino estão no mesmo diretório; reduz chance de deixar JSON parcial se o processo cair. Não é bloqueio contra dois escritores simultâneos.

reset_index

Transforma o índice atual em coluna ou o descarta e cria índice sequencial.

Parâmetros e uso neste projeto: drop=True descarta o índice anterior na ordenação/comparação; sem drop, a data do índice volta a ser coluna após asfreq.

reshape

Muda a forma de um array.

Parâmetros e uso neste projeto: (1,-1) cria uma única linha de atributos com todos os atrasos para prever o próximo dia.

resolve

Resolve caminho absoluto e links simbólicos.

Parâmetros e uso neste projeto: Sem argumentos aqui. Path(__file__).resolve().parents[2] encontra a raiz a partir do módulo.

rglob

Percorre recursivamente caminhos compatíveis com padrão.

Parâmetros e uso neste projeto: pattern='*.py' encontra fontes dentro de src; sorted fixa a ordem antes de compor o hash.

savefig

Exporta uma figura para arquivo.

Parâmetros e uso neste projeto: Path define destino; dpi=150 controla resolução dos PNGs. Não salva previsões ou altera métricas.

set

Cria conjunto de elementos únicos.

Parâmetros e uso neste projeto: iterável são colunas/chaves. Subtração detecta ausentes, interseção detecta nomes proibidos; não preserva ordem como contrato.

set_color

Define cor de uma borda do eixo.

Parâmetros e uso neste projeto: Recebe código de cor da paleta; afeta aparência, não valores.

set_index

Transforma coluna em índice do DataFrame.

Parâmetros e uso neste projeto: keys='timestamp'; drop padrão True remove a coluna enquanto ela é índice. É necessário para asfreq temporal.

set_major_formatter

Define formatação dos rótulos principais.

Parâmetros e uso neste projeto: DateFormatter modifica a apresentação do eixo de datas.

set_title

Define título de eixo.

Parâmetros e uso neste projeto: Texto identifica contexto e métricas; loc e fontsize controlam posição/estilo.

set_visible

Mostra ou oculta uma borda do eixo.

Parâmetros e uso neste projeto: False remove as bordas superior/direita do gráfico.

set_ylabel

Define unidade/rótulo vertical.

Parâmetros e uso neste projeto: Produção é expressa em m³; nas barras, o eixo representa RMSE em m³.

set_ylim

Define limites verticais do eixo.

Parâmetros e uso neste projeto: Nas barras, limite superior reserva espaço acima do maior RMSE; nenhuma barra é recortada.

slice

Representa intervalo, não materializa dados.

Parâmetros e uso neste projeto: start é inclusivo e stop exclusivo; df.iloc[slice(...)] efetua a seleção depois.

sort_values

Ordena linhas pelos valores de colunas.

Parâmetros e uso neste projeto: by é 'timestamp', 'rmse' ou ['origin','lead']; ascending=False coloca erros maiores primeiro. Não modifica inplace nestas chamadas: devolve um novo objeto.

sorted

Devolve uma lista ordenada.

Parâmetros e uso neste projeto: iterável são caminhos, origens ou leads; key/reverse alteram critério/ordem quando fornecidos. Estabilidade ajuda hash e ordem de demonstrações.

split

Divide uma string por separador.

Parâmetros e uso neste projeto: sep='\\nEND HISTORY:' ou marcador CSV identifica segmentos do prompt no teste; a indexação [0]/[1] escolhe trecho esperado.

startswith

Testa prefixo textual exato.

Parâmetros e uso neste projeto: Os argumentos concretos usados nesta edição são descritos junto a cada chamada na listagem do arquivo. Consulte o tipo do objeto receptor para distinguir métodos homônimos.

std

Desvio-padrão do receptor quando usado como método.

Parâmetros e uso neste projeto: Neste projeto a padronização principal usa np.std; não confundir com pandas.std, cujo ddof padrão pode diferir.

str

Produz representação textual.

Parâmetros e uso neste projeto: x pode ser exceção, timestamp ou caminho. Usado em logs/hashes; não faz validação de conteúdo.

strftime

Formata datas como texto.

Parâmetros e uso neste projeto: %Y-%m-%d identifica origens e datas nos artefatos; o corpo narrativo usa date_to_text para datas em inglês.

strip

Remove espaços e quebras nas bordas do texto.

Parâmetros e uso neste projeto: Sem argumentos. Tira a quebra final de to_csv para manter o layout previsível, sem alterar os campos internos.

subTest

Identifica um caso dentro de um loop de teste.

Parâmetros e uso neste projeto: text=text rotula a resposta inválida corrente para diagnóstico individual.

sum

Soma elementos de iterável.

Parâmetros e uso neste projeto: No builtin, conta condições True como 1; no método de Series soma duração ou máscara. Não calcula média por si só.

suptitle

Define título geral da figura.

Parâmetros e uso neste projeto: Identifica cidade, checkpoint e uso do dataset textual.

tempfile.TemporaryDirectory

Cria diretório temporário gerenciado.

Parâmetros e uso neste projeto: Sem argumentos; with remove ao sair. Permite simular resultados sem sobrescrever experimentos reais.

tick_params

Ajusta estilo dos marcadores e rótulos dos eixos.

Parâmetros e uso neste projeto: Os argumentos concretos controlam cores e tamanhos, sem mudar os pontos previstos.

time.monotonic

Relógio crescente para medir duração.

Parâmetros e uso neste projeto: Sem argumentos. A diferença final-inicial mede segundos transcorridos, sem depender da data civil.

time.sleep

Suspende o processo por uma duração.

Parâmetros e uso neste projeto: secs=min(2**attempt,4) aplica espera de 1,2,4... segundos entre tentativas, limitada a quatro.

timesfm.ForecastConfig

Configura os limites e opções de inferência do TimesFM.

Parâmetros e uso neste projeto: max_context arredonda histórico para múltiplo de 32; max_horizon arredonda horizonte para múltiplo de 128. normalize_inputs normaliza internamente; use_continuous_quantile_head ativa cabeça contínua; force_flip_invariance estende invariância de sinal; infer_is_positive preserva não negatividade para entrada não negativa; fix_quantile_crossing corrige cruzamentos; return_backcast disponibiliza estimativas históricas usadas no caminho com regressão.

title

Converte texto para capitalização de título.

Parâmetros e uso neste projeto: Usado para exibir o nome da cidade, sem modificar o dataset.

to_csv

Serializa DataFrame como CSV.

Parâmetros e uso neste projeto: path_or_buf ausente retorna string; um Path grava arquivo. index=False evita coluna extra de índice; float_format='%.6g' usa seis algarismos significativos. Não muda o DataFrame original.

to_numpy

Converte Series/DataFrame em array NumPy.

Parâmetros e uso neste projeto: dtype=float força formato numérico quando fornecido; sem dtype conserva tipo inferido. Não ordena linhas e não corrige alinhamento.

to_string

Formata tabela como texto para terminal/log.

Parâmetros e uso neste projeto: index=False omite índice auxiliar, conservando colunas. Não grava CSV.

tolist

Converte array/índice/Series em lista Python.

Parâmetros e uso neste projeto: Sem argumentos. É usado para serialização e para compatibilizar o retorno dos modelos com o pipeline.

torch.cuda.is_available

Consulta disponibilidade do backend CUDA.

Parâmetros e uso neste projeto: Sem argumentos. Seleciona device_map='cuda' ou 'cpu' para Chronos; não instala drivers.

torch.manual_seed

Define semente do gerador PyTorch.

Parâmetros e uso neste projeto: seed=42. Ajuda reprodutibilidade, mas não habilita sozinho todos os modos determinísticos de GPU.

torch.set_float32_matmul_precision

Configura precisão permitida para multiplicações de matrizes float32.

Parâmetros e uso neste projeto: precision='high' escolhe política do backend para acelerar sob tolerância; não muda a dtype declarada de todos os tensores.

tuple

Materializa sequência imutável.

Parâmetros e uso neste projeto: iterável é histórico/alvo de demonstração. A imutabilidade da tupla ajuda o contrato da dataclass.

type

Obtém a classe concreta do valor.

Parâmetros e uso neste projeto: type(v) not in (int,float) rejeita bool apesar de herdar de int; type(exc).__name__ registra nome da exceção.

unittest.main

Executa testes quando o arquivo é ponto de entrada.

Parâmetros e uso neste projeto: Sem argumentos no código; discover também pode localizar as classes sem executar esse bloco.

update

Acrescenta conteúdo ao objeto mutável.

Parâmetros e uso neste projeto: Em hash.update(data), data são bytes incorporados ao digest; em dict.update(...), argumentos nomeados adicionam/substituem campos como status e audit. O receptor distingue as duas APIs.

urllib.request.Request

Monta uma requisição HTTP.

Parâmetros e uso neste projeto: url é endpoint; data=json.dumps(payload).encode('utf-8') é corpo em bytes e induz POST; headers declara Content-Type application/json. Não envia a rede até urlopen.

urllib.request.urlopen

Abre conexão HTTP e devolve resposta como fluxo.

Parâmetros e uso neste projeto: url ou Request é primeiro argumento; timeout em segundos limita operações de espera da conexão. with fecha a resposta. GET ocorre quando recebe URL sem corpo, como /api/tags.

values

Percorre valores do dicionário.

Parâmetros e uso neste projeto: Sem argumentos. Comparador percorre procedências; testes percorrem datasets/covariáveis.

warnings.catch_warnings

Isola temporariamente a política de avisos.

Parâmetros e uso neste projeto: O bloco with restaura a política anterior ao terminar o ajuste ARIMA.

warnings.simplefilter

Define tratamento de uma categoria de avisos.

Parâmetros e uso neste projeto: action=error e ConvergenceWarning transformam não convergência em exceção registrada como falha.

with_suffix

Produz outro Path trocando a extensão.

Parâmetros e uso neste projeto: suffix='.tmp' transforma run.json em run.tmp, usado antes da substituição atômica.

write_text

Grava uma string em arquivo e retorna a quantidade de caracteres escrita.

Parâmetros e uso neste projeto: data é a string; não cria pais automaticamente. Nestes usos, encoding não é explícito. Substitui conteúdo anterior.

zip

Emparelha iteráveis pela posição.

Parâmetros e uso neste projeto: truth.iterrows() e previsões precisam da mesma ordem. zip padrão para no menor comprimento, então tamanho é validado antes.

Como executar e localizar o resultado

```
conda run -n baseline python -m src.pipelines.classical_baselines
```

Executa os três baselines 355/30/30 nas duas cidades, com seleção por validação e reajuste pré-teste.

```
conda run -n baseline python -m src.visualization.comparison_results
```

Lê resultados existentes e gera CSVs e seis gráficos comparativos, sem executar Qwen.

```
bash experiments/start_ollama_context_server.sh
```

Inicia Ollama local na porta 11435. Se o servidor já estiver ativo, reutilize-o. Não modifica ambientes Conda.

```
conda run -n baseline python -m src.data.preprocessing.calendar_text
```

Gera as duas cópias com calendário em inglês em data/processed/calendar_text.

```
conda run -n promptcast python -m src.pipelines.qwen_text_experiment --prepare-only
```

Gera e verifica os 632 prompts a partir dos CSVs textuais, sem chamar os modelos.

```
conda run -n promptcast python -m src.pipelines.qwen_text_experiment
```

Executa as oito avaliações locais e resume os resultados completos. Reaproveita somente caches válidos.

```
conda run -n promptcast python -m src.pipelines.qwen_text_experiment --summarize-only
```

Recalcula métricas das oito avaliações já concluídas, sem nova inferência.

```
conda run -n baseline python -m src.visualization.qwen_text_results
```

Valida os resultados e gera seis figuras em experiments/qwen_text/plots.

```
conda run -n baseline python -m unittest discover -s tests -v
```

Executa os testes. Mocks não substituem inferência real; o teste textual verifica leitura direta e isolamento temporal.

```
python3 experiments/reports/guide_source/build_guide.py
```

Reconstrói PDF, HTML, manifesto, cobertura de linhas e catálogo de chamadas com o ReportLab já instalado no Python do sistema.

O ambiente selecionado precisa conter dependências já instaladas. O usuário exige consulta antes de modificar ambientes conda. O comando instalado qwen_forecast depende de instalação do pacote naquele ambiente; python -m funciona pela localização de src quando executado a partir da raiz.

Para estudar, não é necessário executar novamente centenas de janelas. Abra uma resposta em responses, compare audit.request.prompt com o .txt correspondente e associe predictions[i] à linha lead=i+1 do CSV. Para mudar código, use validação anterior ao teste; não escolha uma formulação de prompt por desempenho em janelas finais já inspecionadas.

Cobertura, fontes e reprodução do documento

Foram incluídos 43 arquivos de código/configuração/script e 11 inicializadores vazios. Todas as 2566 linhas físicas estão cobertas: linhas com conteúdo recebem código e anotação; linhas em branco são separadores registrados no manifesto. A extração usa os arquivos reais, não trechos reescritos.

A referência primária é o próprio repositório. As explicações descrevem as chamadas e contratos dos adaptadores presentes neste retrato; não afirmam comportamento de versões futuras das bibliotecas. Bibliotecas possuem versões instaladas distintas por ambiente; os parâmetros documentados correspondem às chamadas deste código, não a uma promessa para qualquer versão futura.

Arquivos gerados: guia_qwen_forecast.pdf, guia_qwen_forecast.html, guide_source/manifest.json e guide_source/line_coverage.json. O conteúdo editorial e o gerador estão em guide_source/content.py e guide_source/build_guide.py, fora de src para não invalidar hashes de inferência. O gerador documenta o produto, não integra sua execução.

Para reconstruir neste ambiente, execute python experiments/reports/guide_source/build_guide.py com o Python de sistema, que já possui ReportLab. Isso não altera ambientes conda. O manifesto é a referência para confirmar a qual versão de cada arquivo uma explicação pertence.